BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC NÔNG LÂM TP. HỒ CHÍ MINH

KHOA CÔNG NGHỆ THÔNG TIN

TIỂU LUẬN TỐT NGHIỆP

XÂY DỰNG HỆ THỐNG THƯƠNG MẠI ĐIỆN TỬ THIẾT BỊ ĐIỆN TỬ THÔNG MINH TÍCH HỢP AI VÀ XÁC THỰC QR CODE VÀ KIẾN TRÚC MICROSERVICES

Ngành : Công nghệ thông tin

Niên khóa : 2022–2026

Lớp : DH22DTB

Sinh viên thực hiện : Nguyễn Lê Hoàng Khang

TP. Hồ Chí Minh, tháng 08 năm 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC NÔNG LÂM TP. HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN



TIỂU LUẬN TỐT NGHIỆP

XÂY DỰNG HỆ THỐNG THƯƠNG MẠI ĐIỆN TỬ THIẾT BỊ ĐIỆN TỬ THÔNG MINH TÍCH HỢP AI VÀ XÁC THỰC QR CODE VÀ KIẾN TRÚC MICROSERVICES

CÁN BỘ HƯỚNG DẪN

TS. Nguyễn Thị Phương Trâm

SINH VIÊN THỰC HIỆN

Nguyễn Lê Hoàng Khang (MSSV: 22130116)

TP. HỒ CHÍ MINH, tháng 08 năm 2026

DANH MỤC CHỮ VIẾT TẮT

Viết tắt	Tên đầy đủ	Giải thích
ACID	Atomicity, Consistency, Isolation, Durability	Nhóm tính chất bảo đảm độ tin cậy của giao dịch dữ liệu trong hệ quản trị cơ sở dữ liệu.
AI	Artificial Intelligence	Trí tuệ nhân tạo; được sử dụng cho hệ thống gợi ý và trợ lý ảo.
API	Application Programming Interface	Giao diện lập trình ứng dụng, dùng để kết nối các thành phần trong hệ thống.
CI/CD	Continuous Integration / Continuous Delivery	Quy trình tích hợp và triển khai liên tục.
COD	Cash on Delivery	Hình thức thanh toán khi nhận hàng.
CRUD	Create, Read, Update, Delete	Bốn thao tác cơ bản khi quản lý dữ liệu.
CSS	Cascading Style Sheets	Ngôn ngữ định kiểu dùng để trình bày giao diện web.
DSRM	Design Science Research Methodology	Phương pháp nghiên cứu khoa học thiết kế.
ERD	Entity Relationship Diagram	Sơ đồ thực thể và quan hệ dùng để mô hình hóa cơ sở dữ liệu.
FE	Front-end	Phân hệ giao diện người dùng.
GHN	Giao Hàng Nhanh	Dịch vụ vận chuyển được tích hợp trong hệ thống.
HMAC	Hash-Based Message Authentication Code	Mã xác thực thông điệp dựa trên hàm băm.
HTML	HyperText Markup Language	Ngôn ngữ đánh dấu siêu văn bản.
HTTP	Hypertext Transfer Protocol	Giao thức truyền tải siêu văn bản.
ID	Identifier	Mã định danh của một đối tượng hoặc bản ghi.
IMEI	International Mobile Equipment Identity	Mã định danh thiết bị di động.
JPA	Java Persistence API	Chuẩn ánh xạ và thao tác dữ liệu trong hệ sinh thái Java.
JSON	JavaScript Object Notation	Định dạng dữ liệu dùng để trao đổi thông tin qua API.

Viết tắt	Tên đầy đủ	Giải thích
JWT	JSON Web Token	Chuẩn token dùng cho xác thực và duy trì phiên đăng nhập.
KPI	Key Performance Indicator	Chỉ số đo lường hiệu quả hoạt động.
LLM	Large Language Model	Mô hình ngôn ngữ lớn.
NIM	NVIDIA Inference Microservices	Nền tảng vi dịch vụ suy luận mô hình của NVIDIA.
OAuth2	Open Authorization 2.0	Chuẩn ủy quyền cho ứng dụng và dịch vụ.
POS	Point of Sale	Hệ thống hoặc điểm bán hàng.
QR	Quick Response	Mã phản hồi nhanh dùng để lưu trữ và truy xuất thông tin.
RAG	Retrieval-Augmented Generation	Kỹ thuật sinh nội dung có tăng cường truy xuất thông tin.
RAM	Random Access Memory	Bộ nhớ truy cập ngẫu nhiên.
RBAC	Role-Based Access Control	Cơ chế kiểm soát truy cập dựa trên vai trò.
REST	Representational State Transfer	Phong cách kiến trúc cho hệ thống phân tán.
SEO	Search Engine Optimization	Tối ưu hóa công cụ tìm kiếm.
SOA	Service-Oriented Architecture	Kiến trúc hướng dịch vụ.
SPA	Single-Page Application	Ứng dụng web một trang.
SQL	Structured Query Language	Ngôn ngữ truy vấn có cấu trúc.
TMĐT	Thương mại điện tử	Hoạt động mua bán và giao dịch thông qua nền tảng trực tuyến.
UI	User Interface	Giao diện người dùng.
URI	Uniform Resource Identifier	Chuỗi định danh một tài nguyên.
URL	Uniform Resource Locator	Địa chỉ định vị tài nguyên trên web.
USD	United States Dollar	Đô la Mỹ.
UX	User Experience	Trải nghiệm người dùng.

DANH MỤC HÌNH ẢNH

Hình 2.1 : Minh họa về kiến trúc Microservices 12
Hình 2.2 : So sánh giữa Monolithic và Microservices 16
Hình 2.3 : Mô tả về hệ thống API Gateway 19
Hình 2.4 : Tổng quan kiến trúc hoạt động của RESTful API 23
Hình 2.5: Ánh xạ các phương thức HTTP với thao tác dữ liệu (CRUD) 26
Hình 2.6 : Vai trò của API Gateway trong hệ sinh thái Spring Cloud 29
Hình 2.7 : Logo của Spring Boot 31
Hình 2.8 : Logo của React.js 32
Hình 2.9 : Logo của FastAPI 33
Hình 2.10 : Logo của hệ quản trị cơ sở dữ liệu MySQL 34
Hình 2.11 : Mô hình kiến trúc và các thành phần của Docker 35
Hình 2.12 : Minh họa cơ chế hoạt động của Redis Cache trong hệ thống 37
Hình 2.13 : Logo của Netflix Eureka Service Discovery 38
Hình 2.14 : Minh họa nền tảng NVIDIA NIM và mô hình Llama 3.1 41
Hình 3.1 : Sơ đồ kiến trúc Microservices của hệ thống 53
Hình 3.2 : Mã nguồn tìm tập ứng viên và ước lượng điểm số 62
Hình 3.3 : Dữ liệu phản hồi của Recommendation Service sau xếp hạng 62
Hình 3.4 : Mã nguồn Frontend gọi API lấy danh sách gợi ý sản phẩm 63
Hình 3.5 : Mã nguồn React kết xuất dữ liệu gợi ý bằng Grid CSS 64
Hình 3.6 : Khối sản phẩm "Gợi ý dành cho bạn" trên Trang chủ 64
Hình 3.7 : Mã Nguồn tìm và chọn sản phẩm liên quan đến câu hỏi 68
Hình 3.8 : Mã Nguồn Tạo câu trả lời tư vấn dựa trên thông tin đã tìm được 69
Hình 3.9 : Mã Nguồn đóng gói và gửi câu trả lời về giao diện chat 70

Hình 3.10 : Mã nguồn Frontend gửi câu hỏi và nhận phản hồi Chatbot 71
Hình 3.11 : Mã nguồn React kết xuất luồng hội thoại 72
Hình 3.12 : Cửa sổ trò chuyện Trợ lý Electro tư vấn sản phẩm 73
Hình 3.13 : Lược đồ Use Case tổng quan của hệ thống 77
Hình 3.14 : Lược đồ Use Case vai trò khách hàng 78
Hình 3.15 : Lược đồ Use Case của Admin 90
Hình 3.16 : Lược đồ Use Case của nhân viên quản lý kho 93
Hình 3.17 : Lược đồ Use Case của nhân viên chăm sóc khách hàng 94
Hình 3.18 : Lược đồ Activity của chức năng đăng nhập 95
Hình 3.19 : Lược đồ Activity của luồng mua hàng 96
Hình 3.20 : Lược đồ Sequence của chức năng đăng nhập 97
Hình 3.21 : Lược đồ Sequence của chức năng đăng ký 98
Hình 3.22 : Lược đồ Sequence của chức năng đổi mật khẩu 99
Hình 3.23 : Lược đồ Sequence của chức năng quản lý đơn hàng 100
Hình 3.24 : Lược đồ Sequence của chức năng thanh toán 101
Hình 3.25 : Lược đồ Sequence của chức năng mua hàng 103
Hình 3.27 : Màn hình đăng nhập 113
Hình 3.28 : Màn hình Trang chủ của hệ thống Electro Store 114
Hình 3.29 : Màn hình cập nhật hồ sơ cá nhân 115
Hình 3.30 : Màn hình đổi mật khẩu của hệ thống Electro Store 116
Hình 3.31 : Màn hình giỏ hàng 117
Hình 3.32 : Màn hình thanh toán 118
Hình 3.33 : Màn hình thanh toán Online với VNPay 120
Hình 3.34 : Màn hình thanh toán thành công của hệ thống Electro Store 121
Hình 3.35 : Màn hình trạng thái và lịch sử đơn hàng 122

Hình 3.36 : Màn hình quản lý người dùng của hệ thống Electro Store.....123
Hình 3.37 : Màn hình quản lý đơn hàng của hệ thống Electro Store 124
Hình 3.38 : Màn hình Admin Dashboard báo cáo và thống kê.....126
Hình 4.1 : Màn hình đăng ký tài khoản của hệ thống Electro Store 142
Hình 4.2 : Màn hình đăng nhập của hệ thống Electro Store 143
Hình 4.3 : Màn hình thay đổi ngôn ngữ hiển thị 144
Hình 4.4 : Màn Hình Quên Mật Khẩu của Hệ Thống Electro Store 145
Hình 4.5 : Màn hình tìm kiếm sản phẩm thông minh 146
Hình 4.6 : Màn hình gợi ý sản phẩm cho người dùng.....147
Hình 4.7 :Màn hình chi tiết sản phẩm và thao tác thêm vào giỏ hàng 148
Hình 4.8 : Màn hình quản lý sản phẩm trong giỏ hàng.....149
Hình 4.9 : Màn hình xác nhận thông tin giao hàng và thanh toán 150
Hình 4.10 : Màn hình quản lý hồ sơ cá nhân của người dùng.....151
Hình 4.11 : Màn Hình Hướng dẫn Tra Cứu Bảo Hành 153
Hình 4.12 : Màn Hình Hướng dẫn Đánh Giá Sản Phẩm 154
Hình 4.13 : Giao diện Trợ lý Electro hỗ trợ tư vấn sản phẩm.....156
Hình 4.14 : Dashboard báo cáo và thống kê dành cho Quản trị viên.....158
Hình 4.15 : Màn hình quản lý danh sách sản phẩm dành cho Quản trị viên...159
Hình 4.16 : Màn hình quản lý tài khoản người dùng dành cho Quản trị viên.160
Hình 4.17 : Màn hình quản lý và theo dõi trạng thái đơn hàng.....161
Hình 4.18 : Giao diện tổng quan trang Quản lý Mã giảm giá 163
Hình 4.19 : Màn Hình Quản Lý Danh Mục Sản Phẩm 164
Hình 4.20 : Màn hình quản lý và phản hồi đánh giá khách hàng.....166
Hình 4.21 : Giao diện quản lý và tiếp nhận các Yêu cầu bảo hành online.....168
Hình 4.22 : Màn hình nhập mã định danh Serial theo đơn mua hàng.....169

Hình 4.23 : Màn hình danh sách đơn hàng cần xuất kho 170
Hình 4.24 : Màn hình gom hàng và xuất kho theo FIFO 171
Hình 5.1 : Cấu hình biến môi trường JWT Secret đã che giá trị..... 174
Hình 5.2 : Minh họa cấu hình kết nối MySQL trong tệp môi trường 174
Hình 5.3 : Minh họa cấu hình kết nối Redis trong tệp môi trường 174
Hình 5.4 : Cấu hình kết nối MySQL của Recommendation Service 178

DANH MỤC BẢNG

Bảng 2.1 : So sánh kiến trúc Monolithic và Microservices 16

Bảng 3.1 : Đặc tả Use Case "Đăng ký" 78

Bảng 3.2 : Đặc tả Use Case "Đăng nhập" 80

Bảng 3.3 : Đặc tả Use Case "Cập nhật thông tin cá nhân" 82

Bảng 3.4 : Đặc tả Use Case "Đổi mật khẩu" 83

Bảng 3.5 : Đặc tả Use Case "Mua hàng" 85

Bảng 3.6 : Đặc tả Use Case "Thanh toán" 88

Bảng 3.7 : Đặc tả Use Case "Admin quản lý đơn hàng" 90

Bảng 3.8 : Danh sách API xác thực người dùng 106

Bảng 3.9 : Danh sách API quản trị hệ thống 107

Bảng 3.10 : Danh sách API trí tuệ nhân tạo 110

Bảng 3.11 : Danh sách API dành cho vai trò Sales 110

Bảng 3.12 : Danh sách API cốt lõi dành cho vai trò Nhân viên kho 111

Bảng 3.13 : Kết quả đo lường với 500 người dùng ảo 127

Bảng 3.14 : Kết quả đo lường với 1.000 người dùng ảo 128

Bảng 3.15 : Kết quả đo lường với 2.000 người dùng ảo 128

TÓM TẮT

Sự gia tăng nhanh chóng về số lượng và chủng loại thiết bị điện tử trên các nền tảng thương mại điện tử đặt ra yêu cầu nâng cao khả năng cá nhân hóa trong quá trình tìm kiếm và lựa chọn sản phẩm. Các phương thức gợi ý truyền thống thường dựa trên mức độ phổ biến hoặc các thuộc tính chung của sản phẩm, do đó chưa phản ánh đầy đủ sở thích và hành vi của từng người dùng. Nghiên cứu này đề xuất xây dựng một nền tảng thương mại điện tử thiết bị điện tử theo kiến trúc hướng dịch vụ, kết hợp cơ chế gợi ý sản phẩm dựa trên lịch sử tương tác của người dùng. Hệ thống được thiết kế với các thành phần chức năng độc lập nhằm nâng cao khả năng mở rộng, bảo trì và đáp ứng khi quy mô dữ liệu và số lượng người dùng gia tăng. Cơ chế gợi ý khai thác dữ liệu tương tác của người dùng để xác định mức độ phù hợp giữa người dùng và sản phẩm, từ đó tạo ra danh sách sản phẩm được cá nhân hóa. Kết quả triển khai và đánh giá bước đầu cho thấy kiến trúc đề xuất đáp ứng yêu cầu về tính linh hoạt và khả năng mở rộng của nền tảng, trong khi cơ chế gợi ý giúp nâng cao mức độ phù hợp của sản phẩm được đề xuất so với phương thức hiển thị dựa trên mức độ phổ biến. Kết quả nghiên cứu cung cấp cơ sở cho việc phát triển các hệ thống thương mại điện tử có khả năng cá nhân hóa và tích hợp các phương pháp học máy nhằm nâng cao hiệu quả hỗ trợ ra quyết định của người dùng.

Từ khóa: thương mại điện tử, hệ thống gợi ý, cá nhân hóa, hành vi người dùng, kiến trúc hướng dịch vụ.

MỤC LỤC

DANH MỤC CHỮ VIẾT TẮT i
DANH MỤC HÌNH ẢNH iii
DANH MỤC BẢNG vii
TÓM TẮT viii
MỞ ĐẦU 1
1. LÝ DO CHỌN ĐỀ TÀI 1
2. MỤC TIÊU VÀ PHẠM VI NGHIÊN CỨU 2
3. Ý NGHĨA KHOA HỌC VÀ THỰC TIỄN 3
CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI 5
1.1. Phân tích, đánh giá các công trình nghiên cứu liên quan 5
1.2. Những vấn đề còn tồn tại 8
1.3. Nội dung chính của tiểu luận 10
CHƯƠNG 2. NỘI DUNG VÀ PHƯƠNG PHÁP NGHIÊN CỨU 11
2.1. Cơ sở lý thuyết 11
2.2. Microservices 11
2.2.1. Giới thiệu về Microservices 11
2.2.2. Những đặc trưng của Microservices 13
2.2.2.1. Tính mở rộng độc lập (Scalability) 13
2.2.2.2. Tùy chọn công nghệ linh hoạt 13
2.2.2.3. Cách ly và chịu lỗi 13
2.2.2.4. Phát triển và triển khai nhanh hơn 14
2.2.2.5. Dễ bảo trì hơn 14
2.2.3. So sánh kiến trúc Microservices và kiến trúc nguyên khối (Monolithic) .. 16
2.2.4. Những khó khăn khi triển khai Microservices 17
2.2.4.1. API Gateway và giao tiếp giữa các dịch vụ 18

2.3. API Gateway 18
2.3.1. Khái niệm 18
2.3.2. Ví dụ về cách hoạt động 19
2.3.3. Chức năng và vai trò của API Gateway 20
2.3.3.1. Monitoring (quản lý) và routing (định tuyến) các yêu cầu API 20
2.3.3.2. Bảo mật API 20
2.3.3.3. Quản lý lưu lượng và cân bằng tải 20
2.3.3.4. Giám sát và ghi log 20
2.3.4. Nhược điểm khi dùng API Gateway 21
2.3.4.1. Độ phức tạp và chi phí triển khai 21
2.3.4.2. Độ trễ (latency) 21
2.3.4.3. Điểm nghẽn cổ chai 22
2.3.4.4. Phụ thuộc vào nhà cung cấp dịch vụ lưu trữ API Gateway 22
2.4. REST API 23
2.4.1. Khái niệm và lịch sử hình thành 23
2.4.2. Các phương thức HTTP 25
2.5. Spring Cloud Gateway 27
2.5.1. Giới thiệu về Spring Cloud Gateway 27
2.5.2. Kiến trúc và thành phần của Spring Cloud Gateway 29
2.6. Các công nghệ hỗ trợ 30
2.6.1. Spring Boot Framework 30
2.6.2. React.js 32
2.6.3. FastAPI 33
2.6.4. MySQL 34
2.6.5. Docker 35
2.6.6. Redis 36
2.6.7. Eureka Service Discovery 38

2.6.8. JWT – JSON Web Token 39
2.6.9. Tích hợp Trí tuệ nhân tạo với NVIDIA NIM 41
2.6.9.1. Giới thiệu 41
2.6.9.2. Ứng dụng Chatbot hỗ trợ khách hàng 41
2.6.9.3. Ứng dụng vào đồ án 42
2.6.10. Cổng thanh toán VNPay 43
2.6.10.1. Giới thiệu 43
2.6.10.2. Ứng dụng thực tiễn 43
2.6.10.3. Ứng dụng vào đồ án 43
2.6.11. Dịch vụ vận chuyển GHN (Giao Hàng Nhanh) 44
2.6.11.1. Giới thiệu 44
2.6.11.2. Ứng dụng trong quản lý Logistic 44
2.6.11.3. Ứng dụng vào đồ án 44
2.6.12. Material-UI (MUI) 45
2.6.12.1. Tổng quan 45
2.6.12.2. Vai trò trong phát triển Frontend 45
2.6.12.3. Ứng dụng vào đồ án 46
2.6.13. Các mô hình AI tích hợp trong ứng dụng 46
2.6.13.1. Tổng quan mô hình gợi ý sử dụng 47
2.6.13.2. Lọc cộng tác dựa trên sản phẩm với KNN 47
CHƯƠNG 3. GIẢI PHÁP CHO BÀI TOÁN 50
3.1. Phát biểu bài toán trong đề tài 50
3.2. Giải pháp giải quyết cho bài toán 52
3.2.1. Kiến trúc tổng quan hệ thống 52
3.2.2. Sơ đồ kiến trúc hệ thống 53
3.2.3. Cơ chế giao tiếp giữa các service 55
3.2.4. Cơ chế bảo mật 55
3.2.5. Cơ chế tích hợp AI 57

3.2.5.1. Hệ thống gợi ý sản phẩm (Recommendation System) 57
3.2.5.2. Chatbot tư vấn (RAG) 64
3.3. Hiện thực giải pháp 73
3.3.1. Mô tả chức năng 73
3.3.2. Lược đồ Use case 77
3.3.2.1. Phân hệ các Usecase dành cho Khách hàng (Người dùng) 78
3.3.2.2. Phân hệ các Usecase dành cho Quản trị hệ thống 90
3.3.2.3. Phân hệ các Usecase dành cho Nhân Viên Kho 93
3.3.2.4. Phân hệ các Usecase dành cho Nhân Viên Sales 94
3.3.3. Lược đồ Activity 94
3.3.4. Lược đồ Sequence 96
3.3.5. Lược đồ cơ sở dữ liệu 104
3.3.6. Xây dựng các API Endpoint 105
3.3.6.1. API Xác thực người dùng (Authentication) 105
3.3.6.2. API Quản trị hệ thống (Admin) 107
3.3.6.3. API Trí tuệ nhân tạo (Gợi ý sản phẩm & Chatbot RAG) 110
3.3.6.4. API dành cho vai trò Sales 110
3.3.6.5. API cốt lõi dành cho vai trò Nhân viên kho 111
3.3.7. Các màn hình trong hệ thống 112
3.3.7.1. Màn hình cập nhật hồ sơ cá nhân 114
3.3.7.2. Màn hình đổi mật khẩu 115
3.3.7.3. Màn hình giỏ hàng 116
3.3.7.4. Màn hình thanh toán 117
3.3.7.5. Màn hình thanh toán Online với VNPay 119
3.3.7.6. Màn hình thanh toán thành công 120
3.3.7.7. Màn hình trạng thái và lịch sử đơn hàng 121
3.3.7.8. Màn hình quản lý người dùng 122
3.3.7.9. Màn hình quản lý đơn hàng 124
3.3.7.10. Màn hình Admin Dashboard báo cáo và thống kê 125
3.3.8. Kết quả đo lường các chức năng 126

3.3.8.1. Môi trường và công cụ kiểm thử 126
3.3.8.2. Kết quả thực nghiệm 127
3.3.8.3. Phân tích kết quả đo lường 128
3.3.8.4. Kết luận và hướng cải thiện 130
CHƯƠNG 4. KẾT QUẢ, KẾT LUẬN VÀ KIẾN NGHỊ 132
4.1. KẾT QUẢ ĐẠT ĐƯỢC 132
4.1.1. Về mặt kiến thức và lý thuyết 132
4.1.2. Về mặt sản phẩm và chức năng hệ thống 133
4.2. Đánh giá ưu và nhược điểm 134
4.3. KẾT LUẬN 135
4.4. KIẾN NGHỊ VÀ HƯỚNG PHÁT TRIỂN 136
TÀI LIỆU THAM KHẢO 138
PHỤ LỤC A. HƯỚNG DẪN NGƯỜI DÙNG 141
A.1. HƯỚNG DẪN DÀNH CHO NGƯỜI DÙNG MUA HÀNG 141
A.1.1. Đăng ký tài khoản 141
A.1.2. Đăng nhập tài khoản 142
A.1.3. Thay đổi ngôn ngữ hiển thị 143
A.1.4. Quên Mật Khẩu 144
A.1.5. Tìm kiếm và khám phá sản phẩm 145
A.1.6. Xem sản phẩm gợi ý cá nhân hóa 146
A.1.7. Mua hàng và thanh toán 147
A.1.8. Quản lý hồ sơ cá nhân và lịch sử đơn hàng 150
A.1.9. Hướng dẫn tra cứu bảo hành 151
A.1.10. Hướng dẫn Đánh giá sản phẩm 153
A.1.11. Sử dụng Chatbot AI tư vấn 155
A.2. HƯỚNG DẪN DÀNH CHO QUẢN TRỊ VIÊN (ADMIN) 156

A.2.1. Tổng quan (Dashboard) 156
A.2.2. Quản lý sản phẩm 158
A.2.3. Quản lý người dùng: 159
A.2.4. Quản lý đơn hàng 161
A.2.5. Quản lý Mã giảm giá (Coupon/Voucher) 162
A.2.6. Quản lý Danh mục sản phẩm 163
A.2.7. Duyệt Đánh giá (Reviews) 164
A.2.8. Hướng dẫn Quy trình Bảo hành (Dành cho Sales Staff) 166
A.2.9. Quản lý nhập/xuất kho và theo dõi thiết bị bằng mã IMEI/Serial 168
PHỤ LỤC B. HƯỚNG DẪN CÀI ĐẶT HỆ THỐNG 172
B.1. CẤU HÌNH HỆ THỐNG YÊU CẦU 172
B.2. CÀI ĐẶT BACKEND 172
B.2.1. Công nghệ sử dụng 172
B.2.2. Chuẩn bị môi trường 173
B.2.3. Các bước thực hiện 173
B.3. CÀI ĐẶT FRONTEND 175
B.3.1. Công nghệ sử dụng 175
B.3.2. Chuẩn bị môi trường 176
B.3.3. Các bước thực hiện 176
B.4. CÀI ĐẶT VÀ CHẠY MÔ HÌNH AI (PYTHON API) 177
B.4.1. Công nghệ sử dụng 177
B.4.2. Các bước thực hiện 177

MỞ ĐẦU

1. LÝ DO CHỌN ĐỀ TÀI

Trong những năm gần đây, thương mại điện tử tại Việt Nam phát triển mạnh mẽ cùng với sự phổ biến của Internet, thiết bị di động và các dịch vụ thanh toán trực tuyến. Sự phát triển này làm gia tăng số lượng người dùng, sản phẩm và giao dịch trên các nền tảng thương mại điện tử, đồng thời tạo ra yêu cầu ngày càng cao về khả năng đáp ứng, tính ổn định và trải nghiệm người dùng. Theo Báo cáo Chỉ số Thương mại điện tử Việt Nam 2026 của Hiệp hội Thương mại điện tử Việt Nam (VECOM), quy mô thị trường thương mại điện tử Việt Nam năm 2025 đạt khoảng 38,5 tỷ USD, tăng 21% so với năm 2024 [1]. Điều này cho thấy thương mại điện tử đang trở thành một thành phần quan trọng của nền kinh tế số và tiếp tục có tiềm năng phát triển trong những năm tới.

Trong lĩnh vực kinh doanh thiết bị điện tử, người dùng thường phải lựa chọn giữa số lượng lớn sản phẩm có đặc điểm và thông số kỹ thuật tương đồng. Việc tìm kiếm sản phẩm phù hợp chỉ dựa trên từ khóa, danh mục hoặc mức độ phổ biến có thể chưa đáp ứng tốt nhu cầu và sở thích của từng người dùng. Do đó, các hệ thống thương mại điện tử cần có khả năng khai thác dữ liệu tương tác để cung cấp các đề xuất sản phẩm phù hợp hơn, qua đó hỗ trợ người dùng trong quá trình tìm kiếm và ra quyết định mua hàng.

Bên cạnh vấn đề cá nhân hóa, khả năng mở rộng cũng là một yêu cầu quan trọng đối với các hệ thống thương mại điện tử. Các hệ thống được xây dựng theo kiến trúc nguyên khối (Monolithic) có thể gặp khó khăn khi quy mô người dùng và lưu lượng truy cập tăng cao, do các chức năng được triển khai và quản lý trong cùng một ứng dụng. Kiến trúc Microservices cho phép phân chia hệ thống thành các dịch vụ độc lập, giúp từng dịch vụ có thể được phát triển, triển khai và mở rộng riêng biệt. Đây là hướng tiếp cận phù hợp với các hệ thống có nhiều chức năng và yêu cầu mở rộng theo từng thành phần.

Ngoài ra, người dùng thiết bị điện tử thường cần tìm hiểu nhiều thông tin trước khi mua hàng, chẳng hạn như thông số kỹ thuật, tính năng, giá bán và sự phù hợp với nhu cầu sử dụng. Các phương thức hỗ trợ khách hàng truyền thống hoặc chatbot dựa

trên kịch bản cố định có khả năng xử lý hạn chế đối với những câu hỏi đa dạng. Sự phát triển của các mô hình ngôn ngữ lớn (Large Language Model – LLM) kết hợp với kỹ thuật Retrieval-Augmented Generation (RAG) cho phép xây dựng các trợ lý ảo có khả năng truy xuất thông tin từ nguồn dữ liệu liên quan và tạo câu trả lời theo ngữ cảnh, từ đó nâng cao khả năng hỗ trợ người dùng.

Từ những vấn đề trên, đề tài "Xây dựng hệ thống thương mại điện tử Electro Store áp dụng kiến trúc Microservices, tích hợp hệ thống gợi ý và trợ lý ảo AI" được lựa chọn nhằm nghiên cứu và xây dựng một hệ thống thương mại điện tử chuyên về thiết bị điện tử. Đề tài tập trung kết hợp kiến trúc Microservices với hệ thống gợi ý sản phẩm và trợ lý ảo AI nhằm nâng cao khả năng mở rộng của hệ thống, cá nhân hóa trải nghiệm người dùng và hỗ trợ cung cấp thông tin sản phẩm.

2. MỤC TIÊU VÀ PHẠM VI NGHIÊN CỨU

- **Mục tiêu:**

Mục tiêu tổng quát của đề tài là nghiên cứu và xây dựng hệ thống thương mại điện tử Electro Store theo kiến trúc Microservices, đồng thời tích hợp hệ thống gợi ý sản phẩm và trợ lý ảo AI nhằm nâng cao khả năng mở rộng, cá nhân hóa trải nghiệm và hỗ trợ người dùng.

Các mục tiêu cụ thể gồm:

+ Nghiên cứu cơ sở lý thuyết về kiến trúc Microservices, hệ thống phân tán và các thành phần hỗ trợ như API Gateway, Service Discovery và Distributed Tracing.
+ Phân tích yêu cầu và thiết kế kiến trúc tổng thể của hệ thống thương mại điện tử theo hướng phân tán, đảm bảo khả năng mở rộng và bảo trì.
+ Xây dựng các chức năng cơ bản của hệ thống như quản lý tài khoản, sản phẩm, danh mục, giỏ hàng, đơn hàng, thanh toán và quản lý người dùng.
+ Xây dựng hệ thống gợi ý sản phẩm dựa trên phương pháp Collaborative Filtering, sử dụng lịch sử tương tác của người dùng để đưa ra các sản phẩm phù hợp.
+ Nghiên cứu và tích hợp trợ lý ảo sử dụng mô hình ngôn ngữ lớn kết hợp kỹ thuật RAG nhằm hỗ trợ người dùng tìm kiếm thông tin và tư vấn sản phẩm.

+ Triển khai hệ thống trong môi trường thực nghiệm và thực hiện kiểm thử, đánh giá các chức năng, hiệu năng và khả năng đáp ứng của hệ thống.

- **Phạm vi:**

Đề tài tập trung nghiên cứu, thiết kế và xây dựng hệ thống thương mại điện tử Electro Store dành cho lĩnh vực kinh doanh thiết bị điện tử. Hệ thống bao gồm các chức năng phục vụ khách hàng và quản trị viên.

+ Đối với khách hàng, hệ thống hỗ trợ đăng ký, đăng nhập, quản lý thông tin cá nhân, tìm kiếm và xem thông tin sản phẩm, quản lý giỏ hàng, đặt hàng, theo dõi đơn hàng, đánh giá sản phẩm và nhận các đề xuất sản phẩm cá nhân hóa. Ngoài ra, khách hàng có thể sử dụng trợ lý ảo AI để tìm hiểu thông tin và nhận tư vấn liên quan đến sản phẩm.
+ Đối với quản trị viên, hệ thống hỗ trợ quản lý sản phẩm, danh mục, tài khoản người dùng, đơn hàng và theo dõi các thông tin thống kê liên quan đến hoạt động của hệ thống.
+ Về kiến trúc, đề tài tập trung triển khai Microservices với các thành phần chính như API Gateway, Service Discovery, các dịch vụ nghiệp vụ độc lập và Distributed Tracing. Về trí tuệ nhân tạo, phạm vi nghiên cứu bao gồm hệ thống gợi ý sử dụng Collaborative Filtering và trợ lý ảo sử dụng LLM kết hợp RAG. Các mô hình ngôn ngữ lớn được sử dụng thông qua API và không tập trung vào việc huấn luyện mô hình từ đầu.
+ Về công nghệ, hệ thống sử dụng Java Spring Boot và Spring Cloud để xây dựng các microservice, ReactJS để phát triển giao diện người dùng, MySQL để quản lý dữ liệu nghiệp vụ, Redis làm bộ nhớ đệm, Python/FastAPI cho các dịch vụ xử lý AI và Pinecone để lưu trữ dữ liệu vector. Docker được sử dụng để container hóa hệ thống và Zipkin hỗ trợ theo dõi luồng xử lý giữa các dịch vụ.

Các công nghệ trên được lựa chọn nhằm phục vụ việc xây dựng, triển khai và đánh giá hệ thống trong phạm vi của đề tài, không nhằm mục đích so sánh hoặc đánh giá giữa các nền tảng và công cụ khác.

3. Ý NGHĨA KHOA HỌC VÀ THỰC TIỄN

- **Ý nghĩa khoa học:**

+ Đề tài góp phần hệ thống hóa và vận dụng các kiến thức về kiến trúc Microservices, hệ thống phân tán, hệ thống gợi ý và kỹ thuật Retrieval-Augmented Generation trong bài toán xây dựng nền tảng thương mại điện tử. Thông qua quá trình phân tích, thiết kế và triển khai, đề tài minh họa cách tổ chức các dịch vụ độc lập và phối hợp giữa các thành phần trong một hệ thống phân tán.

+ Bên cạnh đó, đề tài thể hiện khả năng kết hợp phương pháp Collaborative Filtering với dữ liệu tương tác của người dùng để xây dựng hệ thống gợi ý sản phẩm, đồng thời kết hợp mô hình ngôn ngữ lớn với RAG nhằm xây dựng trợ lý ảo có khả năng khai thác dữ liệu sản phẩm. Kết quả nghiên cứu có thể làm cơ sở tham khảo cho các đề tài và hệ thống ứng dụng trí tuệ nhân tạo trong thương mại điện tử.

- **Ý nghĩa thực tiễn:**

+ Đề tài cung cấp một mô hình hệ thống thương mại điện tử có khả năng mở rộng và bảo trì thông qua kiến trúc Microservices. Việc sử dụng Redis làm bộ nhớ đệm góp phần giảm tải cho cơ sở dữ liệu và cải thiện tốc độ phản hồi. Hệ thống gợi ý sản phẩm hỗ trợ cá nhân hóa quá trình tìm kiếm và khám phá sản phẩm dựa trên lịch sử tương tác của người dùng. Trợ lý ảo ứng dụng LLM và RAG hỗ trợ cung cấp thông tin, tư vấn sản phẩm và giải đáp các câu hỏi của khách hàng. Ngoài ra, các chức năng quản trị và thống kê hỗ trợ người quản trị theo dõi hoạt động và quản lý hệ thống hiệu quả hơn.

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Phân tích, đánh giá các công trình nghiên cứu liên quan

Các nghiên cứu về kiến trúc Microservices trong thương mại điện tử. Trong bối cảnh các hệ thống thương mại điện tử ngày càng yêu cầu khả năng mở rộng và độ tin cậy cao, Asrowardi và cộng sự (2020) đã nghiên cứu "Designing Microservice Architectures for Scalability and Reliability in E-Commerce", tập trung vào việc chuyển đổi từ kiến trúc nguyên khối (Monolithic) sang kiến trúc vi dịch vụ (Microservices) nhằm cải thiện khả năng mở rộng, độ tin cậy và hiệu năng của hệ thống [2]. Nhóm tác giả áp dụng Design Science Research Methodology (DSRM) để thiết kế và xây dựng mô hình kiến trúc Microservices, sau đó đánh giá trong môi trường thử nghiệm với nhiều kịch bản tải khác nhau. Kết quả thực nghiệm cho thấy kiến trúc Microservices có khả năng cải thiện hiệu năng so với kiến trúc Monolithic. Mức cải thiện được ghi nhận khoảng 30% đối với kịch bản scripting workload và khoảng 4% đối với rendering/painting workload [2]. Kết quả cho thấy hiệu quả của Microservices phụ thuộc vào đặc điểm workload và cách tổ chức các dịch vụ trong hệ thống. Bổ sung cho nghiên cứu trên, Liu và cộng sự (2020) trong công trình "Microservices: Architecture, Container, and Challenges" đã hệ thống hóa quá trình phát triển từ kiến trúc Monolithic, Service-Oriented Architecture (SOA) đến Microservices, đồng thời phân tích vai trò của container trong việc triển khai và quản lý các dịch vụ độc lập [3]. Nghiên cứu chỉ ra rằng Microservices được áp dụng rộng rãi trong các hệ thống Internet quy mô lớn nhờ khả năng mở rộng và triển khai độc lập. Tuy nhiên, kiến trúc này cũng phát sinh nhiều thách thức như chi phí giao tiếp giữa các dịch vụ, độ trễ mạng, cascading failure, khó khăn trong việc giám sát và gỡ lỗi, cũng như vấn đề nhất quán dữ liệu trong môi trường phân tán [3]. Newman (2021) tiếp tục cung cấp các nguyên tắc thiết kế Microservices theo hướng xây dựng các dịch vụ có phạm vi chức năng rõ ràng, độc lập trong triển khai và có khả năng phát triển riêng biệt [4]. Những nguyên tắc này đặc biệt phù hợp với hệ thống thương mại điện tử, trong đó các nghiệp vụ như quản lý sản phẩm, người dùng, đơn hàng, thanh toán và đề xuất sản phẩm có thể được tổ chức thành các dịch vụ tương đối độc lập. Blinowski và cộng sự (2022) thực hiện nghiên cứu so sánh trực tiếp giữa kiến trúc Monolithic và Microservices trên các khía cạnh hiệu năng và

khả năng mở rộng [5]. Kết quả cho thấy Microservices mang lại lợi thế về khả năng mở rộng độc lập và tính linh hoạt khi hệ thống phát triển, nhưng đồng thời có thể phát sinh overhead do giao tiếp mạng và quản lý nhiều thành phần phân tán. Do đó, việc áp dụng Microservices cần đi kèm với các cơ chế quản lý và giám sát phù hợp. Nhìn chung, các công trình trên cho thấy Microservices là hướng tiếp cận phù hợp cho các hệ thống thương mại điện tử có yêu cầu mở rộng và phát triển nhiều chức năng độc lập. Tuy nhiên, phần lớn các nghiên cứu tập trung vào kiến trúc, hiệu năng và khả năng mở rộng mà chưa kết hợp sâu với các thành phần trí tuệ nhân tạo nhằm cá nhân hóa trải nghiệm người dùng. Đây là một trong những vấn đề được đề tài tiếp tục nghiên cứu.

Các nghiên cứu về hệ thống gợi ý sản phẩm. Bên cạnh khả năng mở rộng của kiến trúc, cá nhân hóa trải nghiệm là một vấn đề quan trọng trong thương mại điện tử. Hệ thống gợi ý (Recommender System) được sử dụng để phân tích dữ liệu người dùng và sản phẩm nhằm đưa ra các đề xuất phù hợp với sở thích hoặc hành vi của từng cá nhân. Một trong những hướng tiếp cận phổ biến là Collaborative Filtering. Linden, Smith và York (2003) đã giới thiệu phương pháp "Item-to-item collaborative filtering" được triển khai trong hệ thống Amazon.com Recommendations [6]. Thay vì chỉ dựa trên đặc điểm nội dung của sản phẩm, phương pháp khai thác mối quan hệ giữa các sản phẩm dựa trên hành vi tương tác của người dùng. Cách tiếp cận này cho phép hệ thống xác định những sản phẩm có khả năng được quan tâm dựa trên các sản phẩm mà người dùng đã xem, mua hoặc tương tác trước đó. Collaborative Filtering có ưu điểm là không yêu cầu xây dựng thủ công các luật gợi ý dựa trên đặc điểm sản phẩm. Tuy nhiên, phương pháp này có thể gặp vấn đề cold-start khi người dùng mới chưa có lịch sử tương tác hoặc sản phẩm mới chưa có đủ dữ liệu đánh giá. Ngoài ra, chất lượng gợi ý phụ thuộc đáng kể vào số lượng và độ chính xác của dữ liệu tương tác. Hug (2020) giới thiệu thư viện Surprise, một thư viện Python dành cho việc xây dựng và đánh giá các thuật toán hệ thống gợi ý [8]. Thư viện cung cấp các thuật toán Collaborative Filtering và các phương pháp đánh giá mô hình, hỗ trợ quá trình xây dựng hệ thống gợi ý từ dữ liệu đánh giá hoặc tương tác của người dùng. Đối với bài toán của đề tài, Collaborative Filtering được lựa chọn làm phương pháp cơ bản để xây dựng hệ thống gợi ý do phù hợp với dữ liệu lịch sử tương tác giữa người dùng và sản phẩm. Tuy nhiên, hệ thống

cần được thiết kế để hạn chế ảnh hưởng của dữ liệu thưa và cold-start, đồng thời có thể mở rộng sang các phương pháp kết hợp nhiều nguồn dữ liệu trong tương lai.

Các nghiên cứu về LLM và Retrieval-Augmented Generation. Sự phát triển của các mô hình ngôn ngữ lớn (Large Language Model – LLM) mở ra khả năng xây dựng các hệ thống hội thoại có khả năng hiểu ngữ cảnh và tạo phản hồi tự nhiên. Trong thương mại điện tử, LLM có thể được sử dụng để hỗ trợ khách hàng tìm kiếm thông tin, tư vấn sản phẩm và giải đáp các câu hỏi liên quan đến sản phẩm. Tuy nhiên, LLM thuần túy có thể gặp hạn chế khi cần cung cấp thông tin cụ thể, cập nhật hoặc thuộc một miền dữ liệu riêng biệt. Mô hình có thể tạo ra thông tin không chính xác hoặc không có trong nguồn dữ liệu thực tế. Retrieval-Augmented Generation (RAG) được đề xuất như một hướng tiếp cận nhằm kết hợp khả năng sinh ngôn ngữ của LLM với cơ chế truy xuất thông tin từ nguồn dữ liệu bên ngoài. Gao và cộng sự (2023) đã thực hiện một nghiên cứu tổng quan về RAG cho các mô hình ngôn ngữ lớn, trong đó mô tả kiến trúc RAG gồm hai thành phần chính: truy xuất thông tin liên quan và sinh câu trả lời dựa trên thông tin được truy xuất [7]. Quy trình này giúp mô hình có thể sử dụng nguồn tri thức bên ngoài thay vì chỉ phụ thuộc vào dữ liệu đã được sử dụng trong quá trình huấn luyện. Trong hệ thống thương mại điện tử, RAG có thể được áp dụng bằng cách xây dựng cơ sở dữ liệu vector chứa thông tin về sản phẩm, chính sách, hướng dẫn sử dụng và các tài liệu liên quan. Khi người dùng đặt câu hỏi, hệ thống thực hiện chuyển đổi truy vấn thành vector, tìm kiếm các tài liệu có mức độ tương đồng cao và cung cấp kết quả truy xuất cho LLM để tạo câu trả lời. Cách tiếp cận này giúp trợ lý ảo có cơ sở thông tin cụ thể và phù hợp hơn với dữ liệu của hệ thống. Tuy nhiên, RAG cũng đặt ra một số vấn đề như chất lượng dữ liệu nguồn, khả năng truy xuất tài liệu phù hợp, độ trễ của quá trình tìm kiếm và chi phí gọi mô hình ngôn ngữ. Do đó, việc tích hợp RAG vào hệ thống thương mại điện tử cần được xem xét đồng thời với kiến trúc hệ thống và yêu cầu hiệu năng.

Từ các công trình đã phân tích có thể nhận thấy ba hướng nghiên cứu chính. Thứ nhất, các nghiên cứu về Microservices chứng minh khả năng mở rộng và tính linh hoạt của kiến trúc phân tán, đồng thời chỉ ra các thách thức liên quan đến giao tiếp liên dịch vụ, độ trễ và nhất quán dữ liệu [2]–[5]. Thứ hai, các nghiên cứu về Recommender

System cho thấy Collaborative Filtering là phương pháp hiệu quả để khai thác lịch sử tương tác nhằm cá nhân hóa đề xuất sản phẩm [6], [8]. Thứ ba, các nghiên cứu về RAG cho thấy khả năng kết hợp LLM với nguồn tri thức bên ngoài để xây dựng các hệ thống hỏi đáp có tính chuyên biệt và phù hợp với dữ liệu miền [7]. Mặc dù các hướng nghiên cứu trên đã được nghiên cứu tương đối độc lập, việc tích hợp đồng thời kiến trúc Microservices, hệ thống gợi ý và trợ lý ảo LLM-RAG trong một nền tảng thương mại điện tử vẫn đặt ra nhiều vấn đề về thiết kế và triển khai. Đặc biệt, cần đảm bảo sự phối hợp giữa các dịch vụ nghiệp vụ với các dịch vụ AI, đồng thời kiểm soát độ trễ và khả năng mở rộng khi các thành phần AI được tích hợp vào hệ thống. Trên cơ sở kế thừa các kết quả nghiên cứu trước, đề tài **"Xây dựng hệ thống thương mại điện tử Electro Store áp dụng kiến trúc Microservices, tích hợp hệ thống gợi ý và trợ lý ảo AI"** tập trung xây dựng một hệ thống thương mại điện tử theo kiến trúc Microservices, trong đó các chức năng nghiệp vụ được phân tách thành các dịch vụ độc lập. Hệ thống gợi ý sử dụng Collaborative Filtering để khai thác lịch sử tương tác của người dùng, trong khi trợ lý ảo sử dụng LLM kết hợp RAG và cơ sở dữ liệu vector để hỗ trợ tư vấn dựa trên dữ liệu sản phẩm. Qua đó, đề tài hướng đến việc kết hợp khả năng mở rộng của kiến trúc Microservices với các kỹ thuật AI nhằm nâng cao khả năng cá nhân hóa và hỗ trợ người dùng trong hệ thống thương mại điện tử.

1.2. Những vấn đề còn tồn tại

Qua phân tích các công trình nghiên cứu liên quan đến kiến trúc Microservices, hệ thống gợi ý và trợ lý ảo ứng dụng trí tuệ nhân tạo, có thể nhận thấy các nghiên cứu trước đây đã cung cấp nhiều cơ sở lý thuyết và thực nghiệm quan trọng. Tuy nhiên, trong phạm vi các công trình được khảo sát, vẫn còn một số vấn đề cần tiếp tục nghiên cứu và hoàn thiện.

Thứ nhất, một số nghiên cứu về Microservices tập trung chủ yếu vào khả năng mở rộng, hiệu năng và độ tin cậy của kiến trúc, trong khi việc tích hợp các thành phần trí tuệ nhân tạo như hệ thống gợi ý và trợ lý ảo LLM-RAG chưa được xem xét một cách đồng bộ. Các thành phần AI có đặc điểm xử lý khác biệt so với các dịch vụ nghiệp vụ thông thường và có thể phát sinh thêm chi phí tính toán, độ trễ cũng như nhu cầu tài

nguyên. Vì vậy, cần nghiên cứu cách tổ chức và tích hợp các dịch vụ AI trong kiến trúc Microservices nhằm hạn chế ảnh hưởng đến hiệu năng chung của hệ thống.

Thứ hai, việc triển khai hệ thống gợi ý trong môi trường thương mại điện tử vẫn tồn tại một số hạn chế liên quan đến dữ liệu người dùng. Khi người dùng mới hoặc sản phẩm mới chưa có đủ lịch sử tương tác, phương pháp Collaborative Filtering có thể gặp vấn đề cold-start và làm giảm chất lượng đề xuất. Do đó, hệ thống cần có cơ chế xử lý phù hợp và được đánh giá trên dữ liệu tương tác thực nghiệm để xác định mức độ phù hợp của phương pháp gợi ý.

Thứ ba, việc ứng dụng LLM trong trợ lý ảo đặt ra yêu cầu về độ chính xác và tính phù hợp của thông tin trả lời. Nếu chỉ sử dụng khả năng sinh nội dung của mô hình ngôn ngữ, hệ thống có thể cung cấp thông tin không phù hợp hoặc không phản ánh dữ liệu sản phẩm hiện tại. Việc kết hợp RAG với cơ sở dữ liệu vector là một hướng tiếp cận có tiềm năng, tuy nhiên hiệu quả của hệ thống còn phụ thuộc vào chất lượng dữ liệu, phương pháp truy xuất và khả năng lựa chọn thông tin liên quan.

Thứ tư, kiến trúc Microservices làm tăng độ phức tạp trong quá trình vận hành so với kiến trúc Monolithic. Việc phân tách hệ thống thành nhiều dịch vụ có thể phát sinh chi phí giao tiếp qua mạng, độ trễ và các vấn đề liên quan đến giám sát, xử lý lỗi và nhất quán dữ liệu. Liu và cộng sự [3] cũng chỉ ra rằng giao tiếp liên dịch vụ và quản lý hệ thống phân tán là những thách thức quan trọng khi áp dụng Microservices. Vì vậy, việc thiết kế cơ chế định tuyến, khám phá dịch vụ, theo dõi luồng xử lý và quản lý lỗi là cần thiết để đảm bảo hệ thống hoạt động ổn định.

Thứ năm, các nghiên cứu được khảo sát chủ yếu đánh giá hệ thống ở một số khía cạnh riêng biệt như hiệu năng kiến trúc, khả năng gợi ý hoặc khả năng trả lời của trợ lý ảo. Việc đánh giá một hệ thống tích hợp đồng thời kiến trúc Microservices, hệ thống gợi ý và trợ lý ảo AI cần xem xét nhiều yếu tố hơn, bao gồm khả năng đáp ứng, độ trễ, khả năng mở rộng, chất lượng gợi ý và khả năng cung cấp thông tin phù hợp cho người dùng.

Từ những vấn đề trên, đề tài tập trung xây dựng hệ thống thương mại điện tử Electro Store theo kiến trúc Microservices, đồng thời tích hợp hệ thống gợi ý sản phẩm

và trợ lý ảo AI. Hệ thống sử dụng Spring Cloud với Eureka Discovery Server và API Gateway nhằm hỗ trợ Service Discovery, định tuyến và quản lý giao tiếp giữa các dịch vụ. Hệ thống gợi ý sử dụng Collaborative Filtering để khai thác lịch sử tương tác của người dùng, trong khi trợ lý ảo kết hợp LLM, RAG và cơ sở dữ liệu vector nhằm cung cấp thông tin dựa trên dữ liệu sản phẩm. Bên cạnh việc triển khai các chức năng, đề tài tiến hành kiểm thử và đánh giá hệ thống trong môi trường thực nghiệm nhằm làm rõ khả năng đáp ứng và mức độ phù hợp của giải pháp đối với bài toán thương mại điện tử.

1.3. Nội dung chính của tiểu luận

Tiểu luận tập trung vào bốn nội dung chính: thiết kế kiến trúc hệ thống, tích hợp trí tuệ nhân tạo, triển khai và giám sát vận hành. Hệ thống Electro Store được xây dựng theo kiến trúc Microservices, sử dụng Spring Cloud cho các dịch vụ backend và FastAPI cho các dịch vụ AI, với các thành phần như API Gateway và Service Discovery.

Về AI, hệ thống gợi ý sử dụng Item-Based Collaborative Filtering thông qua Scikit-Surprise để đề xuất sản phẩm dựa trên lịch sử tương tác. Trợ lý ảo sử dụng LLM kết hợp RAG và Pinecone để truy xuất thông tin sản phẩm và tạo phản hồi phù hợp.

Hệ thống được triển khai bằng Docker Compose, kết hợp MySQL và Redis. Zipkin được sử dụng để Distributed Tracing, hỗ trợ theo dõi luồng xử lý và xác định các điểm có độ trễ cao.

Qua đó, tiểu luận xây dựng một nền tảng thương mại điện tử có khả năng mở rộng, bảo trì và hỗ trợ cá nhân hóa trải nghiệm người dùng.

CHƯƠNG 2. NỘI DUNG VÀ PHƯƠNG PHÁP NGHIÊN CỨU

2.1. Cơ sở lý thuyết

2.2. Microservices

Việc lựa chọn kiến trúc hệ thống là một quyết định quan trọng vì nó ảnh hưởng trực tiếp đến khả năng mở rộng, vận hành và bảo trì ứng dụng. Với hệ thống thương mại điện tử, nơi phải xử lý đồng thời nhiều yêu cầu từ người dùng, đặc biệt trong các thời điểm cao điểm như khuyến mãi hoặc flash sale, yêu cầu về sự ổn định và khả năng đáp ứng là rất lớn.

Trong thực tế, các chức năng như tìm kiếm sản phẩm, quản lý đơn hàng, giỏ hàng hoặc gợi ý sản phẩm thường có mức tải thay đổi theo thời điểm và có thể tăng đột biến. Nếu không có cách tổ chức phù hợp, hệ thống dễ bị quá tải cục bộ và ảnh hưởng đến trải nghiệm người dùng.

Với kiến trúc Monolithic, toàn bộ chức năng được triển khai trong một khối thống nhất. Cách tiếp cận này khiến việc mở rộng trở nên khó khăn và khi một thành phần gặp lỗi, toàn bộ hệ thống có thể bị ảnh hưởng, làm giảm độ ổn định chung.

Vì vậy, kiến trúc Microservices được lựa chọn do khả năng tách hệ thống thành các dịch vụ độc lập theo từng chức năng. Cách tổ chức này giúp mở rộng linh hoạt theo từng phần, hỗ trợ triển khai và bảo trì dễ dàng hơn, đồng thời tăng độ ổn định trong quá trình vận hành.

2.2.1. Giới thiệu về Microservices

Microservices là một phong cách kiến trúc phần mềm, trong đó hệ thống được xây dựng từ nhiều dịch vụ nhỏ, độc lập, cùng phối hợp để tạo nên các chức năng hoàn chỉnh của ứng dụng. Mỗi dịch vụ đảm nhiệm một nhóm nghiệp vụ cụ thể và có thể vận hành độc lập với các dịch vụ khác, giao tiếp qua các giao thức như HTTP, gRPC hoặc hệ thống hàng đợi thông điệp (Message Queue).

Kiến trúc này được xem là bước phát triển tiếp theo của kiến trúc hướng dịch vụ

(SOA). SOA đã đặt nền tảng cho việc phân tách hệ thống thành các dịch vụ nhằm tăng khả năng tái sử dụng, nhưng các dịch vụ trong SOA truyền thống thường có phạm

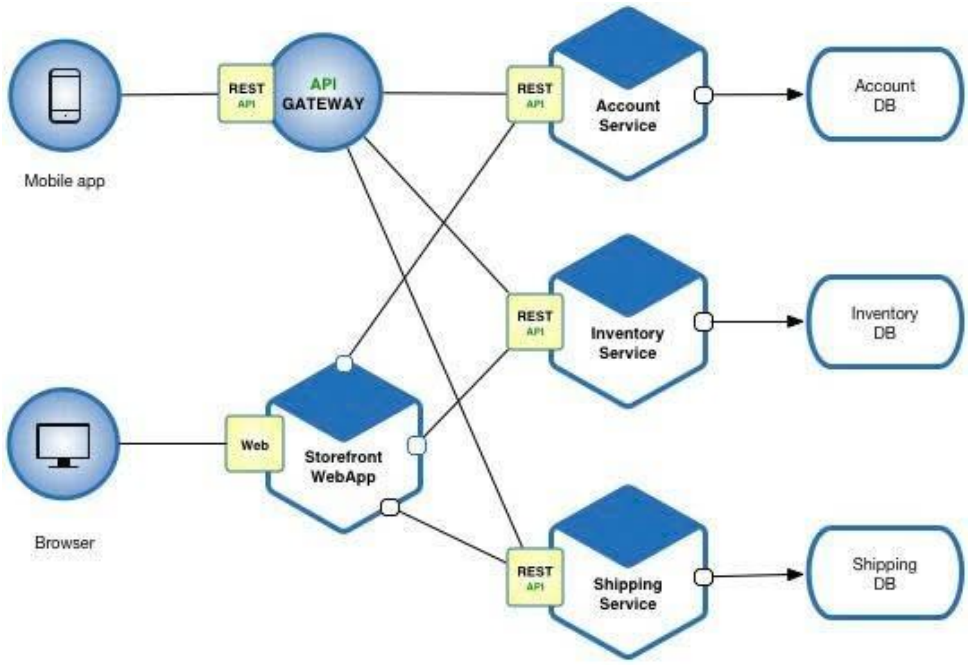
vi chức năng lớn và phụ thuộc vào hạ tầng trung gian phức tạp, khiến việc mở rộng và bảo trì trở nên khó khăn khi hệ thống lớn dần. Microservices giải quyết vấn đề này bằng cách tiếp tục chia nhỏ dịch vụ và giảm mức độ liên kết giữa các thành phần.

Từ đầu thập niên 2010, cùng với sự phát triển của điện toán đám mây, container và DevOps, kiến trúc này được các doanh nghiệp như Amazon, Netflix và Twitter áp dụng cho các hệ thống quy mô lớn, đồng thời dần trở thành một trong những phong cách kiến trúc phổ biến hiện nay [4].

Trong Microservices, mỗi dịch vụ chịu trách nhiệm cho một miền nghiệp vụ

(Business Domain) riêng, có thể sử dụng ngôn ngữ lập trình, cơ sở dữ liệu hoặc công nghệ triển khai khác nhau mà không ảnh hưởng đến các dịch vụ còn lại. Nhờ vậy, một lỗi phát sinh ở một dịch vụ thường chỉ ảnh hưởng cục bộ, thay vì lan ra toàn bộ hệ thống như trong kiến trúc Monolithic.

Đặc điểm này khiến Microservices phù hợp với các hệ thống có lượng người dùng lớn như thương mại điện tử, tài chính hoặc mạng xã hội, nơi từng nhóm chức năng có thể cần được mở rộng độc lập với tốc độ và mức độ khác nhau.



Hình 2.1 : Minh họa về kiến trúc Microservices

(Nguồn: https://microservices.io/i/Microservice_Architecture.png)

2.2.2. Những đặc trưng của Microservices

Kiến trúc Microservices mang lại cách tiếp cận linh hoạt trong xây dựng và phát triển hệ thống phần mềm phức tạp, đặc biệt trong lĩnh vực thương mại điện tử, nơi yêu cầu về khả năng mở rộng và tính sẵn sàng là rất cao. Việc phân tách hệ thống thành các dịch vụ nhỏ, độc lập giúp việc triển khai, mở rộng và bảo trì được thực hiện dễ dàng hơn mà không ảnh hưởng đến toàn bộ hệ thống. Trên cơ sở đó, kiến trúc này mang lại nhiều lợi ích quan trọng, có thể được thể hiện thông qua các đặc trưng tiêu biểu sau đây.

2.2.2.1. Tính mở rộng độc lập (Scalability)

Microservices cho phép từng dịch vụ được mở rộng độc lập theo nhu cầu sử dụng thực tế, thay vì phải tăng tài nguyên cho toàn bộ hệ thống. Ví dụ, trong các đợt siêu khuyến mãi, chỉ dịch vụ xử lý đơn hàng, nơi chịu tải cao nhất, cần được mở rộng để đáp ứng lượng truy cập tăng đột biến, trong khi các dịch vụ khác vẫn giữ nguyên. Cách tiếp cận này giúp tối ưu hóa việc sử dụng tài nguyên hệ thống, đồng thời giảm chi phí vận hành so với việc mở rộng toàn bộ ứng dụng như trong kiến trúc nguyên khối.

2.2.2.2. Tùy chọn công nghệ linh hoạt

Kiến trúc Microservices cho phép mỗi dịch vụ được phát triển bằng ngôn ngữ lập trình, framework và công nghệ phù hợp nhất với đặc thù của từng bài toán cụ thể. Thay vì bị ràng buộc vào một bộ công nghệ duy nhất cho toàn bộ hệ thống như kiến trúc nguyên khối, các nhóm phát triển có thể chủ động lựa chọn công cụ tối ưu cho từng dịch vụ riêng biệt, ví dụ sử dụng ngôn ngữ có hiệu năng xử lý song song tốt cho các dịch vụ chịu tải cao hoặc framework hỗ trợ sẵn thư viện AI cho các dịch vụ xử lý dữ liệu thông minh.

2.2.2.3. Cách ly và chịu lỗi

Một trong những đặc điểm quan trọng của kiến trúc Microservices là khả năng cách ly giữa các dịch vụ. Mỗi dịch vụ được triển khai và vận hành độc lập, do đó khi xảy ra lỗi, sự cố thường chỉ giới hạn trong phạm vi của dịch vụ đó và không ảnh hưởng đến toàn bộ hệ thống.

Nhờ đặc điểm này, hệ thống tránh được hiện tượng "sập dây chuyền" như trong kiến trúc nguyên khối, nơi một lỗi nhỏ có thể làm gián đoạn toàn bộ ứng dụng. Các dịch vụ khác vẫn tiếp tục hoạt động bình thường nhờ mức độ phụ thuộc lỏng lẻo giữa các thành phần.

Cơ chế cách ly giúp hệ thống tăng khả năng chịu lỗi (fault tolerance), giảm thời gian gián đoạn và bảo đảm tính sẵn sàng cao, từ đó duy trì trải nghiệm người dùng ổn định ngay cả khi một số dịch vụ gặp sự cố.

2.2.2.4. Phát triển và triển khai nhanh hơn

Với kiến trúc Microservices, các nhóm phát triển có thể đồng thời làm việc trên các dịch vụ khác nhau và triển khai các bản cập nhật một cách độc lập mà không phụ thuộc lẫn nhau. Cách tiếp cận phát triển song song này giúp rút ngắn đáng kể chu kỳ phát hành tính năng mới, đồng thời tạo điều kiện thuận lợi cho việc thử nghiệm và áp dụng nhanh chóng các công nghệ mới trong từng dịch vụ riêng biệt.

2.2.2.5. Dễ bảo trì hơn

Các dịch vụ trong kiến trúc Microservices được thiết kế theo hướng phân tách độc lập, giúp việc bảo trì, mở rộng và cập nhật trở nên dễ dàng hơn so với kiến trúc nguyên khối. Cách tiếp cận theo hướng mô-đun này không chỉ đơn giản hóa quá trình xử lý sự cố mà còn giảm thiểu ảnh hưởng giữa các thành phần, từ đó tăng tính linh hoạt và khả năng phát triển lâu dài của hệ thống.

Việc ứng dụng kiến trúc Microservices trong thương mại điện tử đòi hỏi sự đầu tư kỹ lưỡng vào cơ sở hạ tầng công nghệ, đặc biệt là các thành phần hỗ trợ điều phối, mở rộng và giao tiếp giữa các dịch vụ. Đối với hệ thống Electro Store, dự án tập trung vào ba nhóm thành phần cốt lõi như sau:

+ **API Gateway và Service Discovery (Định tuyến và khám phá dịch vụ):** Trong môi trường phân tán, API Gateway, được triển khai bằng Spring Cloud Gateway, đóng vai trò là điểm vào duy nhất (entry point), chịu trách nhiệm tiếp nhận, bảo mật và điều hướng các yêu cầu từ phía người dùng đến đúng dịch vụ đích. Bên cạnh đó, Eureka Server đảm nhiệm vai trò Service Discovery như một

"danh bạ" động, giúp hệ thống tự động nhận diện các dịch vụ đang hoạt động. Sự kết hợp này hỗ trợ cơ chế cân bằng tải phía client (Client-side Load Balancing), giúp phân phối lưu lượng hợp lý giữa các instance, từ đó nâng cao tính ổn định và khả năng mở rộng của hệ thống.

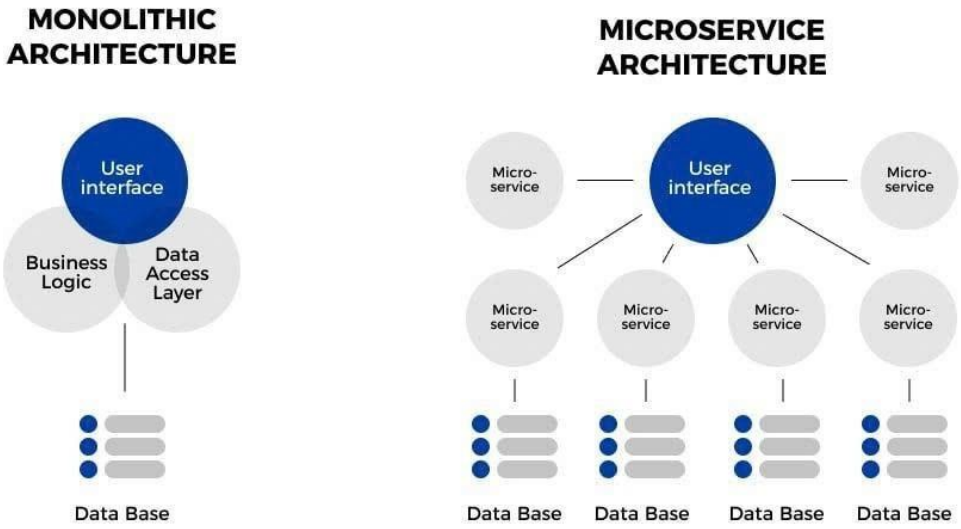
+ **Giao tiếp nội bộ (Internal Communication):** Trong giai đoạn hiện tại, hệ thống sử dụng cơ chế giao tiếp đồng bộ thông qua REST API để trao đổi dữ liệu giữa các dịch vụ, bao gồm cả các dịch vụ được xây dựng bằng Java và Python. Cách tiếp cận này giúp đơn giản hóa việc tích hợp và bảo đảm tính nhất quán trong xử lý nghiệp vụ. Trong tương lai, hệ thống được định hướng mở rộng sang mô hình bất đồng bộ thông qua Message Queue nhằm hỗ trợ kiến trúc hướng sự kiện (Event-Driven Architecture), giảm sự phụ thuộc chặt chẽ giữa các dịch vụ và tăng khả năng chịu lỗi khi hệ thống chịu tải lớn hoặc xảy ra sự cố cục bộ.

+ **Container Orchestration (Điều phối container bằng Docker Compose):**

Container hóa được sử dụng để đóng gói và cô lập môi trường chạy của từng dịch vụ, bao gồm các thành phần sử dụng Java, Python, cùng với các hệ quản trị dữ liệu tự lưu trữ như MySQL và Redis. Riêng cơ sở dữ liệu vector được sử dụng dưới dạng dịch vụ đám mây Pinecone Cloud, được truy cập thông qua API và không nằm trong phạm vi container hóa của hệ thống.

Thay vì triển khai các nền tảng điều phối phức tạp như Kubernetes, hệ thống sử dụng Docker Compose kết hợp với các script tự động hóa để quản lý toàn bộ vòng đời dịch vụ. Cách tiếp cận này cung cấp một cơ chế điều phối tinh gọn nhưng hiệu quả, cho phép khởi tạo, kết nối mạng nội bộ (networking) và quản lý đồng thời nhiều microservice thông qua một tệp cấu hình duy nhất, qua đó bảo đảm tính nhất quán giữa môi trường phát triển và môi trường triển khai thực tế.

2.2.3. So sánh kiến trúc Microservices và kiến trúc nguyên khối (Monolithic)



Hình 2.2 : So sánh giữa Monolithic và Microservices

(Nguồn: https://www.openlegacy.com)

Chương này sẽ so sánh hai kiến trúc phần mềm phổ biến là Microservices và Monolithic Architecture. Đây là hai mô hình phát triển ứng dụng có ảnh hưởng đáng kể đến cách thiết kế, triển khai và quản lý các hệ thống phần mềm.

Bảng 2.1 : So sánh kiến trúc Monolithic và Microservices

Tiêu chí	Monolithic Architecture	Microservices Architecture
Định nghĩa	Trong mô hình này, toàn bộ thành phần của ứng dụng, bao gồm giao diện, xử lý nghiệp vụ và cơ sở dữ liệu, được phát triển và triển khai như một đơn vị thống nhất. Mô hình thường đơn giản trong giai đoạn đầu nhưng trở nên khó quản lý khi quy mô hệ thống tăng lên.	Ứng dụng được chia thành nhiều dịch vụ nhỏ, mỗi dịch vụ đảm nhiệm một chức năng nghiệp vụ cụ thể. Các dịch vụ giao tiếp thông qua API và có thể được phát triển, triển khai hoặc nâng cấp độc lập.
Scalability và Maintenance	Việc mở rộng thường yêu cầu tăng tài nguyên cho toàn bộ ứng dụng. Một thay đổi nhỏ cũng có thể yêu cầu xây dựng và triển khai lại toàn bộ hệ thống, làm tăng thời gian bảo trì và nguy cơ gián đoạn dịch vụ.	Các dịch vụ có thể được mở rộng và cập nhật riêng theo nhu cầu thực tế. Cách tiếp cận này giảm phạm vi ảnh hưởng của thay đổi, nhưng đòi hỏi cơ chế quản lý giao tiếp và điều phối dịch vụ chặt chẽ.

Độ phức tạp và Cost of Development	Quá trình phát triển ban đầu thường nhanh và đơn giản do các thành phần được tích hợp trong cùng một ứng dụng. Tuy nhiên, chi phí bảo trì và độ phức tạp tăng đáng kể khi mã nguồn, dữ liệu và số lượng chức năng phát triển lớn dần.	Quá trình phát triển đòi hỏi kỹ năng cao hơn cùng hạ tầng hỗ trợ hệ thống phân tán, nên chi phí ban đầu có thể lớn. Đổi lại, kiến trúc mang lại khả năng mở rộng, thay đổi và bảo trì tốt hơn trong dài hạn.
Thích hợp với từng loại ứng dụng	Phù hợp với dự án nhỏ đến trung bình, yêu cầu nghiệp vụ tương đối ổn định, độ phức tạp không cao và chưa cần mở rộng độc lập theo từng chức năng.	Phù hợp với hệ thống lớn, yêu cầu khả năng mở rộng và tính linh hoạt cao, có nhiều miền nghiệp vụ hoặc thường xuyên phát hành và cập nhật chức năng.

2.2.4. Những khó khăn khi triển khai Microservices

Mặc dù Microservices mang lại khả năng mở rộng và triển khai độc lập, kiến trúc này cũng làm tăng độ phức tạp trong thiết kế và vận hành hệ thống [5]. Các dịch vụ giao tiếp thông qua mạng như HTTP hoặc Message Broker nên có thể phát sinh độ trễ và lỗi truyền thông, đòi hỏi các cơ chế như Retry và Circuit Breaker để nâng cao khả năng chịu lỗi [4].

Việc mỗi dịch vụ quản lý dữ liệu riêng cũng làm tăng khó khăn trong bảo đảm tính nhất quán. Đối với các nghiệp vụ liên quan đến nhiều dịch vụ, có thể cần áp dụng Saga Pattern và các giao dịch bù để xử lý khi một bước thất bại [5]. Bên cạnh đó, số lượng dịch vụ lớn làm tăng nhu cầu về Service Discovery, cân bằng tải, Distributed Tracing và các công cụ giám sát nhằm theo dõi luồng xử lý và phát hiện sự cố.

Microservices cũng yêu cầu hạ tầng và quy trình triển khai phức tạp hơn Monolithic. Việc vận hành nhiều dịch vụ có thể làm tăng chi phí tài nguyên, cấu hình và quản lý, đồng thời đòi hỏi tự động hóa triển khai, kiểm thử và quản lý phiên bản phù hợp [5].

Đối với Electro Store, các thách thức trên được giảm thiểu thông qua việc sử dụng Spring Cloud, Eureka Service Discovery, Docker Compose và Zipkin. Eureka hỗ trợ đăng ký và phát hiện dịch vụ, Docker Compose hỗ trợ triển khai đồng bộ các thành phần, trong khi Zipkin cung cấp khả năng Distributed Tracing. Ngoài ra, hệ

thống áp dụng kiến trúc đa ngôn ngữ (Polyglot Architecture), sử dụng Java Spring Boot cho các dịch vụ nghiệp vụ và Python/FastAPI cho các dịch vụ AI như hệ thống gợi ý và trợ lý ảo RAG.

2.2.4.1. API Gateway và giao tiếp giữa các dịch vụ

API Gateway đóng vai trò là điểm truy cập trung gian giữa client và các microservice, chịu trách nhiệm tiếp nhận, định tuyến và chuyển tiếp yêu cầu đến dịch vụ phù hợp. Việc sử dụng API Gateway giúp đơn giản hóa giao tiếp với client và tập trung các chức năng dùng chung như xác thực, phân quyền, logging và kiểm soát truy cập.

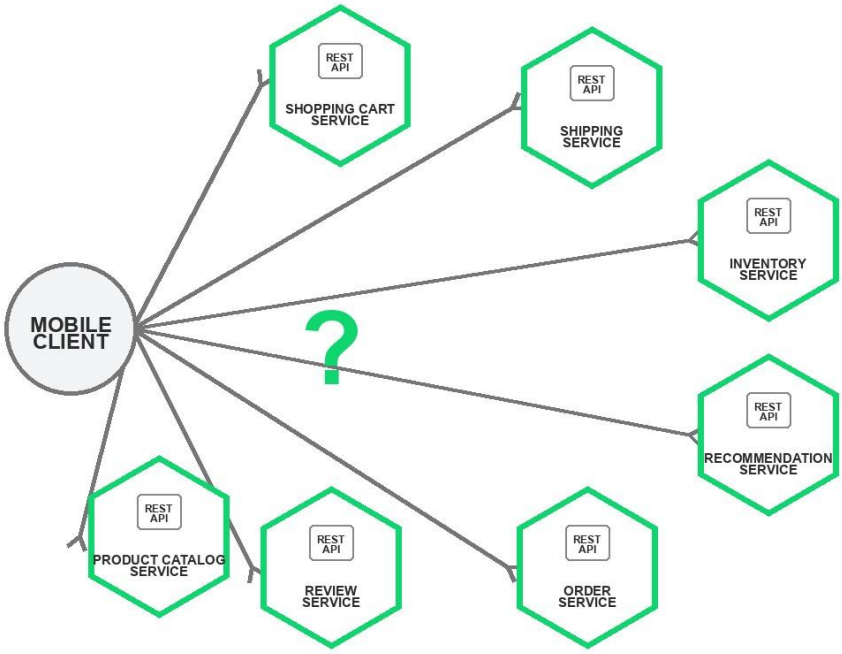
REST API (Representational State Transfer Application Programming Interface) là phương thức giao tiếp chủ yếu giữa các dịch vụ trong hệ thống. REST sử dụng giao thức HTTP và các phương thức như GET, POST, PUT và DELETE để thực hiện các thao tác CRUD trên tài nguyên. Việc sử dụng REST API giúp chuẩn hóa giao tiếp, thuận tiện cho tích hợp và mở rộng hệ thống.

Trong hệ sinh thái Spring, Spring Cloud Gateway được sử dụng để triển khai API Gateway. Công cụ này hỗ trợ định tuyến, bộ lọc yêu cầu và tích hợp với Spring Security và Eureka Service Discovery [9]. Trong Electro Store, Spring Cloud Gateway tiếp nhận yêu cầu từ phía client, xác thực và định tuyến đến microservice tương ứng. Cách tiếp cận này giúp giảm sự phụ thuộc trực tiếp giữa client và các dịch vụ phía sau, đồng thời tạo cơ sở cho việc mở rộng và quản lý hệ thống tập trung.

2.3. API Gateway

2.3.1. Khái niệm

API Gateway là một lớp trung gian giữa các client, chẳng hạn như ứng dụng web hoặc ứng dụng di động, và các dịch vụ back-end trong hệ thống. Thành phần này hoạt động như một reverse proxy, tiếp nhận các yêu cầu API từ client, định tuyến chúng đến dịch vụ back-end phù hợp và trả kết quả xử lý về cho client. Việc sử dụng API Gateway giúp đơn giản hóa quá trình quản lý và bảo mật API, đồng thời hỗ trợ cải thiện hiệu năng, khả năng mở rộng và mức độ ổn định của hệ thống.



Hình 2.3 : Mô tả về hệ thống API Gateway

(Nguồn: https://images.viblo.asia)

2.3.2. Ví dụ về cách hoạt động

Quá trình xử lý một yêu cầu thông qua API Gateway có thể được minh họa bằng trường hợp ứng dụng di động cần truy xuất thông tin người dùng. Luồng xử lý được thực hiện theo các bước sau:

Ứng dụng di động gửi yêu cầu lấy thông tin người dùng đến địa chỉ API được cung cấp bởi API Gateway, thay vì gọi trực tiếp đến dịch vụ quản lý người dùng.

API Gateway tiếp nhận yêu cầu, kiểm tra các điều kiện cần thiết như thông tin xác thực và quyền truy cập, sau đó định tuyến yêu cầu đến User Service tương ứng.

User Service xử lý nghiệp vụ và trả kết quả về API Gateway. Sau khi nhận được phản hồi, API Gateway chuyển dữ liệu kết quả về ứng dụng di động đã gửi yêu cầu ban đầu.

Thông qua cơ chế này, client chỉ cần giao tiếp với một điểm truy cập duy nhất, trong khi việc xác định và kết nối đến dịch vụ bên trong được API Gateway đảm nhiệm.

2.3.3. Chức năng và vai trò của API Gateway

2.3.3.1. Monitoring (quản lý) và routing (định tuyến) các yêu cầu API

API Gateway định tuyến các yêu cầu từ client đến các dịch vụ back-end dựa trên đường dẫn URL, phương thức HTTP và các tham số liên quan. Ví dụ, một yêu cầu GET đến đường dẫn /api/products có thể được định tuyến đến Product Service, trong khi một yêu cầu POST đến đường dẫn /api/orders có thể được chuyển đến Order Service. Cơ chế này giúp client chỉ cần giao tiếp với một điểm truy cập thống nhất mà không cần biết địa chỉ cụ thể của từng dịch vụ bên trong hệ thống.

2.3.3.2. Bảo mật API

API Gateway cung cấp các cơ chế bảo mật như xác thực (authentication), phân quyền (authorization) và mã hóa (encryption). Thành phần này có thể tích hợp với các phương thức xác thực như OAuth 2.0, JWT và API Key nhằm bảo đảm chỉ những yêu cầu hợp lệ mới được chuyển đến các dịch vụ phía sau. Ví dụ, API Gateway có thể yêu cầu mỗi request phải kèm theo một token xác thực, sau đó kiểm tra tính hợp lệ và quyền truy cập của token để xác định người dùng có được phép thực hiện hành động tương ứng hay không. Việc tập trung xử lý bảo mật tại Gateway giúp các chính sách truy cập được áp dụng thống nhất trên toàn hệ thống.

2.3.3.3. Quản lý lưu lượng và cân bằng tải

API Gateway giúp quản lý lưu lượng truy cập và bảo vệ các dịch vụ back-end khỏi tình trạng quá tải thông qua những cơ chế như giới hạn tốc độ (rate limiting) và làm mịn lưu lượng (traffic shaping). Ví dụ, hệ thống có thể giới hạn một client cụ thể chỉ được gửi tối đa 100 yêu cầu trong mỗi phút. Bên cạnh đó, API Gateway hỗ trợ cân bằng tải bằng cách phân phối các yêu cầu đến nhiều instance của dịch vụ back-end, qua đó hạn chế tình trạng một instance phải xử lý quá nhiều yêu cầu và góp phần duy trì sự ổn định của toàn hệ thống.

2.3.3.4. Giám sát và ghi log

API Gateway ghi lại các thông tin liên quan đến request và response, qua đó hỗ trợ giám sát hiệu suất và trạng thái hoạt động của các dịch vụ. Thành phần này cũng có thể tích hợp với các công cụ phân tích log để phát hiện, xác định nguyên nhân và xử lý

lỗi phát sinh trong quá trình vận hành. Ví dụ, API Gateway có thể ghi nhận thông tin chi tiết của mỗi yêu cầu như thời gian phản hồi, mã trạng thái HTTP (status code) và nội dung lỗi, giúp đội ngũ phát triển và quản trị viên dễ dàng theo dõi, phân tích và tối ưu hóa hệ thống.

2.3.4. Nhược điểm khi dùng API Gateway

Mặc dù API Gateway giúp đơn giản hóa việc giao tiếp và quản lý các dịch vụ, việc bổ sung thành phần trung gian này cũng tạo ra một số hạn chế cần được xem xét trong quá trình thiết kế hệ thống.

2.3.4.1. Độ phức tạp và chi phí triển khai

Việc triển khai và cấu hình API Gateway có thể trở nên phức tạp và tốn kém, đặc biệt đối với các tổ chức mới bắt đầu áp dụng kiến trúc này. Trong quá trình triển khai, cần đầu tư đáng kể vào hạ tầng, phần mềm cũng như nguồn nhân lực để xây dựng và vận hành API Gateway một cách hiệu quả.

Cụ thể, hệ thống không chỉ yêu cầu cài đặt và cấu hình ban đầu mà còn cần tích hợp API Gateway với các dịch vụ hiện có. Bên cạnh đó, tổ chức cũng phải dành nguồn lực cho việc đào tạo nhân sự về các công cụ sử dụng, quy trình triển khai và phương thức vận hành nhằm bảo đảm hệ thống hoạt động ổn định và đáp ứng đúng yêu cầu.

2.3.4.2. Độ trễ (latency)

Khi hệ thống sử dụng API Gateway, các yêu cầu từ phía client không đi trực tiếp đến dịch vụ back-end mà phải đi qua một lớp trung gian. Lớp này thực hiện nhiều nhiệm vụ như xác thực người dùng, kiểm tra quyền truy cập, ghi log và trong một số trường hợp còn chuyển đổi định dạng dữ liệu trước khi chuyển tiếp yêu cầu đến dịch vụ đích.

Mỗi bước xử lý đều cần một khoảng thời gian nhất định. Khi các khoảng thời gian này được cộng dồn, tổng thời gian phản hồi của hệ thống có thể tăng lên so với mô hình giao tiếp trực tiếp giữa client và dịch vụ. Ảnh hưởng này trở nên rõ ràng hơn trong các hệ thống có lưu lượng truy cập lớn hoặc yêu cầu phản hồi gần như tức thời, chẳng hạn như ứng dụng tài chính hoặc hệ thống thời gian thực.

Trong thực tế, mức độ ảnh hưởng của độ trễ phụ thuộc vào cách thiết kế API Gateway và mức độ tối ưu của các bước xử lý bên trong. Nếu các chức năng được tổ chức hợp lý và những thao tác không cần thiết được giảm thiểu, tác động đến hiệu năng có thể được kiểm soát ở mức chấp nhận được.

2.3.4.3. Điểm nghẽn cổ chai

API Gateway có thể trở thành một điểm nghẽn trong hệ thống vì mọi yêu cầu từ client đều phải đi qua thành phần này trước khi được chuyển tiếp đến các dịch vụ phía sau. Khi lưu lượng truy cập tăng cao hoặc Gateway gặp sự cố như quá tải tài nguyên hay bị tấn công từ chối dịch vụ, toàn bộ hệ thống có thể bị ảnh hưởng mặc dù các dịch vụ bên trong vẫn đang hoạt động bình thường.

Để giảm thiểu rủi ro này, hệ thống không nên chỉ phụ thuộc vào một Gateway duy nhất. Thay vào đó, có thể triển khai nhiều instance Gateway chạy song song và đặt phía sau một cơ chế cân bằng tải để phân phối request. Khi một instance gặp sự cố hoặc không còn khả năng xử lý, các request sẽ được tự động chuyển sang instance khác đang hoạt động, giúp hệ thống duy trì khả năng phục vụ liên tục và hạn chế tình trạng gián đoạn dịch vụ.

2.3.4.4. Phụ thuộc vào nhà cung cấp dịch vụ lưu trữ API Gateway

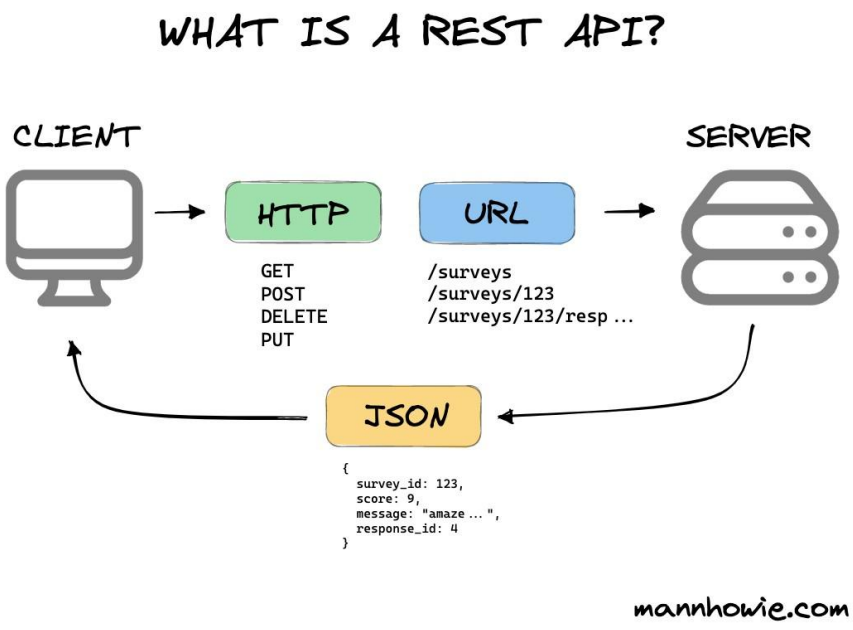
Nếu một tổ chức lựa chọn sử dụng API Gateway từ các nhà cung cấp thương mại hoặc các dịch vụ đám mây, họ có thể phải đối mặt với tình trạng phụ thuộc vào nền tảng đã chọn. Điều này có thể dẫn đến một số rủi ro trong quá trình vận hành hệ thống, chẳng hạn như chi phí sử dụng tăng dần theo thời gian, hạn chế trong việc tùy chỉnh hoặc mở rộng tính năng, cũng như những khó khăn khi muốn chuyển đổi sang một giải pháp khác.

Vì vậy, khi thiết kế và triển khai API Gateway, việc đánh giá kỹ các phương án hiện có là rất quan trọng. Tổ chức cần cân nhắc mức độ phù hợp của từng nhà cung cấp, khả năng hỗ trợ kỹ thuật và độ ổn định của dịch vụ để bảo đảm hệ thống hoạt động hiệu quả trong dài hạn, đồng thời giảm thiểu những rủi ro có thể phát sinh trong quá trình sử dụng.

2.4. REST API

2.4.1. Khái niệm và lịch sử hình thành

REST (Representational State Transfer) là một phong cách kiến trúc được sử dụng để thiết kế các hệ thống mạng, đặc biệt là các dịch vụ web. RESTful API là một loại API tuân theo các nguyên tắc của kiến trúc REST, sử dụng các phương thức HTTP để thực hiện những thao tác CRUD (Create, Read, Update, Delete) trên các tài nguyên web.



Hình 2.4 : Tổng quan kiến trúc hoạt động của RESTful API

(Nguồn: https://images.ctfassets.net/vwq10xzbe6iz/5sBH4Agl614xM7exeLsTo7/9e84dce01735f155911e611c42c9793f/rest-api.png)

REST tuân theo một tập hợp các nguyên tắc kiến trúc nhằm đơn giản hóa quá trình giao tiếp và nâng cao hiệu quả của dịch vụ web. Một API đáp ứng những nguyên tắc này được gọi là RESTful API. Các nguyên tắc cơ bản của REST bao gồm:

+ **Stateless (không lưu trạng thái):** Mỗi yêu cầu từ client đến server phải chứa đầy đủ thông tin cần thiết để server xử lý. Server không lưu trữ trạng thái phiên của client giữa các yêu cầu, nhờ đó giảm sự phụ thuộc và hỗ trợ mở rộng hệ thống thuận lợi hơn.

+ **Client–Server:** REST phân tách rõ trách nhiệm giữa client và server. Client tập trung vào giao diện và trải nghiệm người dùng, trong khi server chịu trách nhiệm xử lý nghiệp vụ và quản lý dữ liệu. Sự phân tách này cho phép hai phía được phát triển và mở rộng tương đối độc lập.

+ **Uniform Interface (giao diện thống nhất):** Các tài nguyên được truy cập thông qua một giao diện và tập quy ước thống nhất. Nguyên tắc này giúp giảm độ phức tạp trong giao tiếp, đồng thời cải thiện khả năng tích hợp giữa các hệ thống khác nhau.

+ **Layered System (hệ thống phân tầng):** Kiến trúc REST cho phép bố trí các tầng trung gian như proxy, API Gateway, bộ cân bằng tải hoặc bộ nhớ đệm giữa client và server. Client không cần biết đang giao tiếp trực tiếp với server hay thông qua một thành phần trung gian, qua đó tăng tính linh hoạt, bảo mật và khả năng mở rộng.

+ **Cacheable (có khả năng lưu đệm):** Phản hồi từ server cần xác định rõ có được phép lưu vào bộ nhớ đệm hay không. Việc tái sử dụng các phản hồi phù hợp giúp giảm số lần xử lý lặp lại, tiết kiệm băng thông và cải thiện thời gian phản hồi của hệ thống.

+ **Code on Demand (mã theo yêu cầu, tùy chọn):** Trong một số trường hợp, server có thể gửi mã thực thi đến client để mở rộng chức năng tại thời điểm chạy.

Đây là nguyên tắc tùy chọn và không bắt buộc đối với mọi RESTful API.

Lịch sử hình thành của REST gắn liền với quá trình phát triển của World Wide

Web vào cuối thập niên 1990 và đầu những năm 2000. Trong giai đoạn đầu, các ứng dụng web chủ yếu cung cấp nội dung HTML tĩnh và sử dụng những cơ chế như CGI (Common Gateway Interface) để tiếp nhận, xử lý các yêu cầu HTTP. Khi quy mô và mức độ tương tác của ứng dụng web ngày càng tăng, nhu cầu về một mô hình kiến trúc đơn giản, linh hoạt và có khả năng mở rộng trở nên rõ ràng hơn.

Từ giữa thập niên 1990, Roy Fielding, người tham gia xây dựng các đặc tả quan trọng của HTTP/1.1, bắt đầu nghiên cứu các nguyên tắc kiến trúc dành cho hệ thống phân tán trên nền Web. Đến năm 2000, ông trình bày luận án tiến sĩ tại Đại học California, Irvine, trong đó chính thức giới thiệu khái niệm REST và hệ thống hóa các

ràng buộc kiến trúc giúp xây dựng dịch vụ web có khả năng mở rộng, dễ bảo trì và ít phụ thuộc giữa các thành phần.

Trong những năm 2000, REST nhanh chóng được cộng đồng phát triển phần mềm tiếp nhận nhờ cách tiếp cận đơn giản và phù hợp với giao thức HTTP. Nhiều nền tảng trực tuyến lớn bắt đầu cung cấp REST API để các ứng dụng bên ngoài có thể truy cập dữ liệu và tích hợp dịch vụ. Đến thập niên 2010, REST API trở thành một tiêu chuẩn được sử dụng phổ biến trong quá trình xây dựng dịch vụ web và hệ thống Microservices. Các framework như Django, Ruby on Rails, Spring và Flask cũng cung cấp nhiều công cụ hỗ trợ, giúp quá trình thiết kế, triển khai và quản lý REST API trở nên thuận tiện hơn.

Hiện nay, REST API vẫn được sử dụng rộng rãi trong các ứng dụng web, ứng dụng di động và hệ thống phân tán. Mặc dù những công nghệ như GraphQL và gRPC đã xuất hiện nhằm giải quyết một số hạn chế của REST trong các trường hợp chuyên biệt, REST vẫn giữ vai trò quan trọng nhờ tính đơn giản, mức độ phổ biến cao và khả năng tương thích tốt với hạ tầng Web. Từ một khái niệm lý thuyết, REST đã phát triển thành một phong cách kiến trúc nền tảng, góp phần đơn giản hóa việc xây dựng, triển khai và quản lý các dịch vụ web hiện đại.

2.4.2. Các phương thức HTTP

RESTful API sử dụng các phương thức HTTP để biểu diễn thao tác cần thực hiện trên tài nguyên. Trong phạm vi hệ thống thương mại điện tử, những phương thức phổ biến bao gồm:

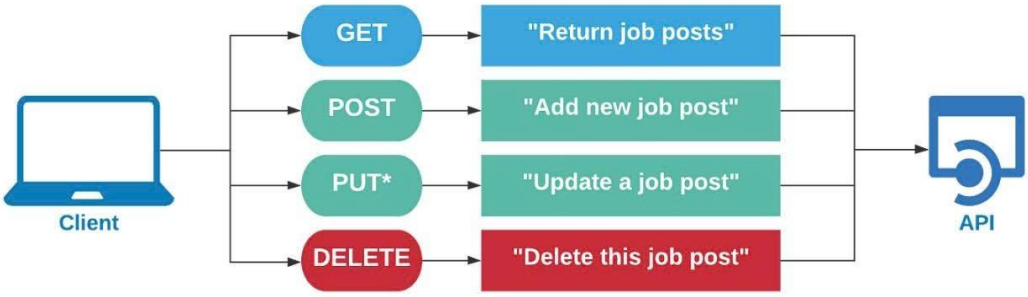
+ **GET:** Được sử dụng để truy xuất thông tin từ server mà không làm thay đổi trạng thái của tài nguyên. Ví dụ, yêu cầu GET /api/products trả về danh sách tất cả sản phẩm trong hệ thống.

+ **POST:** Được sử dụng để gửi dữ liệu đến server nhằm tạo một tài nguyên mới. Ví dụ, yêu cầu POST /api/products kèm dữ liệu sản phẩm trong phần thân yêu cầu sẽ tạo một sản phẩm mới.

+ **PUT:** Được sử dụng để cập nhật toàn bộ thông tin của một tài nguyên hiện có bằng dữ liệu mới. Ví dụ, yêu cầu PUT /api/products/1 kèm dữ liệu cập nhật trong phần thân yêu cầu sẽ thay thế thông tin của sản phẩm có mã định danh là 1.

+ **DELETE:** Được sử dụng để xóa một tài nguyên trên server. Ví dụ, yêu cầu DELETE /api/products/1 sẽ xóa sản phẩm có mã định danh là 1.

+ **PATCH:** Được sử dụng để cập nhật một phần thông tin của tài nguyên hiện có. Ví dụ, yêu cầu PATCH /api/products/1 kèm dữ liệu trong phần thân yêu cầu có thể cập nhật một hoặc một số thuộc tính của sản phẩm có mã định danh là 1 mà không thay thế toàn bộ tài nguyên.



Hình 2.5: Ánh xạ các phương thức HTTP với thao tác dữ liệu (CRUD)

(Nguồn: https://miro.medium.com/v2/resize:fit:1400/1*UXTnlhEg4aua_6eQv3HHSg.png)

Những phương thức và hoạt động này thường được gọi chung là CRUD, tương ứng với bốn thao tác cơ bản: Create, Read, Update và Delete, lần lượt mang ý nghĩa Tạo, Đọc, Cập nhật và Xóa dữ liệu.

Một số mã trạng thái (Status Code) thường gặp khi gửi yêu cầu đến API gồm:

200 OK: Yêu cầu được xử lý thành công, thường được trả về đối với các phương thức GET, PUT, PATCH hoặc DELETE.

201 Created: Một tài nguyên mới đã được tạo thành công.

204 No Content: Yêu cầu được xử lý thành công nhưng phản hồi không chứa nội dung, thường được sử dụng sau thao tác xóa.

304 Not Modified: Tài nguyên chưa thay đổi và client có thể tiếp tục sử dụng dữ liệu đã được lưu trong bộ nhớ đệm.

400 Bad Request: Yêu cầu không hợp lệ hoặc có cấu trúc dữ liệu không đúng.

401 Unauthorized: Yêu cầu chưa cung cấp thông tin xác thực hợp lệ.

403 Forbidden: Server hiểu yêu cầu nhưng từ chối quyền truy cập vào tài nguyên.

404 Not Found: Không tìm thấy tài nguyên tương ứng với URI được yêu cầu.

405 Method Not Allowed: Tài nguyên không hỗ trợ phương thức HTTP được sử dụng trong yêu cầu.

410 Gone: Tài nguyên không còn tồn tại và không có địa chỉ thay thế.

415 Unsupported Media Type: Server không hỗ trợ kiểu dữ liệu được gửi trong yêu cầu.

422 Unprocessable Entity: Dữ liệu có cấu trúc hợp lệ nhưng không đáp ứng điều kiện nghiệp vụ hoặc không thể được xử lý.

429 Too Many Requests: Yêu cầu bị từ chối do client vượt quá giới hạn tần suất truy cập cho phép [24].

Trong tiểu luận này, RESTful API được sử dụng làm cơ chế giao tiếp giữa Frontend và Backend. Frontend gửi yêu cầu HTTP đến các API tương ứng, Backend tiếp nhận và xử lý nghiệp vụ, sau đó trả về dữ liệu cùng mã trạng thái phù hợp để Frontend hiển thị kết quả cho người dùng.

2.5. Spring Cloud Gateway

2.5.1. Giới thiệu về Spring Cloud Gateway

Spring Cloud Gateway là một nền tảng quản lý API mã nguồn mở trong hệ sinh thái Spring, hoạt động như một lớp phần mềm trung gian (middleware) giữa client và các dịch vụ backend. Nền tảng này được thiết kế phù hợp với kiến trúc Microservices và có khả năng mở rộng thông qua hệ thống bộ lọc (Gateway Filters). Nhờ được xây dựng trên Spring WebFlux và mô hình lập trình phản ứng (Reactive Programming),

Spring Cloud Gateway hỗ trợ xử lý bất đồng bộ hiệu quả, đáp ứng tốt các hệ thống có số lượng lớn yêu cầu đồng thời và dễ dàng tích hợp với những ứng dụng Java sử dụng Spring Boot [9].

Spring Cloud Gateway cung cấp nhiều chức năng quản lý API như định tuyến động, xác thực, phân quyền, giới hạn tần suất truy cập (rate limiting), cân bằng tải (load balancing), biến đổi request hoặc response và giám sát trạng thái hoạt động thông qua Spring Boot Actuator. Gateway có thể được triển khai dưới dạng một ứng dụng Spring Boot độc lập trên máy chủ vật lý hoặc máy ảo, đồng thời hỗ trợ đóng gói bằng

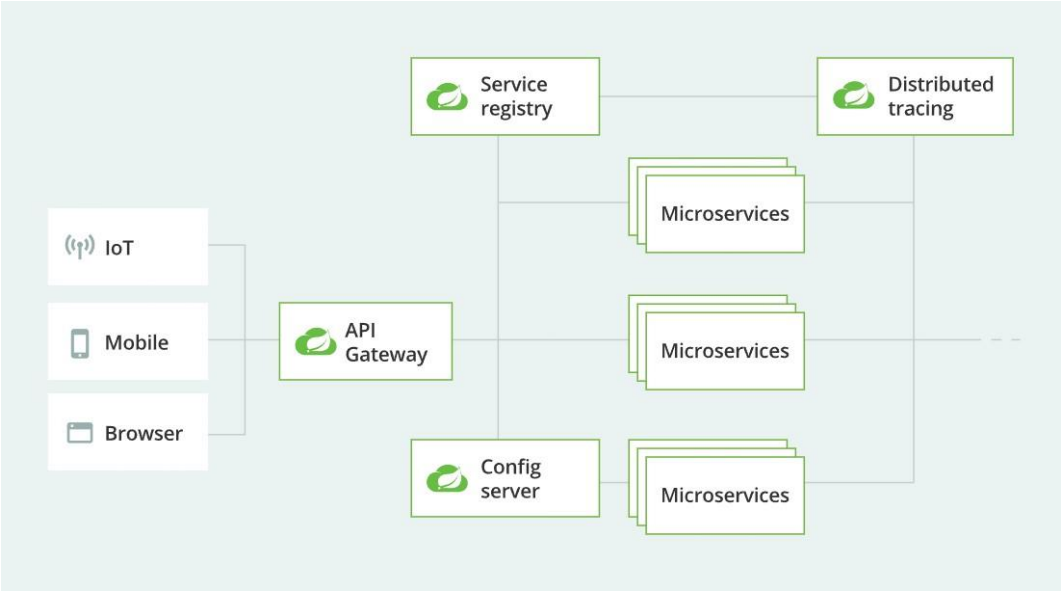
Docker và vận hành trong các cụm Kubernetes. Khả năng triển khai linh hoạt này giúp Gateway phù hợp với cả môi trường truyền thống lẫn hạ tầng cloud-native.

Thông qua các Gateway Actuator Endpoint, quản trị viên có thể truy xuất thông tin về các tuyến đường (routes), kiểm tra bộ lọc đang được áp dụng, làm mới cấu hình định tuyến và theo dõi trạng thái hoạt động của Gateway dưới dạng RESTful API. Khi kết hợp với cơ chế cấu hình động hoặc kho lưu trữ tuyến đường phù hợp, các endpoint quản trị này hỗ trợ quá trình giám sát và điều chỉnh hệ thống thuận tiện hơn. Các chức năng bổ sung được triển khai thông qua Gateway Filters, cho phép xử lý những yêu cầu dùng chung như kiểm tra token, bổ sung header, ghi log, giới hạn lưu lượng hoặc chuẩn hóa dữ liệu mà không cần lặp lại mã nguồn tại từng microservice.

Trong quá trình vận hành, Spring Cloud Gateway được đặt phía trước các microservice và trở thành điểm vào (entry point) thống nhất cho toàn bộ API của hệ thống. Mọi request từ client trước tiên được tiếp nhận tại Gateway và đi qua chuỗi bộ lọc đã cấu hình. Sau khi hoàn tất các bước kiểm tra và xử lý trước (Pre-filters), request được định tuyến và chuyển tiếp đến dịch vụ backend phù hợp. Response do microservice trả về tiếp tục đi qua các bộ lọc sau (Post-filters) trước khi được gửi lại cho client. Cơ chế này cho phép tập trung hóa các chính sách bảo mật, điều phối lưu lượng, giám sát và xử lý giao tiếp tại một thành phần chung, qua đó giúp hệ thống Microservices dễ quản lý và mở rộng hơn.

Hình 2.6 minh họa vị trí của API Gateway trong hệ sinh thái Spring Cloud. Các yêu cầu từ thiết bị IoT, ứng dụng di động và trình duyệt được tập trung tại Gateway

trước khi chuyển đến những microservice phù hợp. Đồng thời, Gateway phối hợp với các thành phần hạ tầng như Service Registry, Config Server và Distributed Tracing để hỗ trợ khám phá dịch vụ, quản lý cấu hình tập trung và theo dõi luồng xử lý trong hệ thống phân tán.



Hình 2.6 : Vai trò của API Gateway trong hệ sinh thái Spring Cloud

(Nguồn: https://spring.io/cloud)

Trong phạm vi đề tài, Spring Cloud Gateway đóng vai trò là cổng vào duy nhất (Single Entry Point), tiếp nhận toàn bộ request từ Frontend và định tuyến đến các microservice tương ứng dựa trên đường dẫn API đã được cấu hình. Các dịch vụ đích bao gồm những phân hệ nghiệp vụ cốt lõi như Sản phẩm, Đơn hàng, Giỏ hàng và Người dùng, đồng thời bao gồm các dịch vụ xử lý Trí tuệ nhân tạo như Gợi ý sản phẩm và Trợ lý ảo AI. Gateway cũng được tích hợp với Eureka Server để hỗ trợ khám phá dịch vụ và cân bằng tải động, qua đó phân phối request giữa nhiều instance của từng service, góp phần duy trì tính ổn định và nâng cao hiệu quả vận hành của hệ thống.

2.5.2. Kiến trúc và thành phần của Spring Cloud Gateway

Spring Cloud Gateway tổ chức cơ chế định tuyến dựa trên các tuyến đường, điều kiện khớp và chuỗi bộ lọc. Mỗi request được Gateway đối chiếu với cấu hình hiện có để xác định dịch vụ đích, sau đó lần lượt đi qua các bộ lọc trước và sau khi được chuyển tiếp. Những thành phần cốt lõi của Spring Cloud Gateway bao gồm [9]:

Route (tuyến đường): Là đơn vị cơ bản của cơ chế định tuyến, bao gồm một mã định danh duy nhất, URI của dịch vụ đích, tập hợp các điều kiện (predicates) và các bộ lọc (filters). Route mô tả request nào được tiếp nhận, dịch vụ nào sẽ xử lý request và những thao tác nào được áp dụng trong quá trình chuyển tiếp.

Predicates (điều kiện định tuyến): Được sử dụng để xác định một request có phù hợp với Route hay không dựa trên các thuộc tính HTTP như đường dẫn, phương thức, header, host, query parameter hoặc thời gian gửi yêu cầu. Một Route chỉ được kích hoạt khi request thỏa mãn các Predicate đã cấu hình.

Gateway Filters (bộ lọc Gateway): Đại diện cho các logic xử lý được tích hợp vào luồng request và response của một Route cụ thể. Các bộ lọc có thể kiểm tra quyền truy cập, bổ sung hoặc xóa header, thay đổi đường dẫn, giới hạn lưu lượng, ghi log và kiểm soát dữ liệu được phép đi vào hoặc đi ra khỏi API.

Global Filters (bộ lọc toàn cục): Là các bộ lọc được áp dụng cho toàn bộ Route trong Gateway thay vì chỉ một tuyến đường riêng lẻ. Thành phần này phù hợp với những xử lý dùng chung như theo dõi request, ghi nhật ký, đo thời gian phản hồi, xử lý lỗi và áp dụng chính sách bảo mật thống nhất cho toàn hệ thống.

DiscoveryClient Route Definition Locator: Là thành phần tích hợp với các hệ thống khám phá dịch vụ như Eureka hoặc Consul. Dựa trên danh sách dịch vụ đã đăng ký, Gateway có thể tự động tạo Route và định tuyến đến các instance microservice tương ứng mà không cần khai báo thủ công từng địa chỉ dịch vụ trong tệp cấu hình.

2.6. Các công nghệ hỗ trợ

2.6.1. Spring Boot Framework

Spring Boot là một framework phát triển ứng dụng Java, được sử dụng để xây dựng tầng backend và xử lý nghiệp vụ trung tâm trong hệ thống thương mại điện tử. Với khả năng hỗ trợ mạnh mẽ kiến trúc RESTful API, Spring Boot cho phép triển khai nhanh chóng các dịch vụ backend theo hướng Microservices, đồng thời bảo đảm tính ổn định, khả năng bảo trì và mở rộng của hệ thống. Trong hệ thống, Spring Boot tiếp nhận các yêu cầu từ tầng giao diện thông qua các endpoint REST, sau đó thực hiện xử lý logic nghiệp vụ, quản lý dữ liệu và điều phối hoạt động giữa các thành phần liên

quan. Kết quả xử lý thường được trả về cho client dưới định dạng JSON, giúp bảo đảm tính tương thích và thuận tiện trong quá trình trao đổi dữ liệu giữa các dịch vụ. Bên cạnh đó, Spring Boot hỗ trợ tích hợp với các công nghệ truy cập dữ liệu như Spring Data JPA và Hibernate, đồng thời cung cấp cơ chế Dependency Injection nhằm đơn giản hóa việc quản lý và liên kết các thành phần trong hệ thống Microservices [10].



Hình 2.7 : Logo của Spring Boot

(Nguồn: https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcSRRdthosAbmOZGTXgtZqa2NUXrXY_By3yAjTbwaA29FbzbmgqllpLYNEA3&s=10)

Trong tiểu luận này, Spring Boot được sử dụng làm nền tảng backend chính, đóng vai trò trung tâm trong quá trình xây dựng và vận hành hệ thống. Backend đảm nhiệm các nghiệp vụ cốt lõi như quản lý người dùng, sản phẩm, đơn hàng, giỏ hàng và xác thực, đồng thời duy trì kết nối ổn định với cơ sở dữ liệu và cung cấp các API phục vụ tầng Frontend.

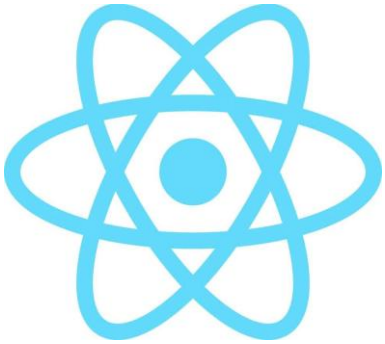
Ngoài vai trò xử lý nghiệp vụ, Spring Boot còn được sử dụng để triển khai các thành phần hạ tầng trong kiến trúc Microservices, bao gồm API Gateway, Service Discovery thông qua Eureka và Config Server. Sự kết hợp này cho phép hệ thống vận hành theo mô hình phân tán nhưng vẫn duy trì khả năng liên kết và phối hợp giữa các dịch vụ, đồng thời tạo điều kiện thuận lợi cho việc mở rộng, cấu hình và bảo trì hệ thống [10].

Bên cạnh đó, Spring Boot được tích hợp với Spring AI để xây dựng dịch vụ Chatbot, đóng vai trò trung gian giữa người dùng và mô hình trí tuệ nhân tạo. Thành

phần này hỗ trợ tiếp nhận yêu cầu, truy xuất dữ liệu liên quan và tạo phản hồi theo thời gian thực, qua đó nâng cao khả năng tương tác và trải nghiệm của người dùng trong hệ thống Electro Store[11].

2.6.2. React.js

React.js là một thư viện JavaScript mã nguồn mở do Facebook, hiện nay là Meta, phát triển và duy trì. Thư viện này được sử dụng phổ biến để xây dựng giao diện người dùng cho các ứng dụng web hiện đại, đặc biệt là ứng dụng một trang (Single Page Application – SPA). React.js tổ chức giao diện dựa trên các component, trong đó mỗi component đại diện cho một thành phần giao diện độc lập, có thể tái sử dụng và bảo trì thuận tiện [12].



Hình 2.8 : Logo của React.js

(Nguồn: https://tinyurl.com/jjx6wjka)

Kiến trúc component giúp phân chia hệ thống giao diện thành các khối chức năng rõ ràng, qua đó nâng cao khả năng mở rộng và giảm độ phức tạp trong quá trình phát triển. Mỗi component có thể tiếp nhận dữ liệu đầu vào thông qua props và quản lý trạng thái riêng thông qua state cùng các React Hooks. Cách tổ chức này giúp luồng dữ liệu trong ứng dụng trở nên minh bạch, nhất quán và dễ kiểm soát hơn [12].

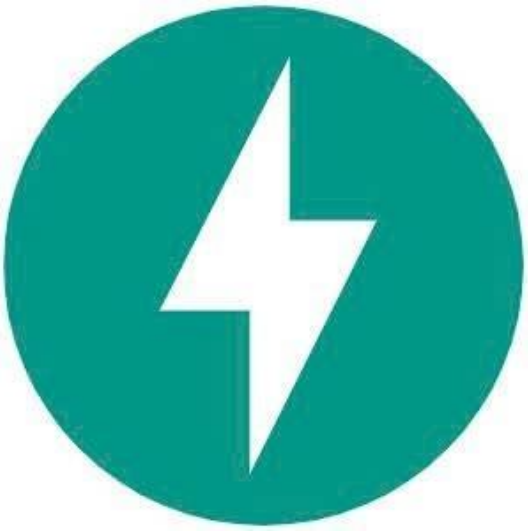
Một đặc điểm nổi bật của React.js là cơ chế kết xuất khai báo kết hợp với Virtual DOM. Khi dữ liệu hoặc trạng thái thay đổi, React tạo biểu diễn giao diện mới trong bộ nhớ, so sánh với phiên bản trước và chỉ cập nhật những thành phần cần thiết trên DOM thực. Cơ chế này hạn chế các thao tác kết xuất không cần thiết, góp phần cải thiện hiệu năng và duy trì trải nghiệm tương tác mượt mà cho người dùng[12].

Trong tiểu luận này, React.js được sử dụng để xây dựng tầng giao diện người dùng(Frontend) theo mô hình SPA. Giao diện có nhiệm vụ tiếp nhận thao tác của người dùng, trao đổi dữ liệu với các dịch vụ backend thông qua REST API và cập nhật nội dung mà không cần tải lại toàn bộ trang. Nhờ đó, hệ thống có thể trình bày

nhất quán các thông tin về sản phẩm, đơn hàng, đánh giá, phản hồi và kết quả gợi ý cá nhân hóa từ các dịch vụ trí tuệ nhân tạo.

2.6.3. FastAPI

FastAPI là một framework hiện đại trên nền tảng Python, có hiệu năng cao và thường được sử dụng để xây dựng các dịch vụ trí tuệ nhân tạo dưới dạng API trong hệ thống thương mại điện tử. Framework hỗ trợ xây dựng các endpoint với cú pháp ngắn gọn, khả năng kiểm tra kiểu dữ liệu và tự động sinh tài liệu API, qua đó đơn giản hóa quá trình phát triển và tích hợp dịch vụ [16].



Hình 2.9 : Logo của FastAPI

(*Nguồn:* https://miro.medium.com/1*3Hf5RE6HpJ3WktzkTPATeA.png)

FastAPI hỗ trợ sẵn cơ chế kiểm tra và chuyển đổi dữ liệu, xử lý bất đồng bộ thông qua cú pháp async/await, đồng thời tự động sinh tài liệu API theo đặc tả OpenAPI. Các giao diện tài liệu tương tác cho phép lập trình viên kiểm tra endpoint, tham số và phản hồi trực tiếp trên trình duyệt, qua đó đơn giản hóa quá trình phát triển, tích hợp và kiểm thử giữa các microservice.

Trong kiến trúc Microservices, FastAPI phù hợp với các dịch vụ cần triển khai độc lập, đặc biệt là những thành phần xử lý dữ liệu, học máy và trí tuệ nhân tạo được xây dựng bằng Python. Các dịch vụ FastAPI có thể giao tiếp với hệ thống Spring Boot thông qua REST API và trao đổi dữ liệu dưới định dạng JSON, giúp duy trì sự phân tách công nghệ nhưng vẫn bảo đảm khả năng phối hợp giữa các thành phần.

Trong tiểu luận này, FastAPI được sử dụng để xây dựng dịch vụ gợi ý sản phẩm dựa trên phương pháp lọc cộng tác. Dịch vụ tiếp nhận dữ liệu hoặc yêu cầu dự đoán từ hệ thống backend, thực hiện xử lý bằng mô hình học máy và trả về danh sách sản phẩm được đề xuất. Cách tổ chức này giúp cô lập quá trình tính toán AI khỏi các dịch vụ nghiệp vụ chính, đồng thời tạo điều kiện thuận lợi cho việc cập nhật mô hình, mở rộng tài nguyên và bảo trì hệ thống.

2.6.4. MySQL

MySQL là hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở phổ biến, được sử dụng trong hệ thống nhằm lưu trữ và quản lý dữ liệu một cách an toàn, nhất quán và có hiệu năng truy xuất cao. Hệ quản trị này hỗ trợ ngôn ngữ truy vấn SQL và cung cấp các cơ chế tổ chức, lập chỉ mục và tối ưu hóa cấu trúc bảng, qua đó góp phần bảo đảm tính toàn vẹn cũng như độ tin cậy của dữ liệu trong quá trình vận hành hệ thống[13].



Hình 2.10 : Logo của hệ quản trị cơ sở dữ liệu MySQL

(Nguồn: https://www.logo.wine/a/logo/MySQL/MySQL-Logo.wine.svg)

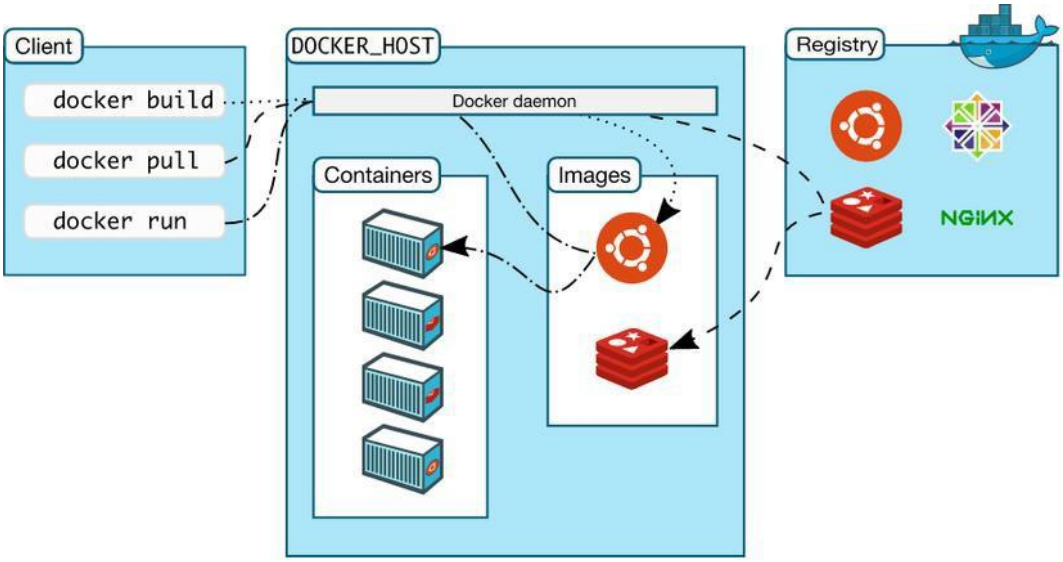
Trong đề tài này, MySQL được sử dụng để lưu trữ toàn bộ dữ liệu có cấu trúc cốt lõi của sàn thương mại điện tử, bao gồm thông tin tài khoản người dùng, danh mục

và sản phẩm, trạng thái đơn hàng cùng lịch sử giao dịch. Bên cạnh đó, cơ sở dữ liệu còn lưu trữ dữ liệu tương tác của khách hàng như nội dung đánh giá và số sao dành cho từng sản phẩm [13].

Nguồn dữ liệu trong MySQL đóng vai trò quan trọng đối với toàn bộ hệ thống và được sử dụng làm đầu vào cho các thành phần xử lý phía Backend. Dữ liệu sau đó được khai thác để phục vụ các dịch vụ trong hệ sinh thái trí tuệ nhân tạo của nền tảng. Đối với hệ thống gợi ý sản phẩm, thông tin đánh giá được sử dụng để xây dựng ma trận tương tác người dùng – sản phẩm, hỗ trợ quá trình huấn luyện mô hình, tính toán độ tương đồng và phát triển thuật toán gợi ý, qua đó cải thiện độ chính xác của các đề xuất sản phẩm.

2.6.5. Docker

Docker là một nền tảng container hóa phổ biến, cho phép đóng gói ứng dụng cùng với các thư viện, môi trường thực thi và cấu hình cần thiết vào những container độc lập. Cơ chế này giúp ứng dụng có thể được phân phối và vận hành nhất quán trên nhiều môi trường triển khai khác nhau[15].



Hình 2.11 : Mô hình kiến trúc và các thành phần của Docker

(Nguồn: Techso, 2018. https://techso.ca/en/docker-introduction/)

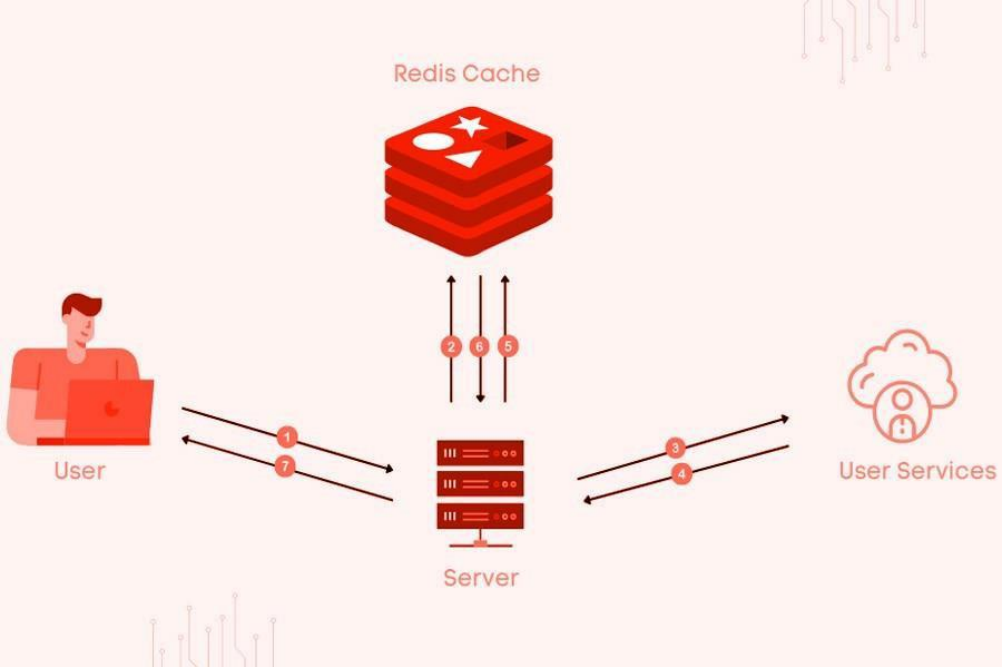
Tính nhất quán: Docker giúp tách biệt ứng dụng khỏi hạ tầng bên dưới, qua đó bảo đảm môi trường vận hành đồng nhất giữa các giai đoạn phát triển, kiểm thử và triển khai thực tế.

Hiệu quả vận hành: Các container hoạt động ở cấp độ hệ điều hành, giúp hạn chế xung đột giữa các phiên bản thư viện, rút ngắn thời gian khởi tạo và tối ưu hóa việc sử dụng tài nguyên máy chủ [15].

Trong tiểu luận này, các thành phần Spring Boot, FastAPI và React.js được đóng gói bằng Docker để hệ thống có thể triển khai ổn định trên nhiều môi trường mà không cần cấu hình lại từng dịch vụ. Mỗi thành phần được vận hành trong một container riêng, giúp giảm thiểu xung đột giữa các thư viện và môi trường thực thi của Java, Python và JavaScript. Đồng thời, Docker Compose được sử dụng để khai báo, kết nối và quản lý tập trung các container, qua đó giúp quá trình triển khai toàn bộ hệ thống Microservices trở nên thống nhất và thuận tiện hơn [15].

2.6.6. Redis

Redis (Remote Dictionary Server) là một hệ quản trị cơ sở dữ liệu NoSQL mã nguồn mở, được thiết kế theo mô hình khóa – giá trị (key-value) và hoạt động chủ yếu trên bộ nhớ trong (in-memory). Do dữ liệu được lưu trực tiếp trên RAM, Redis có khả năng truy xuất với độ trễ thấp, qua đó giúp cải thiện hiệu năng tổng thể của hệ thống trong các tình huống yêu cầu phản hồi nhanh [14].



Hình 2.12 : Minh họa cơ chế hoạt động của Redis Cache trong hệ thống

(Nguồn: Unicode Academy, 2025.

https://online.unicode.vn/bai-viet/kham-pha-redis-co-so-du-lieu-nhanh-nhu-chop-va-linh-hoat)

Các ứng dụng phổ biến của Redis trong hệ thống bao gồm:

Lưu bộ nhớ đệm dữ liệu: Redis lưu trữ các dữ liệu được truy xuất thường xuyên nhằm giảm số lần truy vấn xuống cơ sở dữ liệu chính và cải thiện tốc độ phản hồi của ứng dụng web.

Lưu trữ phiên người dùng: Redis hỗ trợ lưu thông tin phiên làm việc, giúp hệ thống theo dõi trạng thái đăng nhập và các hoạt động của người dùng trong quá trình sử dụng.

Nhắn tin thời gian thực: Cơ chế publish/subscribe(pub/sub) cho phép các thành phần trong hệ thống trao đổi dữ liệu gần như tức thời khi có sự kiện phát sinh [14].

Phân tích dữ liệu thời gian thực: Redis có thể ghi nhận và xử lý các dữ liệu phát sinh liên tục, phục vụ nhu cầu thống kê, xây dựng bộ đếm và giám sát theo thời gian thực.

Trong tiểu luận này, Redis đóng vai trò là lớp lưu trữ tốc độ cao hỗ trợ ứng dụng và cơ sở dữ liệu MySQL. Redis được sử dụng để quản lý Refresh Token trong dịch vụ xác thực, đồng thời lưu trữ các số liệu thống kê và bộ đếm phục vụ bảng điều khiển thời gian thực của dịch vụ thống kê. Cách tiếp cận này giúp giảm tải cho cơ sở dữ liệu chính và cải thiện khả năng đáp ứng khi hệ thống tiếp nhận nhiều yêu cầu đồng thời.

Việc sử dụng Redis theo hướng này giúp hệ thống xử lý hiệu quả hơn các tác vụ có tần suất đọc cao, giảm độ trễ trong quá trình truy xuất dữ liệu và duy trì sự ổn định khi lưu lượng người dùng tăng lên [14].

2.6.7. Eureka Service Discovery

Trong kiến trúc Microservices, mỗi dịch vụ có thể được triển khai dưới dạng nhiều instance và thay đổi liên tục về địa chỉ IP hoặc cổng khi hệ thống mở rộng hay thu hẹp. Vì vậy, việc cấu hình tĩnh địa chỉ của từng dịch vụ không còn phù hợp và có thể làm tăng độ phức tạp trong quá trình triển khai.

Để giải quyết vấn đề này, hệ thống sử dụng Eureka thuộc Spring Cloud Netflix như một cơ chế khám phá dịch vụ (Service Discovery). Eureka đóng vai trò như một "danh bạ trung tâm", cho phép các microservice tự động đăng ký thông tin và tra cứu vị trí của nhau trong hệ thống phân tán, qua đó hỗ trợ giao tiếp linh hoạt mà không phụ thuộc vào địa chỉ IP cố định [9].



Hình 2.13 : Logo của Netflix Eureka Service Discovery

Nguồn:
https://miro.medium.com/v2/resize:fit:1004/1*eHBmngvJz7eumU5jxzso1w.png

Eureka Server được triển khai như một thành phần trung tâm với các cơ chế chính sau:

Đăng ký dịch vụ: Khi các microservice như User, Product và Order khởi động, chúng tự động gửi thông tin tên dịch vụ, địa chỉ IP và cổng hoạt động đến Eureka Server.

Khám phá dịch vụ: Các microservice hoặc API Gateway truyvấn Eureka Server để lấy danh sách những instance đang hoạt động và lựa chọn địa chỉ phù hợp khi định tuyến yêu cầu.

Cập nhật trạng thái: Mỗi dịch vụ gửi tín hiệu heartbeat theo định kỳ để xác nhận rằng instance vẫn đang vận hành bình thường.

Loại bỏ dịch vụ không khả dụng: Nếu một instance không phản hồi trong khoảng thời gian quy định, Eureka sẽ loại bỏ instance đó khỏi registry nhằm duy trì tính chính xác của thông tin định tuyến [9].

Trong tiểu luận này, Eureka Server được sử dụng như thành phần trung tâm để quản lý và điều phối các microservice. Các dịch vụ có thể tự động đăng ký khi khởi động mà không cần cấu hình địa chỉ thủ công, giúp giảm sai sót và đơn giản hóa quá trình triển khai.

Các dịch vụ giao tiếp với nhau thông qua tên dịch vụ và cơ chế khám phá dịch vụ thay vì phụ thuộc vào IP tĩnh. Đồng thời, Eureka hỗ trợ khả năng mở rộng động, cho phép bổ sung hoặc loại bỏ các instance mà không làm gián đoạn toàn bộ hệ thống, qua đó góp phần duy trì hiệu năng và độ sẵn sàng khi lưu lượng truy cập tăng cao.

2.6.8. JWT – JSON Web Token

JSON Web Token (JWT) là một chuẩn mở được quy định trong RFC 7519, thường được sử dụng để hỗ trợ cơ chế xác thực (Authentication) và phân quyền (Authorization) giữa các bên thông qua một chuỗi dữ liệu có cấu trúc JSON. Trong kiến trúc Microservices của sàn thương mại điện tử, JWT được sử dụng làm cơ chế xác thực chính nhằm quản lý phiên đăng nhập và định danh người dùng theo mô hình phi trạng thái (stateless), đồng thời hỗ trợ kiểm soát quyền truy cập giữa các dịch vụ [19].

JWT bao gồm ba thành phần chính là Header, Payload và Signature:

Header: Chứa loại token, thường là JWT, và thuật toán được sử dụng để tạo chữ ký, chẳng hạn như HMAC SHA-256 hoặc RSA.

Payload: Chứa các thông tin (claims) liên quan đến người dùng như user_id, email và role. Trong đó, role đóng vai trò quan trọng trong cơ chế phân quyền, giúp các dịch vụ xác định quyền truy cập của người dùng mà không cần truy vấn lại cơ sở dữ liệu ở mỗi yêu cầu.

Signature: Được tạo từ Header, Payload và khóa ký của máy chủ. Thành phần này cho phép hệ thống kiểm tra tính toàn vẹn và nguồn gốc của token, qua đó phát hiện dữ liệu bị thay đổi hoặc giả mạo trong quá trình truyền tải.

JWT có nhiều đặc điểm phù hợp với kiến trúc Microservices và mô hình giao tiếp RESTful:

Phù hợp với RESTful API: JWT có thể được truyền trong HTTP header dưới dạng Bearer Token ở các yêu cầu từ client React.js, giúp quá trình xác thực được thực hiện thống nhất trên các API.

Khả năng mở rộng: Nhờ đặc tính phi trạng thái, máy chủ không phải lưu trữ session cho từng người dùng. API Gateway và các microservice vì vậy có thể mở rộng theo chiều ngang thuận lợi hơn khi số lượng yêu cầu đồng thời tăng cao.

Hỗ trợ phân quyền linh hoạt: Thông tin vai trò trong JWT cho phép API Gateway hoặc từng microservice kiểm soát quyền truy cập đối với các chức năng như quản lý đơn hàng, giỏ hàng và khu vực quản trị (Admin Dashboard).

+ Bảo đảm tính toàn vẹn: Cơ chế chữ ký giúp máy chủ xác minh token chưa bị chỉnh sửa sau khi được phát hành. Tuy nhiên, Payload của JWT không được mã hóa mặc định, vì vậy không nên lưu trữ trực tiếp các thông tin nhạy cảm trong thành phần này [19].

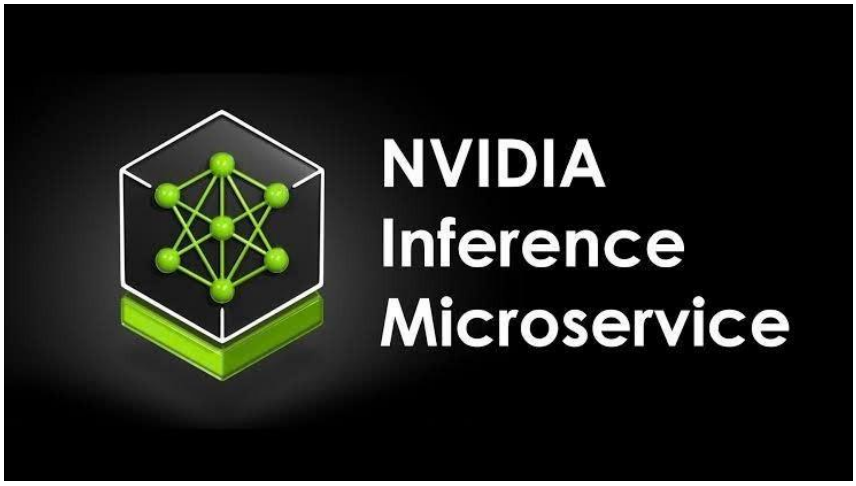
Trong tiểu luận này, JWT được khởi tạo tại dịch vụ xác thực (Authentication Service) sau khi người dùng đăng nhập thành công bằng thông tin tài khoản và mật khẩu hợp lệ. Sau khi được phát hành, token được client lưu trữ và gửi kèm trong các yêu cầu tiếp theo. API Gateway tiếp nhận token, kiểm tra chữ ký, thời hạn hiệu lực và thông tin phân quyền trước khi quyết định cho phép hoặc từ chối chuyển tiếp yêu cầu đến các dịch vụ nghiệp vụ như Order Service hoặc Cart Service.

Thông tin vai trò người dùng được lưu trong Payload của JWT và được sử dụng để xác định phạm vi chức năng mà từng tài khoản được phép truy cập. Cơ chế này đặc biệt cần thiết đối với những chức năng yêu cầu quyền cao như khu vực quản trị. Qua đó, hệ thống có thể kiểm soát truy cập chặt chẽ và áp dụng chính sách phân quyền nhất quán trong toàn bộ kiến trúc Microservices.

2.6.9. Tích hợp Trí tuệ nhân tạo với NVIDIA NIM

2.6.9.1. Giới thiệu

Trong hệ thống, mô hình Llama 3.1 được truy cập thông qua NVIDIA NIM để thực hiện các tác vụ xử lý ngôn ngữ tự nhiên. Mô hình tiếp nhận nội dung hội thoại và ngữ cảnh liên quan, sau đó sinh phản hồi phù hợp với yêu cầu của người dùng. Việc giao tiếp qua API cho phép dịch vụ AI được cập nhật hoặc thay thế độc lập mà không làm thay đổi đáng kể các thành phần Frontend và Backend đang sử dụng kết quả [20].



Hình 2.14 : Minh họa nền tảng NVIDIA NIM và mô hình Llama 3.1

Nguồn: https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcSmpinPVwmvU363lSnC6ipC5fJTdblyaBzgK2FTKsDpGgne7KlAHF_1Vrjz&s=10

2.6.9.2. Ứng dụng Chatbot hỗ trợ khách hàng

Trong hệ thống, chatbot được xây dựng như một công cụ hỗ trợ người dùng tìm kiếm và tra cứu sản phẩm theo phương thức hội thoại tự nhiên. Khác với cơ chế tìm kiếm truyền thống dựa trên các từ khóa cố định, chatbot có khả năng phân tích ý định

và ngữ nghĩa trong câu hỏi, từ đó xác định đồng thời nhiều tiêu chí mà người dùng quan tâm.

Ví dụ, khi người dùng nhập yêu cầu "tìm điện thoại dưới 5 triệu chụp ảnh tốt", hệ thống phân tích đồng thời giới hạn về mức giá và nhu cầu sử dụng camera để truy xuất danh sách sản phẩm phù hợp. Bên cạnh chức năng tư vấn sản phẩm, chatbot còn có thể giải đáp các câu hỏi liên quan đến chính sách giao hàng, đổi trả và bảo hành. Cách tiếp cận này giúp người dùng tiếp cận thông tin nhanh hơn, giảm khối lượng yêu cầu lặp lại cho bộ phận hỗ trợ khách hàng và cải thiện trải nghiệm sử dụng hệ thống.

2.6.9.3. Ứng dụng vào đồ án

Trong đồ án này, nền tảng NVIDIA NIM kết hợp với mô hình Llama 3.1 được sử dụng để bổ sung khả năng xử lý ngôn ngữ tự nhiên cho hệ thống thương mại điện tử, chủ yếu phục vụ cho chức năng chatbot hỗ trợ người dùng.

Chatbot được thiết kế để người dùng có thể đặt câu hỏi liên quan đến sản phẩm, so sánh thông số kỹ thuật hoặc tìm kiếm sản phẩm theo nhu cầu một cách tự nhiên. Thay vì chỉ dựa vào khả năng của mô hình ngôn ngữ, hệ thống sử dụng kiến trúc RAG (Retrieval-Augmented Generation), trong đó thông tin sản phẩm được truy xuất từ cơ sở dữ liệu vector (Pinecone) trước khi đưa vào mô hình để tạo câu trả lời. Cách làm này giúp nội dung phản hồi bám sát dữ liệu thực tế của hệ thống, hạn chế tình trạng trả lời chung chung hoặc không chính xác.

Dữ liệu sản phẩm trong hệ thống được chuyển đổi thành vector embedding và lưu trữ trong vector database để phục vụ cho việc truy vấn theo ngữ nghĩa. Nhờ đó, chatbot có thể hiểu được các câu hỏi mang tính tự nhiên của người dùng và gợi ý những sản phẩm liên quan thay vì chỉ dựa trên từ khóa.

Việc tích hợp mô hình Llama 3.1 thông qua NVIDIA NIM giúp hệ thống tận dụng được khả năng sinh ngôn ngữ tự nhiên của mô hình, đồng thời vẫn đảm bảo tính chính xác nhờ cơ chế truy xuất dữ liệu từ vector store. Sự kết hợp này giúp cải thiện trải nghiệm tìm kiếm và tư vấn sản phẩm, đồng thời làm cho việc tương tác giữa người dùng và hệ thống trở nên linh hoạt và tự nhiên hơn.

2.6.10. Cổng thanh toán VNPay

2.6.10.1. Giới thiệu

VNPay là một trong những đơn vị cung cấp giải pháp thanh toán điện tử phổ biến tại Việt Nam, hỗ trợ kết nối với nhiều ngân hàng và phương thức thanh toán trong nước. Nhờ phạm vi tích hợp rộng, VNPay được ứng dụng trong nhiều hệ thống thương mại điện tử và dịch vụ trực tuyến, giúp người dùng thực hiện giao dịch thuận tiện trên một nền tảng thanh toán thống nhất [21].

2.6.10.2. Ứng dụng thực tiễn

Việc tích hợp VNPay vào đề tài giúp mô phỏng quy trình thanh toán tương tự các hệ thống thương mại điện tử đang vận hành thực tế tại Việt Nam. Thông qua cổng thanh toán trung gian, hệ thống có thể cung cấp trải nghiệm giao dịch trực tuyến thuận tiện mà không phải tự xây dựng kết nối riêng biệt với từng ngân hàng.

Cụ thể, người dùng có thể lựa chọn các phương thức thanh toán quen thuộc như quét mã VNPAY-QR hoặc sử dụng thẻ ngân hàng nội địa. Bên cạnh đó, VNPay hỗ trợ cơ chế xác thực và kiểm tra kết quả giao dịch nhằm tăng tính an toàn trong quá trình thanh toán, qua đó góp phần giảm thiểu rủi ro phát sinh trong các giao dịch trực tuyến [21].

2.6.10.3. Ứng dụng vào đồ án

Trong đồ án, VNPay được tích hợp vào luồng thanh toán của hệ thống với vai trò cổng thanh toán trung gian. Khi người dùng lựa chọn thanh toán trực tuyến, hệ thống tạo yêu cầu giao dịch và chuyển hướng người dùng sang VNPay để lựa chọn phương thức, thực hiện xác thực và hoàn tất quá trình thanh toán.

Sau khi giao dịch được xử lý, VNPay gửi kết quả về hệ thống thông qua các địa chỉ nhận phản hồi đã cấu hình. Backend kiểm tra tính hợp lệ của dữ liệu trả về, đối chiếu mã giao dịch và cập nhật trạng thái đơn hàng từ “Chờ thanh toán” sang “Đã thanh toán” khi giao dịch thành công. Cơ chế này giúp dữ liệu đơn hàng được đồng bộ và phản ánh đúng trạng thái thực tế của giao dịch [21].

2.6.11. Dịch vụ vận chuyển GHN (Giao Hàng Nhanh)

2.6.11.1. Giới thiệu

GHN (Giao Hàng Nhanh) là một đơn vị cung cấp dịch vụ vận chuyển hàng hóa tại Việt Nam, đồng thời hỗ trợ API cho phép các hệ thống bên thứ ba tích hợp các chức năng liên quan đến giao nhận. Thông qua API này, các nền tảng thương mại điện tử có thể thực hiện việc tính phí vận chuyển, tạo đơn hàng và theo dõi trạng thái giao hàng một cách tự động [22].

Hiện nay, GHN là một trong những đơn vị vận chuyển có quy mô lớn tại Việt Nam, với mạng lưới hoạt động trải rộng trên nhiều tỉnh thành, đáp ứng nhu cầu giao nhận hàng hóa trong lĩnh vực thương mại điện tử.

2.6.11.2. Ứng dụng trong quản lý Logistic

Trong tiểu luận này, GHN API được sử dụng để hỗ trợ tự động hóa một số quy trình trong quản lý logistics, đặc biệt là các chức năng liên quan đến tính phí và theo dõi đơn hàng.

Tính phí vận chuyển: Hệ thống sử dụng API của GHN để xác định chi phí giao hàng dựa trên thông tin đơn hàng như địa chỉ nhận và khối lượng sản phẩm. Việc này giúp bảo đảm chi phí được tính toán chính xác và nhất quán.

Cập nhật trạng thái đơn hàng: Trạng thái vận chuyển được đồng bộ từ GHN về hệ thống thông qua API hoặc cơ chế webhook. Nhờ đó, người dùng có thể theo dõi quá trình giao hàng theo từng giai đoạn như tiếp nhận, vận chuyển và hoàn tất giao hàng [22].

2.6.11.3. Ứng dụng vào đồ án

Trong đồ án này, GHN API được tích hợp nhằm hỗ trợ quá trình xử lý đơn hàng và quản lý vận chuyển. Hệ thống kết nối trực tiếp với dịch vụ của GHN để thực hiện việc tính phí giao hàng theo thời gian thực, dựa trên địa chỉ người nhận và thông tin sản phẩm.

Việc tích hợp này giúp giảm thiểu các thao tác thủ công trong quá trình xử lý đơn hàng, đồng thời nâng cao độ chính xác của thông tin phí vận chuyển. Ngoài ra,

người dùng có thể theo dõi chi tiết trạng thái đơn hàng ngay trong hệ thống, từ đó cải thiện trải nghiệm sử dụng và tăng tính minh bạch trong quá trình giao nhận hàng hóa [22].

2.6.12. Material-UI (MUI)

2.6.12.1. Tổng quan

Material-UI (MUI) là một thư viện component dành cho React, được xây dựng dựa trên nguyên tắc thiết kế Material Design của Google. Thư viện này cung cấp sẵn nhiều thành phần giao diện người dùng như nút bấm, biểu mẫu, hộp thoại và bảng dữ liệu, giúp việc xây dựng giao diện trở nên nhanh hơn và có tính thống nhất cao [17].

MUI được phát triển theo hướng mã nguồn mở và hiện được sử dụng rộng rãi trong cộng đồng React. Nhờ hệ sinh thái ổn định và tài liệu đầy đủ, thư viện này thường được lựa chọn trong các dự án thực tế có yêu cầu về giao diện hiện đại và khả năng mở rộng.

2.6.12.2. Vai trò trong phát triển Frontend

Việc sử dụng MUI mang lại nhiều lợi ích trong quá trình phát triển giao diện người dùng. Trước hết, thư viện giúp giảm đáng kể thời gian xây dựng giao diện nhờ hệ thống component có sẵn. Các thành phần như Button, Modal, Dialog và TextField có thể được sử dụng trực tiếp và tùy chỉnh theo yêu cầu mà không cần xây dựng lại từ đầu.

Bên cạnh đó, MUI giúp bảo đảm tính nhất quán trong thiết kế giao diện. Các component được xây dựng theo cùng một hệ quy chuẩn nên giao diện tổng thể có sự đồng bộ về màu sắc, khoảng cách, kiểu chữ và bố cục. Khả năng responsive được hỗ trợ thông qua hệ thống breakpoint và bố cục linh hoạt, giúp giao diện hiển thị phù hợp trên nhiều loại thiết bị mà không cần xử lý quá nhiều bằng CSS thủ công [17].

Trong các hệ thống quản trị, MUI hỗ trợ hiệu quả việc trình bày và thao tác dữ liệu thông qua các component như Table, Pagination, FormControl và Dialog. Những thành phần này phù hợp với các chức năng quản lý sản phẩm, đơn hàng, người dùng và các biểu mẫu cập nhật dữ liệu trong hệ thống thương mại điện tử.

2.6.12.3. Ứng dụng vào đồ án

Trong tiểu luận này, MUI được sử dụng kết hợp với React.js để xây dựng giao diện người dùng và giao diện quản trị. Việc tận dụng các component có sẵn giúp rút ngắn thời gian phát triển, giảm khối lượng mã CSS thủ công và duy trì sự đồng nhất giữa các trang chức năng của hệ thống.

Các thành phần của MUI được áp dụng trong biểu mẫu đăng nhập, quản lý sản phẩm, quản lý đơn hàng, hộp thoại xác nhận, bảng dữ liệu và cơ chế phân trang. Đồng thời, hệ thống breakpoint và bố cục responsive giúp giao diện hiển thị phù hợp trên nhiều thiết bị, từ máy tính để bàn đến điện thoại di động, qua đó cải thiện trải nghiệm sử dụng cho cả khách hàng và người quản trị [17].

2.6.13. Các mô hình AI tích hợp trong ứng dụng

Trong phạm vi tiểu luận, ứng dụng tích hợp nhiều mô hình trí tuệ nhân tạo để hỗ trợ các nhóm chức năng khác nhau, bao gồm xử lý ngôn ngữ tự nhiên, tìm kiếm theo ngữ nghĩa và gợi ý sản phẩm cá nhân hóa. Các mô hình được triển khai tách biệt khỏi các dịch vụ nghiệp vụ chính, giúp hệ thống dễ dàng cập nhật, mở rộng và thay thế từng thành phần khi cần thiết.

Các mô hình AI chính được sử dụng trong ứng dụng gồm:

Mô hình Llama-3.1-8B-Instruct: Được truy cập thông qua nền tảng NVIDIA NIM để xử lý các yêu cầu ngôn ngữ tự nhiên. Mô hình hỗ trợ chatbot tư vấn sản phẩm, giải đáp câu hỏi của khách hàng và tự động tạo nội dung mô tả sản phẩm cho người bán [20].

Mô hình embedding nv-embed-v1: Chuyển đổi nội dung văn bản thành vector ngữ nghĩa để lưu trữ và truy vấn trên Pinecone. Mô hình giúp hệ thống tìm kiếm dữ liệu liên quan ngay cả khi câu hỏi của người dùng không trùng khớp hoàn toàn với từ khóa trong thông tin sản phẩm.

Mô hình lọc cộng tác: Phân tích dữ liệu đánh giá và lịch sử tương tác giữa người dùng với sản phẩm để dự đoán mức độ quan tâm. Mô hình được xây dựng bằng thư viện Surprise và sử dụng kết quả dự đoán để tạo danh sách sản phẩm gợi ý phù hợp với từng người dùng [8].

2.6.13.1. **T**ổng quan mô hình gợi ý sử dụng

Hệ thống gợi ý trong ứng dụng được xây dựng dựa trên phương pháp lọc cộng tác theo hướng dựa trên sản phẩm (Item-based Collaborative Filtering), kết hợp với thuật toán K-Nearest Neighbors (KNN) để tìm ra các sản phẩm có mức độ tương đồng cao. Cách tiếp cận này tận dụng dữ liệu từ lịch sử đánh giá và hành vi mua hàng của người dùng để suy ra mối quan hệ giữa các sản phẩm trong hệ thống [6], [8].

Trong quá trình hoạt động, hệ thống không tập trung vào việc phân tích từng người dùng riêng lẻ mà chủ yếu quan sát cách các sản phẩm được đánh giá tương tự nhau bởi nhiều người dùng khác nhau. Từ đó, những sản phẩm có xu hướng được đánh giá giống nhau sẽ được xem là có liên quan và có khả năng được gợi ý cho người dùng trong các lần truy cập tiếp theo.

Thuật toán KNN được sử dụng như một bước hỗ trợ để xác định nhóm các sản phẩm gần nhau trong không gian đặc trưng. Thay vì phải so sánh toàn bộ tập dữ liệu, mô hình chỉ tập trung vào một số lượng giới hạn các sản phẩm có mức độ tương đồng cao nhất, giúp giảm chi phí tính toán và cải thiện tốc độ phản hồi của hệ thống [8].

Để nâng cao độ chính xác của phép đo tương đồng, hệ thống áp dụng biến thể KNearest Neighbors với Pearson Baseline. Cụ thể, hệ số tương quan Pearson được hiệu chỉnh bằng cách trừ đi giá trị ước lượng Baseline (Baseline Estimate), nhằm loại bỏ các sai lệch do đặc điểm riêng của người dùng hoặc sản phẩm. Cách tiếp cận này giúp giảm thiểu hiện tượng thiên vị trong đánh giá, chẳng hạn như người dùng có xu hướng chấm điểm quá khắt khe hoặc quá dễ dãi, cũng như sự ảnh hưởng của các sản phẩm quá phổ biến hoặc ít được quan tâm [8].

Toàn bộ quá trình gợi ý được triển khai theo hướng tuần tự, bắt đầu từ việc xây dựng ma trận tương đồng giữa các sản phẩm, sau đó sử dụng kết quả này để ước lượng mức độ quan tâm của người dùng đối với từng sản phẩm chưa tương tác. Cuối cùng, hệ thống sắp xếp lại danh sách kết quả để đưa ra các gợi ý phù hợp nhất [6],[8].

2.6.13.2. Lọc cộng tác dựa trên sản phẩm với KNN

Phương pháp Item-based Collaborative Filtering kết hợp độ đo Pearson Baseline không chỉ đơn thuần tính toán góc giữa các vector tương tác, mà tập trung đo lường xu

hướng biến thiên đồng điệu giữa các sản phẩm sau khi đã loại bỏ ước lượng điểm nền (bias) của toàn hệ thống $b_{u,i}$ (gồm trung bình toàn cục, bias người dùng và bias sản phẩm). Cơ chế này giúp kết quả tương đồng phản ánh rõ hơn mối quan hệ thực tế giữa các sản phẩm thay vì bị chi phối bởi xu hướng chấm điểm của từng người dùng [8].

Cấu trúc mô hình gồm:

+ Ma trận tương tác Người dùng - Sản phẩm (User-Item Interaction Matrix).

+ Hệ thống ước lượng điểm nền (Baseline Estimates) cho từng người dùng và từng sản phẩm.

+ Hàm tính toán độ tương đồng Pearson Baseline giữa các cặp sản phẩm.

+ Hàm dự đoán điểm số có hiệu chỉnh sai số thiên vị (Bias Correction).

Ước lượng điểm nền (Baseline Estimate): Điểm nền bu,i thể hiện mức điểm dự đoán mặc định cho người dùng u đối với sản phẩm i khi chưa xét đến yếu tố tương đồng, được xác định bằng công thức:

$$b_{u,i} = \mu + b_u + b_i$$

Trong đó, μ là điểm đánh giá trung bình của toàn hệ thống; bu là độ lệch đánh giá của người dùng u; và bi là độ lệch đánh giá của sản phẩm i.

Độ đo tương đồng Pearson Baseline: Độ tương đồng giữa hai sản phẩm i và j được tính dựa trên tập hợp người dùng Ui,j, gồm những người đã tương tác với cả hai sản phẩm, theo công thức:

$$\text{sim}(i,j) = \frac{\sum_{u \in U_{i,j}} (r_{u,i} - b_{u,i})(r_{u,j} - b_{u,j})}{\sqrt{\sum_{u \in U_{i,j}} (r_{u,i} - b_{u,i})^2} \sqrt{\sum_{u \in U_{i,j}} (r_{u,j} - b_{u,j})^2}}$$

Trong đó, $r_{u,i}$ và $r_{u,j}$ lần lượt là điểm tương tác hoặc mức độ yêu thích thực tế của người dùng u đối với sản phẩm i và sản phẩm j.

Hàm dự đoán điểm số: Mức độ quan tâm dự kiến của người dùng u đối với sản phẩm i được tính bằng điểm nền $b_{u,i}$ cộng với trung bình có trọng số của phần sai lệch từ tập K láng giềng gần nhất, ký hiệu là $N_{u(i)}$:

$$\hat{r}_{u,i} = b_{u,i} + \frac{\sum_{j \in N_u(i)} \text{sim}(i,j) \cdot (r_{u,j} - b_{u,j})}{\sum_{j \in N_u(i)} |\text{sim}(i,j)|}$$

Trong đó, $N_{u(i)}$ là tập K sản phẩm láng giềng của sản phẩm i (theo độ tương đồng $sim(i,j)$) mà người dùng u đã đánh giá.Các công thức trên thể hiện cơ chế của KNN Baseline, trong đó điểm dự đoán được hiệu chỉnh dựa trên điểm nền và mức độ tương đồng giữa các sản phẩm láng giềng [8].

Vai trò ứng dụng: Trong tiểu luận này, thuật toán Item-based Collaborative Filtering kết hợp KNN và Pearson Baseline đóng vai trò cốt lõi trong việc sinh danh sách gợi ý cá nhân hóa. Nhờ cơ chế hiệu chỉnh sai số, mô hình góp phần nâng cao độ chính xác và giảm ảnh hưởng từ dữ liệu nhiễu hoặc thói quen chấm điểm cực đoan của người dùng trên sàn thương mại điện tử.

CHƯƠNG 3. GIẢI PHÁP CHO BÀI TOÁN

3.1. Phát biểu bài toán trong đề tài

Từ khảo sát các hệ thống thương mại điện tử bán lẻ thiết bị điện tử hiện nay, có thể nhận thấy người dùng thường gặp tình trạng quá tải thông tin do số lượng sản phẩm có cấu hình và mức giá tương đồng ngày càng lớn, khiến việc tìm kiếm sản phẩm phù hợp trở nên mất thời gian và phụ thuộc nhiều vào thao tác lọc thủ công.

Phần lớn nền tảng hiện nay vẫn hiển thị sản phẩm theo danh mục cố định hoặc mức độ phổ biến chung, chưa khai thác hiệu quả dữ liệu hành vi để cá nhân hóa trải nghiệm mua sắm. Nhiều hệ thống cũng được xây dựng theo kiến trúc nguyên khối, gây khó khăn khi mở rộng hoặc tích hợp thêm các thành phần thông minh như dịch vụ gợi ý sản phẩm khi lượng dữ liệu và người dùng tăng lên.

Một vấn đề khác liên quan đến chăm sóc khách hàng: nhiều hệ thống vẫn phụ thuộc chủ yếu vào nhân sự điều hành/chăm sóc khách hàng trực tuyến hoặc tổng đài trong giờ hành chính, khó đáp ứng nhu cầu hỗ trợ liên tục. Các yêu cầu đơn giản như tra cứu đơn hàng hay tư vấn sản phẩm cơ bản vẫn phải xử lý thủ công, gây lãng phí nguồn lực vận hành.

Từ thực trạng trên, đề tài xác định ba nhóm vấn đề cần giải quyết: xây dựng một hệ thống có khả năng mở rộng linh hoạt khi dữ liệu và người dùng tăng trưởng; nâng cao trải nghiệm mua sắm thông qua cơ chế gợi ý sản phẩm cá nhân hóa; và cải thiện hiệu quả chăm sóc khách hàng thông qua tự động hóa một phần quy trình hỗ trợ.

Xuất phát từ đó, đề tài xây dựng hệ thống thương mại điện tử Electro Store theo mô hình B2C (Business-to-Consumer), trong đó hệ thống được vận hành bởi một đơn vị bán lẻ duy nhất, phục vụ trực tiếp người tiêu dùng cuối. Hệ thống được thiết kế theo kiến trúc Microservices nhằm tách biệt các nghiệp vụ và cho phép triển khai độc lập từng thành phần.

Trên nền tảng này, hệ thống tích hợp dịch vụ gợi ý sản phẩm để cá nhân hóa trải nghiệm mua sắm, đồng thời tích hợp thêm dịch vụ chatbot AI nhằm hỗ trợ khách hàng trong việc tư vấn sản phẩm và giải đáp các thắc mắc cơ bản. Hai thành phần này được

thiết kế như các dịch vụ độc lập, có khả năng mở rộng và tích hợp linh hoạt trong toàn bộ hệ thống.

Hệ thống phục vụ hai nhóm đối tượng chính trong mô hình B2C. Khách hàng là người dùng cuối có nhu cầu tìm kiếm, xem thông tin và mua sản phẩm thiết bị điện tử; hệ thống hỗ trợ nhóm này thông qua cơ chế gợi ý sản phẩm cá nhân hóa cùng dịch vụ chatbot hỗ trợ tư vấn. Đội ngũ vận hành hệ thống chịu trách nhiệm quản lý sản phẩm, đơn hàng, người dùng và giám sát hoạt động, được hỗ trợ bởi các công cụ thống kê và phân tích dữ liệu.

Để đáp ứng các yêu cầu trên, hệ thống được xây dựng với các nhóm chức năng chính sau:

+ **Dịch vụ quản lý người dùng và phân quyền:** Cho phép đăng ký, đăng nhập và xác thực tài khoản, đồng thời phân quyền truy cập theo mô hình RBAC nhằm phân biệt rõ giữa khách hàng và quản trị viên.

+ **Dịch vụ quản lý sản phẩm và mua sắm trực tuyến:** Hỗ trợ duyệt danh mục, tìm kiếm sản phẩm theo nhiều tiêu chí, quản lý giỏ hàng, đặt hàng và theo dõi trạng thái đơn hàng trong suốt quá trình giao dịch.

+ **Dịch vụ đánh giá sản phẩm:** Ghi nhận các hành vi đánh giá sản phẩm của người dùng sau khi mua hoặc trải nghiệm sản phẩm, làm cơ sở dữ liệu phục vụ cho việc phân tích hành vi người dùng.

+ **Dịch vụ thống kê hệ thống:** Cung cấp các công cụ theo dõi, tổng hợp và phân tích dữ liệu hoạt động của hệ thống, phục vụ cho việc giám sát và ra quyết định vận hành.

+ **Dịch vụ gợi ý sản phẩm:** Sinh danh sách Top-k sản phẩm được cá nhân hóa cho từng người dùng, đồng thời xử lý các trường hợp người dùng mới hoặc thiếu dữ liệu tương tác bằng các chiến lược phù hợp.

+ **Dịch vụ chatbot:** Hỗ trợ khách hàng tra cứu thông tin đơn hàng và tư vấn sản phẩm cơ bản một cách tự động, giảm bớt sự phụ thuộc vào nhân sự chăm sóc khách hàng trực tiếp.

+ **Kiến trúc Microservices:** Toàn bộ hệ thống được tổ chức thành các dịch vụ độc lập bao gồm: Dịch vụ xác thực, Dịch vụ người dùng, Dịch vụ sản phẩm, Dịch vụ giỏ hàng, Dịch vụ đơn hàng, Dịch vụ đánh giá, Dịch vụ thống kê, Dịch vụ gợi ý sản phẩm và Dịch vụ chatbot, nhằm đảm bảo khả năng mở rộng, bảo trì và triển khai linh hoạt.

3.2. Giải pháp giải quyết cho bài toán

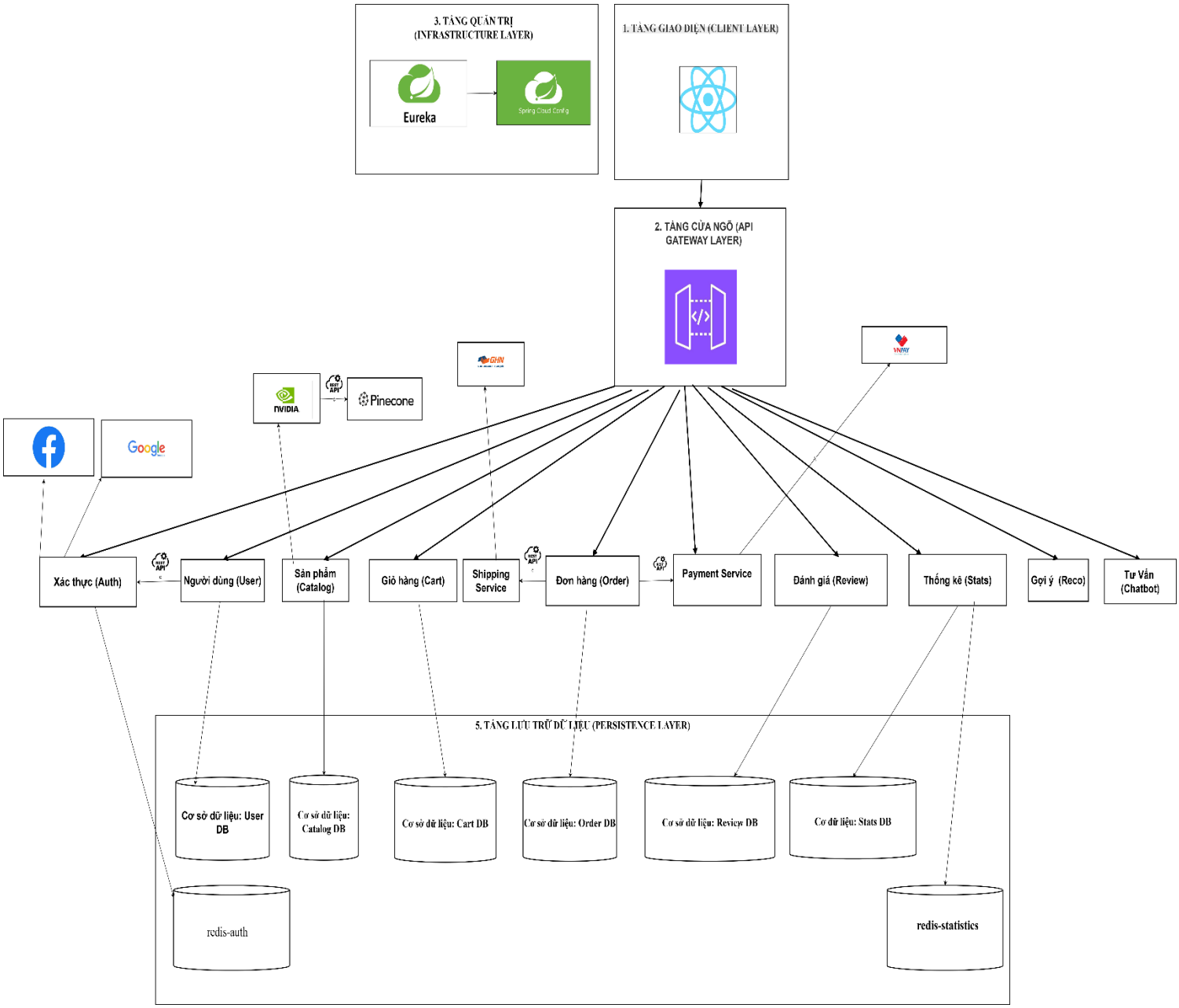
3.2.1. Kiến trúc tổng quan hệ thống

Giải pháp được thiết kế theo mô hình kiến trúc client–server, trong đó:

+ **Client:** Là tầng giao diện người dùng, được xây dựng bằng ReactJS, chịu trách nhiệm hiển thị thông tin và tiếp nhận thao tác của người dùng (duyệt sản phẩm, đặt hàng, quản trị…), đồng thời gửi yêu cầu tới phía máy chủ thông qua mạng.

+ **Server:** Là tầng xử lý nghiệp vụ và dữ liệu, tiếp nhận yêu cầu từ Client, thực hiện logic hệ thống và trả kết quả về dưới dạng API RESTful. Phía Server được tổ chức theo hướng Microservices nhằm tách biệt các chức năng, dễ mở rộng và bảo trì hơn so với kiến trúc nguyên khối.

3.2.2. Sơ đồ kiến trúc hệ thống



Hình 3.1 : Sơ đồ kiến trúc Microservices của hệ thống

Giải pháp được thiết kế theo mô hình kiến trúc client–server, triển khai phía máy chủ theo hướng Microservices. Hệ thống được tổ chức thành các nhóm thành phần chính như sau:

+ **Client (Frontend):** Ứng dụng web được xây dựng bằng ReactJS (Vite), cung cấp giao diện người dùng cho khách hàng thực hiện các thao tác mua sắm như xem sản

phẩm, tìm kiếm, đánh giá, quản lý giỏ hàng và đặt hàng. Đồng thời, hệ thống cũng cung cấp giao diện quản trị dành cho đội ngũ vận hành nhằm quản lý sản phẩm, đơn hàng và người dùng. Client chịu trách nhiệm tiếp nhận tương tác từ người dùng và gửi các yêu cầu HTTP đến Backend thông qua API Gateway.

+ **API Gateway:** Đóng vai trò là "cửa ngõ" duy nhất tiếp nhận toàn bộ request từ phía Client vào hệ thống. Thay vì Client phải giao tiếp trực tiếp với từng microservice riêng lẻ, API Gateway sẽ thực hiện định tuyến (routing) các yêu cầu đến đúng dịch vụ tương ứng như Product Service, Order Service, User Service,…Bên cạnh đó, thành phần này còn đảm nhiệm cân bằng tải (load balancing) giữa các instance dịch vụ, xác thực và kiểm tra token tập trung nhằm đảm bảo an toàn hệ thống, đồng thời áp dụng cơ chế giới hạn tần suất truy cập (rate limiting) để bảo vệ hệ thống trước các tình huống quá tải hoặc tấn công từ chối dịch vụ (DDoS).

+ **Server (tầng Microservices):** Logic nghiệp vụ được thiết kế theo kiến trúc Microservices, trong đó mỗi dịch vụ đảm nhiệm một miền nghiệp vụ riêng theo tư tưởng Domain-Driven Design (DDD). Các dịch vụ lõi bao gồm xác thực (Auth), người dùng (User), sản phẩm (Product), giỏ hàng (Cart), đơn hàng (Order), thanh toán (Payment), đánh giá (Review), thống kê (Analytics) và chatbot hỗ trợ người dùng, được xây dựng bằng Java Spring Boot nhằm đảm bảo tính ổn định, khả năng mở rộng và dễ bảo trì. Riêng dịch vụ gợi ý sản phẩm (Recommendation Service) được phát triển bằng Python (FastAPI) để thuận tiện tích hợp các thư viện học máy phục vụ bài toán cá nhân hóa.

Các dịch vụ trong hệ thống giao tiếp với nhau chủ yếu theo mô hình đồng bộ thông qua giao thức HTTP/REST, đảm bảo luồng xử lý rõ ràng và dễ kiểm soát trong các nghiệp vụ như đặt hàng, thanh toán và truy vấn dữ liệu. Bên cạnh đó, hệ thống sử dụng Service Discovery (Eureka) để các dịch vụ tự động đăng ký và phát hiện lẫn nhau mà không cần cấu hình địa chỉ tĩnh, giúp tăng tính linh hoạt trong môi trường phân tán. Config Server được sử dụng để quản lý cấu hình tập trung, hỗ trợ việc thay đổi cấu hình hệ thống một cách nhất quán và thuận tiện trong quá trình triển khai và bảo trì.

+ **Lưu trữ dữ liệu:** Dữ liệu nghiệp vụ được lưu trên MySQL theo mô hình tách database riêng cho từng dịch vụ (Database-per-service) nhằm đảm bảo tính độc lập và giảm phụ thuộc giữa các module. Mô hình này được áp dụng cho các dịch vụ như User,

Product, Order, Cart và Review, cho phép mỗi dịch vụ tận dụng đặc tính ACID sẵn có của MySQL trong phạm vi database riêng của mình, đảm bảo tính nhất quán dữ liệu cho các nghiệp vụ quan trọng như đặt hàng và quản lý người dùng.

Bên cạnh đó, Redis được sử dụng như một tầng lưu trữ tạm nhằm hỗ trợ tăng tốc độ xử lý và giảm tải cho cơ sở dữ liệu chính trong các tác vụ không mang tính nghiệp vụ cốt lõi. Cụ thể, Redis được sử dụng để lưu trữ refresh token, qua đó hỗ trợ cơ chế xác thực và duy trì phiên đăng nhập của người dùng trong suốt quá trình sử dụng hệ thống mà không cần truy vấn lại cơ sở dữ liệu quan hệ. Đồng thời, một thành phần Redis khác được sử dụng để tiếp nhận và lưu trữ tạm thời các dữ liệu phát sinh từ hành vi người dùng như lượt xem, thao tác tương tác và các sự kiện hệ thống, sau đó phục vụ cho quá trình tổng hợp và phân tích thống kê.

Hai thành phần Redis này được triển khai tách biệt nhằm phân chia rõ ràng mục đích sử dụng: một bên phục vụ xác thực và quản lý phiên đăng nhập, bên còn lại phục vụ thu thập và xử lý dữ liệu phục vụ thống kê và phân tích hành vi. Việc tách biệt này giúp đảm bảo các luồng dữ liệu không ảnh hưởng lẫn nhau, đồng thời tăng tính ổn định của hệ thống khi khối lượng dữ liệu phát sinh lớn. Trong kiến trúc tổng thể, Redis không được sử dụng để thay thế MySQL trong các nghiệp vụ giao dịch chính, mà chỉ đóng vai trò như một lớp lưu trữ trung gian hỗ trợ xử lý nhanh và theo cơ chế bất đồng bộ, góp phần cải thiện hiệu năng và khả năng mở rộng của hệ thống.

+ **Tích hợp bên ngoài:** Hệ thống kết nối với một số dịch vụ bên thứ ba phục vụ thanh toán, vận chuyển, đăng nhập và các dịch vụ trí tuệ nhân tạo liên quan.

3.2.3. Cơ chế giao tiếp giữa các service

3.2.4. Cơ chế bảo mật

Hệ thống sử dụng cơ chế bảo mật dựa trên JSON Web Token (JWT) nhằm thực hiện xác thực và phân quyền người dùng trong toàn bộ hệ thống.

+ **Xác thực và phân quyền (Authentication & Authorization):** Hệ thống sử dụng JWT để xác thực người dùng sau khi đăng nhập và thực hiện phân quyền cho các chức năng như truy cập dịch vụ, thao tác dữ liệu và quản lý tài nguyên. Thay vì phải

truy vấn cơ sở dữ liệu ở mỗi yêu cầu để kiểm tra quyền, thông tin xác thực và phân quyền của người dùng được mã hóa trực tiếp trong token. Điều này giúp tách biệt rõ ràng giữa quá trình xác thực và xử lý nghiệp vụ, đồng thời giảm sự phụ thuộc giữa các thành phần trong hệ thống. Đặc biệt, trong kiến trúc Microservices, JWT cho phép các dịch vụ xác thực người dùng một cách độc lập mà không cần phụ thuộc vào một service trung tâm.

+ **Kiến trúc không trạng thái (Stateless Authentication):** JWT giúp hệ thống hoạt động theo mô hình stateless, tức là máy chủ không cần lưu trữ session của người dùng. Toàn bộ thông tin cần thiết được chứa trong token và được gửi kèm theo mỗi request từ client. Điều này giúp giảm đáng kể tải bộ nhớ trên server, đồng thời tăng khả năng mở rộng theo chiều ngang (horizontal scaling), phù hợp với các hệ thống lớn hoặc triển khai trên môi trường cloud-native.

+ **Tính toàn vẹn và bảo mật dữ liệu (Integrity & Security):** JWT được ký bằng secret key hoặc cơ chế cặp khóa công khai–riêng tư nhằm đảm bảo tính toàn vẹn của dữ liệu. Bất kỳ sự thay đổi nào trong nội dung token đều làm chữ ký không hợp lệ, từ đó bị hệ thống từ chối. Cơ chế này giúp đảm bảo rằng thông tin người dùng và quyền truy cập không bị chỉnh sửa trong quá trình truyền tải giữa client và server.

+ **Tối ưu hiệu năng hệ thống:** Việc không cần truy vấn cơ sở dữ liệu hoặc session store trong mỗi request giúp giảm đáng kể số lượng thao tác I/O, từ đó cải thiện hiệu năng xử lý. Token được gửi trực tiếp trong request và được xác thực nhanh chóng, giúp hệ thống đáp ứng tốt hơn trong các môi trường có lượng truy cập lớn như thương mại điện tử hoặc hệ thống phân tán.

+ **Phân quyền linh hoạt dựa trên Claim (Claim-based Authorization):** JWT cho phép lưu trữ các thông tin như vai trò người dùng (role), quyền hạn (permission), định danh (userId) hoặc các metadata khác. Điều này giúp hệ thống dễ dàng mở rộng cơ chế phân quyền theo nhiều mức độ khác nhau mà không cần thay đổi cấu trúc backend. Cách tiếp cận này đặc biệt phù hợp với các hệ thống có nhiều nhóm người dùng như admin, người dùng thông thường hoặc đối tác.

+ **Quản lý phiên và trải nghiệm người dùng:** JWT cho phép người dùng duy trì trạng thái đăng nhập trong một khoảng thời gian nhất định mà không cần xác thực lại liên tục. Điều này giúp cải thiện trải nghiệm người dùng, đồng thời giảm số lần xử lý xác thực trên server, góp phần tối ưu hiệu suất tổng thể của hệ thống.

+ **Phù hợp với kiến trúc phân tán:** Do không phụ thuộc vào session server, JWT có thể được sử dụng linh hoạt trong các hệ thống phân tán, microservices hoặc các nền tảng tích hợp nhiều dịch vụ. Các service có thể tự xác thực token mà không cần giao tiếp với một hệ thống xác thực trung tâm, từ đó tăng tính độc lập và khả năng mở rộng của toàn hệ thống.

3.2.5. Cơ chế tích hợp AI

3.2.5.1. Hệ thống gợi ý sản phẩm (Recommendation System)

Công nghệ và thư viện sử dụng

Phân hệ gợi ý sản phẩm được xây dựng tách biệt dưới dạng một Python Microservice sử dụng FastAPI và được tổ chức trong thư mục reco-service. Các công nghệ và thư viện chính bao gồm:

+ **scikit-surprise:** Được sử dụng để xây dựng mô hình Item-based Collaborative Filtering với thuật toán KNNBaseline và độ đo Pearson Baseline. Thư viện này hỗ trợ huấn luyện mô hình dựa trên dữ liệu rating của người dùng, từ đó xây dựng ma trận tương đồng giữa các item và ước lượng các giá trị bias (baseline) cho người dùng và sản phẩm. Trong giai đoạn suy luận, hệ thống sử dụng các thông tin đã học để tính toán điểm dự đoán dựa trên mức độ tương đồng giữa các sản phẩm và các giá trị bias tương ứng.

+ **pandas, NumPy và SciPy:** Hỗ trợ đọc, làm sạch và xử lý dữ liệu rating phục vụ quá trình huấn luyện, đồng thời cung cấp các phép tính số học cần thiết khi xây dựng và sử dụng mô hình.

+ **scikit-learn:** Bổ sung các công cụ xử lý dữ liệu và học máy cho môi trường Python khoa học dữ liệu của phân hệ gợi ý.

+ **Tệp lưu trữ mô hình (pickle):** Được sử dụng để tuần tự hóa, lưu trữ và nạp lại mô hình đã huấn luyện. Đây là file chứa mô hình recommendation sau khi huấn

luyện xong, giúp hệ thống có thể tải trực tiếp vào bộ nhớ khi chạy dịch vụ gợi ý mà không cần huấn luyện lại từ đầu.

+ **SQLAlchemy, PyMySQL và cryptography:** Cung cấp cơ chế kết nối an toàn đến MySQL, truy vấn dữ liệu rating trong quá trình huấn luyện và lấy lịch sử mua hàng khi xử lý trường hợp người dùng chưa có đánh giá.

+ **FastAPI và Uvicorn:** Cung cấp REST API như /recommend/{user_id} và /api/retrain để Statistics Service ở Java Backend gọi và nhận danh sách sản phẩm gợi ý.

+ **pydantic và python-dotenv:** Pydantic đảm nhiệm việc kiểm tra và chuẩn hóa cấu trúc dữ liệu đầu vào cũng như phản hồi của API, trong khi python-dotenv hỗ trợ nạp các biến cấu hình môi trường để hệ thống có thể vận hành linh hoạt giữa các môi trường triển khai khác nhau.

+ **httpx và py_eureka_client:** Cung cấp khả năng giao tiếp HTTP giữa các service và hỗ trợ đăng ký Recommendation Service lên Eureka trong kiến trúc Microservices của hệ thống, giúp các dịch vụ có thể tự động phát hiện và tương tác với nhau một cách linh hoạt và ổn định.

Logic xử lý (Code Implementation):

Trong trường hợp người dùng đã có lịch sử tương tác hoặc đã thực hiện đánh giá sản phẩm, hệ thống Recommendation Service sẽ tiến hành xây dựng danh sách gợi ý dựa trên các dữ liệu hành vi sẵn có. Quy trình này được thực hiện theo một chuỗi các bước xử lý nhằm đảm bảo việc khai thác hiệu quả thông tin người dùng và đưa ra các đề xuất sản phẩm phù hợp nhất với nhu cầu và sở thích cá nhân của họ.

- **Bước 1:** Tiếp nhận yêu cầu (Receive Request)

Ở giai đoạn đầu của quy trình gợi ý sản phẩm, hệ thống thực hiện việc tiếp nhận yêu cầu từ người dùng thông qua giao diện phía Backend, cụ thể là **dịch vụ Thống kê (Statistics Service)**. Thành phần này đóng vai trò trung gian giữa giao diện người dùng và dịch vụ gợi ý, giúp chuẩn hóa và điều phối các yêu cầu trước khi chuyển sang các module xử lý chuyên biệt.

Khi người dùng hoặc hệ thống quản trị yêu cầu danh sách sản phẩm gợi ý, Statistics Service nhận các tham số đầu vào gồm user_id để xác định người dùng và top_k để xác định số lượng sản phẩm cần trả về. Dựa trên các tham số này, hệ thống tạo một yêu cầu nội bộ và chuyển tiếp sang Recommendation Service thông qua cơ chế gọi API.

Yêu cầu được gửi đến endpoint /recommend/{user_id} kèm theo tham số top_k. Tại phía Recommendation Service, tầng Route đóng vai trò là lớp tiếp nhận yêu cầu từ client, chịu trách nhiệm định tuyến (routing) request đến đúng chức năng xử lý tương ứng trong hệ thống. Nói cách khác, đây là lớp trung gian giúp tách biệt giữa giao diện API bên ngoài và logic xử lý bên trong hệ thống. Sau khi tiếp nhận request, tầng Route sẽ chuyển tiếp dữ liệu đầu vào cho tầng xử lý nghiệp vụ, nơi thực hiện quá trình xử lý và sinh danh sách gợi ý sản phẩm cho người dùng.

Tại thời điểm này, hệ thống đã hoàn tất việc tiếp nhận và định tuyến yêu cầu. Dữ liệu đầu vào đã được kiểm tra tính hợp lệ ở mức tham số, đồng thời được chuyển đúng đến dịch vụ chịu trách nhiệm xử lý bài toán gợi ý. Đây là tiền đề để hệ thống tiếp tục thực hiện các bước tiếp theo như xác định người dùng, lựa chọn chiến lược gợi ý và tính toán danh sách sản phẩm phù hợp.

- **Bước 2:** Chuẩn bị dữ liệu và xác định nhánh xử lý

Ở bước này, hệ thống thực hiện việc nạp lại mô hình đã được huấn luyện từ bộ nhớ hoặc file lưu trữ nhằm phục vụ cho quá trình suy luận, đảm bảo các kết quả gợi ý được xây dựng dựa trên dữ liệu học trước đó.

Tiếp theo, hệ thống kiểm tra người dùng hiện tại có tồn tại trong tập dữ liệu tương tác (rating) của mô hình hay không. Nếu người dùng đã từng sử dụng hệ thống, hệ thống sẽ truy xuất lịch sử đánh giá và các hành vi tương tác liên quan đến sản phẩm để làm cơ sở cho việc sinh gợi ý theo phương pháp Item-based Collaborative Filtering sử dụng KNN Baseline.

Ngược lại, trong trường hợp người dùng chưa có dữ liệu tương tác, hệ thống sẽ chuyển sang xử lý bài toán cold-start. Khi đó, nếu người dùng đã phát sinh lịch sử mua hàng, hệ thống sẽ khai thác dữ liệu này để suy ra mức độ quan tâm và đề xuất các sản

phẩm có đặc điểm tương đồng. Trong trường hợp không tồn tại cả lịch sử đánh giá lẫn lịch sử mua hàng, hệ thống sẽ sử dụng chiến lược gợi ý dựa trên mức độ phổ biến của sản phẩm trong toàn hệ thống.

Bước này đóng vai trò quan trọng trong việc phân loại trạng thái dữ liệu của người dùng, từ đó xác định chiến lược gợi ý phù hợp cho các bước xử lý tiếp theo trong hệ thống.

- **Bước 3:** Tìm tập ứng viên và suy luận điểm số (Inference)

Luồng gợi ý cho người dùng đã có lịch sử đánh giá được sử dụng trong trường hợp người dùng đã có lịch sử đánh giá. Hệ thống bắt đầu bằng việc xác định các sản phẩm liên quan đến những sản phẩm mà người dùng đã tương tác trước đó. Với mỗi sản phẩm đã được đánh giá, hệ thống truy xuất danh sách các sản phẩm tương đồng thông qua hàm truy xuất các sản phẩm tương đồng, trong đó chỉ giữ lại một số lượng giới hạn các láng giềng có mức độ liên quan cao nhất.

Sau khi thu thập tập ứng viên, hệ thống loại bỏ các sản phẩm mà người dùng đã từng đánh giá để tránh việc gợi ý lặp lại. Đây là bước giúp đảm bảo danh sách đề xuất tập trung vào các sản phẩm mới mà người dùng chưa biết đến.

Tiếp theo, hệ thống tiến hành ước lượng điểm đánh giá cho từng sản phẩm ứng viên. Giá trị dự đoán được xây dựng dựa trên mức đánh giá trung bình của hệ thống, độ lệch của người dùng và độ lệch của từng sản phẩm. Sau đó, mô hình điều chỉnh giá trị này bằng cách kết hợp với mức độ tương đồng giữa sản phẩm ứng viên và các sản phẩm lân cận đã được người dùng đánh giá. Những sản phẩm có mức độ tương đồng cao sẽ có ảnh hưởng lớn hơn trong quá trình tính toán, trong khi các sản phẩm có mức độ tương đồng thấp hoặc không đáng tin cậy sẽ bị loại bỏ khỏi quá trình tổng hợp.

Trong trường hợp không có đủ thông tin từ các láng giềng phù hợp, hệ thống sẽ sử dụng giá trị dự đoán cơ sở dựa trên mức trung bình và các độ lệch đã được ước lượng trước đó. Kết quả cuối cùng được giới hạn trong một khoảng hợp lệ để đảm bảo tính ổn định của đầu ra.

```
def item_based_knn_baseline_predict_single(
    algo: Any,
    trainset: Any,
```

```
    inner_uid: int,
    raw_iid: int,
    k: int = 40,
) -> float:
    global_mean = float(trainset.global_mean)

    try:
        inner_iid = trainset.to_inner_iid(raw_iid)
    except ValueError:
        bu = _get_bu(algo, inner_uid)
        return _clip(global_mean + bu)

    bu = _get_bu(algo, inner_uid)
    bj = _get_bi(algo, inner_iid)
    b_uj = global_mean + bu + bj

    try:
        user_ratings = trainset.ur[inner_uid]
    except (IndexError, KeyError):
        return _clip(b_uj)

    rated_dict = {iid: r for iid, r in user_ratings}
    neighbors = algo.get_neighbors(inner_iid, k=k)

    numerator = 0.0
    denominator = 0.0

    for neighbor_iid in neighbors:
        if neighbor_iid not in rated_dict:
            continue

        sim_ji = float(algo.sim[inner_iid, neighbor_iid])

        if sim_ji <= 0:
            continue

        r_ui = float(rated_dict[neighbor_iid])
        bi_neighbor = _get_bi(algo, neighbor_iid)
        b_ui = global_mean + bu + bi_neighbor
        residual = r_ui - b_ui

        numerator += sim_ji * residual
        denominator += sim_ji

    if denominator > 0:
        est = b_uj + (numerator / denominator)
    else:
        est = b_uj
```

```
    return _clip(est)
```

Hình 3.2 : Mã nguồn tìm tập ứng viên và ước lượng điểm số

- **Bước 4:** Sắp xếp và trả kết quả (Response)

Danh sách các sản phẩm ứng viên sau khi được dự đoán điểm đánh giá sẽ được sắp xếp theo thứ tự giảm dần của điểm số. Hệ thống sau đó chọn ra Top-K sản phẩm có mức độ phù hợp cao nhất để đề xuất cho người dùng.

Kết quả trả về bao gồm danh sách các sản phẩm cùng với điểm dự đoán tương ứng. Phía backend sẽ tiếp nhận danh sách này và thực hiện truy vấn thêm thông tin chi tiết của từng sản phẩm như tên, giá bán và hình ảnh trước khi gửi kết quả hoàn chỉnh về giao diện người dùng.


```
{
    "user_id": 1,
    "status": "success",
    "strategy": "item_knn_baseline",
    "top_k": 10,
    "recommendations": [
        {
            "product_id": 26,
            "predicted_rating": 4.0546
        },
        {
            "product_id": 37,
            "predicted_rating": 3.7194
        },
        {
            "product_id": 14,
            "predicted_rating": 3.5414
        },
        {
            "product_id": 13,
            "predicted_rating": 3.3567
        },
        {
            "product_id": 40,
            "predicted_rating": 3.3507
        },
```


Hình 3.3 : Dữ liệu phản hồi của Recommendation Service sau xếp hạng

+ Để đưa danh sách sản phẩm cá nhân hóa đến với người dùng, phân hệ giao diện trên Trang chủ sẽ chịu trách nhiệm kích hoạt luồng gọi API bất đồng bộ và xử lý toàn bộ dữ liệu phản hồi từ Backend.

+ **Cơ chế gọi API bằng Axios:** Khối xử lý giao diện sử dụng bộ công cụ Axios để gửi yêu cầu đến cổng kết nối dịch vụ nhằm thu thập danh sách mã sản phẩm được hệ thống AI đề xuất riêng cho tài khoản người dùng hiện tại, thông tin chi tiết được thể hiện trong Hình 3.4.

```
useEffect(() => {
  const fetchRecommendations = async () => {
    try {
      let rawProducts = [];

      try {
        // Gọi API Gateway: /api/recommendations/user/{userId}
        // (Vite proxy → :8080 → route statistics-service → reco-service Inference)
        const response = await httpClient.get(
          `${API_PREFIX}/recommendations/user/${currentUserId}`,
        );
        rawProducts = response.data?.data || [];
      } catch (err) {
        console.warn("Recommendations API unavailable:", err.message);
      }

      // BFF thường chỉ trả { id, name } → thiếu price/images thì lấy chi tiết từ catalog
      if (rawProducts.length > 0 && !rawProducts[0].price && !rawProducts[0].images) {
        const enriched = await Promise.all(
          rawProducts.map(async (item) => {
            try {
              return await getProductDetail(item.id);
            } catch {
              return item;
            }
          }),
        );
        rawProducts = enriched.filter(Boolean);
      }
```

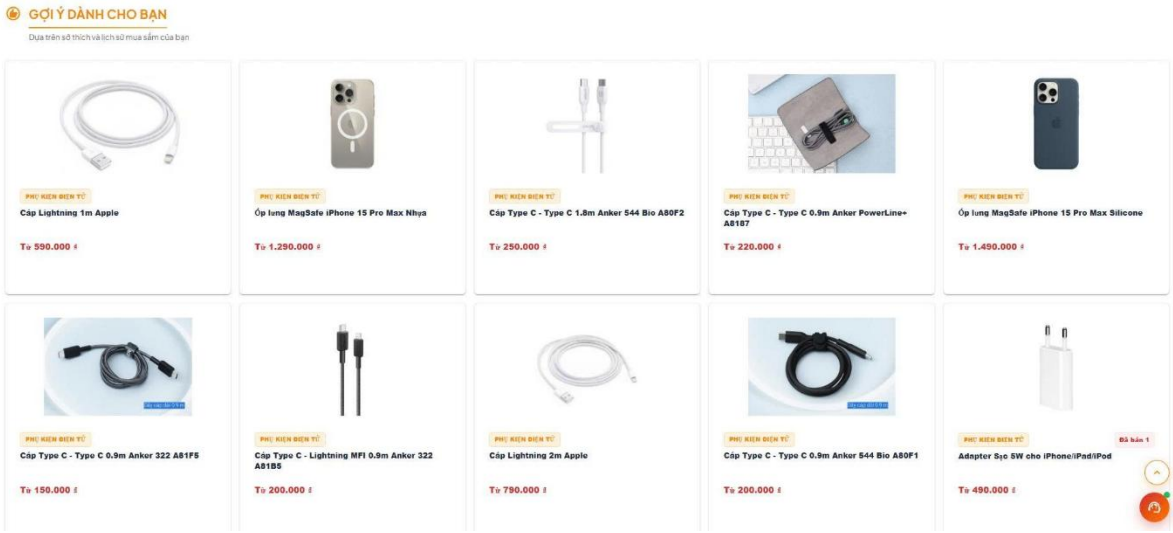
Hình 3.4 : Mã nguồn Frontend gọi API lấy danh sách gợi ý sản phẩm

+ **Cơ chế kết xuất dữ liệu ra giao diện**: Sau khi nhận dữ liệu phản hồi thành công, mảng danh sách sản phẩm sẽ được lưu trữ vào trạng thái nội bộ của ứng dụng. Khối giao diện sẽ áp dụng cấu trúc lưới Grid CSS kết hợp phương thức duyệt mảng động để tự động chuyển đổi và ánh xạ toàn bộ thông tin sản phẩm vào các thẻ hiển thị trực quan trên màn hình, như thể hiện trong Hình 3.5.

```
<div className="product-recommendations__grid">
  {products.map((p) => (
    <div key={p.id} className="product-recommendations__item">
      <ProductCard product={p} />
    </div>
  ))}
</div>
```

Hình 3.5 : Mã nguồn React kết xuất dữ liệu gợi ý bằng Grid CSS

+ Sau đó, danh sách các sản phẩm công nghệ thông minh này sẽ tự động thay đổi và cá nhân hóa theo từng tài khoản dựa trên lịch sử tương tác thời gian thực của khách hàng. Giao diện thực tế được minh họa trong Hình 3.6.



Hình 3.6 : Khối sản phẩm "Gợi ý dành cho bạn" trên Trang chủ

3.2.5.2. Chatbot tư vấn (RAG)

Công nghệ và thư viện sử dụng

Phân hệ Chatbot được triển khai dưới dạng một Spring Boot Microservice độc lập với tên chatbot-service, hoạt động tại cổng 8092. Dịch vụ được xây dựng theo kiến trúc Retrieval-Augmented Generation, kết hợp cơ chế tìm kiếm lai trên Pinecone để

truy xuất dữ liệu phù hợp trước khi sinh phản hồi. Các công nghệ và thư viện chính bao gồm:

+ **Spring Boot, Spring Cloud, Eureka và API Gateway:** Cung cấp nền tảng triển khai Chatbot Service, đăng ký và phát hiện dịch vụ, đồng thời định tuyến các yêu cầu từ Client đến đúng microservice.

+ **OpenFeign:** Các Feign Client như CatalogClient và ReviewClient được sử dụng để lấy dữ liệu sản phẩm và đánh giá từ các dịch vụ nghiệp vụ, phục vụ quá trình thu thập và đưa dữ liệu vào kho vector.

+ **Spring AI:** Hỗ trợ tích hợp mô hình ngôn ngữ lớn, mô hình embedding và Vector Store vào ứng dụng Spring Boot.

+ **NVIDIA NIM:** Cung cấp mô hình nv-embed-v1 để tạo vector embedding và mô hình llama-3.1-8b-instruct để sinh nội dung tư vấn.

+ **Pinecone:** Lưu trữ và truy vấn vector theo cơ chế hybrid, kết hợp vector dense và sparse nhằm nâng cao khả năng tìm kiếm ngữ nghĩa và từ khóa.

+ **Hybrid Retrieval và Rerank:** Kết quả truy xuất được xếp hạng lại bằng mô hình bge-reranker-v2-m3 với hệ số kết hợp $\alpha \approx 0{,}75$, giúp lựa chọn ngữ cảnh phù hợp trước khi gửi yêu cầu đến mô hình ngôn ngữ.

Nhờ sự kết hợp trên, Chatbot có thể truy xuất đúng ngữ cảnh dữ liệu của cửa hàng và sinh câu trả lời tư vấn phù hợp với yêu cầu của người dùng.

Logic xử lý (Code Implementation)

- Bước 1: Kiểm tra trạng thái vector store

Luồng xử lý của Chatbot được khởi tạo tại ChatbotController khi Frontend gửi yêu cầu thông qua endpoint POST /api/chatbot/chat. Tại đây, hệ thống tiếp nhận nội dung tin nhắn và chuyển tiếp sang tầng dịch vụ thông qua lớp xử lý nghiệp vụ của hệ thống để thực hiện quy trình xử lý theo kiến trúc RAG. Trong bước này, service đóng vai trò điều phối toàn bộ luồng xử lý, từ truy xuất dữ liệu liên quan đến việc tạo phản hồi dựa trên ngữ cảnh được tổng hợp.

Khi hệ thống nhận yêu cầu chat từ frontend thông qua endpoint POST /api/chatbot/chat, yêu cầu được chuyển đến tầng dịch vụ xử lý chatbot, nơi điều phối toàn bộ quy trình theo kiến trúc Retrieval-Augmented Generation (RAG). Trước khi thực hiện truy xuất ngữ cảnh và sinh phản hồi từ mô hình ngôn ngữ, hệ thống cần kiểm tra xem kho dữ liệu vector (Pinecone index) đã được khởi tạo và có dữ liệu sản phẩm hay chưa.

Cụ thể, hệ thống kiểm tra trạng thái của Pinecone index để xác định mức độ sẵn sàng của dữ liệu embedding. Bước này đảm bảo quá trình truy xuất thông tin có thể diễn ra chính xác, vì nếu không tồn tại dữ liệu vector thì hệ thống sẽ không thể tìm kiếm ngữ cảnh liên quan để hỗ trợ sinh câu trả lời.

Trong trường hợp index chưa được tạo hoặc chưa có dữ liệu, quy trình RAG sẽ dừng tại đây. Hệ thống trả về phản hồi cho người dùng, thông báo rằng dữ liệu chưa được đồng bộ và cần thực hiện quá trình indexing trước khi sử dụng chatbot. Khi đó, yêu cầu sẽ không được chuyển tiếp sang mô hình ngôn ngữ, nhằm tránh việc sinh ra câu trả lời thiếu ngữ cảnh hỗ trợ.

- **Bước 2:** Truy xuất và xếp hạng ngữ cảnh

Sau khi hệ thống xác nhận kho vector (Pinecone index) đã sẵn sàng, yêu cầu của người dùng được chuyển sang giai đoạn truy xuất thông tin liên quan nhằm cung cấp ngữ cảnh cho mô hình ngôn ngữ. Giai đoạn này được thực hiện thông qua một dịch vụ truy xuất, đóng vai trò trung gian giữa truy vấn người dùng và kho dữ liệu vector.

Cụ thể, hệ thống tiến hành chuyển đổi câu hỏi đầu vào thành hai dạng biểu diễn: **dense vector** và **sparse vector**. Dense vector được sinh ra từ mô hình embedding ngữ nghĩa, giúp nắm bắt mối quan hệ ngữ nghĩa sâu giữa các thực thể trong câu hỏi. Trong khi đó, sparse vector (thường dựa trên TF-IDF hoặc BM25) giúp duy trì khả năng khớp chính xác theo từ khóa, đặc biệt hữu ích trong các trường hợp truy vấn chứa tên sản phẩm hoặc thuộc tính kỹ thuật cụ thể.

Hai dạng biểu diễn này được sử dụng đồng thời để thực hiện truy vấn **hybrid search** trên Pinecone. Cơ chế hybrid cho phép kết hợp ưu điểm của tìm kiếm ngữ nghĩa (semantic search) và tìm kiếm từ khóa (lexical search), từ đó nâng cao khả năng thu hồi

các đoạn dữ liệu liên quan trong bối cảnh dữ liệu sản phẩm có tính đa dạng và nhiều thuộc tính kỹ thuật.

Sau khi Pinecone trả về tập kết quả ứng viên ban đầu, hệ thống không sử dụng trực tiếp các kết quả này mà tiếp tục thực hiện bước **reranking**. Mục tiêu của reranking là sắp xếp lại các đoạn ngữ cảnh dựa trên mức độ liên quan thực sự với truy vấn, thay vì chỉ dựa trên độ tương đồng vector ban đầu. Quá trình này giúp loại bỏ các kết quả nhiễu và ưu tiên các đoạn thông tin có giá trị cao nhất đối với câu hỏi của người dùng.

Kết quả cuối cùng sau khi rerank là một tập hợp các đoạn ngữ cảnh có độ liên quan cao nhất, được đưa vào bước tiếp theo của pipeline RAG để làm đầu vào cho mô hình ngôn ngữ sinh phản hồi.



```
private List<Document> hybridRetrieve(String userQuery) {
        try {
            List<Float> dense = toFloatList(embeddingModel.embed(userQuery));
            dense = HybridScoreNormalizer.l2Normalize(dense);

            String sparseModel = ragProperties.getHybrid().getSparseModel();
            SparseVectorValues sparse = pineconeRagClient
                    .embedSparse(sparseModel, List.of(userQuery), "query")
                    .get(0);

            double alpha = ragProperties.getHybrid().getAlpha();
            dense = HybridScoreNormalizer.scaleDense(dense, alpha);
            sparse = HybridScoreNormalizer.scaleSparse(sparse, alpha);

            List<PineconeRagClient.HybridQueryMatch> matches = pineconeRagClient.queryHybrid(
                    dense,
                    sparse,
                    ragProperties.getRetrievalTopK()
            );

            log.info("Hybrid search: {} candidates (alpha={}, retrievalTopK={})",
                    matches.size(), alpha, ragProperties.getRetrievalTopK());

            return matches.stream()
                    .map(this::matchToDocument)
                    .collect(Collectors.toCollection(ArrayList::new));
        } catch (Exception e) {
            log.warn("Hybrid search thất bại (index cần metric dotproduct + sparse?). Fallback dense-
only: {}",
                    e.getMessage());
            return denseRetrieve(userQuery);
        }
    }
```

Hình 3.7 : Mã Nguồn tìm và chọn sản phẩm liên quan đến câu hỏi

- **Bước 3:** Xây dựng prompt và sinh phản hồi

Sau khi truy xuất và xếp hạng ngữ cảnh, hệ thống xây dựng prompt cho LLM bằng cách kết hợp: câu hỏi người dùng, các đoạn ngữ cảnh liên quan và danh sách sản phẩm hợp lệ từ cơ sở dữ liệu. Mục tiêu là đảm bảo mô hình chỉ sinh câu trả lời dựa trên dữ liệu nội bộ, hạn chế sai lệch thông tin.

Prompt sau đó được gửi đến mô hình Llama 3.1 thông qua dịch vụ suy luận của NVIDIA NIM. Mô hình sử dụng các thông tin được cung cấp để tạo phản hồi mang tính tư vấn, đồng thời tuân thủ các ràng buộc về thực thể sản phẩm.

Kết quả trả về là câu trả lời tự nhiên, phù hợp ngữ cảnh hội thoại và đảm bảo tính chính xác theo dữ liệu của hệ thống thương mại điện tử.

```
// ----- Bước 3: Lấy context + tên sản phẩm sau Rerank -----
String context = similarDocuments.stream()
        .map(Document::getText)
        .collect(Collectors.joining("\n---\n"));

String allowedProductNames = extractAllowedProductNames(similarDocuments);

// ----- Bước 4: Prompt Engineering (chống ảo giác) -----
String systemInstructions = """
        Bạn là "Electro Store Assistant" - nhân viên tư vấn của cửa hàng Electro Store.
        Bạn CHỈ được trả lời dựa trên THÔNG TIN BỐI CẢNH và DANH SÁCH SẢN PHẨM ĐƯỢC PHÉP bên dưới.
        Tuyệt đối KHÔNG dùng kiến thức có sẵn của mô hình để thêm sản phẩm, model, hãng hoặc giá
        không có trong bối cảnh.

        DANH SÁCH SẢN PHẨM ĐƯỢC PHÉP NHẮC TỚI (chỉ các tên sau – không thêm tên khác):
        %s

        QUY TẮC BẮT BUỘC:
        1. Chỉ mô tả/đề xuất sản phẩm có trong DANH SÁCH và THÔNG TIN BỐI CẢNH.
           Không liệt kê nhiều model giả nếu cửa hàng chỉ có một mục trong danh sách.
        2. Giá, thông số, ưu/nhược điểm phải khớp với bối cảnh; không bịa số liệu.
        3. Nếu chỉ có một sản phẩm trong danh sách, chỉ tư vấn một sản phẩm đó.
        4. Giọng điệu lịch sự, Tiếng Việt, Markdown gọn (in đậm tên/giá).
        5. Nếu bối cảnh không đủ trả lời, nói rõ Electro Store chưa có thêm thông tin
           trên hệ thống – KHÔNG đề xuất sản phẩm ngoài danh sách.

        THÔNG TIN BỐI CẢNH:
        %s
        """.formatted(allowedProductNames, context);

// ----- Bước 5: Gọi NVIDIA NIM (Llama 3.1) -----
log.info("Đang gửi yêu cầu tới NVIDIA NIM để sinh câu trả lời...");

String aiResponse = chatClient.prompt()
        .system(systemInstructions)
        .user(userQuery)
        .call()
        .content();

log.info("Đã nhận được câu trả lời từ AI.");
return aiResponse;
```

Hình 3.8 : Mã Nguồn Tạo câu trả lời tư vấn dựa trên thông tin đã tìm được

- **Bước 4:** Trả kết quả về Frontend

Sau khi mô hình LLM sinh ra câu trả lời dựa trên ngữ cảnh truy xuất và prompt đã xây dựng, hệ thống tiến hành đóng gói kết quả để trả về cho tầng giao diện người dùng (Frontend). Câu trả lời được bao bọc trong một đối tượng chuẩn hóa **ApiResponse**, nhằm đảm bảo tính thống nhất trong toàn bộ hệ thống khi giao tiếp giữa các dịch vụ.

Đối tượng ApiResponse thường bao gồm các thành phần như mã trạng thái (status), thông điệp phản hồi (message) và dữ liệu trả về (data). Trong trường hợp này, dữ liệu chính là câu trả lời được sinh ra từ mô hình RAG. Việc chuẩn hóa phản hồi giúp Frontend dễ dàng xử lý và hiển thị nội dung hội thoại cho người dùng mà không cần quan tâm đến logic xử lý phía Backend.

```
    @PostMapping("/chat")
    @Operation(summary = "Gửi tin nhắn trò chuyện với Chatbot RAG", description = "AI tự động tìm kiếm
ngữ cảnh sản phẩm trong Pinecone để tư vấn chính xác nhất")
    public ResponseEntity<ApiResponse<ChatResponse>> chat(@RequestBody ChatRequest request) {
        if (request == null || request.getMessage() == null || request.getMessage().trim().isEmpty()) {
            return ResponseEntity.badRequest().body(ApiResponse.error(400, "Nội dung tin nhắn không được
để trống"));
        }

        String aiResponse = ragChatbotService.chatWithRag(request.getMessage());
        return ResponseEntity.ok(ApiResponse.success("Phản hồi thành công", new
ChatResponse(aiResponse)));
    }
```

Hình 3.9 : Mã Nguồn đóng gói và gửi câu trả lời về giao diện chat

- **Bước 5:** Xử lý truyền nhận và kết xuất dữ liệu hội thoại tại tầng Giao diện (Frontend)

+ Để người dùng có thể tương tác trực tiếp với hệ thống trí tuệ nhân tạo, phân hệ giao diện Chatbot sẽ chịu trách nhiệm quản lý trạng thái luồng tin nhắn và thực hiện các kết nối bất đồng bộ đến cổng dịch vụ AI

+ **Cơ chế gửi yêu cầu và nhận phản hồi qua Axios:** Khi người dùng gửi nội dung câu hỏi, khối giao diện sử dụng bộ công cụ Axios gửi yêu cầu dạng POST

/api/chatbot/chat kèm theo dữ liệu câu hỏi của khách hàng. Quá trình này kích hoạt luồng xử lý RAG tại máy chủ và nhận về chuỗi văn bản câu trả lời đã được chuẩn hóa, như minh họa trong Hình 3.10.



```
// Gọi REST API của Java chatbot-service
const res = await fetch(`${CHATBOT_BASE}/api/chatbot/chat`, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    message: question
  }),
  signal
});
```

Hình 3.10 : Mã nguồn Frontend gửi câu hỏi và nhận phản hồi Chatbot

+ **Cơ chế kết xuất luồng hội thoại lên UI**: Khi dữ liệu phản hồi trả về thành công, hệ thống sử dụng các hàm cập nhật trạng thái ứng dụng (useState/setState) để đẩy nội dung tin nhắn mới vào danh sách. Phân hệ giao diện áp dụng phương thức duyệt mảng động để tự động sinh cấu trúc các bong bóng chat (chat bubbles) phân biệt giữa khách hàng và Trợ lý AI, như minh họa trong Hình 3.11



```
<Box className="store-chatbot-messages">
  <div className="store-chatbot-date-divider">
    <span>{t("chatbot.today")}</span>
  </div>
  {messages.map((msg, idx) => (
    <ChatMessage key={idx} msg={msg} />
  ))}
```

Hình 3.11 : Mã nguồn React kết xuất luồng hội thoại

+ Sau đó, người dùng có thể tiếp tục đặt các câu hỏi tiếp theo dựa trên ngữ cảnh hội thoại cũ để được Trợ lý AI phân tích ý định và đưa ra phản hồi chính xác. Giao diện hiển thị thực tế trên trình duyệt được minh họa trong Hình 3.12



Hình 3.12 : Cửa sổ trò chuyện Trợ lý Electro tư vấn sản phẩm

3.3. Hiện thực giải pháp

3.3.1. Mô tả chức năng

Hệ thống thương mại điện tử được xây dựng nhằm mục tiêu cung cấp giải pháp bán lẻ thiết bị điện tử trực tuyến theo mô hình B2C, khắc phục những hạn chế thường gặp của các hệ thống truyền thống như thiếu khả năng cá nhân hóa, khó mở rộng khi lượng dữ liệu và người dùng tăng lên, và phụ thuộc nhiều vào thao tác thủ công trong quá trình vận hành. Hệ thống cho phép người dùng thực hiện các thao tác cơ bản như tìm kiếm sản phẩm, xem thông tin chi tiết, đặt hàng và đánh giá sản phẩm. Ở phía vận hành, hệ thống hỗ trợ đội ngũ quản trị trong việc quản lý sản phẩm, đơn hàng, người dùng và theo dõi các chỉ số thống kê liên quan đến hoạt động kinh doanh.

Hệ thống được phát triển theo kiến trúc client–server. Frontend sử dụng ReactJS kết hợp Material UI để xây dựng giao diện người dùng. Backend được tổ chức theo hướng Microservices và triển khai bằng Java Spring Boot. Dữ liệu được lưu trữ trên MySQL theo nguyên tắc Database-per-service, giúp giảm sự phụ thuộc giữa các dịch vụ và tăng khả năng mở rộng. Hệ thống tích hợp thêm dịch vụ gợi ý sản phẩm được

xây dựng bằng Python FastAPI, sử dụng thuật toán Item-based KNNBaseline để phân tích hành vi đánh giá và đề xuất sản phẩm phù hợp. Bên cạnh đó, hệ thống còn tích hợp chatbot AI dựa trên Spring AI, NVIDIA NIM và cơ chế RAG với Pinecone nhằm hỗ trợ người dùng trong việc tra cứu và tư vấn sản phẩm.

Hệ thống sử dụng JWT kết hợp RBAC để xác thực và phân quyền người dùng.

Access token được thiết kế theo hướng stateless, trong khi refresh token được lưu trên

Redis để hỗ trợ việc làm mới phiên đăng nhập. Cách tiếp cận này giúp giảm phụ thuộc vào session tập trung và phù hợp với đặc trưng phân tán của kiến trúc microservices.

Các chức năng chính của hệ thống được thiết kế nhằm đáp ứng đầy đủ quy trình mua sắm trực tuyến, đồng thời tích hợp các thành phần trí tuệ nhân tạo để nâng cao trải nghiệm người dùng và tối ưu hiệu quả vận hành của hệ thống.

+ **Đăng ký / đăng nhập và quản lý tài khoản:** Hỗ trợ người dùng tạo tài khoản, đăng nhập, cập nhật thông tin cá nhân; đồng thời tích hợp cơ chế đăng nhập nhanh thông qua các nền tảng bên thứ ba như Google và Facebook (OAuth), giúp đơn giản hóa quá trình xác thực và tăng tính tiện lợi.

+ **Duyệt và tìm kiếm sản phẩm:** Cung cấp khả năng xem danh mục sản phẩm thiết bị điện tử, tìm kiếm theo từ khóa, lọc theo nhiều tiêu chí (giá, thương hiệu, cấu hình, v.v.) và xem thông tin chi tiết của từng sản phẩm, giúp người dùng dễ dàng tiếp cận sản phẩm phù hợp nhu cầu.

+ **Giỏ hàng và đặt hàng:** Cho phép người dùng thêm hoặc loại bỏ sản phẩm trong giỏ hàng, tiến hành đặt hàng và theo dõi trạng thái đơn hàng trong suốt quá trình xử lý, từ lúc xác nhận đến khi giao hàng hoàn tất.

+ **Thanh toán và vận chuyển:** Tích hợp cổng thanh toán trực tuyến VNPay nhằm hỗ trợ giao dịch an toàn, đồng thời kết nối với đơn vị vận chuyển GHN để tự động hóa quy trình giao nhận và cập nhật trạng thái vận chuyển theo thời gian thực.

+ **Tra cứu và Quản lý Bảo hành:** Cho phép khách hàng dễ dàng tra cứu thông tin và thời hạn bảo hành điện tử của thiết bị dựa trên mã đơn hàng hoặc sản phẩm đã mua, giúp nâng cao chất lượng dịch vụ hậu mãi, đồng thời hỗ trợ doanh nghiệp quản lý tập trung thông tin bảo hành và giảm thiểu sai sót trong quá trình xử lý thủ công. Bên cạnh đó, hệ thống còn cho phép khách hàng gửi yêu cầu bảo hành trực tuyến khi sản phẩm gặp sự cố, giúp rút ngắn thời gian tiếp nhận, chuẩn hóa quy trình xử lý và theo dõi trạng thái bảo hành một cách minh bạch từ lúc tiếp nhận đến khi hoàn tất.

+ **Đánh giá sản phẩm:** Người dùng có thể đánh giá bằng số sao và để lại nhận xét sau khi mua hàng. Các dữ liệu này đóng vai trò là nguồn thông tin quan trọng phục vụ cho việc huấn luyện và cải thiện các mô hình gợi ý sản phẩm.

+ **Gợi ý sản phẩm cá nhân hóa:** Hệ thống tự động sinh danh sách Top-K sản phẩm phù hợp với từng người dùng dựa trên thuật toán KNN Baseline. Đối với người dùng đã có lịch sử đánh giá, hệ thống khai thác hành vi tương tác để xác định mức độ tương đồng giữa các sản phẩm và đưa ra gợi ý phù hợp. Trong trường hợp người dùng mới hoặc thiếu dữ liệu, hệ thống áp dụng chiến lược Cold-Start bằng cách sử dụng lịch sử mua hàng hoặc các sản phẩm phổ biến để đảm bảo vẫn cung cấp được kết quả gợi ý hợp lý.

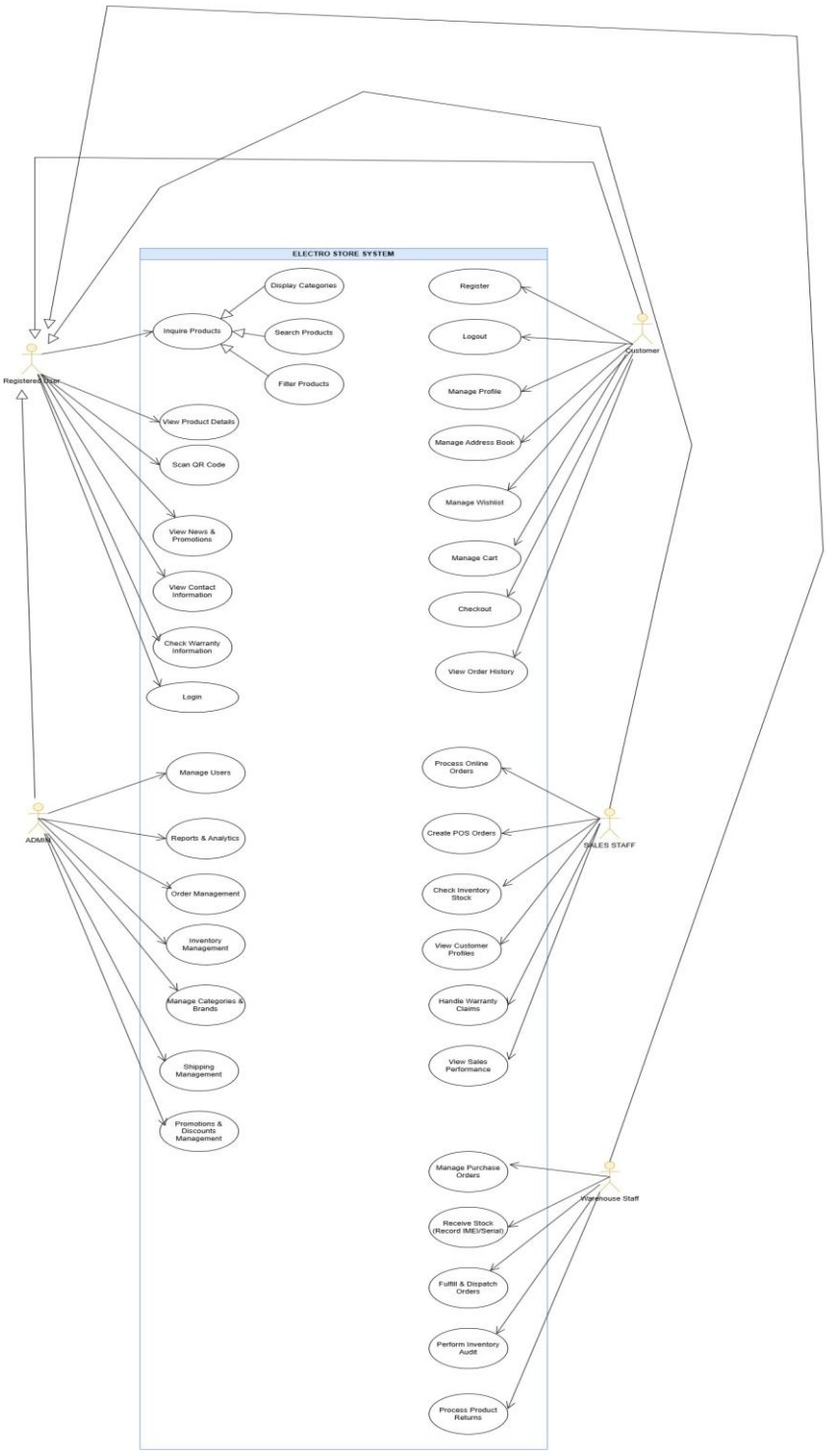
+ **Chatbot tư vấn:** Hỗ trợ người dùng trong việc tìm kiếm thông tin và tư vấn sản phẩm thông qua cơ chế truy hồi ngữ cảnh (RAG), kết hợp dữ liệu nội bộ của hệ thống để tạo ra các phản hồi tự nhiên, chính xác và mang tính hỗ trợ 24/7.

+ **Quản trị hệ thống (phía Admin/Sales/Kho):** Cung cấp chức năng quản lý toàn diện bao gồm sản phẩm, đơn hàng, người dùng, tồn kho, mã giảm giá, đánh giá và thống kê doanh thu. Đồng thời hỗ trợ kiểm duyệt nội dung và theo dõi hoạt động hệ thống thông qua các bảng điều khiển (Dashboard) trực quan.

Backend được tổ chức thành các microservice độc lập, giao tiếp thông qua API

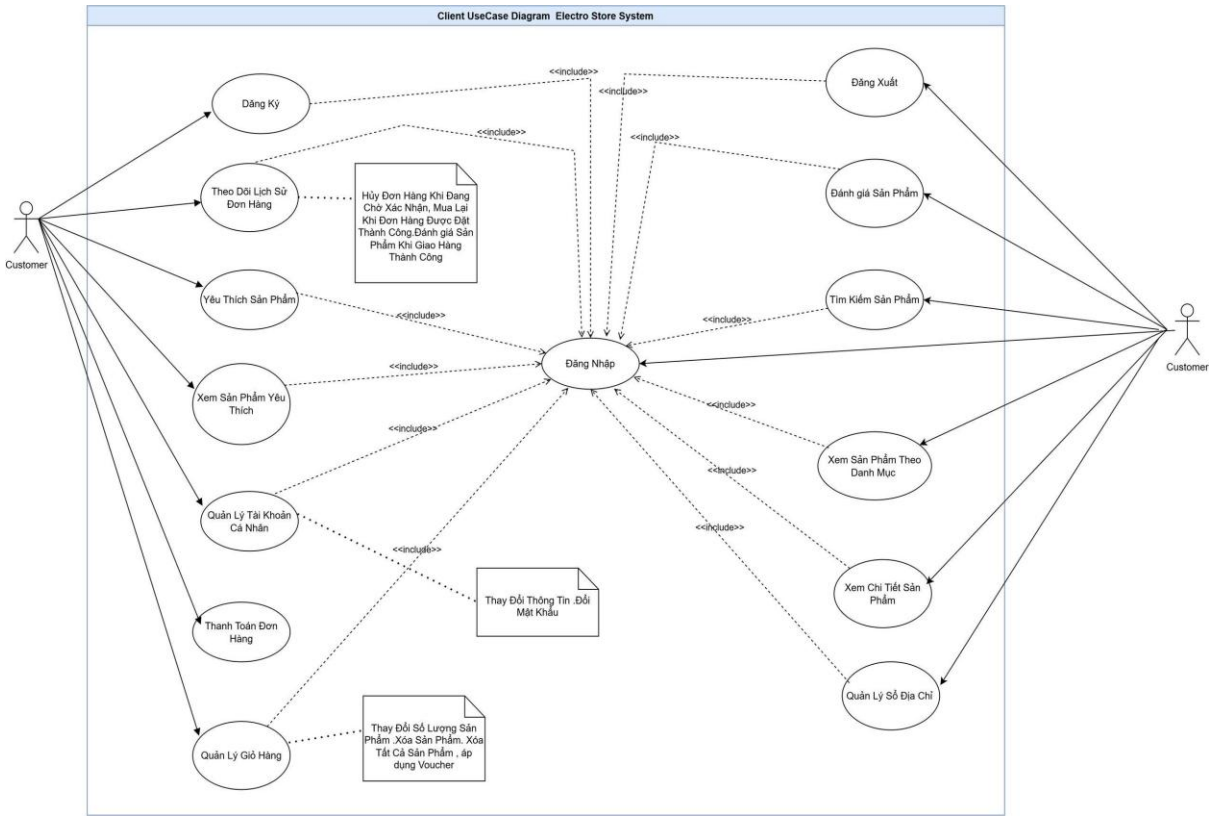
RESTful. Client truy cập hệ thống thông qua API Gateway. Các dịch vụ được đăng ký và khám phá thông qua Eureka, cấu hình tập trung tại Config Server, và giao tiếp nội bộ chủ yếu bằng OpenFeign. Dữ liệu nghiệp vụ được lưu trữ trên MySQL. Redis được sử dụng như tầng lưu trữ tạm cho refresh token và một số tác vụ trung gian. Pinecone được sử dụng để lưu trữ vector phục vụ cơ chế truy hồi trong chatbot. Dịch vụ gợi ý sản phẩm xử lý dữ liệu đánh giá để huấn luyện và suy luận mô hình, trong khi dịch vụ chatbot kết hợp NVIDIA NIM và Pinecone để truy xuất ngữ cảnh và sinh phản hồi cho người dùng.

3.3.2. Lược đồ Use case



Hình 3.13 : Lược đồ Use Case tổng quan của hệ thống

3.3.2.1. Phân hệ các Usecase dành cho Khách hàng (Người dùng)



Hình 3.14 : Lược đồ Use Case vai trò khách hàng

Đối với người dùng chưa tồn tại trong hệ thống, việc khởi tạo tài khoản được thực hiện thông qua use case Đăng ký, nhằm thiết lập thông tin định danh ban đầu cho người dùng. Quy trình chi tiết của use case này được trình bày trong Bảng 3.1.

Bảng 3.1 : Đặc tả Use Case “Đăng ký”

Thuộc tính	**Chi tiết đặc tả**
Tên Use Case	Đăng ký tài khoản
Actor chính	Người dùng chưa xác thực (Guest).
Tiền điều kiện	Người dùng đang truy cập hệ thống và chưa đăng nhập.
Hậu điều kiện	Tài khoản mới được tạo thành công trong hệ thống với vai trò mặc định là khách hàng. Phiên đăng nhập được khởi tạo (cấp JWT) và trạng thái người dùng chuyển sang "Đã xác thực".
Luồng chính (Main Flow)	Quy trình đăng ký tài khoản được thực hiện như sau: 1. Người dùng chọn chức năng "Đăng ký".

	2. Hệ thống hiển thị biểu mẫu yêu cầu nhập thông tin (Tên đăng nhập, email, mật khẩu, họ tên, số điện thoại). 3. Người dùng nhập đầy đủ thông tin và nhấn "Đăng ký". 4. Hệ thống kiểm tra tính hợp lệ của dữ liệu, kiểm tra trùng lặp email/tên đăng nhập và tiến hành lưu thông tin tài khoản. 5. Hệ thống khởi tạo phiên đăng nhập (sinh mã token), tự động đăng nhập cho người dùng. 6. Hệ thống hiển thị thông báo đăng ký thành công và chuyển hướng người dùng về Trang chủ.
Luồng thay thế (Alternative Flow)	Trong trường hợp người dùng lựa chọn đăng ký qua bên thứ ba, quy trình được thực hiện như sau: **Alt 1:** Đăng ký qua bên thứ ba (Google OAuth2) 1. Tại bước 3 luồng chính, người dùng chọn đăng ký bằng tài khoản Google. 2. Hệ thống chuyển hướng sang trang xác thực của Google. 3. Người dùng đăng nhập và chấp nhận chia sẻ thông tin trên Google. 4. Hệ thống nhận thông tin (email, họ tên), tự động tạo tài khoản với vai trò mặc định (không cần mật khẩu) và tiếp tục từ bước 5 của luồng chính.
Luồng ngoại lệ (Exception Flow)	Trong quá trình xử lý, một số tình huống ngoại lệ có thể xảy ra như sau: - **Exc 1:** Email hoặc Tên đăng nhập đã tồn tại 1. Tại bước 4 luồng chính, hệ thống phát hiện thông tin đã tồn tại. 2. Hệ thống hiển thị thông báo lỗi (Lỗi trùng lặp/Xung đột dữ liệu) và giữ nguyên thông tin đã nhập trên biểu mẫu để người dùng chỉnh sửa. - **Exc 2:** Dữ liệu đầu vào không hợp lệ 1. Tại bước 4 luồng chính, hệ thống phát hiện dữ liệu sai định dạng (ví dụ: email sai cú pháp, mật khẩu yếu).

	2. Hệ thống hiển thị thông báo lỗi chi tiết tại các trường dữ liệu tương ứng để người dùng điều chỉnh. - **Exc 3:** Lỗi hệ thống/Lưu trữ thất bại (Bao gồm lỗi kết nối Redis/Cơ sở dữ liệu) 1. Tại bước 4 hoặc 5 luồng chính, hệ thống gặp sự cố không thể ghi dữ liệu hoặc khởi tạo phiên. 2. Hệ thống hủy giao dịch hiện tại, hiển thị thông báo "Hệ thống bận, vui lòng thử lại sau".

Ngay khi các hậu điều kiện trong Bảng 3.1 được thỏa mãn, hệ thống sẽ cấp cho người dùng một định danh hợp lệ. Ở các lần truy cập tiếp theo, nhằm đảm bảo an toàn và xác thực danh tính, người dùng cần thực hiện chức năng đăng nhập để hệ thống tiến hành kiểm tra thông tin và cấp lại token truy cập.

Bảng 3.2 : Đặc tả Use Case “Đăng nhập”

Thuộc tính	**Chi tiết đặc tả**
Tên Use Case	Đăng nhập hệ thống
Actor chính	Người dùng chưa xác thực (Guest).
Tiền điều kiện	Người dùng đang truy cập hệ thống và chưa đăng nhập.
Hậu điều kiện	Phiên làm việc mới được khởi tạo thành công. Trạng thái tài khoản chuyển sang "Đã xác thực". Giao diện được cập nhật và hiển thị các chức năng tương ứng với vai trò (Role) của người dùng.
Luồng chính (Main Flow)	Quy trình đăng nhập hệ thống được thực hiện như sau: 1. Người dùng truy cập vào trang đăng nhập. 2. Người dùng nhập thông tin đăng nhập (Email hoặc Tên đăng nhập) và Mật khẩu. 3. Người dùng nhấn nút "Đăng nhập". 4. Hệ thống kiểm tra tính chính xác của thông tin đăng nhập và trạng thái hoạt động của tài khoản. 5. Hệ thống khởi tạo phiên làm việc (cấp mã token định danh) cho người dùng. 6. Hệ thống chuyển hướng người dùng đến giao diện trang chủ hoặc trang quản lý phù hợp với vai trò của họ.

Luồng thay thế (Alternative Flow)	Trong trường hợp người dùng lựa chọn phương thức thay thế, quy trình được thực hiện như sau: **Alt 1:** Đăng nhập bằng mã QR 1. Tại bước 2 của luồng chính, người dùng chọn phương thức "Đăng nhập bằng QR". 2. Hệ thống tạo và hiển thị mã QR tạm thời trên màn hình. 3. Người dùng sử dụng ứng dụng di động (đã đăng nhập) để quét mã QR và xác nhận đăng nhập. 4. Hệ thống ghi nhận xác nhận thành công từ thiết bị di động, kích hoạt phiên làm việc trên trình duyệt và tiếp tục từ bước 6 của luồng chính.
Luồng ngoại lệ (Exception Flow)	Trong quá trình xử lý, một số tình huống ngoại lệ có thể xảy ra như sau: - **Exc 1:** Thông tin đăng nhập không chính xác 1. Tại bước 4 luồng chính, hệ thống phát hiện sai tên đăng nhập/email hoặc mật khẩu. 2. Hệ thống hiển thị thông báo lỗi "Thông tin đăng nhập không chính xác" và yêu cầu người dùng nhập lại. - **Exc 2:** Tài khoản bị khóa/vô hiệu hóa 1. Tại bước 4 luồng chính, hệ thống phát hiện tài khoản đang bị khóa. 2. Hệ thống từ chối truy cập và hiển thị thông báo "Tài khoản của bạn đã bị vô hiệu hóa, vui lòng liên hệ quản trị viên". - **Exc 3:** Sự cố kết nối/Hệ thống lỗi 1. Tại bước 4 hoặc 5 luồng chính, hệ thống không thể xác thực do sự cố kỹ thuật (mất kết nối cơ sở dữ liệu/dịch vụ xác thực ngắt quãng). 2. Hệ thống hiển thị thông báo lỗi chung: "Hệ thống bận, vui lòng thử lại sau".

Sau khi người dùng đăng nhập thành công, hệ thống tiến hành truy xuất và hiển thị thông tin hồ sơ cá nhân tương ứng. Thông tin người dùng đóng vai trò là dữ liệu nền quan trọng, được sử dụng xuyên suốt trong các quy trình nghiệp vụ của hệ thống, đặc biệt là trong xử lý đơn hàng và tính toán chi phí vận chuyển. Việc đảm bảo tính chính xác và đầy đủ của dữ liệu cá nhân giúp hệ thống thực hiện các thao tác nghiệp vụ một

cách chính xác, đồng thời hạn chế các sai sót phát sinh trong quá trình giao nhận và vận hành.

Bảng 3.3 : Đặc tả Use Case "Cập nhật thông tin cá nhân"

Thuộc tính	**Chi tiết đặc tả**
Tên Use Case	Cập nhật thông tin cá nhân
Actor chính	Khách hàng (Customer)
Tiền điều kiện	Khách hàng đã đăng nhập thành công vào hệ thống.
Hậu điều kiện	Thông tin cá nhân mới được lưu lại vào hệ thống và áp dụng làm thông tin mặc định cho các tương tác/đơn hàng tiếp theo.
Luồng chính (Main Flow)	Quy trình cập nhật thông tin cá nhân được thực hiện như sau: 1. Khách hàng chọn chức năng "Hồ sơ của tôi". 2. Hệ thống hiển thị thông tin cá nhân hiện tại của khách hàng lên biểu mẫu. 3. Khách hàng chỉnh sửa các trường thông tin cho phép (Họ tên, Số điện thoại, Địa chỉ, Ngày sinh, Giới tính) và nhấn "Lưu thay đổi". 4. Hệ thống kiểm tra tính hợp lệ của dữ liệu mới và tiến hành cập nhật. 5. Hệ thống hiển thị thông báo "Cập nhật hồ sơ thành công" và làm mới giao diện với dữ liệu mới.
Luồng rẽ nhánh (Alternative Flow)	Trong trường hợp khách hàng muốn hủy thao tác, quy trình thực hiện như sau: **Alt 1:** Khách hàng hủy bỏ thao tác 1. Tại bước 3 luồng chính, khách hàng chọn "Hủy" hoặc thoát khỏi trang mà không nhấn "Lưu thay đổi". 2. Hệ thống không lưu các thông tin vừa chỉnh sửa, giữ nguyên dữ liệu cũ và quay về trang trước đó.
Luồng ngoại lệ (Exception Flow)	Trong quá trình xử lý, một số tình huống ngoại lệ có thể xảy ra như sau: **- Exc 1:** Phiên đăng nhập hết hạn 1. Tại bước 1 hoặc bước 3 luồng chính, hệ thống phát hiện phiên làm việc của khách hàng đã hết hạn.

	2. Hệ thống tự động đăng xuất, thông báo "Phiên đăng nhập đã hết hạn" và điều hướng người dùng về trang Đăng nhập. **- Exc 2:** Dữ liệu đầu vào sai quy chuẩn 1. Tại bước 4 luồng chính, hệ thống phát hiện dữ liệu không hợp lệ (Ví dụ: Số điện thoại sai định dạng, ngày sinh ở tương lai). 2. Hệ thống từ chối lưu, hiển thị thông báo lỗi chi tiết tại các trường dữ liệu sai và yêu cầu khách hàng sửa lại. **- Exc 3:** Lỗi lưu trữ hệ thống 1. Tại bước 4 luồng chính, hệ thống gặp sự cố kết nối dữ liệu không thể ghi nhận thay đổi. 2. Hệ thống hủy bỏ thao tác cập nhật, giữ nguyên dữ liệu trên biểu mẫu và hiển thị thông báo "Hệ thống bận, vui lòng thử lại sau".

Nhằm đảm bảo tính liên tục trong trải nghiệm người dùng đồng thời duy trì mức độ an toàn và bảo mật thông tin cá nhân, sau khi hoàn tất quá trình cập nhật thông tin cá nhân, hệ thống tiếp tục cung cấp chức năng Đổi mật khẩu như một bước nghiệp vụ kế tiếp trong chuỗi quản lý tài khoản. Chức năng này được thiết kế như một quy trình khép kín, trong đó mọi yêu cầu thay đổi mật khẩu đều bắt buộc phải trải qua bước xác thực lại danh tính của người dùng nhằm đảm bảo rằng chỉ chủ sở hữu hợp pháp của tài khoản mới có quyền thực hiện thao tác này.

Bảng 3.4 : Đặc tả Use Case “Đổi mật khẩu”

Thuộc tính	**Chi tiết đặc tả**
Tên Use Case	Đổi mật khẩu
Actor chính	Người dùng đã đăng nhập (Authenticated User).
Tiền điều kiện	Người dùng đang truy cập hệ thống và đã đăng nhập thành công.
Hậu điều kiện	Mật khẩu mới được cập nhật thành công vào hệ thống. Phiên làm việc hiện tại bị hủy bỏ (đăng xuất tự động) trên các thiết bị để đảm bảo an toàn bảo mật.
Luồng chính (Main Flow)	Quy trình đổi mật khẩu được thực hiện như sau: 1. Người dùng chọn chức năng "Đổi mật khẩu" tại trang quản lý tài khoản.

	2. Hệ thống hiển thị biểu mẫu yêu cầu nhập các thông tin: Mật khẩu hiện tại, Mật khẩu mới, Nhập lại mật khẩu mới. 3. Người dùng nhập đầy đủ thông tin và nhấn nút "Xác nhận". 4. Hệ thống kiểm tra tính chính xác của mật khẩu hiện tại, độ trùng khớp của mật khẩu mới và độ mạnh của mật khẩu mới. 5. Hệ thống tiến hành cập nhật mật khẩu mới vào cơ sở dữ liệu. 6. Hệ thống thực hiện đăng xuất tài khoản, hiển thị thông báo "Đổi mật khẩu thành công" và điều hướng người dùng về trang Đăng nhập để đăng nhập lại bằng mật khẩu mới.
Luồng thay thế (Alternative Flow)	Trong trường hợp người dùng muốn hủy thao tác, quy trình thực hiện như sau: **Alt 1:** Người dùng hủy bỏ thao tác 1. Tại bước 3 luồng chính, người dùng chọn "Hủy" hoặc thoát khỏi trang. 2. Hệ thống hủy bỏ yêu cầu thay đổi, giữ nguyên mật khẩu cũ và quay về trang trước đó.
Luồng ngoại lệ (Exception Flow)	Trong quá trình xử lý, một số tình huống ngoại lệ có thể xảy ra như sau: - **Exc 1:** Nhập lại mật khẩu mới không trùng khớp 1. Tại bước 4 luồng chính, hệ thống phát hiện mật khẩu mới và trường nhập lại không giống nhau. 2. Hệ thống từ chối xử lý, hiển thị cảnh báo lỗi tại trường dữ liệu tương ứng và yêu cầu người dùng nhập lại. - **Exc 2:** Mật khẩu hiện tại không chính xác 1. Tại bước 4 luồng chính, hệ thống xác định mật khẩu cũ người dùng vừa nhập không đúng. 2. Hệ thống từ chối cập nhật, hiển thị thông báo lỗi "Mật khẩu hiện tại không chính xác" và giữ nguyên giao diện để người dùng thử lại. - **Exc 3:** Mật khẩu mới không đạt độ phức tạp (Mật khẩu yếu) 1. Tại bước 4 luồng chính, hệ thống phát hiện mật khẩu mới không đáp ứng tiêu chuẩn an toàn (ví dụ: quá ngắn, thiếu ký tự đặc biệt). 2. Hệ thống hiển thị hướng dẫn quy chuẩn mật khẩu mạnh và yêu cầu người dùng thay đổi lại.

	- Exc 4: Lỗi kết nối hệ thống 1. Tại bước 5 luồng chính, hệ thống gặp sự cố kỹ thuật không thể ghi nhận dữ liệu mới. 2. Hệ thống hủy giao dịch đổi mật khẩu và thông báo lỗi: "Hệ thống bận, vui lòng thử lại sau".

Luồng Mua hàng (Checkout) là thành phần trung tâm trong toàn bộ quy trình giao dịch của hệ thống, nơi các dịch vụ như giỏ hàng, hồ sơ khách hàng, vận chuyển và quản lý tồn kho được phối hợp để hình thành đơn hàng. Đây là bước then chốt quyết định việc khởi tạo giao dịch thanh toán, đồng thời đảm bảo dữ liệu được nhất quán và toàn vẹn trong suốt quá trình xử lý.

Bảng 3.5 : Đặc tả Use Case “Mua hàng”

Thuộc tính	**Chi tiết đặc tả**
Tên Use Case	Mua hàng (Checkout)
Actor chính	Khách hàng (Customer)
Tiền điều kiện	Khách hàng đã đăng nhập và có ít nhất một sản phẩm còn hàng trong giỏ hàng.
Hậu điều kiện	Một đơn hàng mới được tạo thành công sẽ được ghi nhận ở trạng thái “Chờ xử lý” đối với phương thức thanh toán COD hoặc “Chờ thanh toán” đối với phương thức VNPay. Đồng thời, số lượng sản phẩm tương ứng sẽ được trừ khỏi giỏ hàng của người dùng và tạm thời khấu trừ vào số lượng tồn kho thực tế của sản phẩm nhằm đảm bảo tính chính xác trong quản lý số lượng hàng hóa.
Luồng chính (Main Flow)	Quy trình mua hàng được thực hiện như sau: 1. Khách hàng tại trang Giỏ hàng nhấn nút "Tiến hành thanh toán". 2. Hệ thống chuyển hướng sang giao diện Checkout và tự động hiển thị thông tin người nhận (Họ tên, Số điện thoại, Địa chỉ) lấy từ hồ sơ khách hàng. 3. Hệ thống tự động tính toán phí vận chuyển dựa trên địa chỉ nhận hàng của khách hàng. 4. Hệ thống hiển thị bảng tóm tắt chi phí bao gồm: Tiền hàng, Phí vận chuyển và Tổng số tiền cần thanh toán.

	5. Khách hàng kiểm tra lại thông tin và lựa chọn phương thức thanh toán là "Thanh toán khi nhận hàng (COD)". 6. Khách hàng nhấn nút "Đặt hàng ngay". 7. Hệ thống kiểm tra số lượng tồn kho lần cuối, tiến hành trừ sản phẩm trong giỏ hàng và tạo đơn hàng mới. 8. Hệ thống hiển thị giao diện "Đặt hàng thành công" cùng mã đơn hàng để Khách hàng theo dõi.
Luồng thay thế (Alternative Flow)	Trong quá trình mua hàng, một số luồng thay thế có thể xảy ra như sau: - **Alt 1:** Khách hàng thay đổi thông tin nhận hàng 1. Tại bước 2 luồng chính, khách hàng chỉnh sửa trực tiếp thông tin người nhận hoặc địa chỉ giao hàng trên biểu mẫu. 2. Hệ thống ghi nhận địa chỉ mới và thực hiện lại bước 3 luồng chính để cập nhật lại phí vận chuyển. - **Alt 2:** Khách hàng áp dụng mã giảm giá (Coupon) 1. Tại bước 3 luồng chính, khách hàng nhập mã giảm giá và nhấn áp dụng. 2. Hệ thống kiểm tra tính hợp lệ của mã. Nếu hợp lệ, hệ thống tính toán lại số tiền giảm trừ và cập nhật lại Tổng thanh toán ở bước 4. - **Alt 3:** Khách hàng chọn thanh toán điện tử (VNPay) 1. Tại bước 5 luồng chính, khách hàng lựa chọn phương thức thanh toán qua VNPay. 2. Sau khi khách hàng nhấn "Đặt hàng", hệ thống vẫn thực hiện bước 7 (trừ tồn kho, tạo đơn hàng với trạng thái "Chờ thanh toán"). Trong trường hợp cần thiết, hệ thống sẽ thực hiện lại quy trình của bước 7 để đảm bảo trạng thái đơn hàng được cập nhật chính xác. 3. Hệ thống điều hướng khách hàng sang cổng thanh toán VNPay thay vì hiển thị ngay trang thành công.
Luồng ngoại lệ (Exception Flow)	Trong quá trình xử lý, một số tình huống ngoại lệ có thể xảy ra như sau: - **Exc 1:** Hệ thống tính phí vận chuyển của bên thứ ba gặp sự cố 1. Tại bước 3 luồng chính, hệ thống không thể kết nối tới dịch vụ vận chuyển để tính phí.

	2. Hệ thống tự động áp dụng mức phí vận chuyển đồng giá dự phòng (Fallback) để khách hàng tiếp tục mua sắm mà không bị ngắt quãng. - **Exc 2:** Mã giảm giá không hợp lệ 1. Tại bước 2 của Alt 2 (Luồng rẽ nhánh áp dụng mã), hệ thống phát hiện mã hết hạn hoặc không đủ điều kiện. 2. Hệ thống hiển thị thông báo lỗi "Mã giảm giá không hợp lệ" và giữ nguyên mức giá ban đầu. - **Exc 3:** Sản phẩm hết hàng vào phút chót (Hết hàng thời gian thực) 1. Tại bước 7 luồng chính, hệ thống phát hiện sản phẩm đã bị người khác mua hết ngay trước khi khách hàng bấm nút Đặt hàng. 2. Hệ thống hủy bỏ giao dịch đặt hàng, trả về thông báo lỗi "Sản phẩm [Tên sản phẩm] đã hết hàng. Vui lòng cập nhật lại giỏ hàng" và đưa người dùng quay lại trang Giỏ hàng để điều chỉnh. - **Exc 4:** Giao dịch thanh toán trực tuyến thất bại, bị hủy hoặc quá hạn 1. Trong quá trình người dùng thực hiện thanh toán thông qua cổng VNPay sau khi hoàn tất bước xác nhận đặt hàng và chuyển sang giai đoạn thanh toán, hệ thống có thể ghi nhận một số tình huống phát sinh như giao dịch không thành công từ phía VNPay, người dùng chủ động hủy thanh toán, hoặc không nhận được phản hồi xác nhận trong khoảng thời gian quy định, ví dụ 15 phút kể từ thời điểm đơn hàng được tạo và chuyển sang trạng thái chờ thanh toán. 2. Khi một trong các trường hợp trên xảy ra, hệ thống sẽ cập nhật trạng thái đơn hàng sang “đã hủy” để đảm bảo dữ liệu phản ánh đúng tình trạng thực tế của giao dịch. Việc cập nhật này giúp tránh tình trạng đơn hàng bị treo ở trạng thái chờ thanh toán trong khi giao dịch thực tế đã không còn hiệu lực. 3. Song song với việc cập nhật trạng thái đơn hàng, hệ thống sẽ thực hiện hoàn trả số lượng sản phẩm đã được tạm giữ trong đơn về lại kho hàng. Cơ chế này đảm bảo tính chính xác của tồn kho và giúp sản phẩm sớm được đưa trở lại trạng thái sẵn sàng bán cho các đơn hàng tiếp theo. Sau khi hoàn tất quá trình xử lý, hệ thống gửi thông báo đến

	người dùng để xác nhận rằng đơn hàng đã được hủy do thanh toán không thành công hoặc vượt quá thời gian xử lý cho phép.

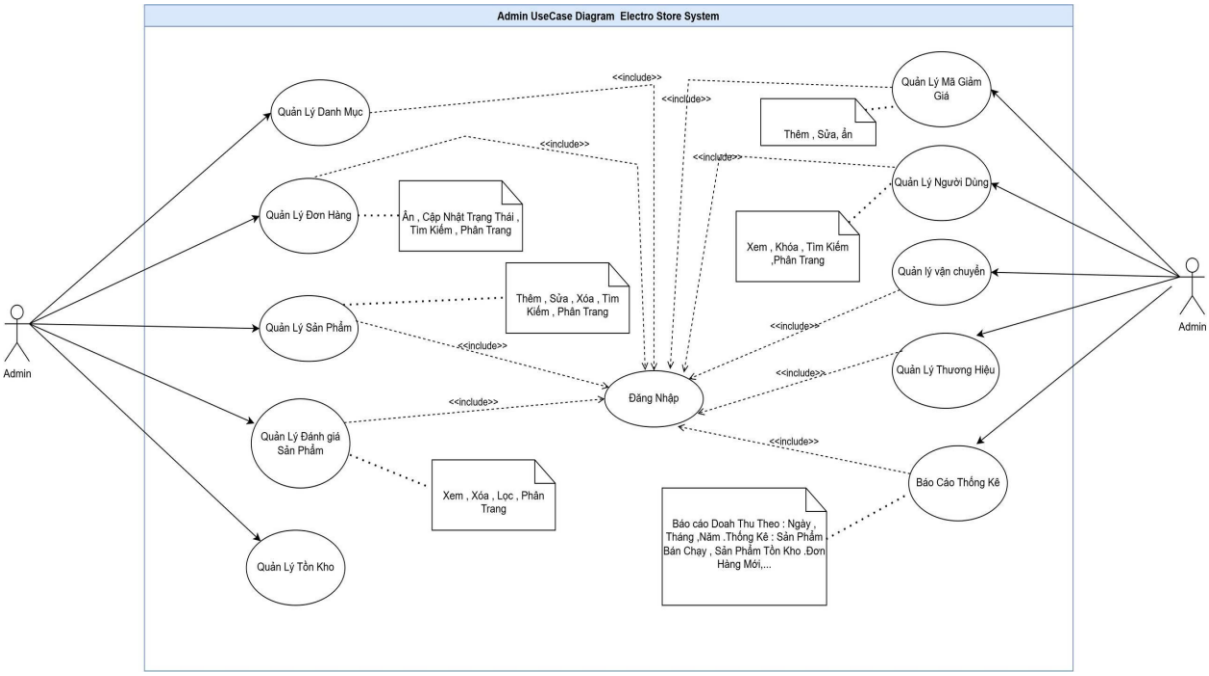
Luồng thanh toán trực tuyến là minh chứng tiêu biểu cho cơ chế tích hợp dịch vụ bên ngoài (external service integration) và đảm bảo tính toàn vẹn dữ liệu thông qua cơ chế xác thực chữ ký HMAC trong hệ thống.

Bảng 3.6 : Đặc tả Use Case “Thanh toán”

Thuộc tính	**Chi tiết đặc tả**
Tên Use Case	Thanh toán trực tuyến
Actor chính	Khách hàng (Customer), Cổng thanh toán (VNPay)
Tiền điều kiện	Đơn hàng đã được tạo thành công ở trạng thái "Chờ xử lý". Khách hàng lựa chọn phương thức thanh toán qua VNPay.
Hậu điều kiện	Trạng thái thanh toán của Đơn hàng được cập nhật thành "Đã thanh toán" (PAID). Đơn hàng đủ điều kiện chuyển sang giai đoạn chuẩn bị hàng và giao nhận.
Luồng chính (Main Flow)	Quy trình thanh toán trực tuyến được thực hiện như sau: 1. Hệ thống tự động khởi tạo thông tin giao dịch trực tuyến và tạo liên kết (URL) thanh toán tương ứng với đơn hàng. 2. Hệ thống chuyển hướng Khách hàng sang giao diện của Cổng thanh toán VNPay. 3. Tại giao diện VNPay, Khách hàng lựa chọn hình thức thanh toán (Quét mã QR, Thẻ nội địa, Thẻ quốc tế) và hoàn tất các thủ tục xác thực bảo mật của ngân hàng. 4. VNPay xử lý trừ tiền thành công, tiến hành gửi thông báo xác nhận kết quả thanh toán trực tiếp về cho hệ thống và đồng thời chuyển hướng Khách hàng quay lại website. 5. Hệ thống tiếp nhận thông báo từ VNPay, thực hiện xác thực tính toàn vẹn và hợp lệ của dữ liệu giao dịch. 6. Xác minh thành công, hệ thống cập nhật trạng thái đơn hàng thành "Đã thanh toán". 7. Hệ thống hiển thị giao diện "Thanh toán thành công" kèm theo thông tin biên lai điện tử cho Khách hàng.

Luồng thay thế (Alternative Flow)	Use Case này tập trung hoàn toàn vào luồng thanh toán điện tử trực tiếp, không có luồng rẽ nhánh nghiệp vụ khác.
Luồng ngoại lệ (Exception Flow)	Trong quá trình xử lý, một số tình huống ngoại lệ có thể xảy ra như sau: - **Exc 1:** Khách hàng chủ động hủy giao dịch 1. Tại bước 3 luồng chính, Khách hàng đổi ý và nhấn nút "Hủy thanh toán" trên giao diện VNPay. 2. VNPay gửi thông báo hủy về hệ thống và chuyển hướng người dùng quay lại website. 3. Hệ thống giữ nguyên trạng thái đơn hàng là "Chờ xử lý", thông báo "Thanh toán bị hủy" và lưu đơn hàng vào danh sách để Khách hàng có thể thực hiện thanh toán lại sau. - **Exc 2:** Giao dịch thất bại từ phía Ngân hàng (Tài khoản không đủ tiền, lỗi OTP...) 1. Tại bước 3 luồng chính, quá trình xác thực ngân hàng thất bại (ví dụ: thẻ không đủ số dư, nhập sai mã OTP, lỗi kết nối ngân hàng). 2. VNPay gửi thông báo giao dịch thất bại về hệ thống. 3. Hệ thống thông báo lỗi chi tiết đến Khách hàng, giữ nguyên trạng thái đơn hàng chưa thanh toán và hiển thị tùy chọn "Thanh toán lại". - **Exc 3:** Xác thực dữ liệu giao dịch thất bại (Cảnh báo bảo mật) 1. Tại bước 5 luồng chính, hệ thống phát hiện dữ liệu phản hồi từ cổng thanh toán có dấu hiệu bị can thiệp hoặc sai lệch thông tin chữ ký bảo mật. 2. Hệ thống lập tức từ chối ghi nhận giao dịch, ghi nhận log cảnh báo rủi ro hệ thống và không cập nhật trạng thái đơn hàng sang "Đã thanh toán" để bảo vệ an toàn tài chính.

3.3.2.2. Phân hệ các Usecase dành cho Quản trị hệ thống



Hình 3.15 : Lược đồ Use Case của Admin

Sau khi đơn hàng được thanh toán thành công, hệ thống chuyển sang giai đoạn xử lý và vận hành logistics. Phân hệ quản lý đơn hàng cho phép quản trị viên và nhân viên bán hàng theo dõi, xác nhận và điều phối trạng thái đơn hàng trong toàn bộ vòng đời từ đóng gói đến giao nhận.

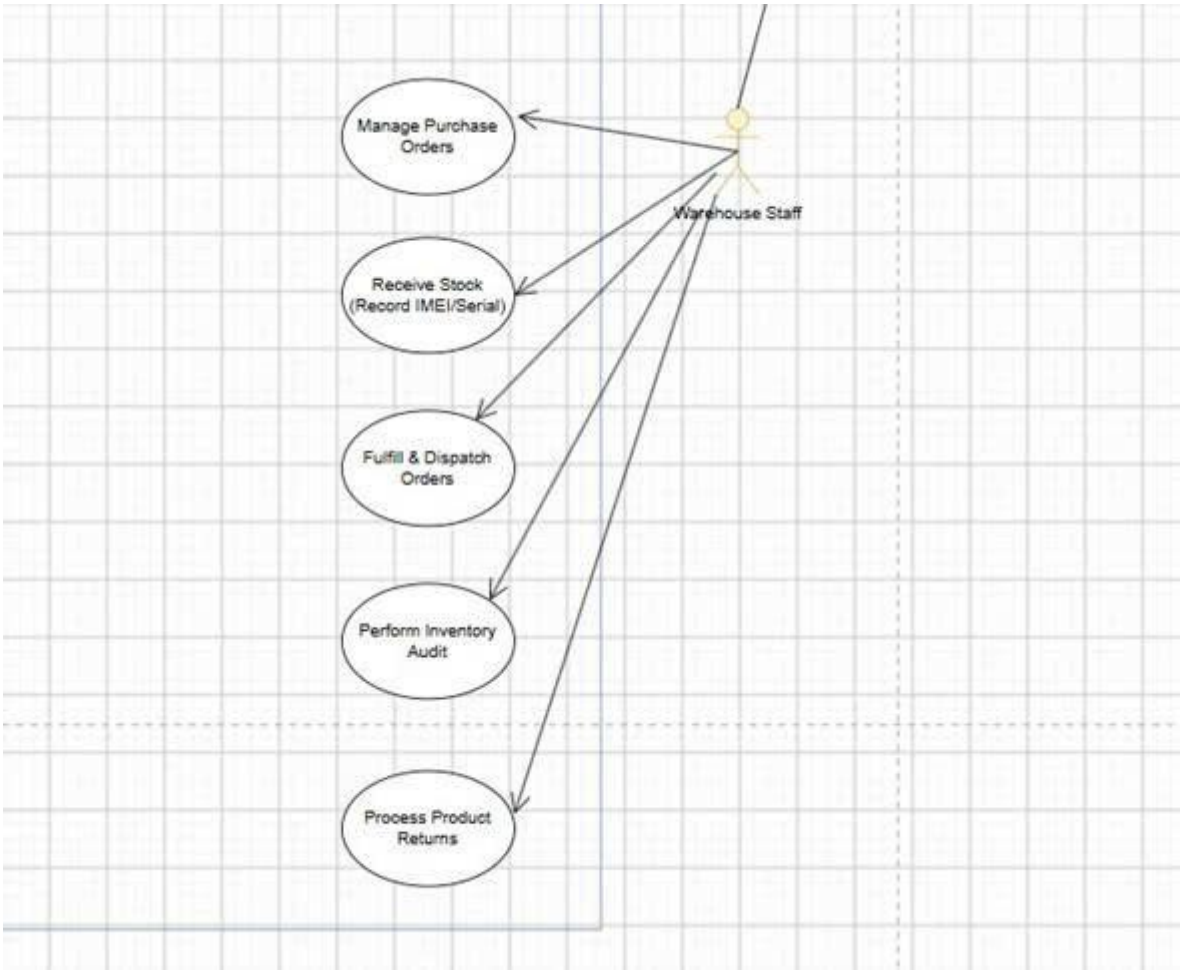
Bảng 3.7 : Đặc tả Use Case "Admin quản lý đơn hàng"

Thuộc tính	**Chi tiết đặc tả**
Tên Use Case	Admin quản lý đơn hàng
Actor chính	Quản trị viên (Admin), Nhân viên Bán hàng (Sales Staff).
Tiền điều kiện	Người dùng đã đăng nhập vào hệ thống quản trị và có quyền quản lý đơn hàng.
Hậu điều kiện	Trạng thái đơn hàng được cập nhật sang bước tiếp theo trong quy trình xử lý. Thông tin trạng thái mới được đồng bộ hiển thị cho khách hàng. Hệ thống ghi nhận mã IMEI/Serial của sản phẩm (nếu có) để phục vụ bảo hành.
Luồng chính (Main Flow)	Quy trình quản lý đơn hàng được thực hiện như sau:

	1. Admin truy cập vào mục "Quản lý Đơn hàng" trên trang quản trị Dashboard. 2. Hệ thống kiểm tra quyền hạn và hiển thị danh sách các đơn hàng hiện có dưới dạng phân trang. 3. Admin nhấp chọn một đơn hàng đang ở trạng thái "Chờ xác nhận" để xem thông tin chi tiết. 4. Admin kiểm tra các thông tin liên quan (Thông tin thanh toán, địa chỉ giao hàng) 5. Admin nhấn nút "Xác nhận đơn hàng" để chuyển trạng thái sang "Đang đóng gói". 6. Hệ thống tiến hành cập nhật trạng thái đơn hàng, đồng thời gửi thông tin thông báo tới bộ phận kho để chuẩn bị đóng gói hàng hóa. 7. Hệ thống làm mới giao diện hiển thị, thông báo cập nhật thành công và gắn nhãn trạng thái mới cho đơn hàng.
Luồng thay thế (Alternative Flow)	Trong quá trình quản lý, một số luồng thay thế có thể xảy ra như sau: - **Alt 1:** Gán mã định danh sản phẩm (IMEI/Serial) để kích hoạt bảo hành 1. Tại bước 4 luồng chính, trong quá trình soạn hàng xuất kho, Admin thực hiện nhập hoặc quét mã IMEI/Serial của từng thiết bị vào hệ thống. 2. Hệ thống ghi nhận các mã này vào thông tin chi tiết đơn hàng để tự động kích hoạt chế độ bảo hành điện tử khi đơn hàng hoàn tất. - **Alt 2:** Tìm kiếm và bộ lọc đơn hàng 1. Tại bước 2 luồng chính, Admin sử dụng thanh công cụ để tìm kiếm đơn hàng theo Số điện thoại khách hàng hoặc lọc nhanh theo trạng thái (Ví dụ: "Đang giao", "Chờ xác nhận"). 2. Hệ thống hiển thị danh sách các đơn hàng khớp với điều kiện lọc để Admin tiện theo dõi tiến độ.
Luồng ngoại lệ (Exception Flow)	Trong quá trình xử lý, một số tình huống ngoại lệ có thể xảy ra như sau: - **Exc 1:** Phiên làm việc của Admin hết hạn (Lỗi Token)

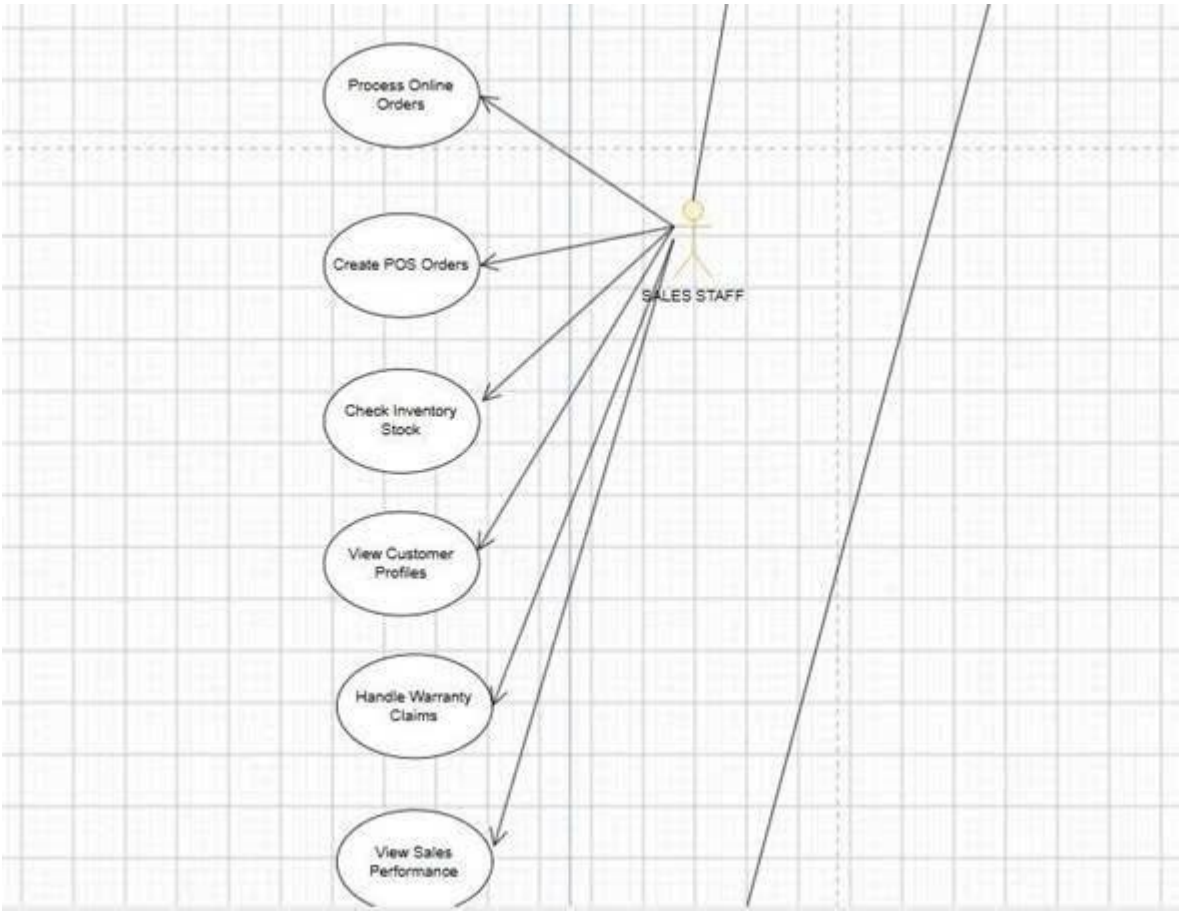
	1. Tại bước 2 hoặc bước 6 luồng chính, hệ thống phát hiện phiên đăng nhập bảo mật của Admin đã hết hiệu lực. 2. Hệ thống từ chối thực hiện thao tác, tự động đăng xuất và yêu cầu Admin đăng nhập lại. - **Exc 2:** Hủy đơn hàng do sự cố hoặc gian lận 1. Tại bước 4 luồng chính, Admin phát hiện thông tin khách hàng có dấu hiệu gian lận hoặc kho báo mất mát/hết hàng vật lý. 2. Admin chọn chức năng "Hủy đơn hàng" và bắt buộc phải nhập lý do hủy vào biểu mẫu. 3. Hệ thống chuyển trạng thái đơn hàng sang "Đã hủy", đồng thời kích hoạt quy trình hoàn tiền tự động (nếu trước đó khách hàng đã thanh toán trực tuyến qua VNPay). - **Exc 3:** Xung đột trạng thái đơn hàng (Thao tác đồng thời) 1. Tại bước 5 luồng chính, Admin nhấn xác nhận đơn hàng, nhưng thực tế đơn hàng này đã được một Admin khác xử lý trước đó vài giây. 2. Hệ thống từ chối cập nhật, hiển thị thông báo cảnh báo: "Đơn hàng đã thay đổi trạng thái bởi người dùng khác, vui lòng tải lại trang" để tránh trùng lặp thao tác.

3.3.2.3. Phân hệ các Usecase dành cho Nhân Viên Kho



Hình 3.16 : Lược đồ Use Case của nhân viên quản lý kho

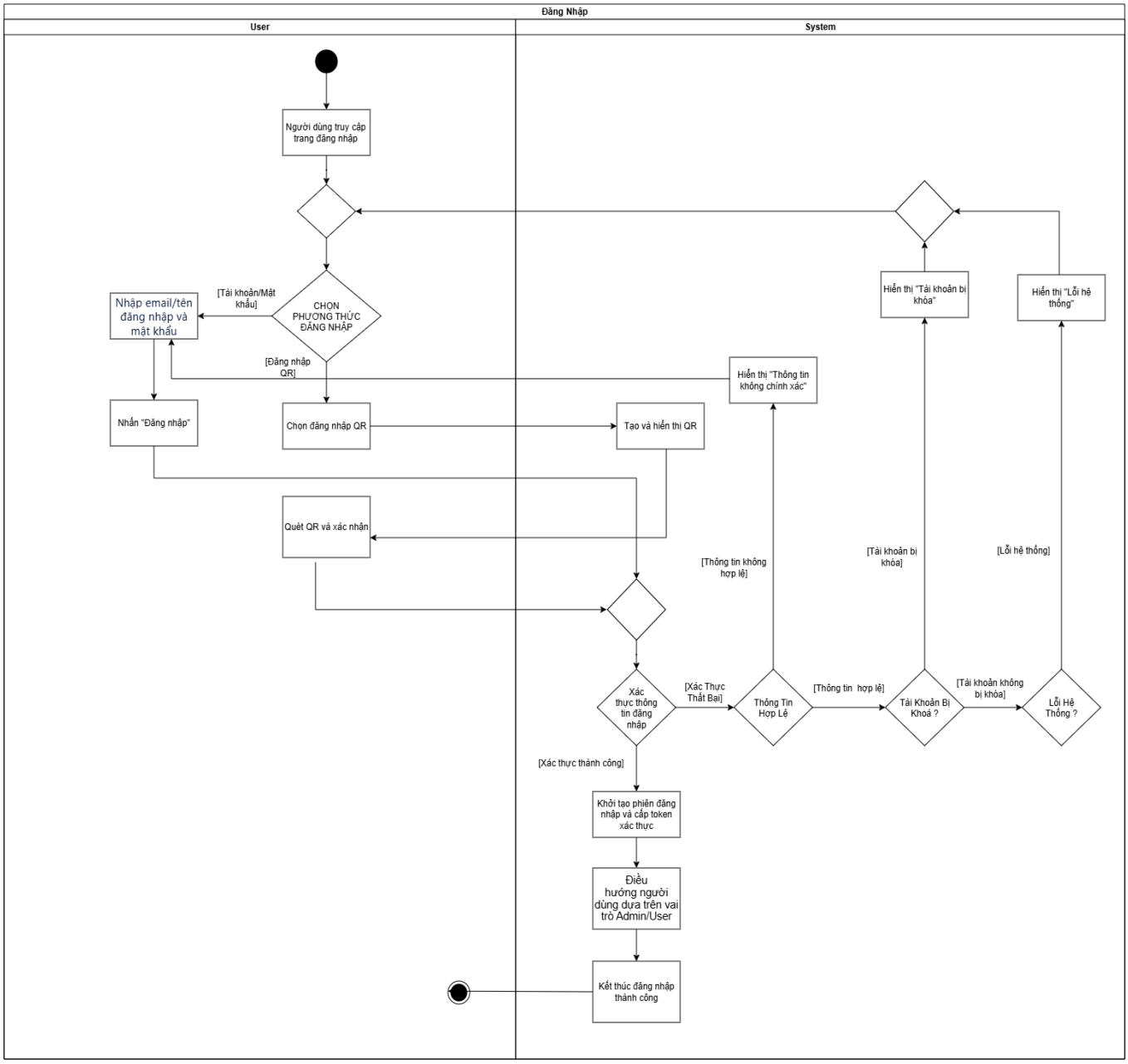
3.3.2.4. Phân hệ các Usecase dành cho Nhân Viên Sales



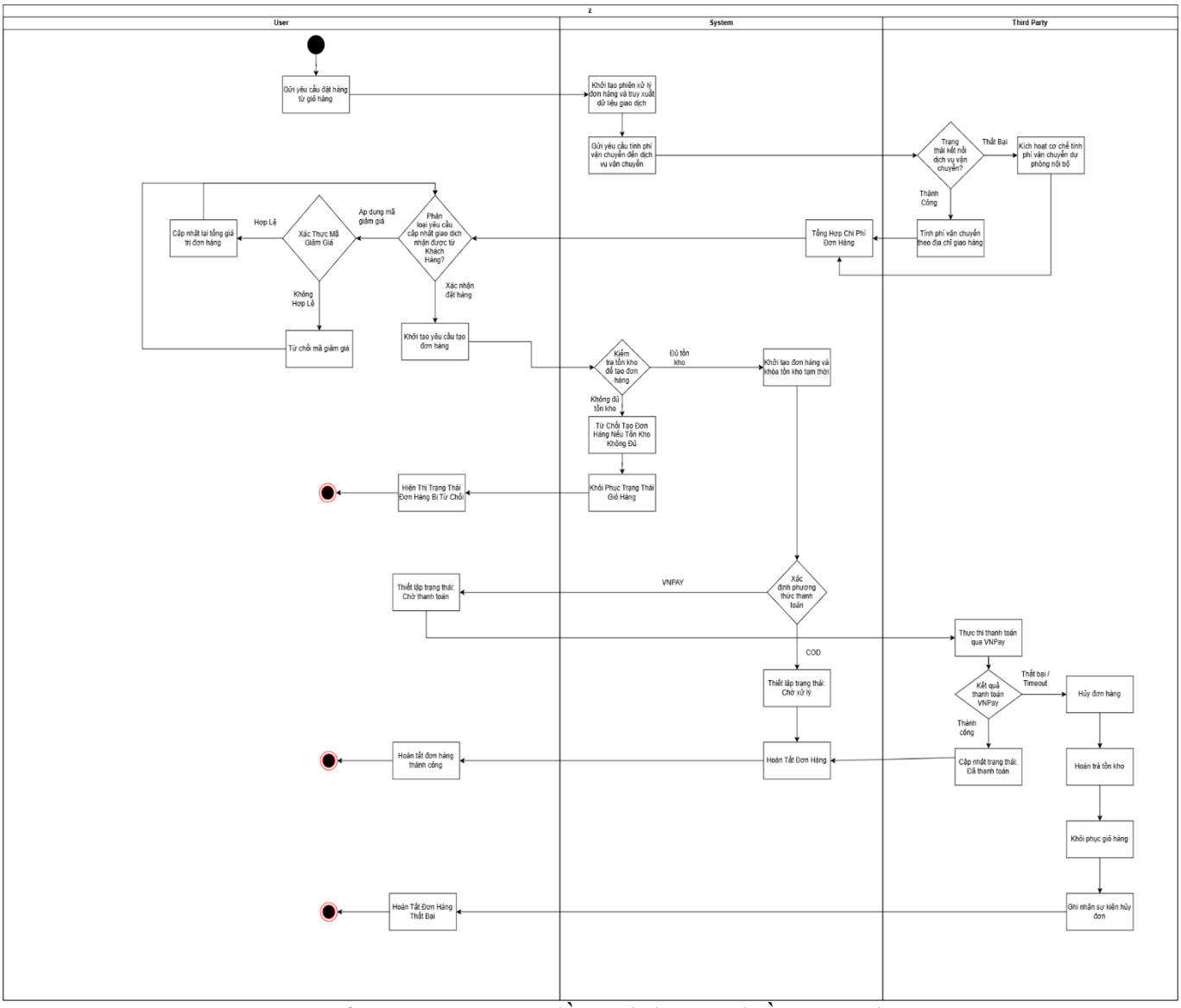
Hình 3.17 : Lược đồ Use Case của nhân viên chăm sóc khách hàng

3.3.3. Lược đồ Activity

Lược đồ Activity của chức năng đăng nhập mô tả luồng xử lý giữa người dùng và hệ thống, từ khi người dùng nhập thông tin xác thực đến khi hệ thống kiểm tra tài khoản, cấp token, lưu phiên đăng nhập và điều hướng đến giao diện phù hợp với vai trò.



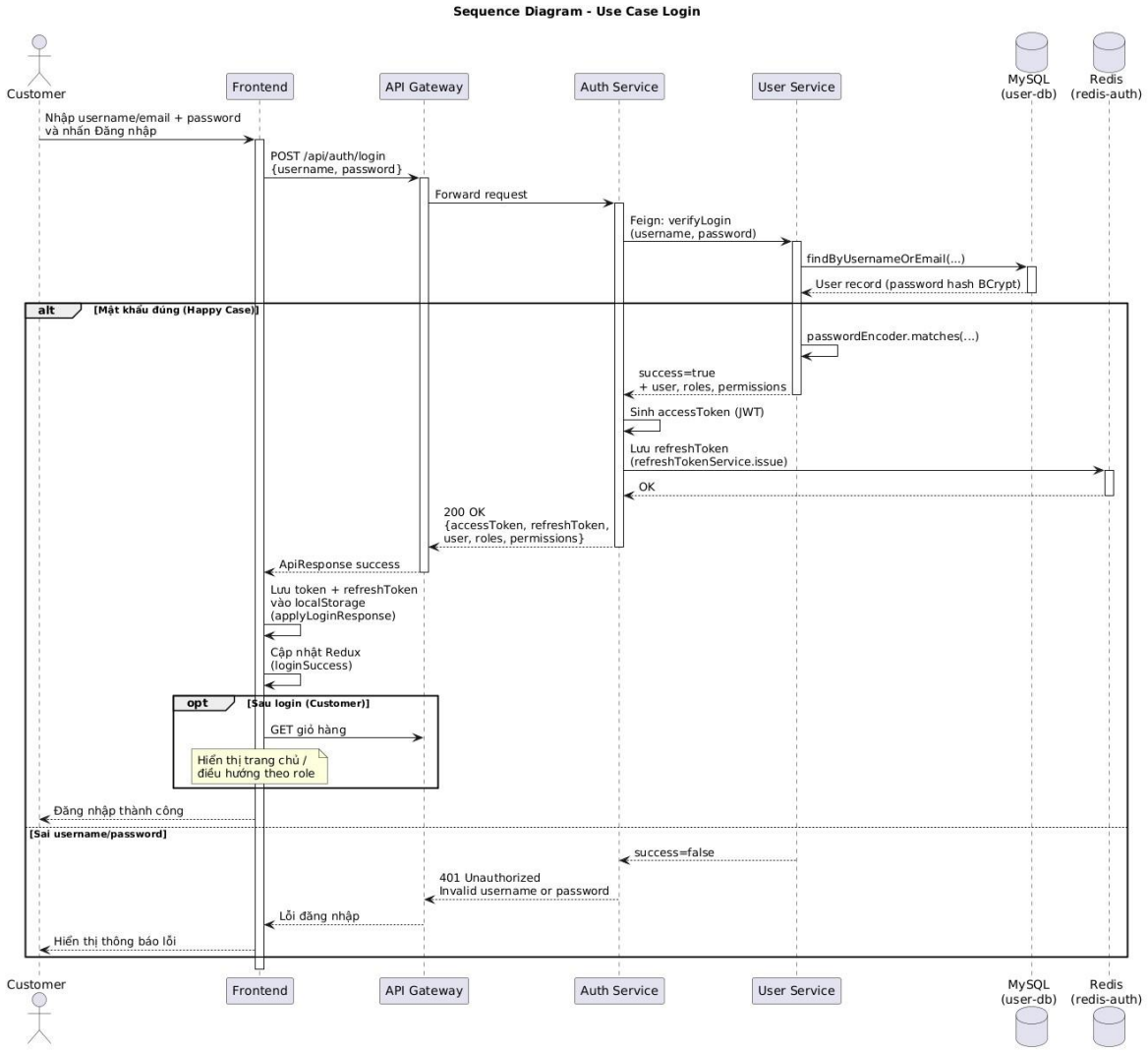
Hình 3.18 : Lược đồ Activity của chức năng đăng nhập



Hình 3.19 : Lược đồ Activity của luồng mua hàng

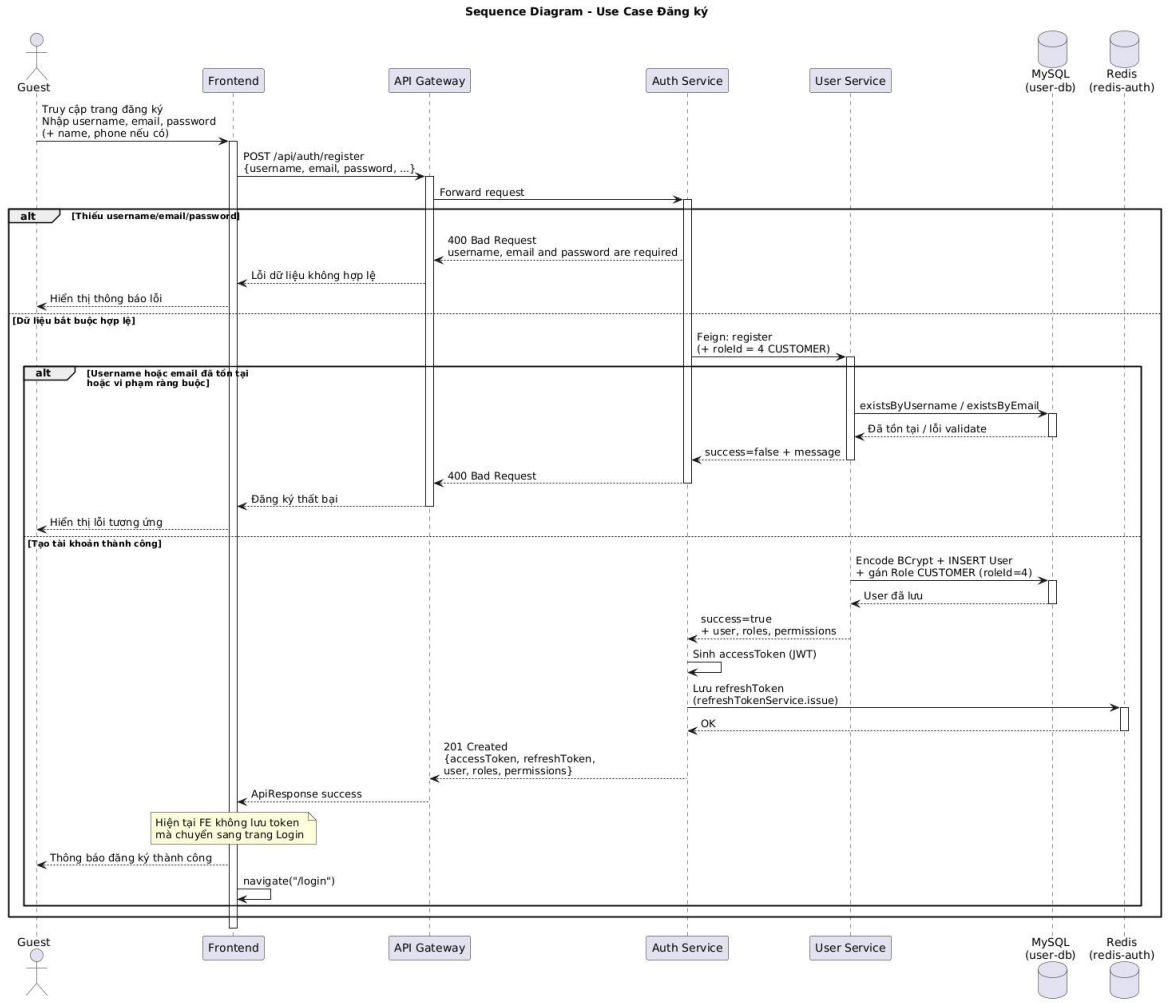
3.3.4. Lược đồ Sequence

Lược đồ Sequence của chức năng đăng nhập mô tả trình tự tương tác giữa khách hàng, Frontend, API Gateway, Auth Service, User Service, MySQL và Redis trong quá trình xác thực tài khoản, cấp token và phản hồi kết quả đăng nhập.



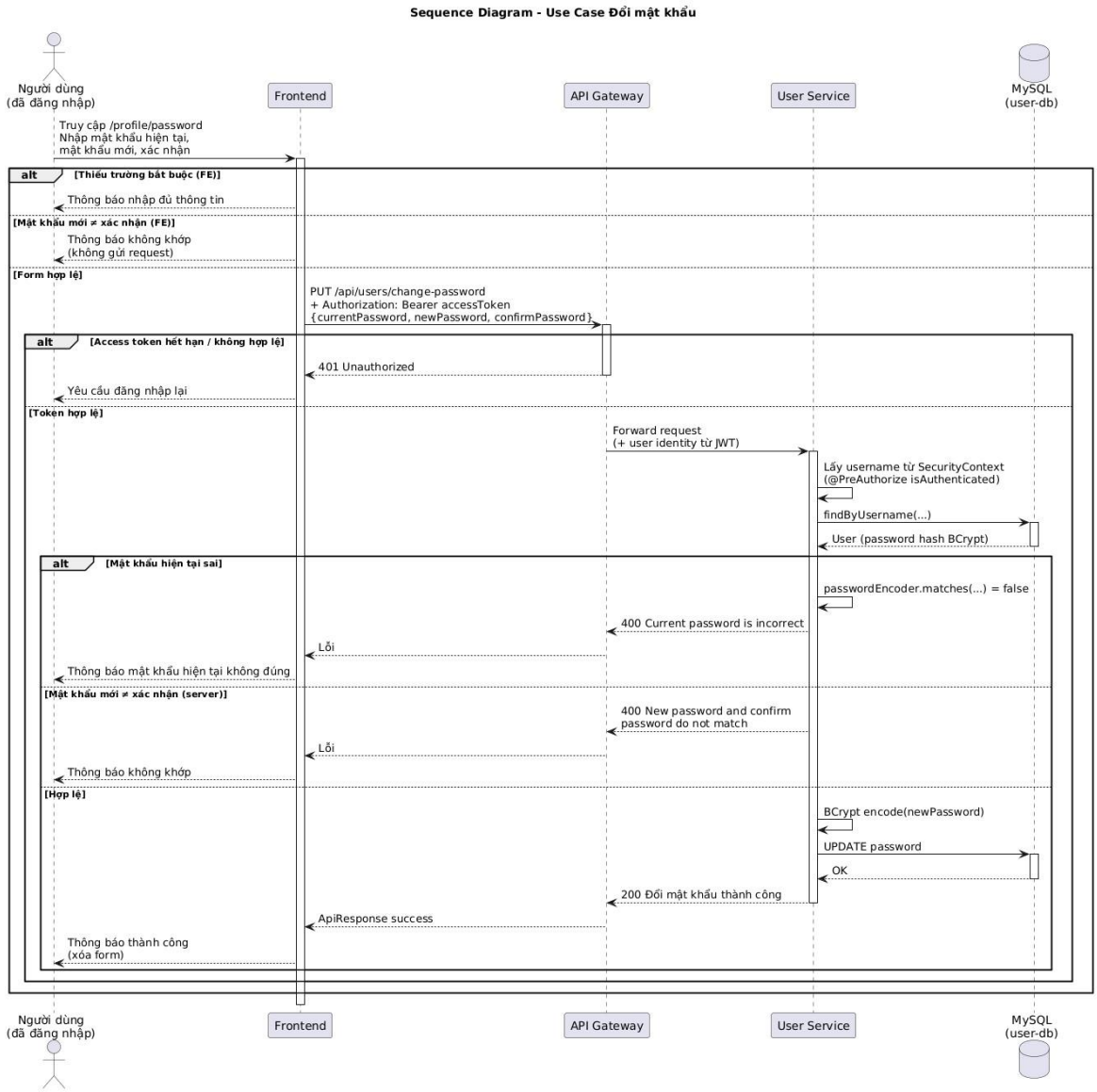
Hình 3.20 : Lược đồ Sequence của chức năng đăng nhập

Lược đồ Sequence của chức năng đăng ký mô tả quá trình Guest gửi thông tin tạo tài khoản qua Frontend và API Gateway; hệ thống kiểm tra dữ liệu, xác minh username hoặc email, tạo người dùng với vai trò Customer, cấp token và phản hồi kết quả đăng ký.



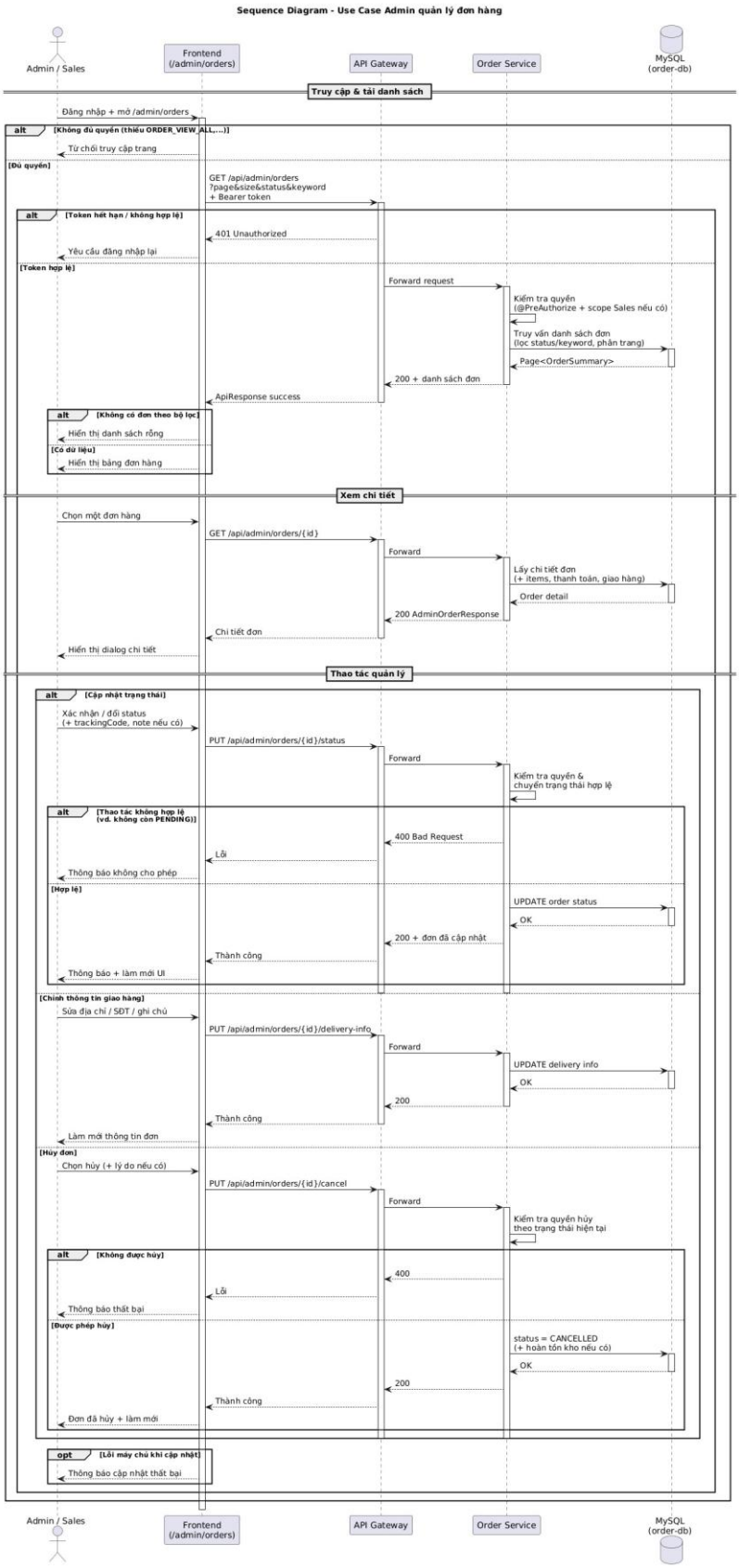
Hình 3.21 : Lược đồ Sequence của chức năng đăng ký

Lược đồ Sequence của chức năng đổi mật khẩu mô tả quá trình người dùng nhập mật khẩu hiện tại và mật khẩu mới; Frontend kiểm tra dữ liệu, API Gateway xác thực access token, User Service kiểm tra mật khẩu hiện tại, mã hóa mật khẩu mới và cập nhật dữ liệu trên MySQL.



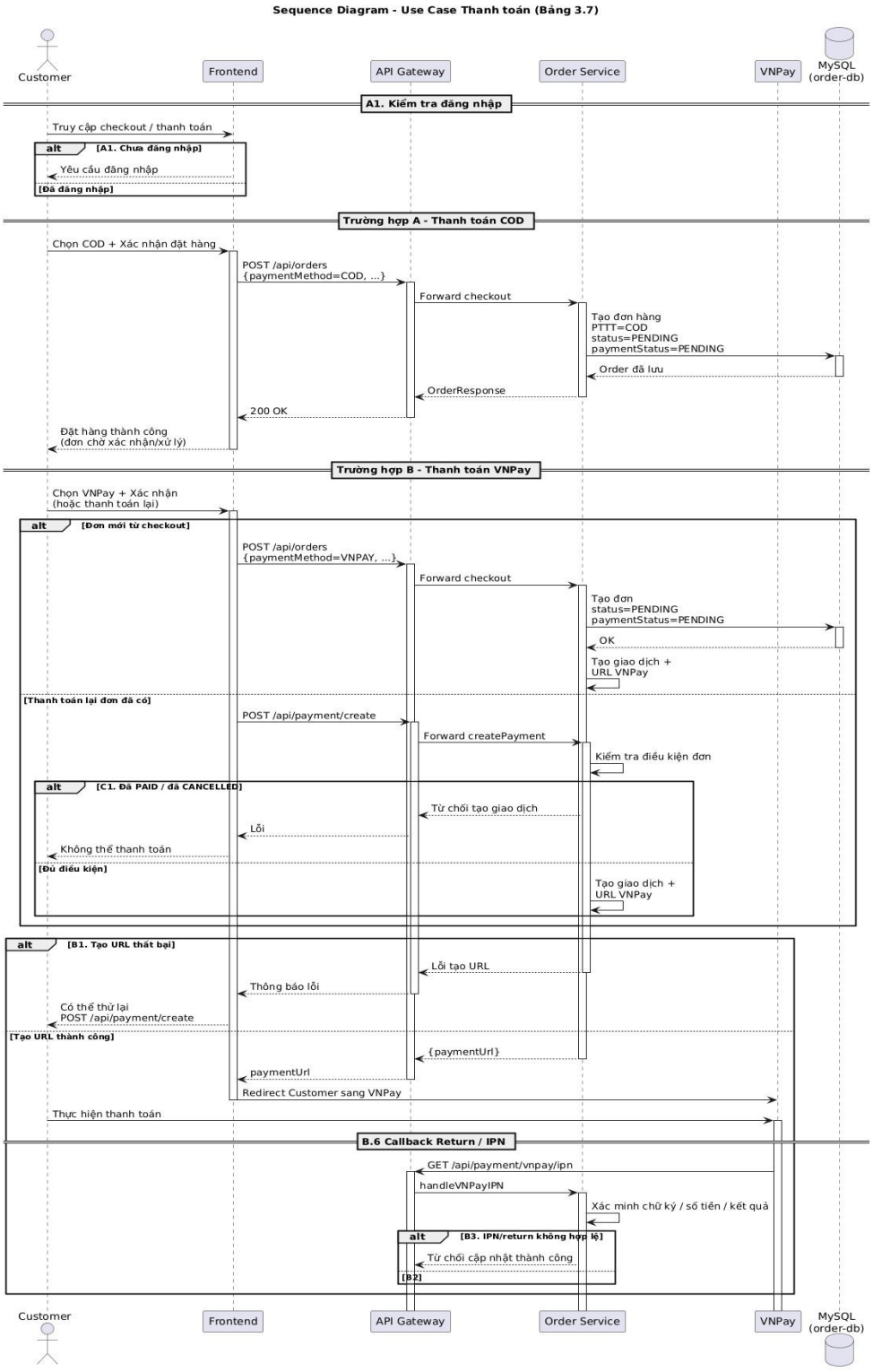
Hình 3.22 : Lược đồ Sequence của chức năng đổi mật khẩu

Lược đồ Sequence của chức năng quản lý đơn hàng mô tả quá trình Admin hoặc Sales tải danh sách đơn hàng, xem chi tiết và thực hiện các thao tác nghiệp vụ như cập nhật trạng thái, chỉnh sửa thông tin giao hàng hoặc hủy đơn thông qua API Gateway và Order Service.



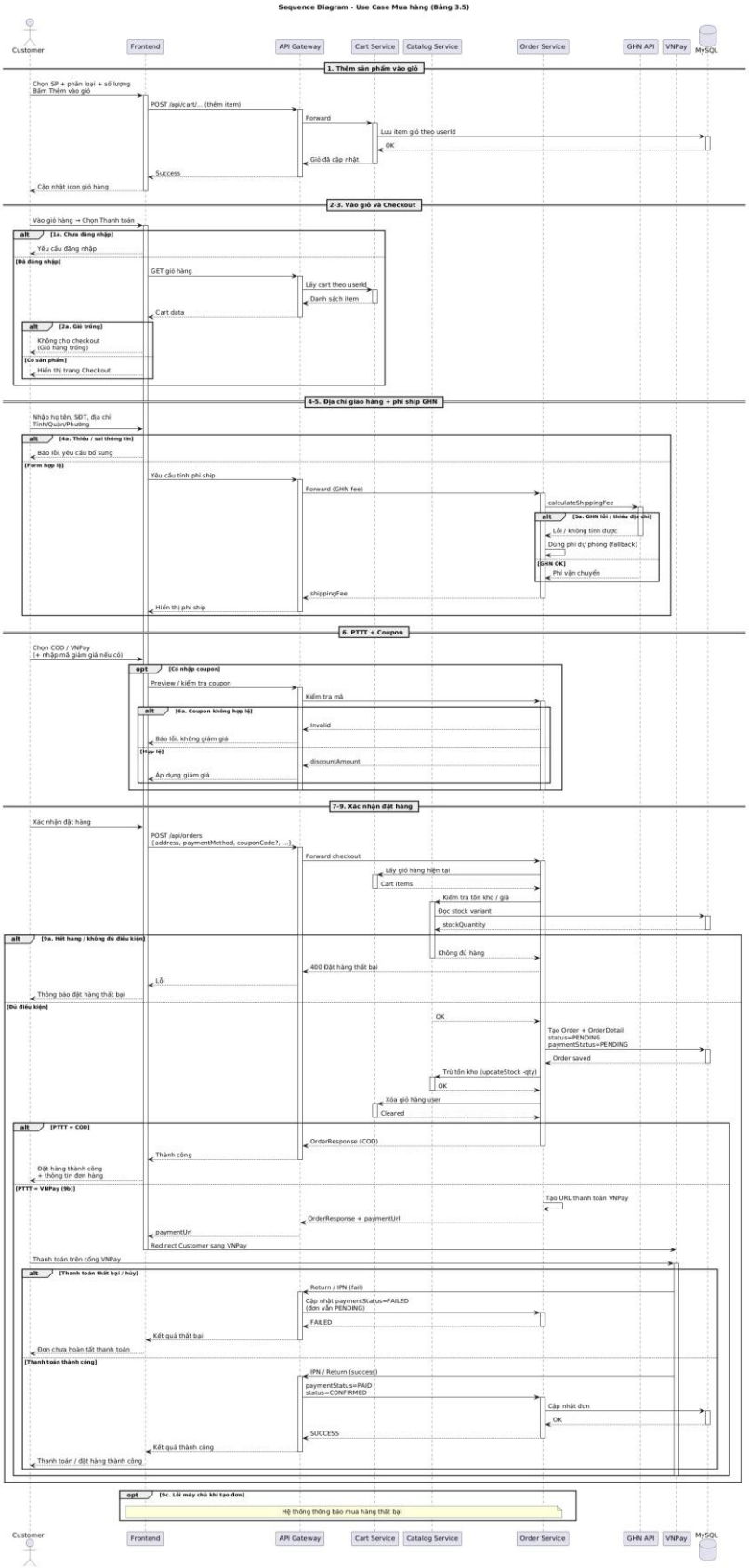
Hình 3.23 : Lược đồ Sequence của chức năng quản lý đơn hàng

Lược đồ Sequence của chức năng thanh toán mô tả quá trình Customer lựa chọn thanh toán COD hoặc VNPay; hệ thống tạo đơn hàng, khởi tạo giao dịch, chuyển hướng đến VNPay và xử lý kết quả trả về thông qua return URL hoặc IPN.



Hình 3.24 : Lược đồ Sequence của chức năng thanh toán

Lược đồ Sequence của chức năng mua hàng mô tả toàn bộ quá trình Customer thêm sản phẩm vào giỏ hàng, thực hiện checkout, nhập địa chỉ giao hàng, tính phí vận chuyển, áp dụng mã giảm giá, kiểm tra tồn kho, tạo đơn và xử lý kết quả thanh toán COD hoặc VNPay.

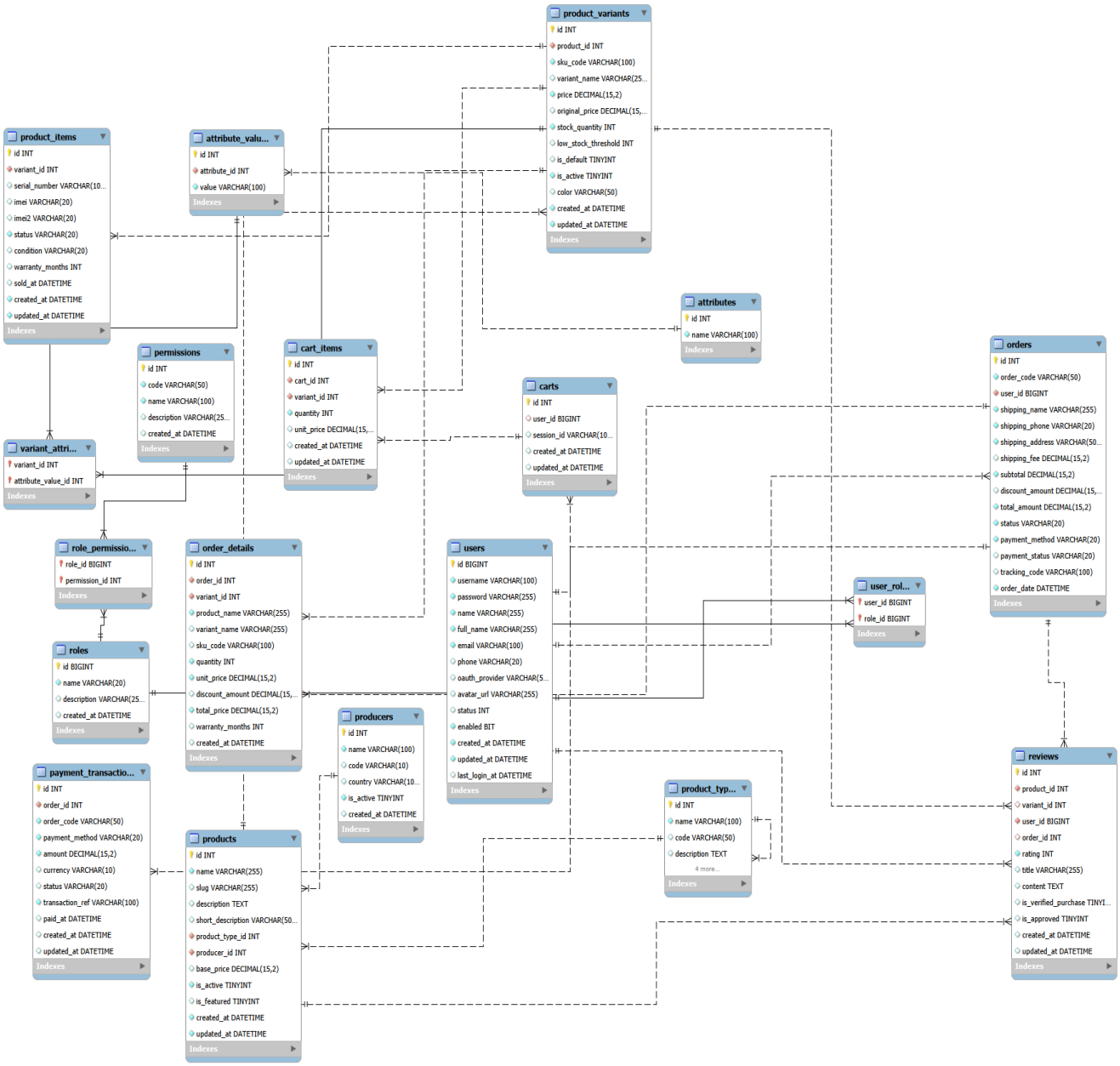


Hình 3.25 : Lược đồ Sequence của chức năng mua hàng

3.3.5. Lược đồ cơ sở dữ liệu

Lược đồ thực thể–liên kết (Entity–Relationship Diagram – ERD) mô tả cấu trúc dữ liệu và mối quan hệ giữa các nhóm bảng chính của hệ thống Electro Store. Các thực thể được tổ chức theo từng miền nghiệp vụ, bao gồm người dùng và phân quyền, sản phẩm và biến thể, giỏ hàng, đơn hàng, thanh toán, quản lý sản phẩm theo IMEI hoặc số serial và đánh giá sản phẩm.

Trong lược đồ, các khóa chính được sử dụng để định danh từng bản ghi, trong khi khóa ngoại thể hiện mối liên kết giữa các thực thể. Việc phân tách các bảng trung gian như user_roles, role_permissions và variant_attribute_values giúp biểu diễn các quan hệ nhiều–nhiều, đồng thời hạn chế dữ liệu lặp và hỗ trợ mở rộng hệ thống.



Hình 3.26 : Lược đồ cơ sở dữ liệu ERD của hệ thống Electro Store

3.3.6. Xây dựng các API Endpoint

3.3.6.1. API Xác thực người dùng (Authentication)

Nhóm API xác thực người dùng chịu trách nhiệm đăng ký, đăng nhập, quản lý token, khôi phục mật khẩu và hỗ trợ các phương thức đăng nhập mở rộng như Google SSO hoặc mã QR. Danh sách các endpoint chính được trình bày trong

Bảng 3.8 : Danh sách API xác thực người dùng

Phương thức	Endpoint	Mô tả
POST	/api/auth/login	Đăng nhập: Nhận thông tin đăng nhập (username, password), trả về Access Token (JWT), Refresh Token và thông tin cơ bản của người dùng.
POST	/api/auth/register	Đăng ký: Tạo tài khoản người dùng mới với username, email, password, name, phone và mặc định gán vai trò cơ bản (roleId = 4).
POST	/api/auth/refresh	Làm mới Token: Sử dụng Refresh Token hợp lệ và chưa hết hạn để xoay vòng, đồng thời cấp lại một cặp Access Token và Refresh Token mới.
POST	/api/auth/google	Đăng nhập Google: Nhận idToken từ Google, kiểm tra tính hợp lệ và trả về token truy cập của hệ thống nhằm hỗ trợ cơ chế đăng nhập một lần (SSO).
POST	/api/auth/logout	Đăng xuất: Nhận Refresh Token và thu hồi token khỏi Redis để vô hiệu hóa phiên đăng nhập.
POST	/api/auth/ forgot-password	Quên mật khẩu: Nhận email của người dùng và gửi liên kết hoặc mã OTP hỗ trợ quá trình đặt lại mật khẩu.
POST	/api/auth/ reset-password	Đặt lại mật khẩu: Cập nhật mật khẩu mới thông qua token được cấp từ email quên mật khẩu, kèm theo newPassword và confirmPassword.

GET	/api/auth/qr/generate	Tạo mã QR: Cấp phát một chuỗi qrToken ngẫu nhiên để Frontend tạo mã QR cho chức năng đăng nhập không cần mật khẩu.
GET	/api/auth/qr/status/{token}	Kiểm tra trạng thái QR: Frontend gọi liên tục theo cơ chế polling để kiểm tra trạng thái QR Token, chẳng hạn PENDING hoặc VERIFIED. Khi trạng thái là VERIFIED, hệ thống trả về Access Token.
POST	/api/auth/qr/verify	Xác thực mã QR: Dành cho Mobile App hoặc thiết bị đã đăng nhập để quét và xác nhận qrToken kèm userId, qua đó phê duyệt phiên đăng nhập trên web.

3.3.6.2. API Quản trị hệ thống (Admin)

Nhóm API quản trị hệ thống cung cấp các endpoint dành cho Admin nhằm quản lý người dùng, vai trò, quyền hạn, sản phẩm, đơn hàng và các tài nguyên vận hành khác. Các endpoint thuộc nhóm này yêu cầu người dùng đã được xác thực và có quyền quản trị tương ứng theo cơ chế RBAC.

Bảng 3.9 : Danh sách API quản trị hệ thống

Phương thức	**Endpoint**	**Mô tả**
Quản lý người dùng và hỗ trợ khách hàng		
GET	/api/admin/users	Lấy danh sách người dùng
PUT	/api/admin/users/{id}/status	Bật/tắt tài khoản
POST	/api/admin/live-chat/conversations/{id}/messages	Gửi tin nhắn trả lời khách
Quản lý danh mục sản phẩm và nội dung cửa hàng		

POST	/api/admin/products	Tạo sản phẩm
PUT	/api/admin/products/{id}	Cập nhật sản phẩm
POST	/api/admin/products/{id}/ variants	Thêm biến thể (màu, dung lượng…)
GET	/api/admin/categories/all	Lấy danh sách danh mục
GET	/api/admin/producers/all	Lấy danh sách thương hiệu

Phương thức	**Endpoint**	**Mô tả**
Quản lý kho, nhập hàng và kiểm kê		
POST	/api/admin/inventory/import	Nhập kho
POST	/api/admin/inventory/imei	Thêm serial/IMEI
POST	/api/admin/inventory/imei/ upload-excel	Import IMEI bằng Excel
POST	/api/admin/purchase-orders	Tạo đơn mua hàng (PO)
POST	/api/admin/purchase-orders/ {id}/approve	Duyệt PO
POST	/api/admin/purchase-orders/ {id}/confirm-receive	Xác nhận nhận hàng PO
GET	/api/admin/inventory/serials/fifo	Lấy serial xuất kho theo FIFO
POST	/api/admin/inventory-audits/ {id}/scan	Quét serial khi kiểm kê
POST	/api/admin/inventory-audits/ {id}/approve	Duyệt phiếu kiểm kê
Quản lý đơn hàng, hoàn tiền, bán hàng và bảo hành		
PUT	/api/admin/orders/{id}/status	Cập nhật trạng thái đơn hàng

PUT	/api/admin/orders/{id}/ assign-imei	Gán IMEI cho đơn hàng
POST	/api/admin/orders/ return-requests/{id}/approve	Duyệt yêu cầu trả hàng
PUT	/api/admin/refunds/{id}/ approve	Duyệt hoàn tiền
PUT	/api/admin/refunds/{id}/ confirm-transfer	Xác nhận đã chuyển khoản hoàn tiền
GET	/api/admin/sales/pipeline	Xem pipeline Sales
PUT	/api/admin/sales/orders/ {orderId}/attribution	Gán đơn cho nhân viên Sales
PUT	/api/admin/warranty/claims/ {id}/approve	Duyệt claim bảo hành
PUT	/api/admin/warranty/claims/ {id}/resolve	Giải quyết claim bảo hành
Quản lý mã giảm giá		
POST	/api/admin/coupons	Tạo mã giảm giá
Phương thức	**Endpoint**	**Mô tả**
Quản lý đánh giá sản phẩm		
PUT	/api/admin/reviews/{id}/status	Duyệt / ẩn đánh giá
Quản lý báo cáo và thống kê kinh doanh (Dashboard)		
GET	/api/admin/statistics/overview	Tổng quan dashboard
GET	/api/admin/statistics/revenue/ chart	Biểu đồ doanh thu
GET	/api/admin/statistics/finance/ summary	Tóm tắt tài chính

3.3.6.3. API Trí tuệ nhân tạo (Gợi ý sản phẩm & Chatbot RAG)

Nhóm API trí tuệ nhân tạo cung cấp các endpoint phục vụ gợi ý sản phẩm cá nhân hóa, huấn luyện lại mô hình và chatbot tư vấn dựa trên cơ chế RAG. Danh sách các endpoint được trình bày trong Bảng 3.10.

Bảng 3.10 : Danh sách API trí tuệ nhân tạo

Phương thức	Endpoint	Mô tả
Hệ thống gợi ý sản phẩm (Recommender System)		
GET	/recommend/{user_id}?top_k=	Sinh gợi ý sản phẩm (CF inference)
POST	/api/retrain	Huấn luyện lại mô hình gợi ý
GET	/api/recommendations/user/ {userId}	Điểm tích hợp FE ↔ AI service
Chatbot tư vấn sản phẩm (RAG)		
POST	/api/chatbot/chat	Trả lời câu hỏi qua RAG (Pinecone + LLM)
POST	/api/chatbot/ingest	Đồng bộ catalog vào vector store

3.3.6.4. API dành cho vai trò Sales

Nhóm API dành cho vai trò Sales hỗ trợ theo dõi, tiếp nhận, phê duyệt và hoàn tất các yêu cầu bảo hành của khách hàng. Danh sách các endpoint được trình bày trong Bảng 3.11.

Bảng 3.11 : Danh sách API dành cho vai trò Sales

Phương thức	Endpoint	Mô tả
Theo dõi và quyết định tiếp nhận yêu cầu bảo hành		
GET	/api/admin/warranty/claims	Lấy danh sách yêu cầu bảo hành đang chờ xử lý
GET	/api/admin/warranty/claims/ {id}	Xem chi tiết yêu cầu (khách hàng, sản phẩm, lý do bảo hành)

PUT	/api/admin/warranty/claims/{id}/approve	Duyệt yêu cầu hợp lệ, chuyển sang xử lý kỹ thuật
PUT	/api/admin/warranty/claims/{id}/reject	Từ chối yêu cầu không đủ điều kiện bảo hành
Hoàn tất hồ sơ sau xử lý kỹ thuật		
PUT	/api/admin/warranty/claims/{id}/resolve	Xác nhận kết quả xử lý (sửa chữa, đổi máy, …) sau khi kỹ thuật hoàn tất
PUT	/api/admin/warranty/claims/{id}/close	Đóng hồ sơ bảo hành và hoàn tất bàn giao cho khách hàng

3.3.6.5. API cốt lõi dành cho vai trò Nhân viên kho

Nhóm API dành cho vai trò Nhân viên kho hỗ trợ tiếp nhận hàng từ nhà cung cấp, quản lý Serial/IMEI, xuất kho theo nguyên tắc FIFO, xử lý hàng đợi đóng gói và tiếp nhận thiết bị bảo hành. Danh sách các endpoint được trình bày trong Bảng 3.12.

Bảng 3.12 : Danh sách API cốt lõi dành cho vai trò Nhân viên kho

Phương thức	Endpoint	Mô tả
POST	/api/admin/purchase-orders/{id}/confirm-receive	Xác nhận nhận hàng thực tế từ nhà cung cấp, đối chiếu với đơn mua hàng (PO) đã duyệt
POST	/api/admin/inventory/imei	Gắn định danh Serial/IMEI cho từng thiết bị
GET	/api/admin/inventory/serials/fifo	Lấy serial xuất kho theo nguyên tắc FIFO
GET	/api/admin/orders/warehouse/fulfillment-queue	Xem hàng đợi đơn cần đóng gói / xuất kho
PUT	/api/admin/orders/{id}/ assign-imei	Gán thiết bị cụ thể (theo IMEI) cho đơn hàng khách
PUT	/api/admin/warranty/claims/{id}/received	Xác nhận đã nhận máy bảo hành tại kho để xử lý kỹ thuật

3.3.7. Các màn hình trong hệ thống

Các màn hình trong hệ thống được thiết kế phù hợp với từng nhóm người dùng, bao gồm giao diện mua sắm dành cho khách hàng và giao diện quản trị dành cho Admin, Sales và nhân viên quản lý kho. Nội dung trình bày tập trung vào những màn hình chính và cách các chức năng nghiệp vụ được thể hiện trực quan trên giao diện.

Màn hình đăng nhập

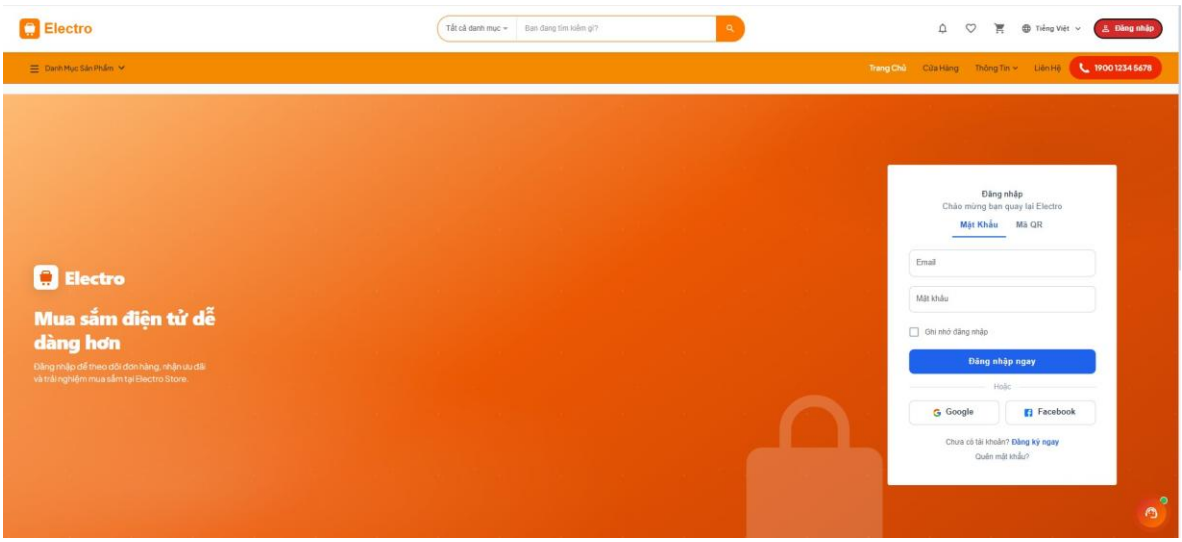
Mô tả: Màn hình đăng nhập được xây dựng để người dùng có thể truy cập vào hệ thống thông qua một số phương thức xác thực khác nhau như email kèm mật khẩu, tài khoản Google, tài khoản Facebook hoặc quét mã QR. Giao diện được trình bày theo hướng đơn giản và dễ hiểu, giúp người dùng thực hiện thao tác một cách thuận tiện ngay từ lần đầu sử dụng.

Chức năng:

Người dùng có thể nhập email và mật khẩu để đăng nhập vào hệ thống, đồng thời có tùy chọn lưu lại phiên đăng nhập để không phải thực hiện lại thao tác trong những lần truy cập tiếp theo.

Bên cạnh phương thức truyền thống, hệ thống hỗ trợ đăng nhập thông qua Google, Facebook hoặc mã QR, giúp tăng tính linh hoạt trong quá trình sử dụng.

Ngoài ra, từ màn hình này người dùng có thể chuyển sang chức năng đăng ký tài khoản mới hoặc khôi phục mật khẩu khi cần thiết.



Hình 3.27 : Màn hình đăng nhập

Màn hình Trang chủ

Mô tả: Màn hình Trang chủ của ứng dụng Electro được thiết kế theo bố cục lưới (Grid layout) hiện đại với nền màu trắng làm chủ đạo, kết hợp màu cam (Primary color) để tạo điểm nhấn thương hiệu và tăng khả năng nhận diện. Giao diện được tối ưu theo hướng trải nghiệm người dùng (UX), trong đó các khu vực chức năng được chia thành các component rõ ràng, giúp người dùng dễ quan sát, điều hướng và khám phá sản phẩm một cách mượt mà.

Chức năng:

Thanh công cụ (Header & Navigation): Cung cấp thanh tìm kiếm ở vị trí trung tâm để người dùng tra cứu sản phẩm một cách nhanh chóng, đi kèm các biểu tượng chức năng như tài khoản, danh sách yêu thích, giỏ hàng và nút đăng nhập. Bên cạnh đó là thanh điều hướng chính giúp truy cập các mục như danh mục sản phẩm, trang chủ, sản phẩm, tin tức và liên hệ.

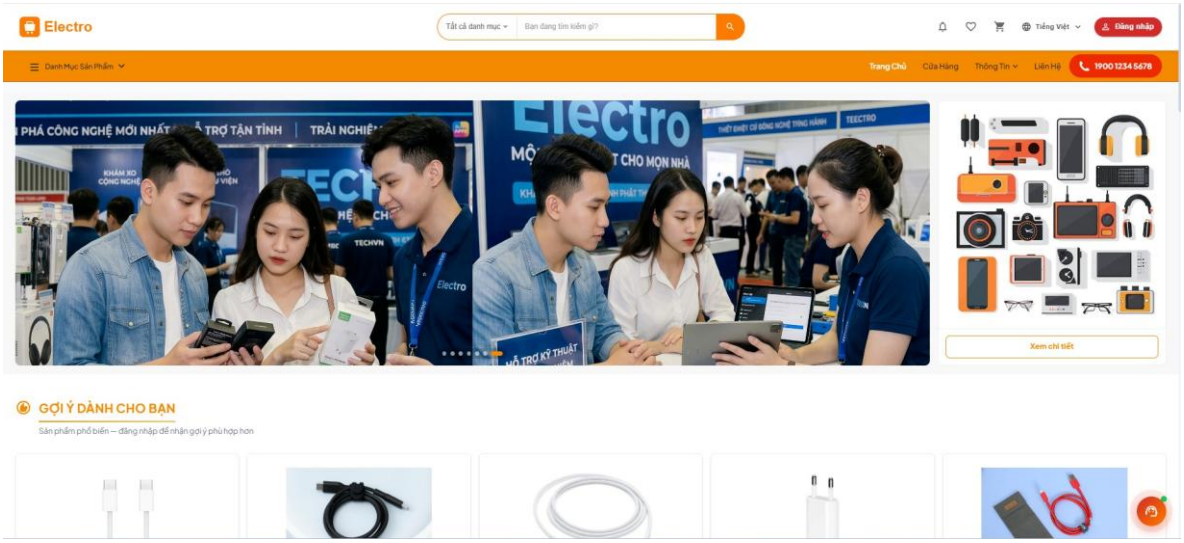
Khu vực Banner (Hero Section): Hiển thị các slide hình ảnh về sản phẩm công nghệ nổi bật cùng các banner khuyến mãi, tạo điểm nhấn trực quan ngay khi người dùng truy cập trang.

Khu vực "Gợi ý dành cho bạn": Trình bày danh sách sản phẩm được đề xuất như cáp sạc, sạc dự phòng và củ sạc, dựa trên hành vi hoặc sở thích mua sắm của người dùng.

Khu vực "Sản phẩm bán chạy": Hiển thị các sản phẩm có lượng mua cao như đồng hồ thông minh và máy tính bảng, kèm nhãn nổi bật để thể hiện mức độ phổ biến.

Khu vực "Sản phẩm của chúng tôi": Cung cấp danh sách sản phẩm đa dạng với hệ thống tab lọc nhanh gồm tất cả, Macbook, máy tính và tai nghe. Người dùng có thể nhấn "Xem thêm" để tải thêm sản phẩm.

Chân trang (Footer): Bao gồm thông tin chính sách dịch vụ, biểu mẫu đăng ký nhận bản tin, các liên kết hỗ trợ, phương thức thanh toán và các kênh mạng xã hội.



Hình 3.28 : Màn hình Trang chủ của hệ thống Electro Store

3.3.7.1. Màn hình cập nhật hồ sơ cá nhân

Mô tả: Màn hình "Hồ Sơ Của Tôi" được thiết kế với bố cục gồm hai phần chính: thanh menu bên trái và khu vực nội dung bên phải. Cách sắp xếp này giúp người dùng dễ quan sát, dễ thao tác và chỉnh sửa thông tin cá nhân trong hệ thống Electro. Giao diện được giữ ở mức đơn giản để đảm bảo trải nghiệm sử dụng ổn định và dễ hiểu.

Chức năng:

Sidebar quản lý tài khoản: Cho phép người dùng truy cập nhanh các mục như Thông báo, Hồ sơ, Ngân hàng, Địa chỉ, Đổi mật khẩu, Đơn mua, Bảo hành và Kho

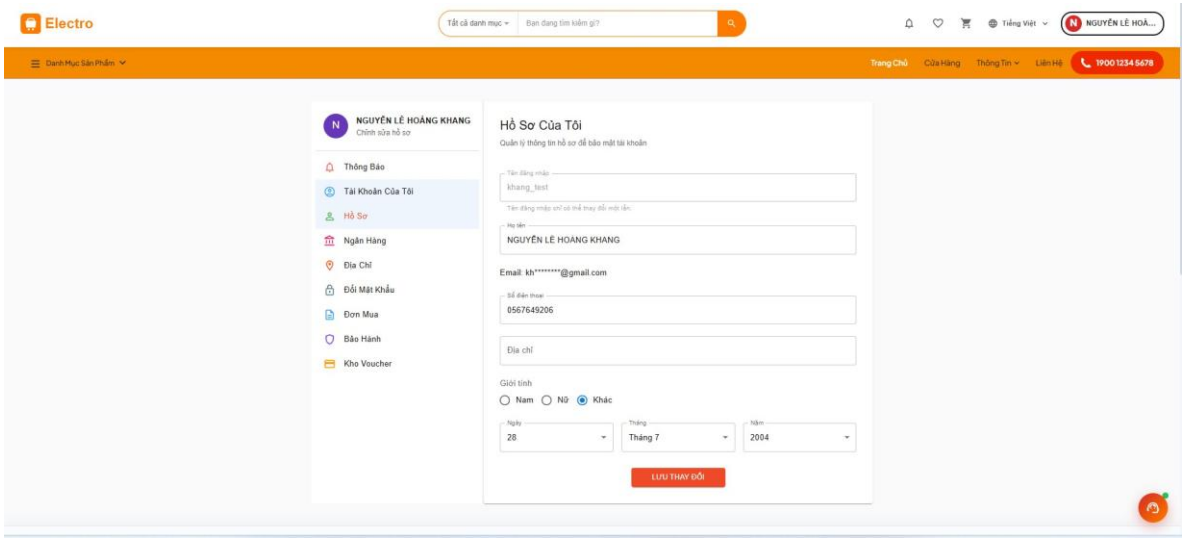
voucher. Mục Hồ sơ được hiển thị trạng thái đang chọn để người dùng nhận biết vị trí hiện tại.

Hiển thị thông tin tài khoản: Cung cấp tên đăng nhập và email của người dùng, trong đó email được che bớt ký tự để tăng tính bảo mật.

Cập nhật thông tin cá nhân: Cho phép chỉnh sửa họ tên, số điện thoại và địa chỉ.

Thiết lập thông tin cá nhân: Hỗ trợ lựa chọn giới tính và ngày sinh thông qua các thành phần nhập liệu phù hợp.

Lưu thay đổi: Ghi nhận và cập nhật dữ liệu mới của người dùng vào hệ thống.



Hình 3.29 : Màn hình cập nhật hồ sơ cá nhân

3.3.7.2. Màn hình đổi mật khẩu

Mô tả: Màn hình Đổi Mật Khẩu được thiết kế theo bố cục hai phần gồm thanh menu bên trái và khu vực nội dung chính bên phải. Cách sắp xếp này giúp người dùng dễ quan sát và thao tác trong toàn bộ hệ thống quản lý tài khoản của Electro. Nội dung tập trung vào chức năng thay đổi mật khẩu, từ đó đảm bảo tính bảo mật và giữ cho trải nghiệm sử dụng đơn giản, rõ ràng.

Chức năng:

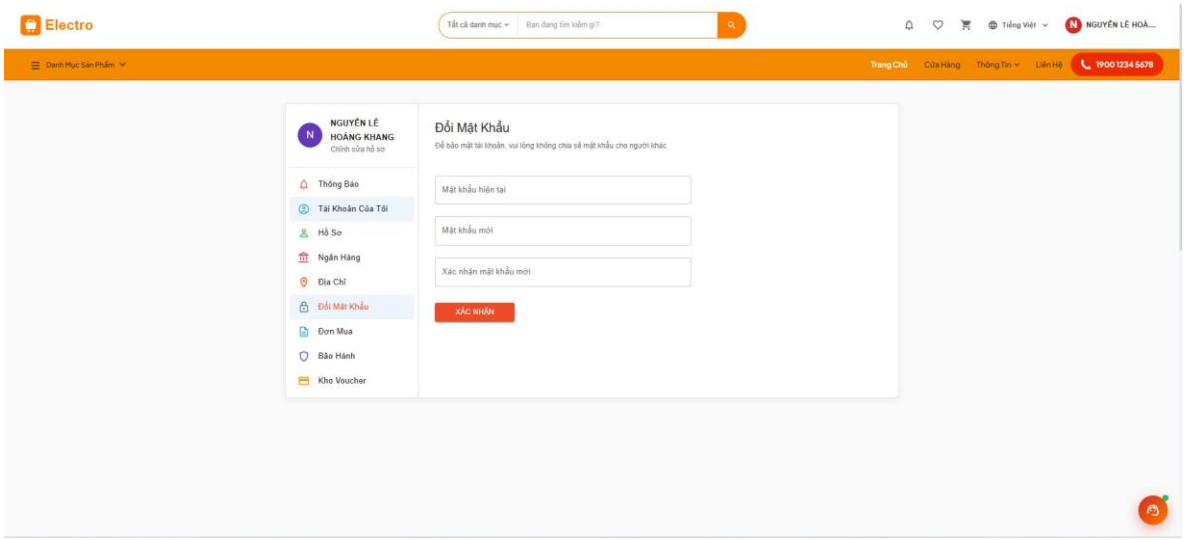
Thanh điều hướng tài khoản (Sidebar): Hiển thị các mục quản lý cá nhân như Thông báo, Hồ sơ, Ngân hàng, Địa chỉ, Đổi mật khẩu, Đơn mua, Bảo hành và Kho

voucher. Mục Đổi Mật Khẩu được làm nổi bật bằng màu nền xám nhạt và chữ cam đỏ để thể hiện trạng thái đang được chọn, giúp người dùng nhận biết vị trí hiện tại trong hệ thống.

Tiêu đề và cảnh báo bảo mật: Phần tiêu đề "Đổi Mật Khẩu" được đặt ở đầu nội dung để xác định rõ chức năng của màn hình. Bên dưới là thông báo nhắc nhở người dùng không chia sẻ mật khẩu với người khác nhằm tăng mức độ an toàn cho tài khoản.

Khu vực biểu mẫu (Form nhập liệu): Gồm ba trường nhập theo thứ tự: mật khẩu hiện tại, mật khẩu mới và xác nhận mật khẩu mới. Các trường được thiết kế đơn giản, có gợi ý nhập liệu rõ ràng để người dùng thao tác chính xác và hạn chế sai sót.

Nút hành động chính: Nút XÁC NHẬN được đặt ở cuối biểu mẫu, sử dụng màu cam đỏ và bo góc nhẹ. Khi người dùng nhấn nút này, hệ thống sẽ tiếp nhận yêu cầu và tiến hành xử lý việc thay đổi mật khẩu sau khi xác thực thông tin.



Hình 3.30 : Màn hình đổi mật khẩu của hệ thống Electro Store

3.3.7.3. Màn hình giỏ hàng

Mô tả: Màn hình giỏ hàng được thiết kế theo bố cục hai cột. Bên trái hiển thị danh sách sản phẩm đã chọn, bên phải là phần tóm tắt chi phí. Cách sắp xếp này giúp người dùng dễ theo dõi thông tin đơn hàng và kiểm tra lại trước khi thanh toán.

Chức năng:

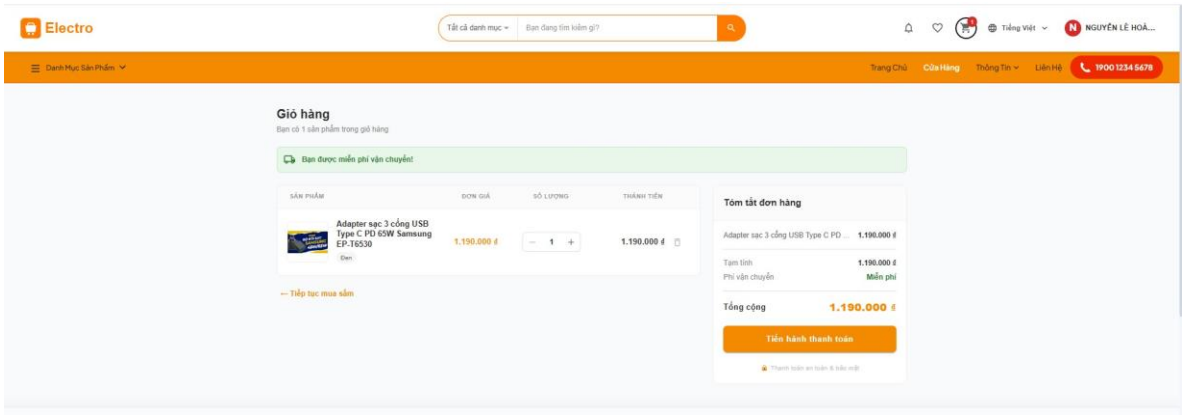
Danh sách sản phẩm: Hiển thị hình ảnh, tên sản phẩm, phân loại, đơn giá và tổng tiền tương ứng.

Điều chỉnh số lượng: Người dùng có thể tăng hoặc giảm số lượng sản phẩm hoặc xóa sản phẩm khỏi giỏ hàng khi không còn nhu cầu.

Thông báo ưu đãi: Khu vực thông tin hiển thị các chương trình khuyến mãi đang áp dụng, ví dụ như miễn phí vận chuyển.

Tóm tắt đơn hàng: Hệ thống tự động tính toán các khoản chi phí gồm tạm tính, phí vận chuyển và tổng thanh toán.

Thao tác thanh toán: Người dùng có thể tiếp tục mua sắm hoặc chuyển sang bước thanh toán để hoàn tất đơn hàng.



Hình 3.31 : Màn hình giỏ hàng

3.3.7.4. Màn hình thanh toán

Mô tả: Màn hình "Thanh toán" được thiết kế với bố cục hai cột (Two-column layout) trực quan và tối ưu tỷ lệ chuyển đổi. Cột bên trái tập trung vào việc thu thập các thông tin thiết yếu của khách hàng như địa chỉ giao hàng và phương thức thanh toán, trong khi cột bên phải được cố định để luôn hiển thị minh bạch tổng chi phí và các tùy chọn ưu đãi, giúp khách hàng an tâm xác nhận đơn hàng.

Chức năng:

Sản phẩm trong đơn hàng: Bảng hiển thị các mặt hàng đã chọn từ giỏ hàng gồm hình ảnh, tên sản phẩm, phân loại, đơn giá, số lượng và thành tiền, giúp khách hàng kiểm tra lại trước khi đặt hàng.

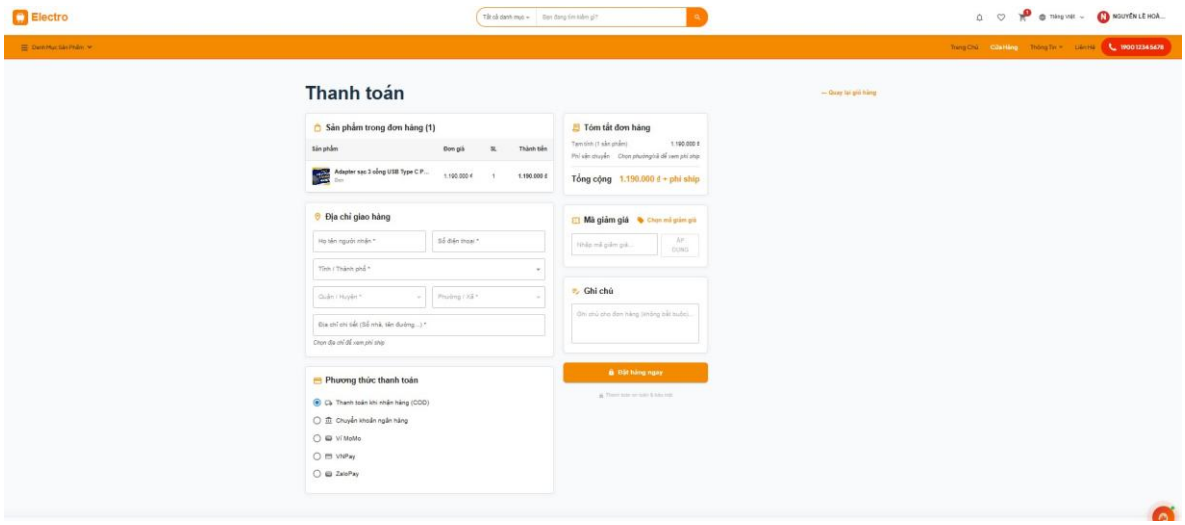
Địa chỉ giao hàng: Biểu mẫu yêu cầu các thông tin bắt buộc như họ tên, số điện thoại, địa chỉ theo ba cấp gồm tỉnh hoặc thành phố, quận hoặc huyện, phường hoặc xã, cùng số nhà chi tiết. Phí vận chuyển được cập nhật theo địa chỉ được chọn.

Phương thức thanh toán: Người dùng có thể lựa chọn thanh toán khi nhận hàng (COD), chuyển khoản ngân hàng hoặc các ví điện tử như MoMo, VNPay và ZaloPay thông qua các lựa chọn radio.

Tóm tắt đơn hàng: Khu vực này hiển thị tổng chi phí, bao gồm tạm tính và phí vận chuyển sau khi hệ thống xác định xong địa chỉ giao hàng.

Mã giảm giá và ghi chú: Cung cấp ô nhập mã khuyến mãi hoặc lựa chọn Voucher có sẵn, kèm theo phần ghi chú để khách hàng gửi thêm yêu cầu cho cửa hàng hoặc đơn vị vận chuyển

Nút hành động chính: Nút "Đặt hàng ngay" màu cam được bố trí nổi bật, đi kèm biểu tượng ổ khóa và thông điệp cam kết thanh toán an toàn, bảo mật.



Hình 3.32 : Màn hình thanh toán

3.3.7.5. Màn hình thanh toán Online với VNPay

Mô tả: Sau khi lựa chọn VNPay và xác nhận đặt hàng, người dùng được chuyển hướng đến cổng thanh toán VNPay để hoàn tất giao dịch trực tuyến. Giao diện thanh toán được trình bày tập trung, hiển thị rõ các phương thức được hỗ trợ và thông tin nhận diện của cổng thanh toán, giúp người dùng lựa chọn hình thức phù hợp trong môi trường thử nghiệm.

Chức năng:

Điều hướng giao dịch: Cung cấp nút quay lại để người dùng trở về hệ thống Electro khi chưa muốn tiếp tục thanh toán.

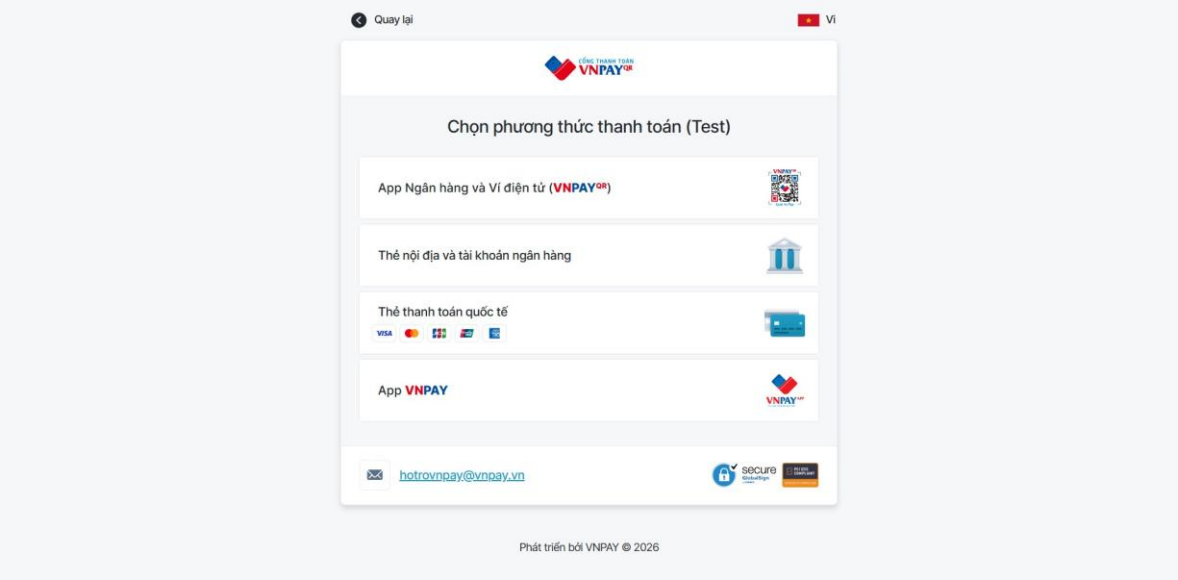
Lựa chọn ngôn ngữ: Hiển thị tùy chọn ngôn ngữ tiếng Việt nhằm hỗ trợ người dùng thao tác thuận tiện trên cổng thanh toán.

Thanh toán bằng VNPay QR: Cho phép sử dụng ứng dụng ngân hàng hoặc ví điện tử hỗ trợ VNPay QR để quét mã và xác nhận giao dịch.

Thẻ nội địa và tài khoản ngân hàng: Hỗ trợ thanh toán thông qua thẻ ATM nội địa hoặc tài khoản ngân hàng đã đăng ký dịch vụ thanh toán trực tuyến.

Thẻ thanh toán quốc tế: Hỗ trợ các loại thẻ quốc tế phổ biến như Visa, Mastercard, JCB, UnionPay và American Express.

Ứng dụng VNPay và bảo mật: Cho phép thanh toán trực tiếp bằng ứng dụng VNPay, đồng thời hiển thị thông tin hỗ trợ và các chứng nhận bảo mật của cổng thanh toán.



Hình 3.33 : Màn hình thanh toán Online với VNPay

3.3.7.6. Màn hình thanh toán thành công

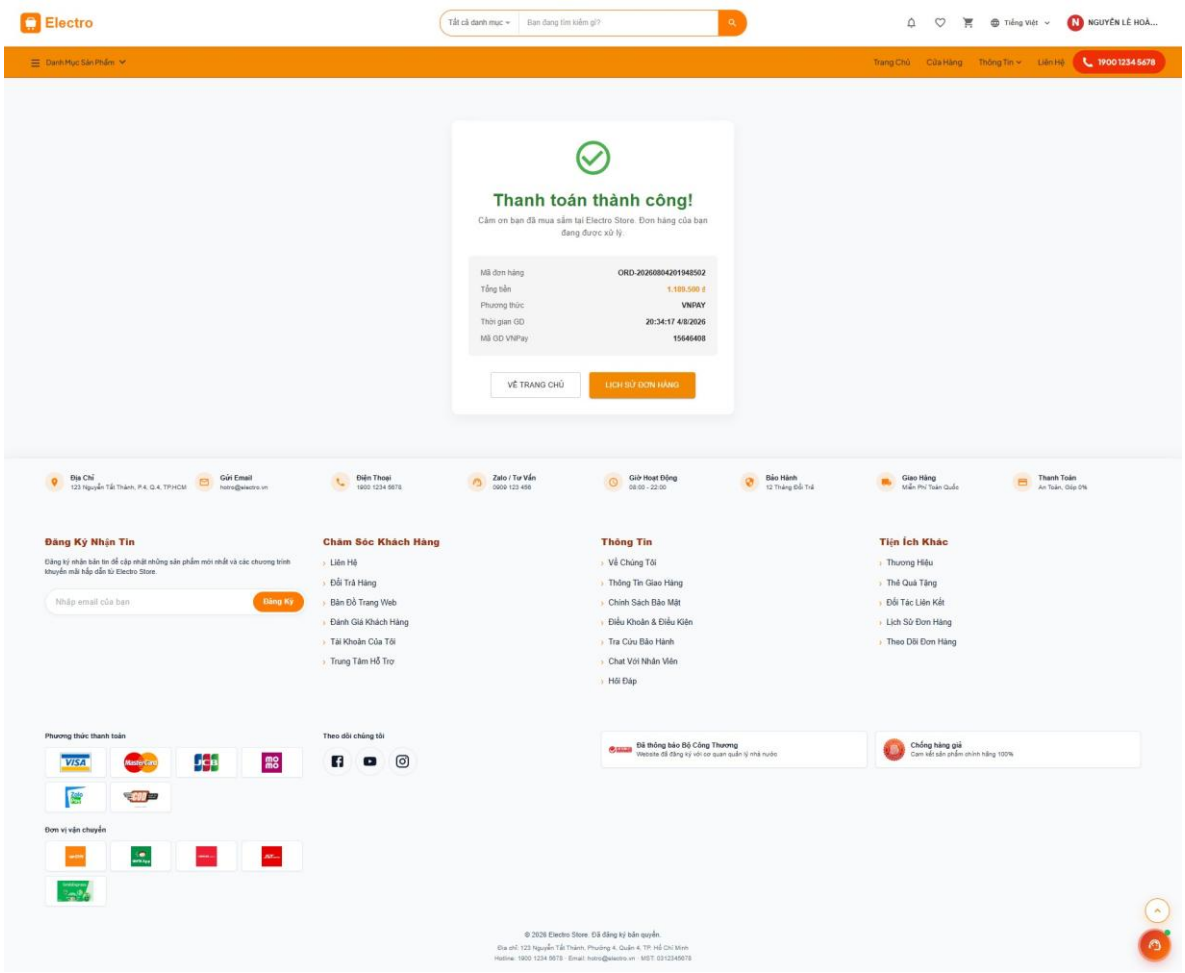
Mô tả: Màn hình "Thanh toán thành công" là bước cuối trong luồng mua hàng của người dùng. Giao diện được thiết kế đơn giản với một khung thông báo đặt ở giữa màn hình. Sau khi giao dịch hoàn tất, hệ thống hiển thị thông tin xác nhận cùng biên lai điện tử để người dùng có thể kiểm tra lại các chi tiết thanh toán và yên tâm rằng đơn hàng đã được xử lý.

Chức năng:

Xác nhận giao dịch: Biểu tượng dấu tick màu xanh được hiển thị cùng thông báo thanh toán thành công và lời cảm ơn từ hệ thống, giúp người dùng nhận biết giao dịch đã hoàn tất.

Biên lai giao dịch: Cung cấp các thông tin quan trọng được ghi nhận từ hệ thống và cổng thanh toán, bao gồm mã đơn hàng, tổng số tiền đã thanh toán, phương thức thanh toán VNPay, thời gian ghi nhận giao dịch và mã giao dịch VNPay phục vụ việc đối soát hoặc xử lý khiếu nại khi cần thiết.

Điều hướng sau thanh toán: Cung cấp nút quay về trang chủ để người dùng tiếp tục xem sản phẩm và nút truy cập lịch sử đơn hàng để theo dõi các giao dịch đã thực hiện.



Hình 3.34 : Màn hình thanh toán thành công của hệ thống Electro Store

3.3.7.7. Màn hình trạng thái và lịch sử đơn hàng

Mô tả: Màn hình "Đơn Hàng Của Tôi" giúp người dùng xem lại toàn bộ lịch sử mua hàng, theo dõi trạng thái và quản lý đơn hàng một cách nhanh chóng, trực quan.

Chức năng:

Sidebar tài khoản: Mục "Đơn Mua" được hiển thị nổi bật, giúp người dùng dễ dàng xác định vị trí đang xem trong hệ thống.

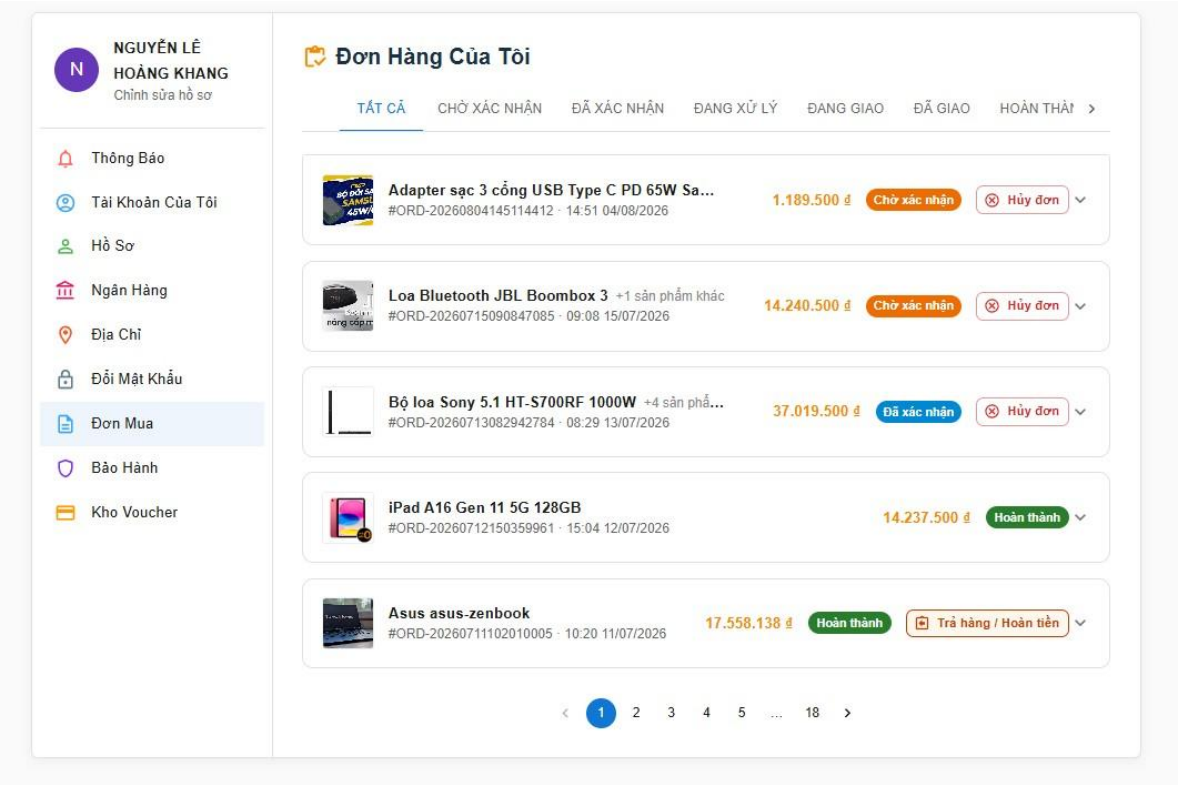
Tab trạng thái: Người dùng có thể lọc và theo dõi đơn hàng theo từng trạng thái như chờ xác nhận, đã xác nhận, đang xử lý, đang giao, đã giao và hoàn thành.

Danh sách đơn hàng: Hiển thị danh sách đơn dưới dạng thẻ, bao gồm hình sản phẩm, tên sản phẩm, mã đơn, thời gian đặt và tổng tiền.

Trạng thái đơn: Giúp người dùng dễ dàng nhận biết tiến trình xử lý đơn hàng thông qua các màu sắc được phân biệt rõ ràng.

Thao tác đơn hàng: Người dùng có thể thực hiện các hành động như hủy đơn hoặc yêu cầu trả hàng tùy theo trạng thái của đơn hàng.

Phân trang: Cho phép người dùng chuyển trang để xem thêm các đơn hàng khi số lượng đơn lớn.



Hình 3.35 : Màn hình trạng thái và lịch sử đơn hàng

3.3.7.8. Màn hình quản lý người dùng

Mô tả: Màn hình "Quản Lý Người Dùng" thuộc phân hệ quản trị (Back-office) dành cho Admin. Giao diện được thiết kế theo bố cục rộng, tối ưu cho màn hình máy tính để bàn, với thanh điều hướng nằm bên trái và khu vực xử lý dữ liệu chính nằm bên phải. Màu sắc được sử dụng nhằm phân tách rõ các nhóm thông tin và trạng thái khác nhau trong hệ thống.

Chức năng:

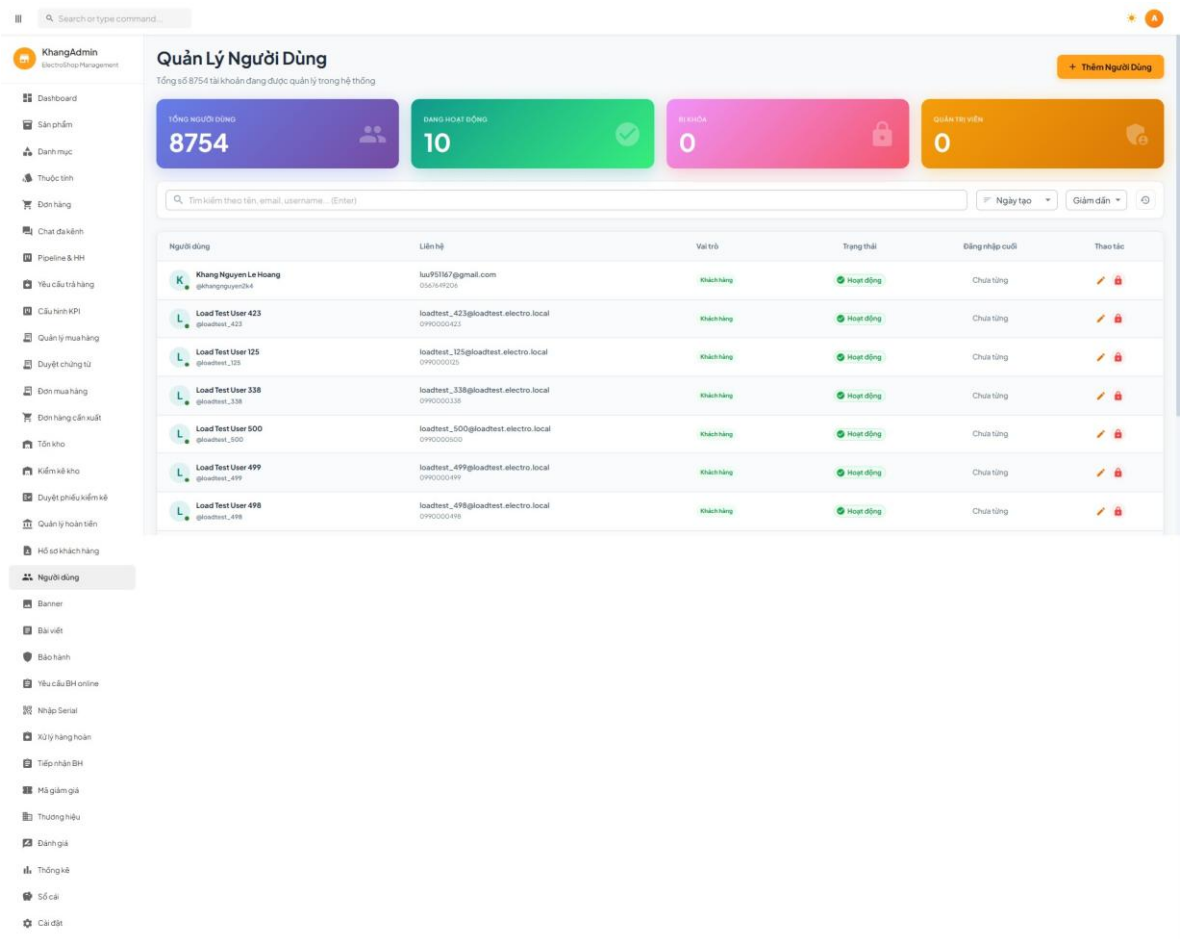
Theo dõi tổng quan: Giúp quản trị viên theo dõi nhanh tình hình người dùng trong hệ thống thông qua các chỉ số thống kê trực quan như tổng người dùng, số tài khoản đang hoạt động, tài khoản bị khóa và số quản trị viên.

Tìm kiếm và lọc dữ liệu: Tăng hiệu quả quản lý người dùng nhờ khả năng tìm kiếm theo tên, email hoặc tên đăng nhập, đồng thời hỗ trợ lọc và sắp xếp dữ liệu linh hoạt.

Thao tác quản lý: Tiết kiệm thời gian với các chức năng thêm người dùng, chỉnh sửa thông tin và quản lý trạng thái tài khoản được bố trí rõ ràng.

Cập nhật dữ liệu: Hỗ trợ quản trị viên ra quyết định chính xác hơn nhờ dữ liệu người dùng và trạng thái tài khoản được cập nhật kịp thời.

Giao diện trực quan: Giảm sai sót trong quá trình quản lý nhờ cách trình bày dễ sử dụng, bảng dữ liệu rõ ràng và các nhóm chức năng được phân chia hợp lý.



Hình 3.36 : Màn hình quản lý người dùng của hệ thống Electro Store

3.3.7.9. Màn hình quản lý đơn hàng

Mô tả: Màn hình Quản Lý Đơn Hàng thuộc khu vực quản trị hệ thống, được sử dụng bởi quản trị viên để theo dõi toàn bộ hoạt động mua bán trong ứng dụng Electro. Giao diện được tổ chức theo dạng bảng dữ liệu, hiển thị đầy đủ thông tin của từng đơn hàng theo thời gian thực, giúp việc kiểm soát trạng thái xử lý trở nên rõ ràng và dễ theo dõi trong quá trình vận hành.

Chức năng:

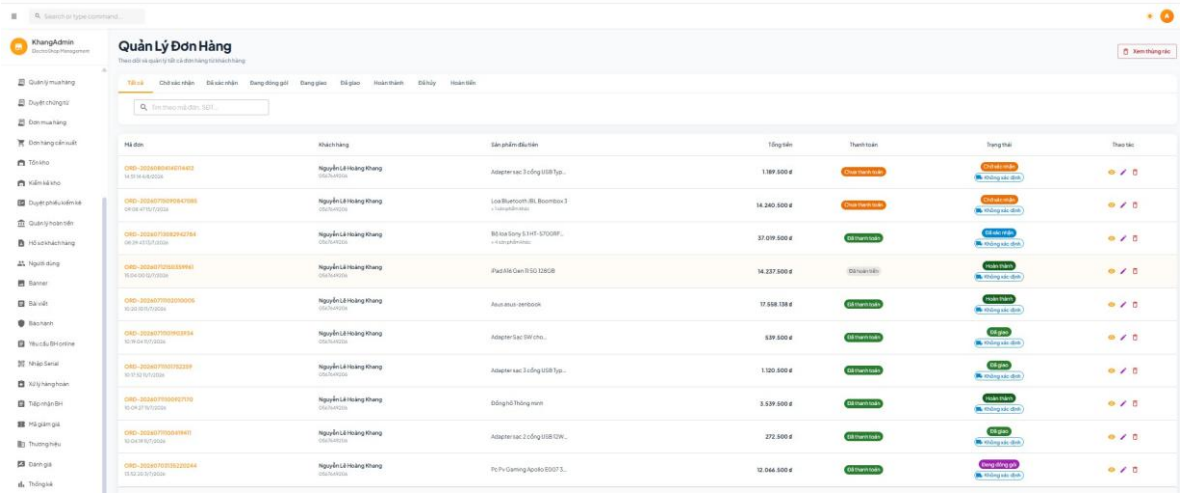
Thanh điều hướng trạng thái đơn hàng: Cho phép lọc đơn hàng theo từng giai đoạn như Tất cả, Chờ xác nhận, Đã xác nhận, Đang đóng gói, Đang giao, Đã giao, Hoàn thành, Đã hủy và Hoàn tiền, giúp quản trị viên nhanh chóng nắm bắt tình hình xử lý.

Thanh tìm kiếm: Hỗ trợ tra cứu đơn hàng theo mã đơn hoặc số điện thoại khách hàng, giúp rút ngắn thời gian tìm kiếm khi số lượng đơn lớn.

Bảng dữ liệu đơn hàng: Hiển thị thông tin chi tiết của từng đơn bao gồm mã đơn, thời gian đặt, thông tin khách hàng, sản phẩm, tổng tiền, trạng thái thanh toán và trạng thái xử lý đơn hàng.

Khu vực thao tác: Cung cấp các chức năng xem chi tiết, chỉnh sửa thông tin đơn hàng và xóa đơn hàng khi cần thiết.

Nút quản lý thùng rác: Cho phép truy cập các đơn hàng đã xóa để xem lại hoặc khôi phục khi có yêu cầu.



Hình 3.37 : Màn hình quản lý đơn hàng của hệ thống Electro Store

3.3.7.10. Màn hình Admin Dashboard báo cáo và thống kê

Mô tả: Màn hình Dashboard là trung tâm điều khiển dành cho quản trị viên. Giao diện được thiết kế theo dạng bảng tổng hợp, hiển thị các chỉ số quan trọng của hệ thống theo thời gian thực. Nhờ cách trình bày bằng thẻ thông tin và biểu đồ, người quản trị có thể nhanh chóng theo dõi tình hình kinh doanh và phát hiện các vấn đề phát sinh.

Chức năng:

Thanh điều hướng quản trị: Nằm bên trái và đóng vai trò trung tâm điều hướng hệ thống, cho phép truy cập nhanh đến các phân hệ nghiệp vụ khác nhau. Mục Dashboard được làm nổi bật để thể hiện trạng thái đang được chọn.

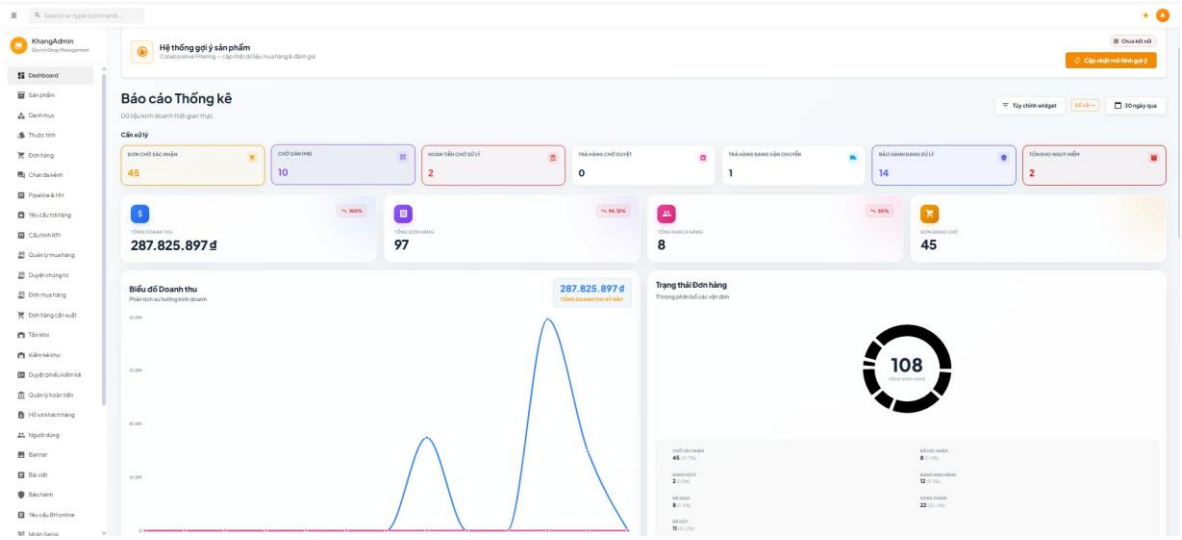
Khu vực quản trị và thông báo AI: Cho phép Admin kiểm soát hệ thống trí tuệ nhân tạo, bao gồm cập nhật và huấn luyện lại mô hình gợi ý sản phẩm nhằm tối ưu hóa khả năng đề xuất dựa trên dữ liệu hành vi người dùng mới nhất.

Thanh bộ lọc dữ liệu: Hỗ trợ lựa chọn phạm vi phân tích theo chi nhánh và khoảng thời gian như hôm nay, tuần này hoặc tháng này, giúp việc đánh giá trở nên linh hoạt và đa chiều hơn.

Khu vực cảnh báo vận hành: Hiển thị các tác vụ cần xử lý ngay như đơn hàng chờ xác nhận, đơn chờ giao, yêu cầu hoàn tiền và tình trạng tồn kho sản phẩm ở mức nguy hiểm, hỗ trợ Admin xử lý kịp thời các vấn đề phát sinh.

Các thẻ chỉ số KPI tổng quan: Cung cấp thông tin nhanh về doanh thu, số lượng đơn hàng, số lượng khách hàng và các đơn đang xử lý, giúp đánh giá nhanh tình trạng hoạt động của hệ thống.

Khu vực biểu đồ: Trực quan hóa dữ liệu bằng biểu đồ doanh thu theo thời gian và biểu đồ phân bố trạng thái đơn hàng, hỗ trợ phân tích xu hướng và cơ cấu hoạt động một cách trực quan, dễ hiểu.



Hình 3.38 : Màn hình Admin Dashboard báo cáo và thống kê

3.3.8. Kết quả đo lường các chức năng

Sau khi hoàn thiện việc xây dựng các chức năng và triển khai hệ thống, quá trình kiểm thử hiệu năng (Performance Testing) được thực hiện nhằm đánh giá khả năng chịu tải, thời gian phản hồi và độ ổn định của hệ thống dưới áp lực truy cập cao.

3.3.8.1. Môi trường và công cụ kiểm thử

+ **Công cụ thực hiện:**

+ **Apache JMeter 5.6.3:** Được sử dụng để giả lập số lượng người dùng ảo (Virtual Users), gửi đồng thời nhiều yêu cầu đến hệ thống và tạo tải phục vụ quá trình đo lường hiệu năng.

+ **Môi trường máy chủ:** Hệ thống được triển khai và kiểm thử trên môi trường localhost của máy tính cá nhân.

+ **Backend:** Được xây dựng theo kiến trúc Microservices bằng Spring Boot. Toàn bộ yêu cầu kiểm thử được gửi qua API Gateway sử dụng Spring Cloud Gateway tại cổng 8080.

+ **Dịch vụ AI gợi ý sản phẩm:** Recommendation Service (reco-service) được phát triển bằng Python FastAPI và hoạt động tại cổng 5003.

+ **Cơ sở dữ liệu:** Hệ thống sử dụng MySQL 8.0 để lưu trữ và truy xuất dữ liệu trong quá trình kiểm thử.

+ **Kịch bản kiểm thử (Test Plan):**

Hệ thống được kiểm thử theo kịch bản chịu tải cao với số lượng từ 500 đến 2.000 người dùng ảo truy cập liên tục vào năm API cốt lõi. Các yêu cầu được gửi đồng thời thông qua API Gateway nhằm mô phỏng tình huống nhiều người dùng sử dụng hệ thống trong cùng một khoảng thời gian. Nội dung kiểm thử cụ thể gồm:

+ **Login:** Kiểm tra khả năng xác thực thông tin đăng nhập và cấp token cho người dùng khi số lượng yêu cầu đồng thời tăng cao.

+ **Products**: Kiểm tra tốc độ truy vấn và đọc danh sách sản phẩm trong kịch bản có tần suất đọc dữ liệu lớn (read-heavy).

+ **Cart:** Kiểm tra khả năng truy vấn dữ liệu giỏ hàng đối với các yêu cầu đã được xác thực bằng token.

+ **Recommend:** Kiểm tra khả năng tính toán và thời gian phản hồi của mô hình gợi ý sản phẩm AI trong tác vụ phụ thuộc nhiều vào tài nguyên CPU (CPU-bound).

+ **Chatbot:** Kiểm tra khả năng phản hồi của chatbot RAG trong quá trình truy xuất thông tin và sinh câu trả lời (retrieval and generation).

3.3.8.2. Kết quả thực nghiệm

Kết quả kiểm thử tại từng mức tải được tổng hợp theo các chỉ số do Apache JMeter cung cấp, bao gồm số lượng mẫu, thời gian phản hồi trung bình, thời gian phản hồi nhỏ nhất và lớn nhất, tỷ lệ lỗi và thông lượng xử lý. Ba bảng dưới đây tương ứng với các mốc 500, 1.000 và 2.000 người dùng ảo.

Bảng 3.13 : Kết quả đo lường với 500 người dùng ảo

Tên chức năng	Số mẫu	Thời gian TB (ms)	Min (ms)	Max (ms)	Tỷ lệ lỗi	Thông lượng (/sec)
POST – Login	500	134	104	218	0,00%	4,18
GET – Products	500	9	7	42	0,00%	4,15
GET – Cart	500	19	16	36	0,00%	4,13
GET – Recommend	500	6	5	16	0,00%	4,14

POST – Chatbot	501	568.446	152.027	950.008	0,00%	0,40
TOTAL	**2.501**	**113.905**	**5**	**950.008**	**0,00%**	**2,01**

Bảng 3.14 : Kết quả đo lường với 1.000 người dùng ảo

Tên chức năng	**Số mẫu**	**Thời gian TB (ms)**	**Min (ms)**	**Max (ms)**	**Tỷ lệ lỗi**	**Thông lượng (/sec)**
POST – Login	1.000	138	105	524	0,00%	8,24
GET – Products	1.000	12	7	566	0,00%	8,23
GET – Cart	1.000	22	17	308	0,00%	8,19
GET – Recommend	1.000	7	5	80	0,00%	8,17
POST – Chatbot	1.000	503.313	150.167	1.124.821	0,00%	0,80
TOTAL	**5.000**	**100.698**	**5**	**1.124.821**	**0,00%**	**3,99**

Bảng 3.15 : Kết quả đo lường với 2.000 người dùng ảo

Tên chức năng	**Số mẫu**	**Thời gian TB (ms)**	**Min (ms)**	**Max (ms)**	**Tỷ lệ lỗi**	**Thông lượng (/sec)**
POST – Login	2.000	146	104	1.767	0,00%	16,54
GET – Products	2.000	11	7	114	0,00%	16,54
GET – Cart	2.000	28	17	1.007	0,00%	16,55
GET – Recommend	2.000	7	5	261	0,00%	16,45
POST – Chatbot	2.000	234.324	13.724	472.184	0,00%	3,38
TOTAL	**10.000**	**46.903**	**5**	**472.184**	**0,00%**	**16,66**

3.3.8.3. Phân tích kết quả đo lường

+ Đối với các chức năng nghiệp vụ:

Đối với các chức năng nghiệp vụ được triển khai bằng Spring Boot và truy cập thông qua API Gateway, nhóm API Products và Cart cho thấy hiệu năng ổn định. Trong cả ba mức tải 500, 1.000 và 2.000 người dùng ảo, hệ thống không ghi nhận lỗi. Ở mức

tải 2.000 người dùng, thời gian phản hồi trung bình của API Products vẫn duy trì ở khoảng 11 ms, trong khi API Cart có thời gian phản hồi trung bình dao động từ khoảng 19 đến 28 ms qua các kịch bản. Kết quả này phù hợp với đặc điểm của các truy vấn đọc tương đối đơn giản được chuyển tiếp từ API Gateway đến các microservice Spring Boot và cơ sở dữ liệu MySQL trên môi trường localhost. Bên cạnh đó, API Products được hỗ trợ bởi cơ chế lưu bộ nhớ đệm danh sách sản phẩm bằng Caffeine [23], qua đó giảm số lần truy vấn cơ sở dữ liệu và góp phần duy trì độ trễ ổn định.

Đối với API Login, hệ thống cũng hoạt động ổn định và không phát sinh lỗi trong toàn bộ các kịch bản kiểm thử. Thời gian phản hồi trung bình nằm trong khoảng từ 134 đến 146 ms, phù hợp với quy trình xác thực gồm kiểm tra mật khẩu bằng BCrypt, tạo JWT và lưu refresh token trên Redis. Khi tải tăng, nhóm API nghiệp vụ cốt lõi vẫn duy trì khả năng phục vụ và không xảy ra hiện tượng gián đoạn hệ thống. Tuy nhiên, độ trễ tăng rõ rệt hơn ở các thành phần liên quan đến trí tuệ nhân tạo, đặc biệt là Chatbot; nội dung này được phân tích chi tiết trong phần tiếp theo.

+ **Đối với chức năng gợi ý thông minh:**

Kết quả kiểm thử cho thấy Recommendation Service hoạt động ổn định trong toàn bộ các kịch bản đánh giá, với tỷ lệ lỗi ghi nhận bằng 0%. Thời gian phản hồi trung bình dao động trong khoảng từ 6 đến 7 ms, đáp ứng tốt yêu cầu xử lý gần thời gian thực của hệ thống thương mại điện tử.

Trong điều kiện tải cao với 2.000 người dùng ảo, dịch vụ vẫn xử lý đầy đủ 2.000 mẫu yêu cầu mà không ghi nhận lỗi. Kết quả này cho thấy việc kết hợp mô hình Collaborative Filtering sử dụng thuật toán KNNBaseline với cơ chế nạp sẵn mô hình (preloaded model) giúp hạn chế chi phí khởi tạo và tính toán trong quá trình suy luận, từ đó duy trì độ trễ ổn định khi tải tăng.

Nhìn chung, module gợi ý sản phẩm đáp ứng tốt yêu cầu về tốc độ phản hồi và khả năng mở rộng trong phạm vi môi trường kiểm thử. Kết quả thực nghiệm cho thấy kiến trúc dịch vụ độc lập và phương pháp triển khai được lựa chọn phù hợp với nhu cầu cung cấp danh sách sản phẩm gợi ý cho người dùng mà không gây ảnh hưởng đáng kể đến các dịch vụ nghiệp vụ khác.

+ Đối với chức năng chatbot tư vấn bằng AI (RAG):

Đối với chức năng chatbot tư vấn sử dụng kiến trúc RAG, kết quả kiểm thử cho thấy độ trễ xử lý là điểm nghẽn chính của hệ thống. Thời gian phản hồi trung bình luôn ở mức cao, dao động từ khoảng 234 đến 568 giây, trong khi tỷ lệ lỗi bằng 0%. Điều này cho thấy dịch vụ duy trì được tính ổn định trong quá trình xử lý yêu cầu nhưng chưa đáp ứng tốt yêu cầu về thời gian phản hồi của một hệ thống tư vấn trực tuyến.

Khi so sánh giữa các mức tải 500, 1.000 và 2.000 người dùng ảo, thời gian phản hồi không biến đổi theo xu hướng tuyến tính. Sự chênh lệch giữa các lần đo không nên được xem là bằng chứng cho thấy hiệu năng được cải thiện khi tải tăng, mà có thể chịu ảnh hưởng bởi các yếu tố thực nghiệm như trạng thái bộ nhớ đệm, độ ổn định của kết nối đến cơ sở dữ liệu vector Pinecone và thời điểm gửi yêu cầu đến mô hình ngôn ngữ lớn (LLM).

Nguyên nhân chính của độ trễ xuất phát từ đặc thù của pipeline RAG, bao gồm các bước tạo embedding cho truy vấn, tìm kiếm vector tương đồng, tổng hợp ngữ cảnh và sinh phản hồi bằng LLM. Đây là các tác vụ có chi phí tính toán và vào/ra dữ liệu cao, khác với các nghiệp vụ CRUD hoặc Collaborative Filtering vốn có độ phức tạp thấp hơn và phần lớn được xử lý cục bộ.

Nhìn chung, Chatbot Service đảm bảo khả năng hoàn thành yêu cầu mà không ghi nhận lỗi, nhưng hiệu năng về thời gian phản hồi vẫn chưa được tối ưu. Trong các phiên bản tiếp theo, hệ thống có thể cải thiện bằng cách tối ưu chiến lược truy xuất, giới hạn số lượng tài liệu ngữ cảnh, giảm số token đầu vào cho LLM, áp dụng cơ chế lưu bộ nhớ đệm cho các truy vấn tương tự hoặc lựa chọn mô hình sinh phản hồi có độ trễ thấp hơn.

3.3.8.4. Kết luận và hướng cải thiện

+ Kết luận chung:

Nhìn chung, hệ thống đáp ứng tốt các yêu cầu chức năng và hiệu năng của một ứng dụng thương mại điện tử thiết bị điện tử trong điều kiện tải trung bình đến cao trên môi trường localhost. Kiến trúc Microservices duy trì hoạt động ổn định, tỷ lệ lỗi ghi nhận bằng 0% và các API nghiệp vụ cốt lõi có thời gian phản hồi tương đối nhất quán

khi số lượng người dùng ảo tăng từ 500 lên 2.000. Bên cạnh đó, module gợi ý sản phẩm hoạt động hiệu quả với độ trễ thấp ở mức mili giây. Tuy nhiên, Chatbot RAG vẫn cần được tối ưu thêm về hiệu năng khi chịu tải cao do đặc thù của quy trình xử lý nhiều bước, bao gồm truy xuất dữ liệu, tổng hợp ngữ cảnh và sinh phản hồi bằng mô hình ngôn ngữ lớn.

+ **Hướng cải thiện:**

+ **Mở rộng môi trường kiểm thử:** Triển khai hệ thống trên máy chủ hoặc nền tảng điện toán đám mây có cấu hình gần với môi trường thực tế để đánh giá chính xác hơn khả năng chịu tải, độ trễ mạng và mức sử dụng tài nguyên.

+ **Tăng khả năng mở rộng dịch vụ:** Áp dụng cơ chế nhân bản dịch vụ, cân bằng tải và tự động mở rộng cho các microservice có lưu lượng truy cập cao nhằm phân phối yêu cầu hiệu quả hơn.

+ **Tối ưu truy vấn và bộ nhớ đệm:** Tiếp tục tối ưu truy vấn MySQL, chỉ mục dữ liệu và phạm vi sử dụng cache để giảm số lần truy cập cơ sở dữ liệu đối với các dữ liệu được đọc thường xuyên.

+ **Tối ưu Chatbot RAG:** Giảm số lượng tài liệu truy xuất, giới hạn token đầu vào, lưu bộ nhớ đệm cho truy vấn tương tự, hỗ trợ phản hồi dạng streaming và lựa chọn mô hình ngôn ngữ có độ trễ thấp hơn.

+ **Bổ sung giám sát hệ thống:** Theo dõi các chỉ số CPU, bộ nhớ, thời gian phản hồi, thông lượng và tỷ lệ lỗi của từng dịch vụ để phát hiện sớm điểm nghẽn và hỗ trợ điều chỉnh cấu hình phù hợp.

CHƯƠNG 4. KẾT QUẢ, KẾT LUẬN VÀ KIẾN NGHỊ

4.1. KẾT QUẢ ĐẠT ĐƯỢC

4.1.1. Về mặt kiến thức và lý thuyết

Trong quá trình thực hiện đồ án, đề tài đã tổng hợp và vận dụng các kiến thức về kiến trúc phần mềm hiện đại, đặc biệt là mô hình Microservices. Các thành phần API Gateway, Eureka và OpenFeign được áp dụng nhằm tổ chức hệ thống theo hướng phân tán, giảm sự phụ thuộc giữa các dịch vụ và hỗ trợ mở rộng linh hoạt.

Về bảo mật, hệ thống sử dụng JWT kết hợp refresh token được lưu trữ trên Redis để xác thực và duy trì phiên đăng nhập của người dùng. Đồng thời, mô hình phân quyền RBAC được áp dụng để kiểm soát quyền truy cập theo từng vai trò, bao gồm khách hàng, quản trị viên và nhân viên vận hành.

Về tổ chức dữ liệu,mô hình database-per-service được triển khai với MySQL,trong đó mỗi microservice quản lý cơ sở dữ liệu thuộc phạm vi nghiệp vụ riêng. Cách tiếp cận này giúp giảm mức độ phụ thuộc giữa các dịch vụ, tăng tính độc lập khi triển khai và tạo điều kiện thuận lợi cho việc mở rộng hệ thống.

Trong lĩnh vực thương mại điện tử, quá trình xây dựng hệ thống giúp củng cố kiến thức về luồng nghiệp vụ đầu cuối, từ quản lý sản phẩm, giỏ hàng, đặt hàng, thanh toán bằng COD hoặc VNPay, tích hợp vận chuyển GHN đến theo dõi trạng thái đơn hàng và đánh giá sản phẩm.

Về học máy, đề tài triển khai dịch vụ gợi ý sản phẩm dựa trên phương pháp Itembased Collaborative Filtering với thuật toán KNNBaseline. Dịch vụ được xây dựng độc lập bằng FastAPI và có nhiệm vụ sinh danh sách Top-N sản phẩm phù hợp cho từng người dùng.

Bên cạnh đó, hệ thống tích hợp chatbot tư vấn sản phẩm theo mô hình Retrieval Augmented Generation (RAG), sử dụng Pinecone để lưu trữ và truy xuất vector, kết hợp mô hình ngôn ngữ được cung cấp thông qua NVIDIA NIM để sinh câu trả lời dựa trên ngữ cảnh tìm kiếm được.

Cuối cùng, đề tài vận dụng kiến thức về kiểm thử hiệu năng thông qua Apache

JMeter. Hệ thống được đánh giá dựa trên các chỉ số như thời gian phản hồi, tỷ lệ lỗi và thông lượng, qua đó xác định khả năng chịu tải cũng như các điểm nghẽn cần tiếp tục tối ưu.

4.1.2. Về mặt sản phẩm và chức năng hệ thống

Đề tài đã xây dựng thành công hệ thống thương mại điện tử Electro Store theo kiến trúc Microservices. Các nhóm chức năng chính được tổ chức theo từng phân hệ người dùng và nghiệp vụ như sau:

Phân hệ Người dùng (User):

Hỗ trợ đăng ký, đăng nhập và xác thực tài khoản bằng JWT, góp phần bảo vệ thông tin người dùng và kiểm soát truy cập vào hệ thống.

Cung cấp quy trình mua sắm trực tuyến gồm tìm kiếm sản phẩm, quản lý giỏ hàng, áp dụng mã giảm giá, thanh toán bằng COD hoặc VNPay và tích hợp dịch vụ vận chuyển GHN.

Cho phép theo dõi trạng thái đơn hàng, đánh giá sản phẩm, quản lý danh sách yêu thích và nhận hỗ trợ thông qua kênh chat hoặc Chatbot AI.

Cung cấp danh sách sản phẩm gợi ý được cá nhân hóa dựa trên hành vi và lịch sử tương tác của người dùng thông qua Recommendation Service.

Phân hệ Nhân viên bán hàng (Sales Staff):

Hỗ trợ xử lý quy trình đơn hàng từ tiếp nhận, xác nhận, đóng gói đến bàn giao cho đơn vị vận chuyển và theo dõi quá trình giao nhận.

Cho phép tiếp nhận và xử lý các yêu cầu trả hàng, bảo hành, đồng thời tương tác trực tiếp với khách hàng qua kênh chat.

Cung cấp thông tin theo dõi tình hình bán hàng và hiệu suất xử lý nghiệp vụ phục vụ công tác vận hành.

Phân hệ Kho vận (Warehouse):

Quản lý quy trình nhập và xuất hàng hóa, theo dõi số lượng tồn kho của từng sản phẩm theo mã IMEI hoặc số serial.

Hỗ trợ lựa chọn hàng hóa xuất kho theo nguyên tắc nhập trước, xuất trước (FIFO).

Tiếp nhận và xử lý các yêu cầu liên quan đến bảo hành, đồng thời đồng bộ thông tin giữa kho và các bộ phận liên quan.

Phân hệ Quản trị (Admin):

Cho phép quản lý người dùng, sản phẩm, đơn hàng, mã giảm giá và bảo hành với cơ chế phân quyền theo vai trò rõ ràng.

Cung cấp Dashboard trực quan để theo dõi các chỉ số quan trọng như doanh thu, số lượng đơn hàng, tỷ lệ chuyển đổi và tình hình hoạt động tổng thể của hệ thống.

Hỗ trợ giám sát hoạt động của các phân hệ, cập nhật dữ liệu và xử lý những cảnh báo phát sinh trong quá trình vận hành.

Dịch vụ AI và gợi ý sản phẩm (AI Service):

Xây dựng Recommendation Service dưới dạng microservice độc lập bằng Python và FastAPI, duy trì hoạt động ổn định trong các kịch bản kiểm thử từ 500 đến 2.000 người dùng ảo.

Tích hợp phương pháp Item-based Collaborative Filtering với thuật toán KNNBaseline để sinh danh sách Top-N sản phẩm phù hợp cho từng người dùng.

Hỗ trợ xử lý trường hợp người dùng mới hoặc thiếu dữ liệu tương tác bằng chiến lược gợi ý thay thế phù hợp, qua đó giảm ảnh hưởng của bài toán khởi động lạnh (cold-start).

Triển khai Chatbot AI tư vấn bằng ngôn ngữ tự nhiên theo kiến trúc RAG, kết hợp cơ sở dữ liệu vector Pinecone và mô hình ngôn ngữ được cung cấp thông qua NVIDIA NIM.

4.2. Đánh giá ưu và nhược điểm

- **Ưu điểm:**

  Kiến trúc hiện đại: Hệ thống được tổ chức theo kiến trúc Microservices với ranh giới nghiệp vụ giữa các dịch vụ được phân tách rõ ràng, tạo điều kiện thuận lợi cho việc bảo trì, triển khai độc lập và mở rộng trong tương lai.

Nghiệp vụ tương đối hoàn chỉnh: Hệ thống đáp ứng các phân hệ vận hành chính gồm Customer, Sales Staff, Warehouse và Admin, đồng thời tích hợp những tiện ích cốt lõi của thương mại điện tử như vận chuyển qua GHN, thanh toán bằng VNPay hoặc COD, quản lý IMEI, bảo hành và thống kê hoạt động.

Tích hợp các giải pháp AI: Hệ thống kết hợp Collaborative Filtering và RAG trong các dịch vụ độc lập. Recommendation Service có tốc độ phản hồi thấp ở mức mili giây và duy trì hoạt động ổn định trong các kịch bản kiểm thử tải cao.

- **Nhược điểm:**

Độ trễ của Chatbot AI: Chatbot sử dụng kiến trúc RAG vẫn có thời gian phản hồi cao khi số lượng người dùng truy cập đồng thời tăng, chủ yếu do quy trình xử lý nhiều bước và sự phụ thuộc vào các dịch vụ AI bên ngoài.

Ràng buộc nghiệp vụ: Một số tình huống phức tạp như khấu trừ tồn kho đồng thời, hoàn tác giao dịch hoặc xử lý thanh toán thất bại cần được kiểm soát và tối ưu chặt chẽ hơn để đáp ứng yêu cầu của hệ thống thương mại điện tử quy mô lớn.

Giới hạn môi trường kiểm thử: Quá trình đánh giá hiệu năng và kiểm thử tải mới được thực hiện trên môi trường localhost, vì vậy chưa phản ánh đầy đủ khả năng chịu tải, độ trễ mạng và mức độ ổn định của hệ thống khi triển khai production trên nhiều node.

4.3. KẾT LUẬN

Đồ án đã hoàn thành mục tiêu đề ra là nghiên cứu và xây dựng hệ thống Electro Store:

Một nền tảng thương mại điện tử B2C có khả năng cá nhân hóa và hỗ trợ nâng cao trải nghiệm mua sắm của người dùng thông qua các thành phần trí tuệ nhân tạo. Kết quả thực hiện cho thấy tính khả thi của việc kết hợp kiến trúc phân tán Microservices với phương pháp Item-based Collaborative Filtering và kiến trúc Chatbot RAG trong một hệ thống thương mại điện tử.

Việc phân tách các miền nghiệp vụ gồm khách hàng, bán hàng, kho vận và quản trị thành những dịch vụ tương đối độc lập giúp hệ thống dễ tổ chức, bảo trì và mở rộng hơn. Bên cạnh đó, Recommendation Service được triển khai như một API độc lập, có khả năng cung cấp danh sách sản phẩm gợi ý với độ trễ thấp. Kết quả kiểm thử tải trên môi trường localhost cho thấy các dịch vụ duy trì hoạt động ổn định và không ghi nhận lỗi trong phạm vi các kịch bản đã thực hiện. Tuy nhiên, kết quả này chỉ phản ánh môi trường thực nghiệm và cần được kiểm chứng thêm khi triển khai trên hạ tầng production.

Sản phẩm của đồ án không chỉ dừng lại ở việc nghiên cứu lý thuyết mà đã được hiện thực hóa thành một hệ thống có các luồng nghiệp vụ chính của thương mại điện tử, tích hợp thanh toán, vận chuyển, quản lý kho, gợi ý sản phẩm và chatbot hỗ trợ. Đây là cơ sở để tiếp tục hoàn thiện và triển khai thử nghiệm trong điều kiện thực tế. Đồng thời, việc xác định hạn chế về độ trễ của Chatbot RAG khi tải tăng đã cung cấp định hướng rõ ràng cho quá trình tối ưu hóa và phát triển hệ thống trong các giai đoạn tiếp theo.

4.4. KIẾN NGHỊ VÀ HƯỚNG PHÁT TRIỂN

Mặc dù hệ thống Electro Store đã triển khai được các chức năng cốt lõi của một hệ thống thương mại điện tử theo kiến trúc Microservices, đề tài xác định một số hướng cải tiến nhằm nâng cao hiệu năng, độ tin cậy và khả năng mở rộng trong tương lai như sau:

- **Tối ưu hóa Chatbot:**

+ Áp dụng cơ chế giới hạn tần suất truy cập (rate limiting) đối với các yêu cầu gửi đến Chatbot Service nhằm kiểm soát chi phí gọi API và hạn chế tình trạng lạm dụng dịch vụ.
+ Xây dựng lớp bộ nhớ đệm cho các câu hỏi phổ biến hoặc có nội dung tương tự, qua đó giảm số lượt gọi đến mô hình ngôn ngữ và rút ngắn thời gian phản hồi.
+ Tách riêng tài nguyên và cơ chế mở rộng cho Chatbot Service so với các dịch vụ nghiệp vụ khác, phù hợp với đặc thù tải không đồng đều và chi phí xử lý cao của tính năng hội thoại.

- **Tối ưu hóa hiệu năng hệ thống:**

+ Bổ sung cơ chế lưu bộ nhớ đệm danh sách Top-N sản phẩm gợi ý theo mã người dùng trên Redis với thời gian sống (TTL) phù hợp, nhằm giảm các phép tính lặp lại tại Recommendation Service trong những lượt truy cập gần nhau.
+ Tinh chỉnh cấu hình HikariCP và các tham số kết nối MySQL để sử dụng hiệu quả connection pool, giảm độ trễ truy vấn khi số lượng kết nối đồng thời tăng cao.
+ Hoàn thiện luồng phối hợp giữa thanh toán và tồn kho trong các trường hợp giao dịch VNPay thất bại hoặc hết thời gian chờ, đảm bảo dữ liệu tồn kho và trạng thái đơn hàng được hoàn tác hoặc cập nhật nhất quán.

- **Triển khai hạ tầng và vận hành hệ thống:**

+ Container hóa các dịch vụ bằng Docker và nghiên cứu điều phối bằng Kubernetes để hỗ trợ triển khai nhất quán, cô lập tài nguyên và mở rộng theo chiều ngang khi lưu lượng truy cập tăng.
+ Bổ sung các công cụ quan sát hệ thống thông qua metrics, logging tập trung và distributed tracing nhằm theo dõi hiệu năng, phân tích luồng yêu cầu và phát hiện sớm điểm nghẽn giữa các dịch vụ.
+ Xây dựng quy trình CI/CD để tự động hóa các bước kiểm thử, đóng gói và triển khai, qua đó giảm sai sót thủ công và rút ngắn thời gian đưa thay đổi vào môi trường thực tế.

- **Nâng cấp mô hình gợi ý:**

+ Kết hợp thêm các tín hiệu hành vi như lượt xem sản phẩm, thời gian ở lại trang chi tiết và lịch sử tìm kiếm bên cạnh dữ liệu đánh giá và mua hàng hiện có, nhằm xây dựng hồ sơ sở thích người dùng đầy đủ hơn.
+ Thiết lập cơ chế đánh giá mô hình định kỳ và thực hiện thử nghiệm A/B để so sánh hiệu quả giữa các phiên bản, làm cơ sở lựa chọn mô hình phù hợp trước khi triển khai chính thức.

TÀI LIỆU THAM KHẢO

[1] Vietnam E-Commerce Association (VECOM), *Vietnam E-Commerce Index Report 2026*, 2026. [Online]. Available: https://vecom.vn/bao-cao-chi-so-thuong-mai-dien-tu-viet-nam-2026.

[2] I. Asrowardi, S. D. Putra, and E. Subyantoro, "Designing microservice architectures for scalability and reliability in e-commerce," *Journal of Physics: Conference Series*, vol. 1450, no. 1, p. 012077, 2020, doi: 10.1088/1742-6596/1450/1/012077.

[3] G. Liu, B. Huang, Z. Liang, M. Qin, H. Zhou, and Z. Li, "Microservices: Architecture, container, and challenges," in *2020 IEEE 20th International Conference on Software Quality, Reliability and Security Companion (QRS-C)*, IEEE, 2020, pp. 629–635.

[4] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2021.

[5] G. Blinowski, A. Ojdowska, and A. Przybyłek, "Monolithic vs. microservice architecture: A performance and scalability evaluation," *IEEE Access*, vol. 10, pp. 20357–20374, 2022, doi: 10.1109/ACCESS.2022.3152803.

[6] G. Linden, B. Smith, and J. York, "Amazon.com recommendations: Item-to-item collaborative filtering," *IEEE Internet Computing*, vol. 7, no. 1, pp. 76–80, 2003, doi: 10.1109/MIC.2003.1167344.

[7] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2023. [Online]. Available: https://arxiv.org/abs/2312.10997.

[8] N. Hug, "Surprise: A Python library for recommender systems," *Journal of Open Source Software*, vol. 5, no. 52, p. 2174, 2020, doi: 10.21105/joss.02174.

[9] Spring Framework, "Spring Cloud Documentation," 2026. [Online]. Available: https://spring.io/projects/spring-cloud/.

[10] Spring Framework, "Spring Boot Reference Documentation," 2026. [Online]. Available: https://docs.spring.io/spring-boot/.

[11] Spring Framework, "Spring AI Reference," 2026. [Online]. Available: https://docs.spring.io/spring-ai/reference/.

[12] Meta Platforms, Inc., "React Documentation," 2026. [Online]. Available: https://react.dev/.

[13] Oracle Corporation, "MySQL Documentation," 2026. [Online]. Available: https://dev.mysql.com/doc/.

[14] Redis Ltd., "Redis Documentation," 2026. [Online]. Available: https://redis.io/docs/latest/.

[15] Docker Inc., "Docker Overview," 2026. [Online]. Available: https://docs.docker.com/get-started/docker-overview/.

[16] S. Ramírez, "FastAPI Documentation," 2024. [Online]. Available: https://fastapi.tiangolo.com/.

[17] MUI, "MUI: The React Component Library," 2026. [Online]. Available: https://mui.com/.

[18] N. Hug, "Surprise Documentation," 2026. [Online]. Available: https://surprise.readthedocs.io/en/stable/.

[19] Auth0, "JSON Web Token Introduction," 2026. [Online]. Available: https://jwt.io/introduction.

[20] NVIDIA Corporation, "NVIDIA NIM API Documentation," 2026. [Online]. Available: https://docs.api.nvidia.com/nim/docs/introduction.

[21] VNPAY, "VNPAY Payment Gateway Integration Documentation," 2026. [Online]. Available: https://sandbox.vnpayment.vn/apis/.

[22] GHN, "GHN API Documentation," 2026. [Online]. Available: https://api.ghn.vn/.

[23] Spring Framework, "Spring Boot Caching Reference," 2026. [Online]. Available: https://docs.spring.io/spring-boot/reference/io/caching.html.

[24] Mozilla, “HTTP Response Status Codes,” *MDN Web Docs*, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTTP/Status.

PHỤ LỤC A. HƯỚNG DẪN NGƯỜI DÙNG

A.1. HƯỚNG DẪN DÀNH CHO NGƯỜI DÙNG MUA HÀNG

A.1.1. Đăng ký tài khoản

Mục đích: Chức năng đăng ký cho phép người dùng tạo một tài khoản cá nhân để sử dụng đầy đủ các tính năng của hệ thống Electro Store như thêm sản phẩm vào giỏ hàng, đặt hàng và quản lý thông tin cá nhân.

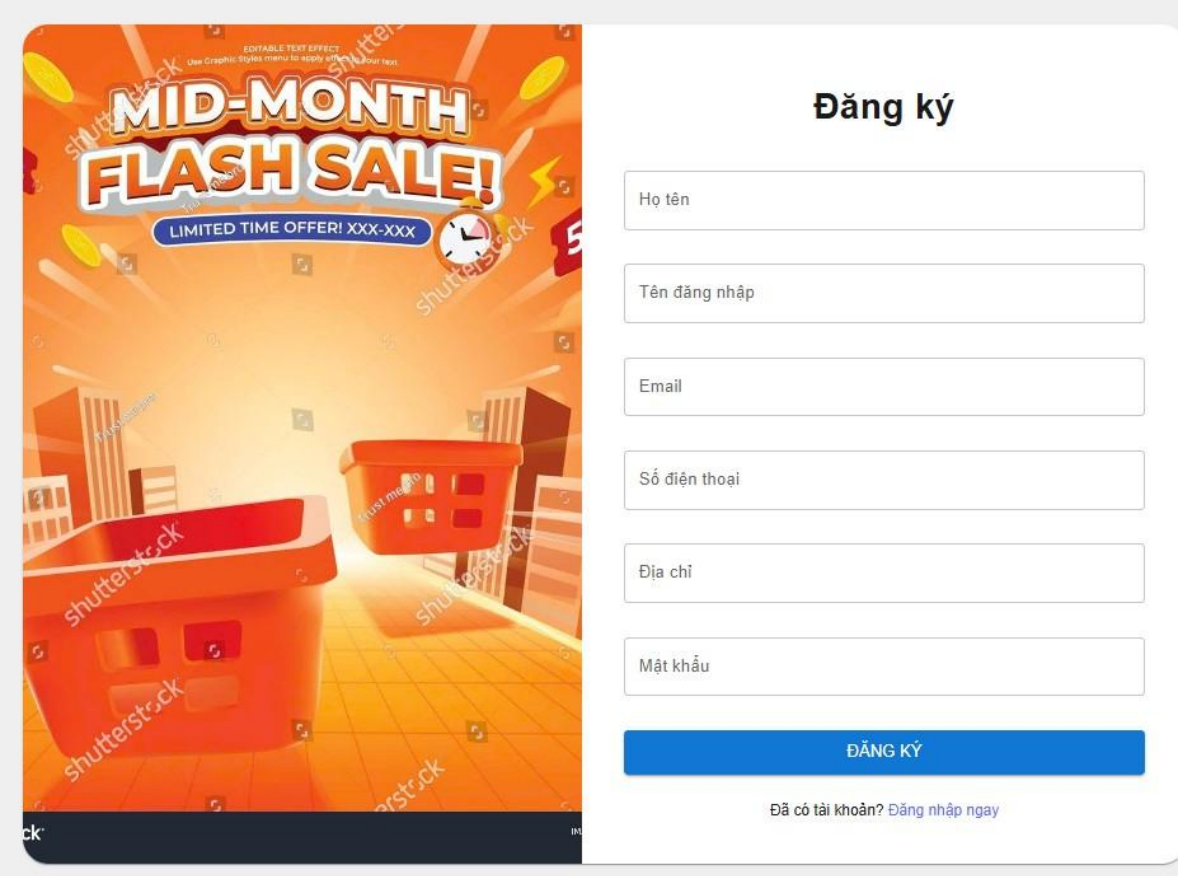
Bước 1: Tại trang chủ, người dùng nhấn vào nút Đăng nhập (màu đỏ) nằm ở góc trên bên phải màn hình.

Tại giao diện đăng nhập, người dùng chọn liên kết "Đăng ký ngay" để chuyển sang trang tạo tài khoản.

Bước 2: Tại trang đăng ký, người dùng tiến hành nhập đầy đủ các thông tin được yêu cầu bao gồm: họ tên, tên đăng nhập, email, số điện thoại, địa chỉ và mật khẩu.

Bước 3: Sau khi hoàn tất việc nhập thông tin, người dùng nhấn nút Đăng ký. Hệ thống sẽ tiến hành kiểm tra dữ liệu đã nhập. Trong trường hợp thông tin chưa đầy đủ hoặc không hợp lệ, hệ thống sẽ hiển thị thông báo để người dùng chỉnh sửa.

Bước 4: Khi tất cả thông tin hợp lệ, tài khoản sẽ được tạo thành công và hệ thống hiển thị thông báo xác nhận. Sau đó, người dùng được chuyển đến trang đăng nhập để bắt đầu sử dụng hệ thống.



Hình 4.1 : Màn hình đăng ký tài khoản của hệ thống Electro Store

A.1.2. Đăng nhập tài khoản

Mục đích: Cho phép người dùng đăng nhập vào hệ thống Electro Store để sử dụng các chức năng cá nhân như theo dõi đơn hàng và nhận gợi ý phù hợp

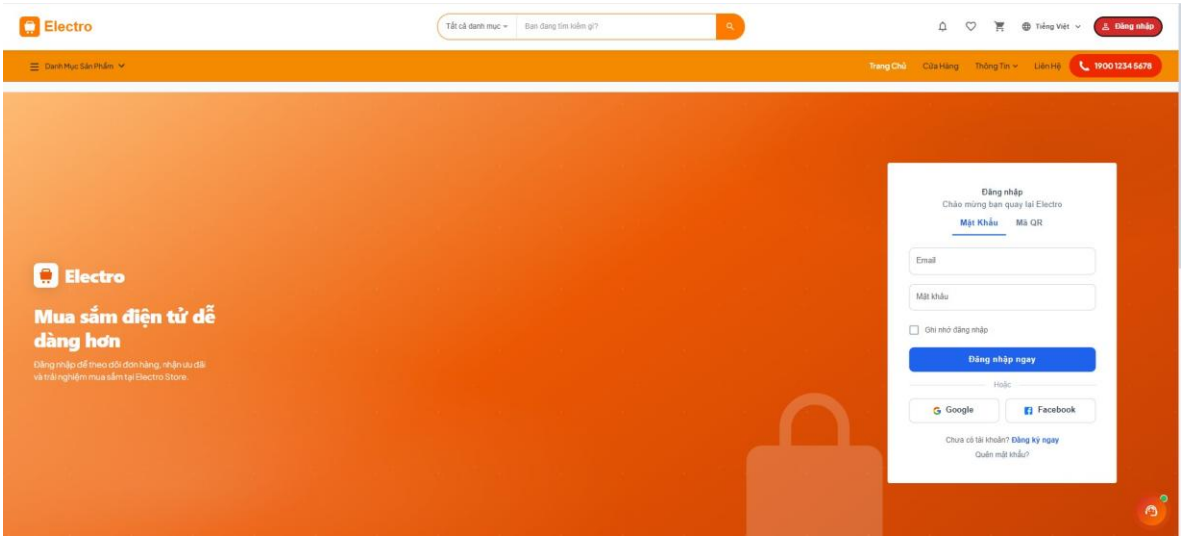
Bước 1: Tại trang chủ, người dùng chọn nút "Đăng nhập" ở góc trên bên phải màn hình.

Bước 2: Hệ thống hiển thị giao diện đăng nhập. Người dùng nhập email và mật khẩu đã đăng ký.

Bước 3: Người dùng có thể chọn tùy chọn ghi nhớ đăng nhập nếu cần.

Bước 4: Nhấn nút "Đăng nhập ngay" để hệ thống xác thực thông tin.

Bước 5: Nếu thông tin hợp lệ, hệ thống chuyển người dùng đến trang chính. Ngược lại, hệ thống hiển thị thông báo lỗi để người dùng kiểm tra lại.



Hình 4.2 : Màn hình đăng nhập của hệ thống Electro Store

A.1.3. Thay đổi ngôn ngữ hiển thị

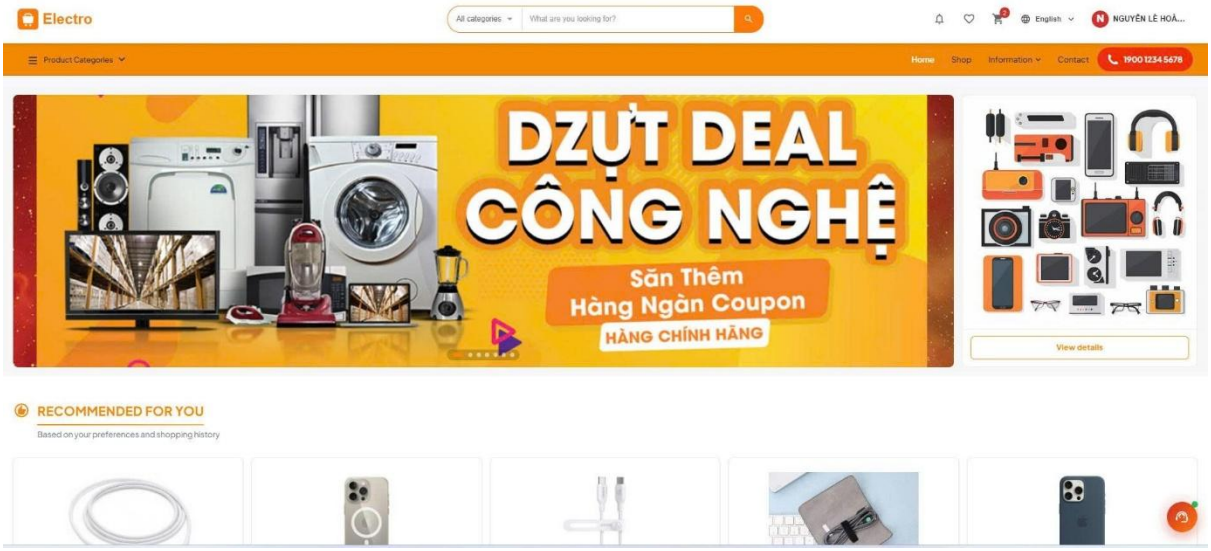
Mục đích: Hỗ trợ người dùng thay đổi ngôn ngữ giao diện của hệ thống (ví dụ: từ Tiếng Việt sang Tiếng Anh) để thao tác và trải nghiệm mua sắm trên Electro Store một cách thuận tiện nhất.

Bước 1: Tại góc trên bên phải của thanh công cụ (cạnh biểu tượng Giỏ hàng và Tài khoản), người dùng nhấn vào biểu tượng Ngôn ngữ (hình quả địa cầu kèm tên ngôn ngữ hiện tại, ví dụ: Tiếng Việt hoặc English).

Bước 2: Một danh sách thả xuống (dropdown menu) sẽ xuất hiện, hiển thị các tùy chọn ngôn ngữ đang được hệ thống hỗ trợ.

Bước 3: Người dùng nhấn chọn vào ngôn ngữ mong muốn cần chuyển đổi.

Bước 4: Hệ thống sẽ ngay lập tức xử lý và tải lại trang. Toàn bộ giao diện, bao gồm danh mục, thanh tìm kiếm, các nút chức năng và thông báo sẽ được đồng bộ sang ngôn ngữ vừa được chọn.



Hình 4.3 : Màn hình thay đổi ngôn ngữ hiển thị

A.1.4. Quên Mật Khẩu

Mục đích: Hỗ trợ khách hàng khôi phục quyền truy cập vào tài khoản Electro Store khi quên mật khẩu đăng nhập.

Bước 1: Tại giao diện Đăng nhập, người dùng nhấn chọn liên kết Quên mật khẩu? để chuyển sang trang khôi phục.

Bước 2: Tại form Quên mật khẩu, người dùng nhập địa chỉ email đã dùng để đăng ký tài khoản vào trường Email.

Bước 3: Người dùng nhấn nút Gửi yêu cầu (màu xanh). Hệ thống tiến hành kiểm tra dữ liệu; nếu email hợp lệ và tồn tại, hệ thống sẽ tự động gửi một email chứa hướng dẫn đặt lại mật khẩu đến hộp thư của người dùng.

Bước 4: Hệ thống hiển thị thông báo xác nhận đã gửi yêu cầu thành công trên màn hình.

Bước 5: Người dùng kiểm tra email, nhấp vào liên kết đính kèm và làm theo hướng dẫn để thiết lập mật khẩu mới.



Hình 4.4 : Màn Hình Quên Mật Khẩu của Hệ Thống Electro Store

A.1.5. Tìm kiếm và khám phá sản phẩm

Mục đích: Hỗ trợ khách hàng tra cứu và tiếp cận sản phẩm mong muốn một cách nhanh chóng, chính xác trên hệ thống Electro Store thông qua từ khóa, bộ lọc danh mục và cơ chế gợi ý tự động.

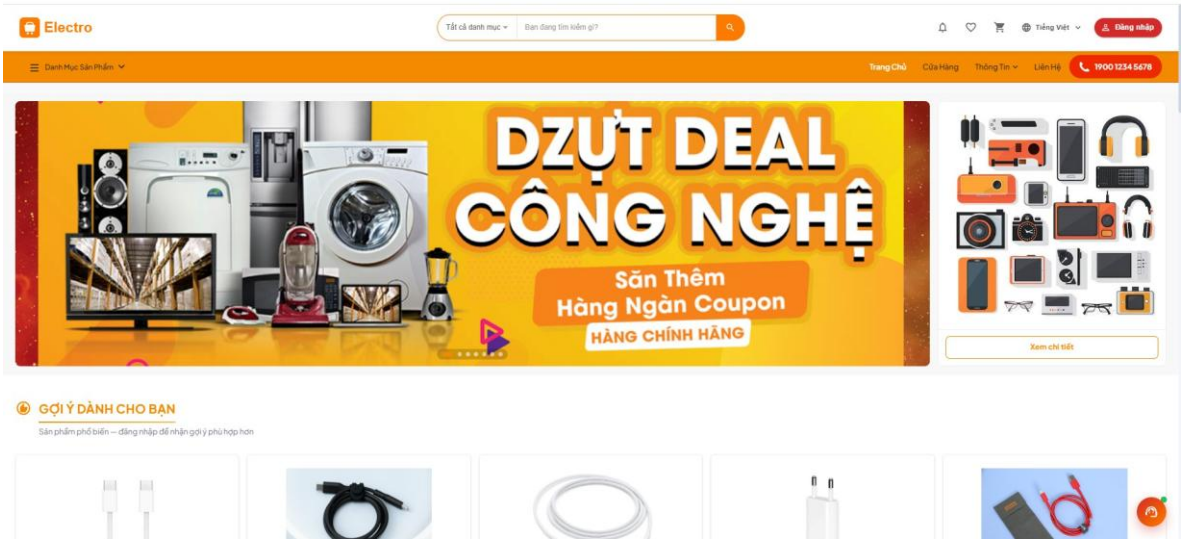
Bước 1: Tại thanh công cụ ở khu vực trên cùng của màn hình, người dùng nhấn vào ô thả xuống Tất cả danh mục để giới hạn phạm vi tìm kiếm theo một nhóm sản phẩm cụ thể (Tùy chọn).

Bước 2: Người dùng nhập tên sản phẩm, thương hiệu hoặc từ khóa liên quan vào ô nhập liệu có dòng chữ Bạn đang tìm kiếm gì?.

Bước 3: Ngay khi người dùng nhập liệu, hệ thống tự động phân tích và hiển thị một danh sách thả xuống chứa các từ khóa gợi ý hoặc sản phẩm liên quan nhất (Smart Suggestion). Người dùng có thể nhấp trực tiếp vào kết quả gợi ý này để xem ngay sản phẩm.

Bước 4: Nếu không chọn từ danh sách gợi ý, người dùng nhấn phím Enter trên bàn phím hoặc nhấp vào biểu tượng Kính lúp để gửi yêu cầu truy vấn toàn diện.

Bước 5: Hệ thống xử lý từ khóa và tự động chuyển hướng người dùng sang trang Kết quả tìm kiếm, hiển thị danh sách toàn bộ các sản phẩm phù hợp.



Hình 4.5 : Màn hình tìm kiếm sản phẩm thông minh

A.1.6. Xem sản phẩm gợi ý cá nhân hóa

Mục đích: Hệ thống tự động đề xuất các sản phẩm phù hợp với nhu cầu và sở thích của khách hàng nhằm tối ưu hóa trải nghiệm mua sắm trên Electro Store.

Bước 1: Tại giao diện Trang chủ, người dùng cuộn chuột xuống khu vực có tiêu đề "GỢI Ý DÀNH CHO BẠN".

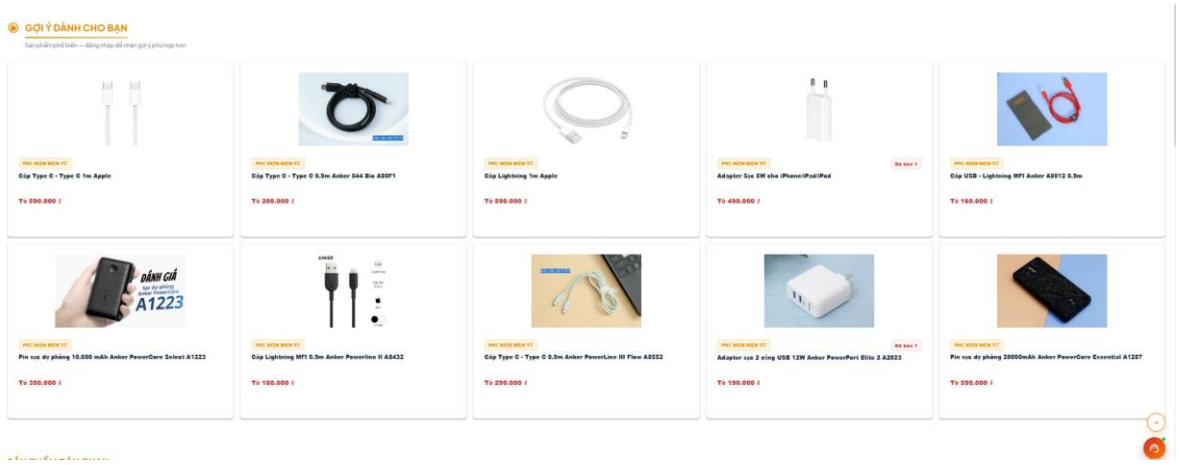
Bước 2: Hệ thống sẽ tự động điều chỉnh nội dung hiển thị dựa trên trạng thái sử dụng của người dùng:

+ **Trường hợp chưa đăng nhập:** Người dùng sẽ thấy danh sách các "Sản phẩm phổ biến" đang được nhiều khách hàng quan tâm nhất. Bên cạnh đó, hệ thống hiển thị thông điệp gợi ý: "Sản phẩm phổ biến – đăng nhập để nhận gợi ý phù hợp hơn", giúp người dùng dễ dàng nhận biết và khuyến khích đăng nhập để trải nghiệm cá nhân hóa tốt hơn.
+ **Trường hợp đã đăng nhập:** Người dùng sẽ được hiển thị danh sách sản phẩm được gợi ý riêng dựa trên sở thích và hành vi mua sắm trước đó, giúp việc tìm kiếm sản phẩm trở nên nhanh chóng và phù hợp hơn với nhu cầu cá nhân.

Bước 3: Người dùng có thể lướt xem danh sách các thẻ sản phẩm (Product Card). Mỗi sản phẩm được hiển thị trực quan với hình ảnh minh họa, các nhãn nổi bật (ví dụ:

Phụ kiện gen Z), tên sản phẩm và giá bán (được hiển thị màu đỏ để dễ nhận diện). Cách trình bày này giúp người dùng dễ dàng nắm bắt thông tin chính và so sánh giữa các sản phẩm một cách thuận tiện.

Bước 4: Người dùng chỉ cần nhấn vào hình ảnh hoặc tên của bất kỳ sản phẩm nào để được chuyển đến trang chi tiết sản phẩm, nơi cung cấp đầy đủ thông tin và lựa chọn mua hàng.

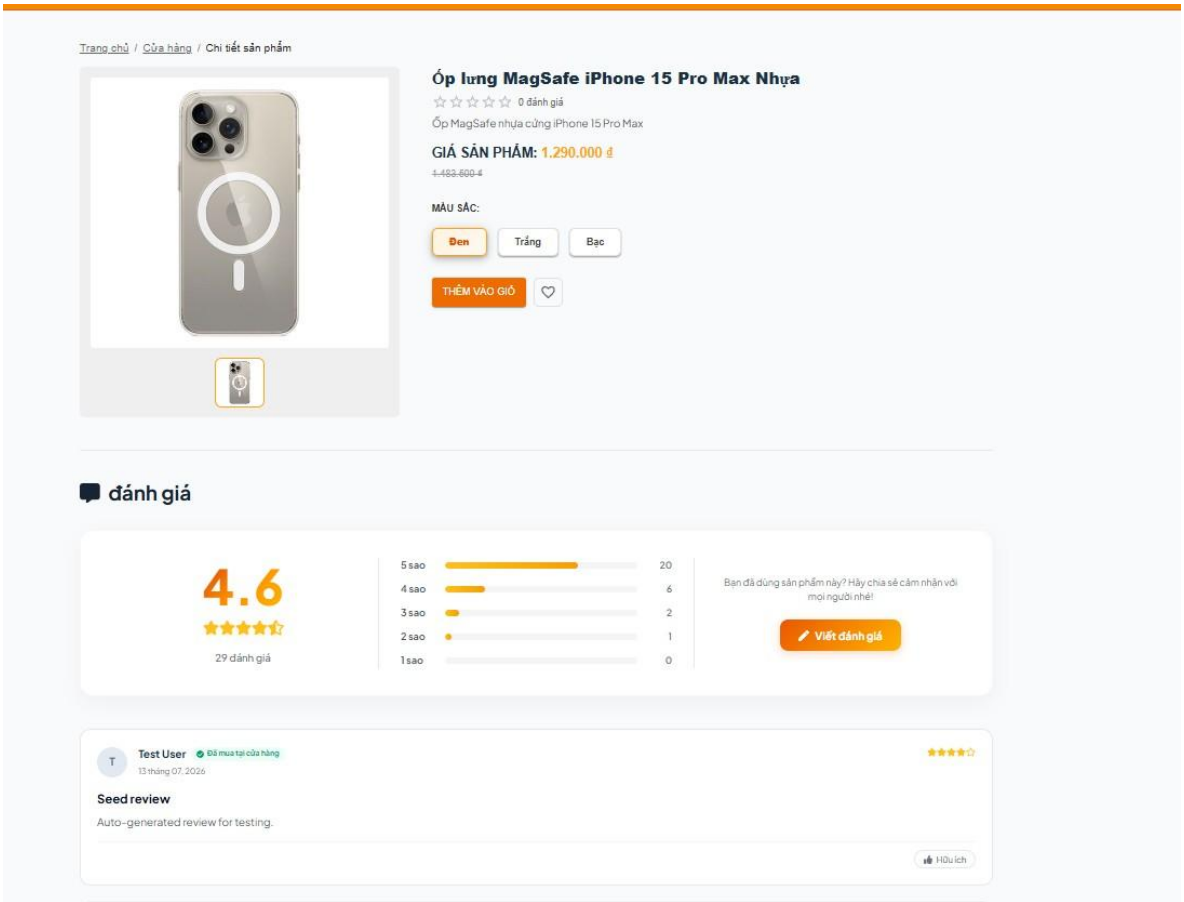


Hình 4.6 : Màn hình gợi ý sản phẩm cho người dùng

A.1.7. Mua hàng và thanh toán

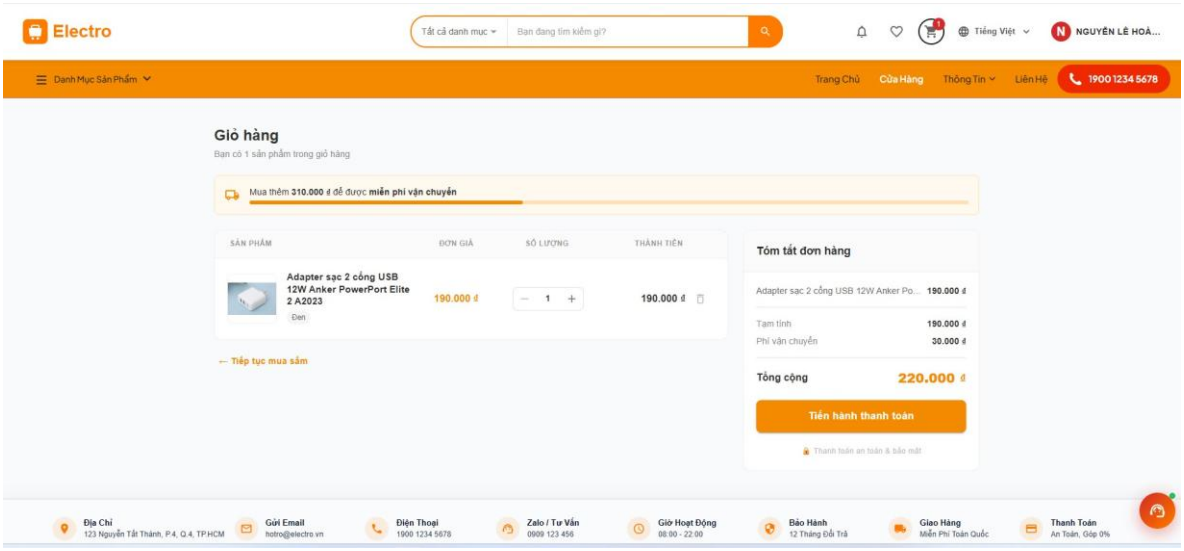
Quy trình mua sắm trên hệ thống được thiết kế liền mạch qua ba bước cơ bản, giúp người dùng dễ dàng lựa chọn sản phẩm, kiểm tra đơn hàng và hoàn tất giao dịch.

Bước 1 – Thêm sản phẩm vào giỏ hàng: Tại trang chi tiết sản phẩm, người dùng có thể xem hình ảnh, thông số, giá bán và các đánh giá từ khách hàng khác. Sau đó, người dùng lựa chọn thuộc tính sản phẩm như màu sắc, phiên bản và số lượng mong muốn, rồi nhấn nút “Thêm vào giỏ” để lưu sản phẩm vào giỏ hàng.



Hình 4.7 :Màn hình chi tiết sản phẩm và thao tác thêm vào giỏ hàng

Bước 2 – Quản lý giỏ hàng và tiến hành đặt hàng: Tại giao diện giỏ hàng, người dùng kiểm tra danh sách sản phẩm đã chọn, đơn giá, số lượng, thành tiền và phí vận chuyển dự kiến. Người dùng có thể tăng hoặc giảm số lượng, xóa sản phẩm không còn nhu cầu hoặc tiếp tục mua sắm. Sau khi kiểm tra tổng giá trị đơn hàng, người dùng chọn “Tiến hành thanh toán” để chuyển sang bước xác nhận thông tin.



Hình 4.8 : Màn hình quản lý sản phẩm trong giỏ hàng

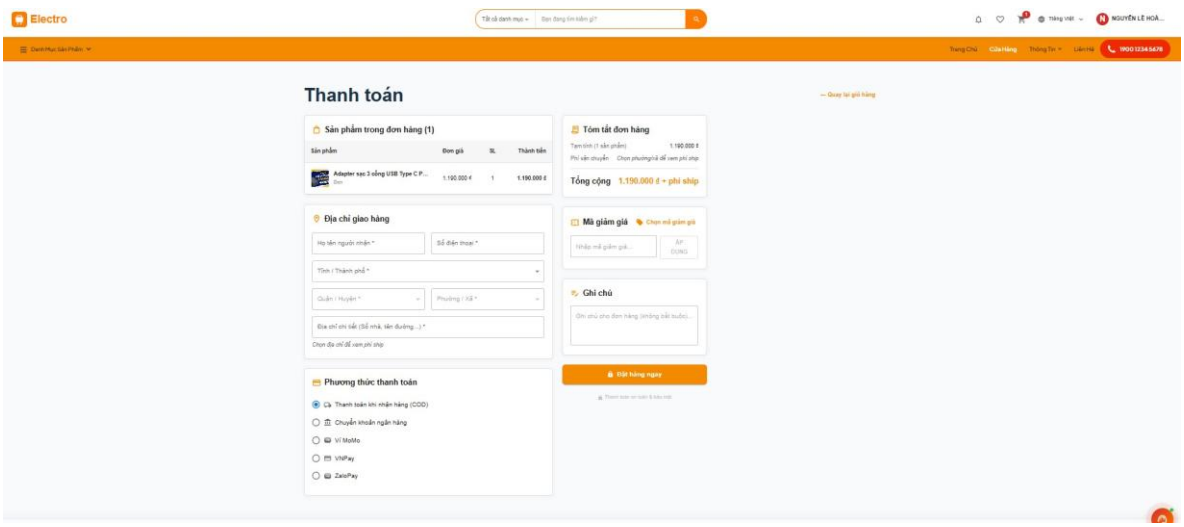
Bước 3 – Xác nhận thông tin và thanh toán: Tại giao diện thanh toán, người dùng rà soát sản phẩm trong đơn hàng và thực hiện các thao tác sau:

+ **Cung cấp thông tin giao hàng:** Nhập họ tên, số điện thoại và địa chỉ nhận hàng gồm tỉnh hoặc thành phố, quận hoặc huyện, phường hoặc xã và địa chỉ chi tiết. Thông qua API của Giao Hàng Nhanh (GHN), hệ thống tính toán phí vận chuyển dựa trên địa chỉ được lựa chọn.

+ **Áp dụng mã giảm giá:** Người dùng nhập hoặc lựa chọn mã giảm giá phù hợp. Nếu mã hợp lệ và đáp ứng điều kiện áp dụng, giá trị khuyến mãi được trừ trực tiếp vào tổng giá trị đơn hàng.

+ **Chọn phương thức thanh toán:** Hệ thống hỗ trợ thanh toán khi nhận hàng (COD), chuyển khoản ngân hàng và thanh toán trực tuyến qua các cổng hoặc ví điện tử như VNPay, MoMo và ZaloPay.

+ **Hoàn tất đặt hàng:** Sau khi kiểm tra toàn bộ thông tin, người dùng nhấn "Đặt hàng ngay". Hệ thống xử lý giao dịch, tạo đơn hàng và cung cấp mã đơn hàng để người dùng tra cứu, theo dõi trạng thái giao nhận.



Hình 4.9 : Màn hình xác nhận thông tin giao hàng và thanh toán

A.1.8. Quản lý hồ sơ cá nhân và lịch sử đơn hàng

Mục đích: Giúp khách hàng xem, quản lý và cập nhật thông tin cá nhân trên hệ thống Electro Store nhằm đảm bảo tính chính xác trong quá trình mua sắm và giao nhận.

Các bước thực hiện:

Bước 1: Người dùng truy cập vào hệ thống Electro Store và đăng nhập vào tài khoản cá nhân.

Bước 2: Tại giao diện chính, người dùng chọn mục Tài Khoản Của Tôi.

Bước 3: Trong khu vực tài khoản, người dùng tiếp tục chọn mục Hồ Sơ để truy cập trang quản lý thông tin cá nhân.

Bước 4: Hệ thống hiển thị giao diện Hồ Sơ Của Tôi, bao gồm các thông tin cá nhân hiện tại của người dùng.

Bước 5: Người dùng tiến hành kiểm tra và chỉnh sửa các trường thông tin được phép cập nhật, bao gồm:

+ Họ và tên
+ Email (hiển thị một phần và được bảo mật)
+ Số điện thoại
+ Địa chỉ giao hàng
+ Giới tính

+ Ngày sinh

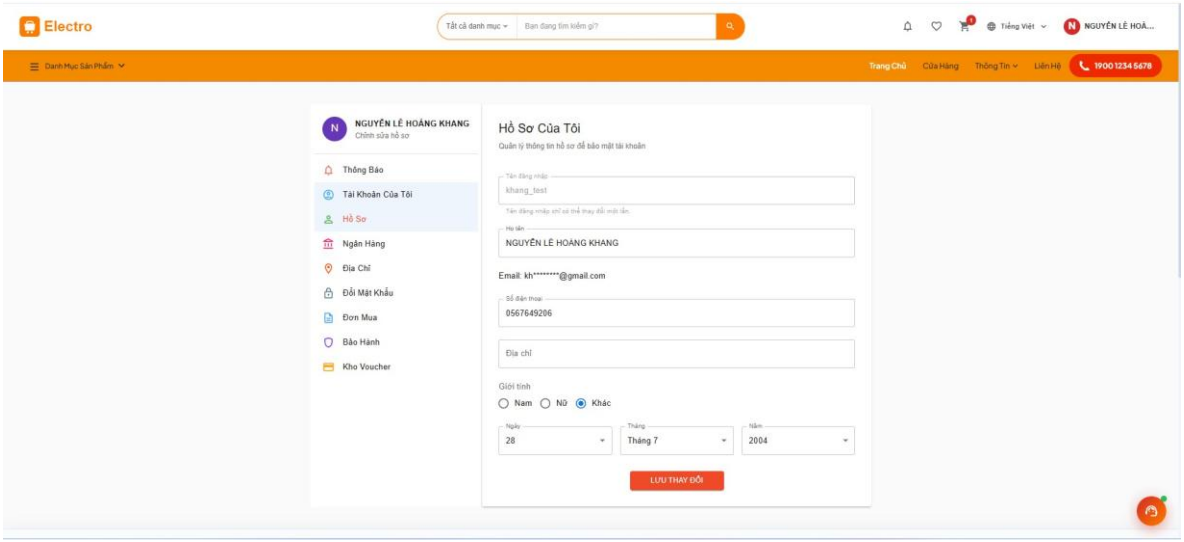
Lưu ý: Tên đăng nhập chỉ được phép thay đổi một lần theo quy định của hệ thống.

Bước 6: Sau khi hoàn tất việc chỉnh sửa, người dùng nhấn nút **LƯU THAY ĐỔI** để gửi yêu cầu cập nhật thông tin.

Bước 7: Hệ thống tiếp nhận yêu cầu, tiến hành kiểm tra tính hợp lệ của dữ liệu (định dạng email, số điện thoại, các trường bắt buộc,...).

Bước 8: Nếu dữ liệu hợp lệ, hệ thống cập nhật thông tin mới vào cơ sở dữ liệu.

Bước 9: Hệ thống hiển thị thông báo xác nhận cập nhật thông tin thành công cho người dùng.



Hình 4.10 : Màn hình quản lý hồ sơ cá nhân của người dùng

A.1.9. Hướng dẫn tra cứu bảo hành

Mục đích: Chức năng Tra cứu bảo hành giúp người dùng kiểm tra nhanh tình trạng và thời hạn bảo hành của thiết bị, đồng thời hỗ trợ hệ thống xác thực thông tin sản phẩm trước khi tiếp nhận yêu cầu bảo hành, góp phần tối ưu hóa quy trình xử lý và giảm tải cho bộ phận hỗ trợ kỹ thuật.

Bước 1: Người dùng vào trang Tra Cứu Bảo Hành thông qua thanh điều hướng hoặc liên kết ở chân trang.

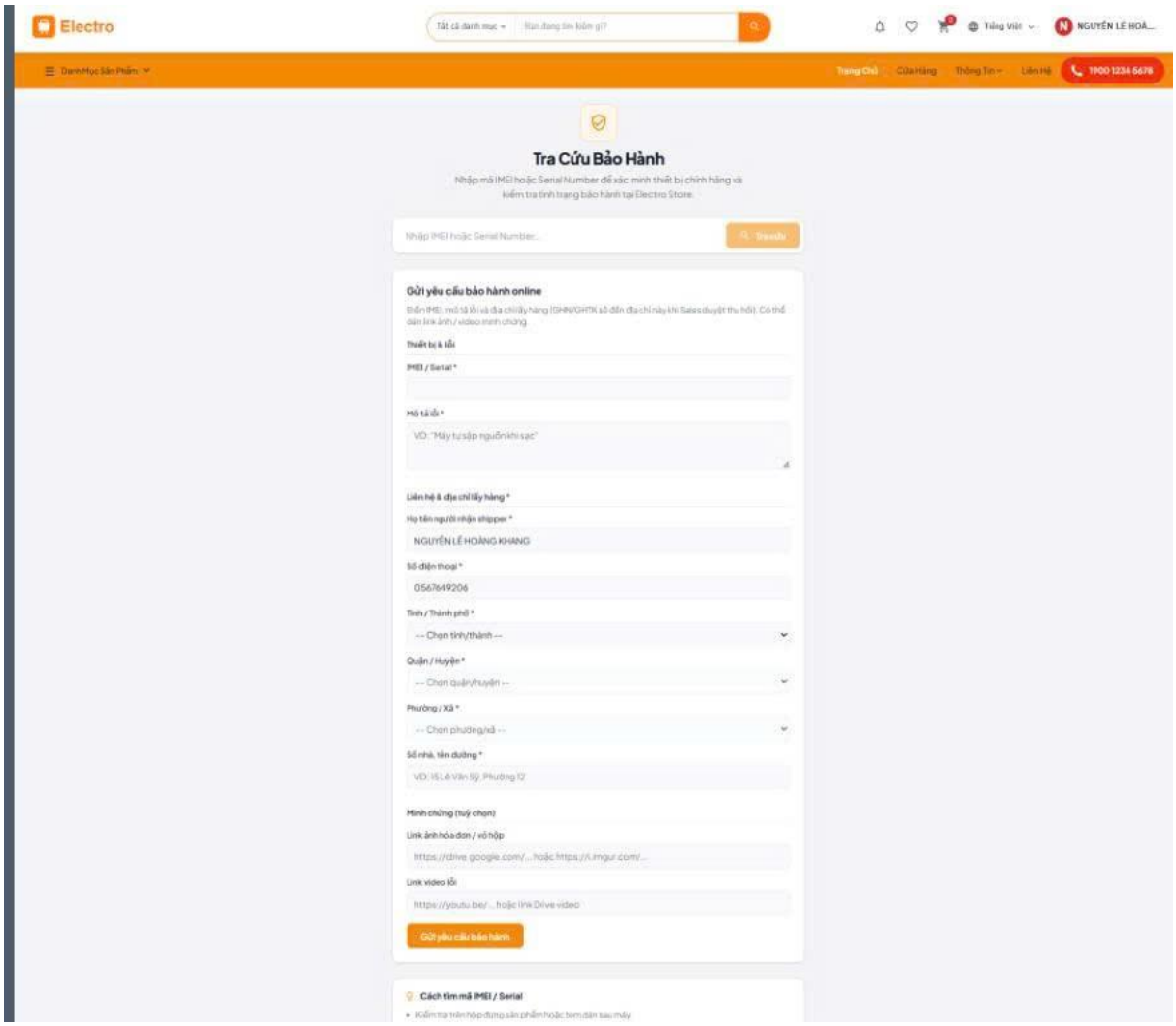
Bước 2: Nhập mã IMEI hoặc Serial Number của sản phẩm vào ô nhập liệu (có thể xem hướng dẫn tìm mã ở cuối trang), sau đó nhấn nút Tra cứu (màu cam). Hệ thống sẽ truy xuất dữ liệu và hiển thị tình trạng cũng như thời hạn bảo hành của thiết bị.

Bước 3: Điền mã IMEI/Serial của thiết bị gặp lỗi và mô tả chi tiết tình trạng hư hỏng (ví dụ: "Máy tự sập nguồn khi sạc").

Bước 4: Nhập họ tên, số điện thoại người nhận; chọn Tỉnh/Thành phố, Quận/Huyện, Phường/Xã và điền số nhà, tên đường để phục vụ việc thu hồi thiết bị.

Bước 5: Đính kèm link (Google Drive, Imgur, YouTube...) chứa hình ảnh hóa đơn, vỏ hộp hoặc video ghi nhận lỗi thiết bị.

Bước 6: Rà soát lại toàn bộ thông tin và nhấn nút Gửi yêu cầu bảo hành (màu cam). Hệ thống tiếp nhận hồ sơ, gửi thông báo xác nhận và chuyển yêu cầu đến bộ phận Sales để xử lý thu hồi thiết bị.



Hình 4.11 : Màn Hình Hướng dẫn Tra Cứu Bảo Hành

A.1.10. Hướng dẫn Đánh giá sản phẩm

Mục đích: Khách hàng chia sẻ trải nghiệm, cảm nhận thực tế và chấm điểm chất lượng sản phẩm sau khi mua hàng tại Electro Store, từ đó cung cấp thêm thông tin tham khảo khách quan cho cộng đồng người mua.

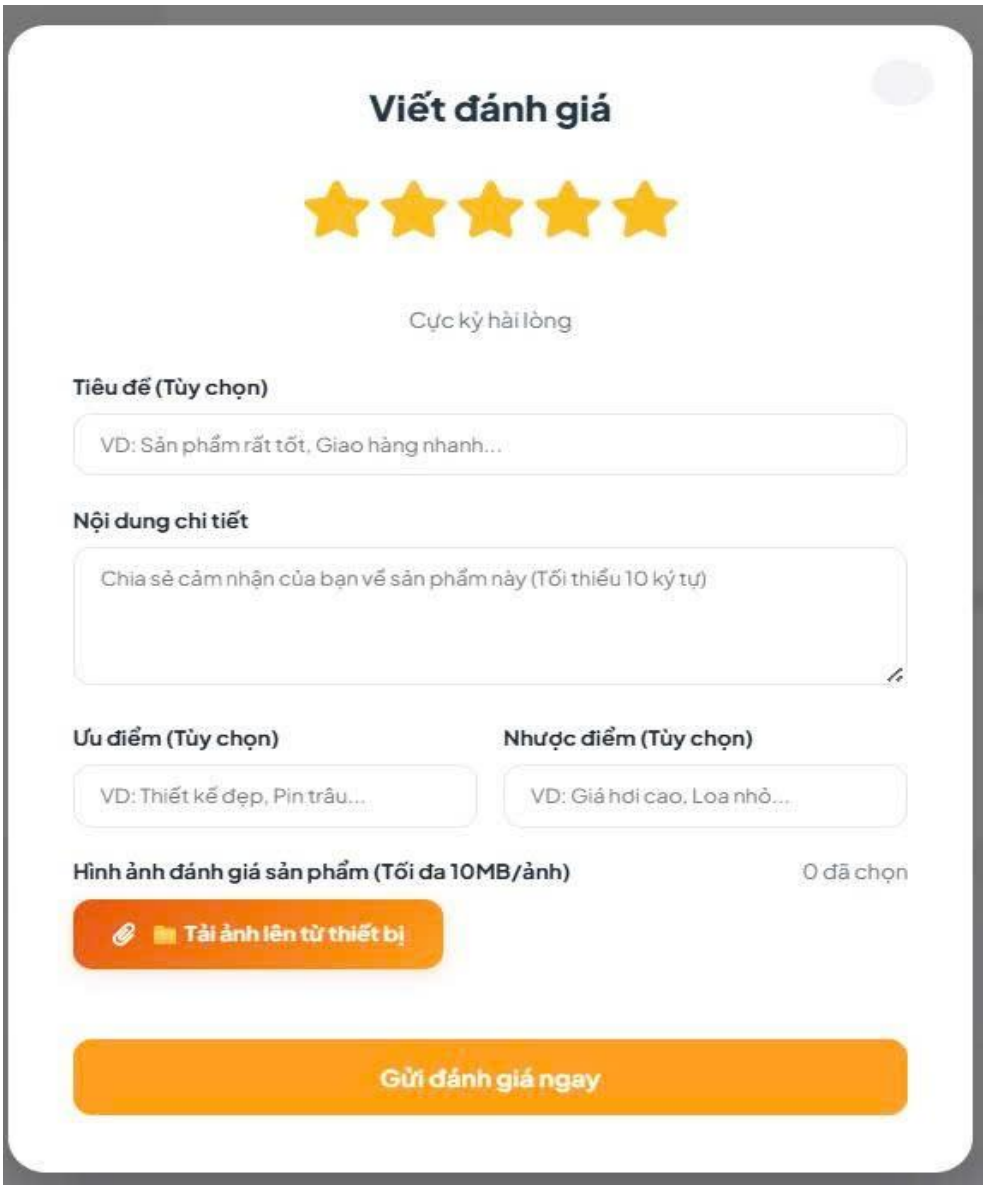
Bước 1: Tại trang Chi tiết sản phẩm hoặc trong danh sách Đơn mua, người dùng nhấn nút Viết đánh giá. Hệ thống sẽ hiển thị một cửa sổ nhỏ (hộp thoại) ngay trên màn hình chứa biểu mẫu đánh giá.

Bước 2: Người dùng lựa chọn số lượng ngôi sao (từ 1 đến 5 sao) tương ứng với mức độ hài lòng, trong đó mức 5 sao thể hiện mức độ “Cực kỳ hài lòng”.

Bước 3: Người dùng nhập Tiêu đề (không bắt buộc, dùng để tóm tắt ngắn gọn cảm nhận), Nội dung chi tiết (bắt buộc, tối thiểu 10 ký tự) và có thể cung cấp thêm Ưu điểm/Nhược điểm (không bắt buộc) để mô tả rõ hơn trải nghiệm thực tế.

Bước 4: Người dùng nhấn nút Tải ảnh lên (màu cam) để bổ sung hình ảnh thực tế của sản phẩm (tối đa 10MB/ảnh).

Bước 5: Người dùng kiểm tra lại nội dung và nhấn Gửi đánh giá ngay (màu cam). Hệ thống tiếp nhận, xử lý và hiển thị bài đánh giá tại khu vực bình luận của trang sản phẩm.



Hình 4.12 : Màn Hình Hướng dẫn Đánh Giá Sản Phẩm

A.1.11. Sử dụng Chatbot AI tư vấn

Mục đích: Trợ lý Electro là chatbot AI tích hợp trên hệ thống, giúp khách hàng nhanh chóng tìm được sản phẩm phù hợp với nhu cầu mà không cần tự tìm kiếm và so sánh thủ công. Khách hàng chỉ cần mô tả nhu cầu bằng ngôn ngữ tự nhiên, chatbot sẽ phân tích và đề xuất sản phẩm tương ứng.

Bước 1: Tại bất kỳ trang nào trên hệ thống, người dùng nhấn vào biểu tượng Trợ lý Electro ở góc dưới bên phải màn hình. Cửa sổ trò chuyện sẽ mở ra ngay trên trang hiện tại mà không cần chuyển sang trang khác.

Bước 2: Người dùng gõ trực tiếp câu hỏi hoặc mô tả nhu cầu vào ô nhập liệu, ví dụ: "Laptop học lập trình dưới 15 triệu". Nếu chưa biết bắt đầu từ đâu, người dùng có thể chọn một trong các câu hỏi gợi ý có sẵn trong giao diện để hệ thống hỗ trợ định hướng nhu cầu.

Bước 3: Sau khi nhập xong, người dùng nhấn nút Gửi. Hệ thống sẽ xử lý yêu cầu và trả về danh sách sản phẩm phù hợp, kèm theo thông tin cấu hình, giá bán và một số lựa chọn thay thế nếu sản phẩm chính chưa đáp ứng đầy đủ tiêu chí. Người dùng có thể xem chi tiết từng sản phẩm hoặc thêm trực tiếp vào giỏ hàng ngay trong giao diện chat.

Bước 4: Khi muốn bắt đầu một nhu cầu tư vấn mới, người dùng nhấn biểu tượng Làm mới để xóa toàn bộ nội dung hội thoại trước đó và khởi tạo một phiên trò chuyện mới.

Bước 5: Người dùng nhấn nút X ở góc cửa sổ chat để đóng chatbot khi không còn nhu cầu sử dụng. Cửa sổ có thể được mở lại bất kỳ lúc nào bằng cách nhấn lại biểu tượng Trợ lý Electro.



Hình 4.13 : Giao diện Trợ lý Electro hỗ trợ tư vấn sản phẩm

A.2. HƯỚNG DẪN DÀNH CHO QUẢN TRỊ VIÊN (ADMIN)

A.2.1. Tổng quan (Dashboard)

Bước 1: Quản trị viên truy cập vào trang đăng nhập của hệ thống. Đây là trang đăng nhập dùng chung cho tất cả người dùng, bao gồm cả khách hàng và nhân viên quản trị, nên sẽ không có trang đăng nhập riêng dành cho Admin. Tại màn hình đăng nhập, nhập thông tin tài khoản đã được cấp quyền quản trị, bao gồm Email (hoặc Tên đăng nhập) và Mật khẩu, sau đó nhấn nút "Đăng nhập ngay" để tiếp tục. Sau khi đăng nhập, hệ thống sẽ tự động kiểm tra quyền của tài khoản. Nếu tài khoản thuộc nhóm khách hàng, hệ thống sẽ chuyển về trang chủ. Nếu tài khoản thuộc nhóm quản trị hoặc nhân viên vận hành, hệ thống sẽ tự động chuyển đến khu vực quản trị. Đối với tài khoản Quản trị viên, sau khi xác thực thành công, hệ thống sẽ điều hướng trực tiếp đến giao diện Dashboard (Báo cáo Thống kê). Đây là màn hình tổng quan, hiển thị các chỉ số quan trọng như doanh thu, số lượng đơn hàng, tình trạng sản phẩm và các hoạt động gần nhất, giúp quản trị viên nhanh chóng nắm bắt tình hình vận hành của hệ thống ngay sau khi đăng nhập. Lưu ý: Người dùng không cần tìm hoặc truy cập một trang Admin

riêng biệt. Chỉ cần đăng nhập đúng tài khoản được cấp quyền, hệ thống sẽ tự động hiển thị giao diện và chức năng phù hợp với vai trò của người dùng.

Bước 2: Cập nhật hệ thống AI (Tuỳ chọn). Tại khu vực trên cùng, Quản trị viên có thể nhấn nút Cập nhật mô hình gợi ý (màu cam) để chủ động kích hoạt quá trình làm mới mô hình Collaborative Filtering khi hệ thống có sự thay đổi về dữ liệu mua hàng và đánh giá.

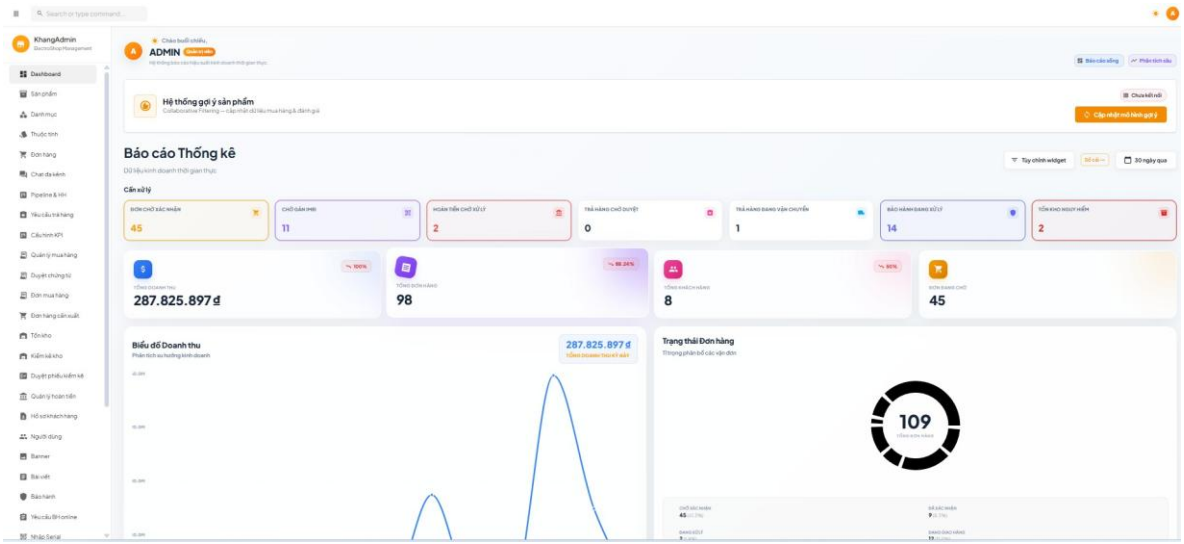
Bước 3: Xử lý các công việc ưu tiên. Quản trị viên kiểm tra khu vực Cần lưu ý. Hệ thống tổng hợp sẵn các nghiệp vụ khẩn cấp thành các thẻ (ví dụ: Đơn chờ xác nhận, Chờ gán IMEI, Hoàn tiền chờ xử lý, Tồn kho nguy hiểm). Nhấn trực tiếp vào các thẻ này để chuyển đến trang chi tiết và nhanh chóng điều phối công việc.

Bước 4: Theo dõi các chỉ số trọng yếu. Rà soát các thẻ chỉ số lớn ở khu vực giữa màn hình. Hệ thống cung cấp nhanh các số liệu về Tổng doanh thu, Tổng đơn hàng, Số lượng khách hàng và Số đơn chờ duyệt giúp Quản trị viên đánh giá khái quát tình trạng hoạt động trong khoảng thời gian được lựa chọn.

Bước 5: Phân tích dữ liệu chuyên sâu. Quản trị viên cuộn xuống khu vực biểu đồ bên dưới để quan sát:

Biểu đồ Doanh thu: Trực quan hóa xu hướng kinh doanh và sự biến động theo thời gian.

Trạng thái Đơn hàng: Phân tích tỷ trọng các đơn hàng hiện tại thông qua biểu đồ tròn để nhận diện những trạng thái đang chiếm tỷ lệ cao.



Hình 4.14 : Dashboard báo cáo và thống kê dành cho Quản trị viên

A.2.2. Quản lý sản phẩm

Mục đích: Cung cấp công cụ giúp Quản trị viên kiểm soát toàn diện danh mục hàng hóa, quản lý vòng đời sản phẩm, cập nhật giá cả, tồn kho và các chương trình khuyến mãi trong toàn bộ hệ thống Electro Store.

Điều kiện thực hiện: Quản trị viên đã đăng nhập thành công và đang ở khu vực quản trị của hệ thống.

Bước 1: Tại thanh điều hướng bên trái màn hình, Quản trị viên nhấn chọn mục Sản phẩm. Hệ thống sẽ truy xuất dữ liệu và hiển thị giao diện Quản Lý Sản Phẩm.

Bước 2: Theo dõi số liệu tổng quan. Ở khu vực trên cùng của giao diện, Quản trị viên quan sát các thẻ chỉ số lớn để nhanh chóng nắm bắt trạng thái danh mục hàng hóa. Hệ thống phân loại rõ ràng thành: Tổng sản phẩm, Đang hoạt động và Đã ẩn.

Bước 3: Tìm kiếm và lọc sản phẩm. Để rút ngắn thời gian tra cứu và xử lý dữ liệu, Quản trị viên nhập từ khóa vào ô Tìm kiếm sản phẩm... kết hợp cùng các menu thả xuống để lọc danh sách theo các tiêu chí cụ thể như: Danh mục, Nhà sản xuất, hoặc Trạng thái.

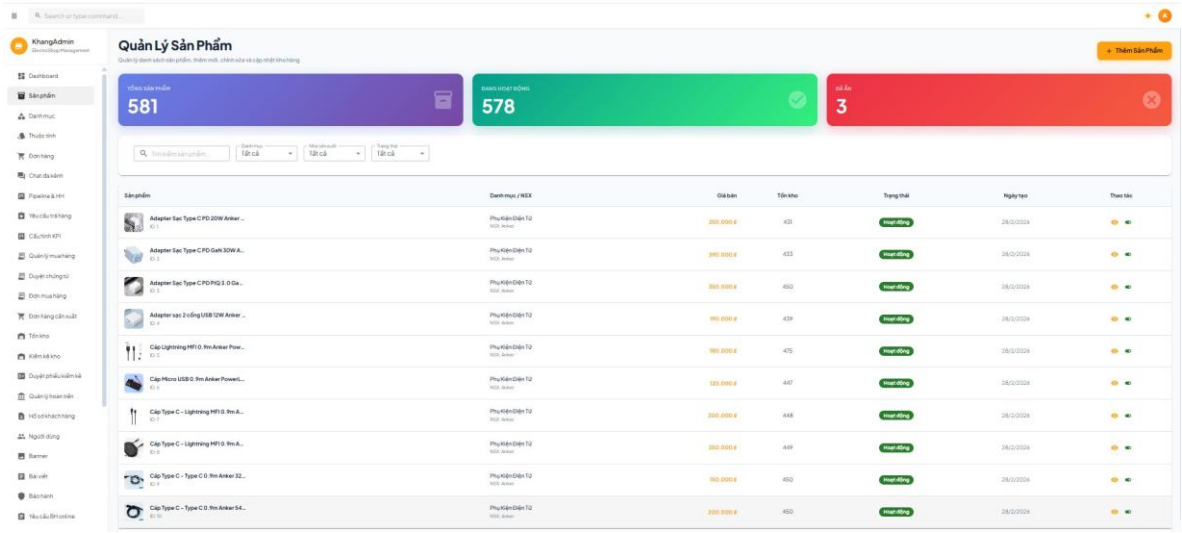
Bước 4: Thêm mới sản phẩm. Khi có nhu cầu cập nhật hàng hóa mới lên hệ thống, Quản trị viên nhấn vào nút Thêm Sản Phẩm ở góc trên bên phải màn hình.

Bước 5: Quản lý vòng đời và trạng thái kinh doanh. Tại bảng danh sách sản phẩm, Quản trị viên chú ý đến cột Thao tác ở cuối mỗi dòng. Nhấn vào các biểu tượng chức năng tương ứng để:

Xem và chỉnh sửa thông tin chi tiết (cập nhật giá bán, giá khuyến mãi, số lượng tồn kho, thuộc tính).

Thay đổi trạng thái hiển thị (tạm ẩn những sản phẩm ngừng kinh doanh).

Xóa dữ liệu sản phẩm khi đáp ứng đủ các điều kiện nghiệp vụ của hệ thống



Hình 4.15 : Màn hình quản lý danh sách sản phẩm dành cho Quản trị viên

A.2.3. Quản lý người dùng:

Mục đích: Cung cấp công cụ giúp Quản trị viên kiểm soát hồ sơ tài khoản trong toàn bộ hệ thống, điều chỉnh quyền hạn và xử lý các trường hợp vi phạm nhằm đảm bảo an toàn cho nền tảng Electro Store.

Điều kiện thực hiện: Quản trị viên đã đăng nhập thành công và đang ở khu vực quản trị của hệ thống.

Bước 1: Tại thanh điều hướng bên trái màn hình, Quản trị viên nhấn chọn mục Người dùng. Hệ thống sẽ truy xuất dữ liệu và hiển thị giao diện Quản Lý Người Dùng.

Bước 2: Theo dõi trạng thái tài khoản. Ở khu vực trên cùng của giao diện, Quản trị viên quan sát các thẻ chỉ số để nhanh chóng nắm bắt tình hình nhân sự và khách

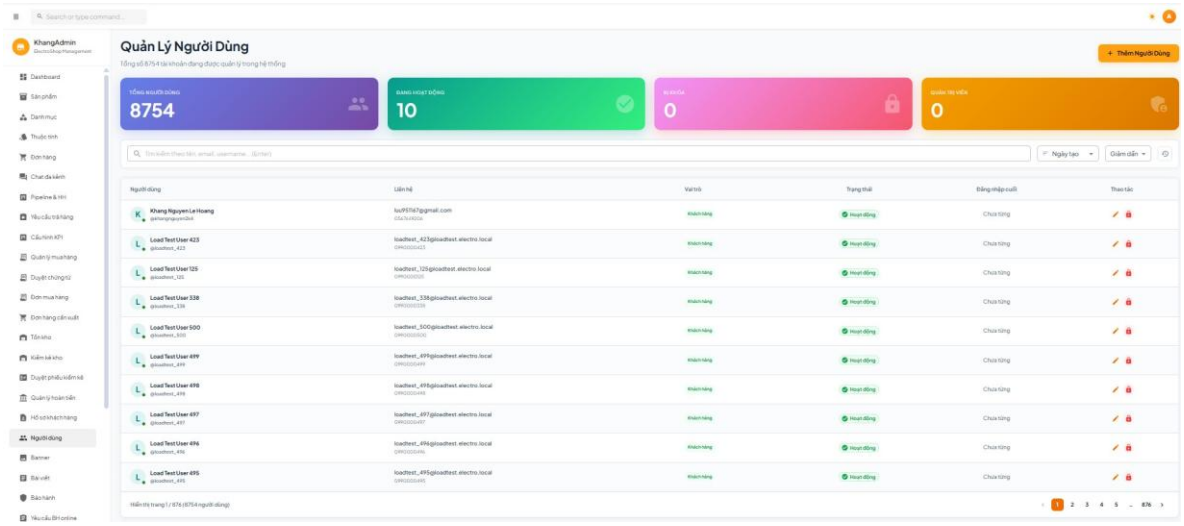
hàng. Hệ thống hiển thị chi tiết: Tổng người dùng, Đang hoạt động, Bị khóa và Quản trị viên.

Bước 3: Tra cứu thông tin người dùng. Để tìm kiếm nhanh, Quản trị viên nhập từ khóa vào thanh tìm kiếm (có thể tìm theo họ tên, email hoặc tên đăng nhập). Bảng dữ liệu phía dưới sẽ lập tức hiển thị các thông tin liên quan như: thông tin liên hệ, vai trò, trạng thái hoạt động và thời điểm đăng nhập cuối cùng của tài khoản.

Bước 4: Phân quyền và Kiểm soát tài khoản. Tại bảng danh sách người dùng, Quản trị viên chú ý đến cột Thao tác ở cuối mỗi dòng. Nhấn vào các biểu tượng chức năng tương ứng để thực hiện:

+ **Phân quyền (Biểu tượng cây bút):** Điều chỉnh vai trò của tài khoản sao cho phù hợp với phạm vi trách nhiệm (ví dụ: khách hàng, nhân viên, hoặc Quản trị viên).
+ **Kiểm soát vi phạm (Biểu tượng ổ khóa):** Khóa các tài khoản có hành vi vi phạm chính sách hoặc gây rủi ro cho hệ thống. Trạng thái tài khoản sẽ được cập nhật và Quản trị viên có thể mở khóa trở lại sau khi đã xử lý xong vi phạm.

Bước 5: Thêm người dùng mới (Tùy chọn). Quản trị viên có thể nhấn vào nút + Thêm Người dùng ở góc trên bên phải màn hình để chủ động cấp mới một tài khoản vào hệ thống mà không cần thông qua luồng đăng ký của khách hàng.



Hình 4.16 : Màn hình quản lý tài khoản người dùng dành cho Quản trị viên

A.2.4. Quản lý đơn hàng

Mục đích: Cho phép Quản trị viên theo dõi, tra cứu và xử lý toàn bộ đơn hàng trong hệ thống Electro Store.

Điều kiện thực hiện: Quản trị viên đã đăng nhập thành công và đang ở khu vực quản trị của hệ thống.

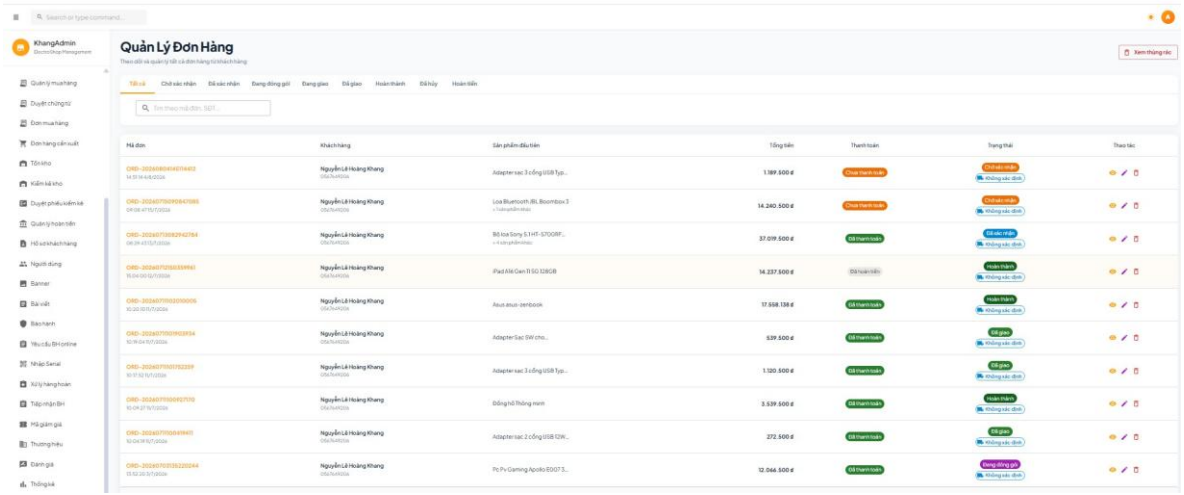
Bước 1: Tại thanh điều hướng, chọn mục Đơn hàng để truy cập giao diện quản lý.

Bước 2: Quản trị viên có thể lọc và tìm kiếm đơn hàng bằng cách chọn các tab trạng thái như Chờ xác nhận, Đã xác nhận, Đang giao, Hoàn thành hoặc Hủy để hiển thị danh sách đơn hàng tương ứng, đồng thời có thể sử dụng ô tìm kiếm để tra cứu theo mã đơn, số điện thoại hoặc thông tin khách hàng.

Bước 3: Nhấn Xem chi tiết để kiểm tra thông tin đơn hàng như sản phẩm, người nhận, địa chỉ, phương thức thanh toán và trạng thái xử lý.

Bước 4: Sử dụng chức năng Chỉnh sửa để cập nhật trạng thái đơn hàng theo quy trình: Chờ xác nhận, Đã xác nhận, Đang đóng gói, Đang giao và Hoàn thành.

Bước 5: Trong trường hợp cần thiết, quản trị viên có thể thực hiện hủy đơn hàng nếu đơn không hợp lệ hoặc theo yêu cầu của khách hàng, hệ thống sẽ tự động cập nhật và ghi nhận lại thay đổi.



Hình 4.17 : Màn hình quản lý và theo dõi trạng thái đơn hàng

A.2.5. Quản lý Mã giảm giá (Coupon/Voucher)

Mục đích: Cung cấp công cụ giúp Quản trị viên khởi tạo, theo dõi và kiểm soát các chương trình khuyến mãi, mã giảm giá áp dụng trên hệ thống Electro Store nhằm kích cầu mua sắm hiệu quả.

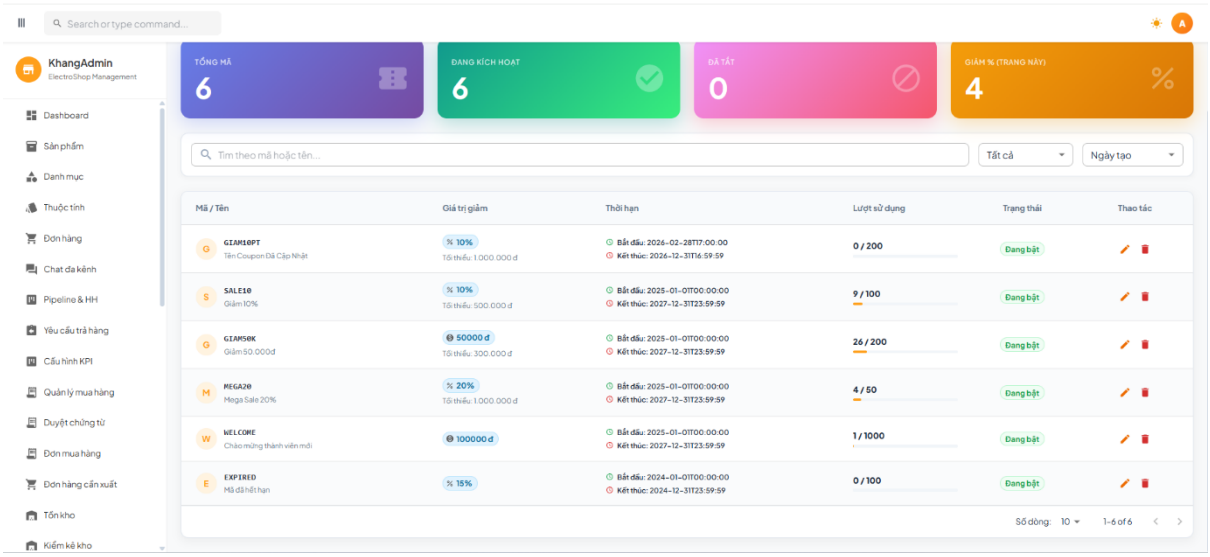
Điều kiện thực hiện: Quản trị viên đã đăng nhập thành công và đang ở khu vực quản trị của hệ thống.

Bước 1: Quản trị viên chọn mục quản lý mã giảm giá ở thanh điều hướng bên trái. Hệ thống tải dữ liệu và hiển thị giao diện tổng quan.

Bước 2: Phía trên giao diện có các thẻ số liệu cho biết Tổng mã, số mã Đang kích hoạt, số mã Đã tắt và số mã dạng Giảm %. Nhìn vào đây, Quản trị viên biết ngay tình trạng chung mà không cần kéo xuống xem từng dòng.

Bước 3: Nếu danh sách quá dài, có thể gõ mã hoặc tên vào ô tìm kiếm, hoặc lọc theo trạng thái, sắp xếp theo ngày tạo để thu hẹp kết quả.

Bước 4: Bảng danh sách hiển thị đầy đủ thông tin từng mã: tên/mã, mức giảm và điều kiện áp dụng, thời gian hiệu lực, số lượt đã dùng trên tổng số lượt cho phép (kèm thanh tiến trình), và trạng thái hiện tại.



Hình 4.18 : Giao diện tổng quan trang Quản lý Mã giảm giá

Bước 5: Ở cột Thao tác, nhấn biểu tượng bút để chỉnh sửa hoặc gia hạn mã, nhấn biểu tượng thùng rác để xóa những mã lỗi hoặc không dùng nữa.

A.2.6. Quản lý Danh mục sản phẩm

Mục đích: Cung cấp công cụ để Quản trị viên tổ chức sản phẩm theo hệ thống cấp bậc, kiểm soát trạng thái hiển thị trên cửa hàng và hỗ trợ tối ưu hóa SEO cho hệ thống Electro Store.

Điều kiện thực hiện: Quản trị viên đã đăng nhập thành công và đang ở khu vực quản trị của hệ thống.

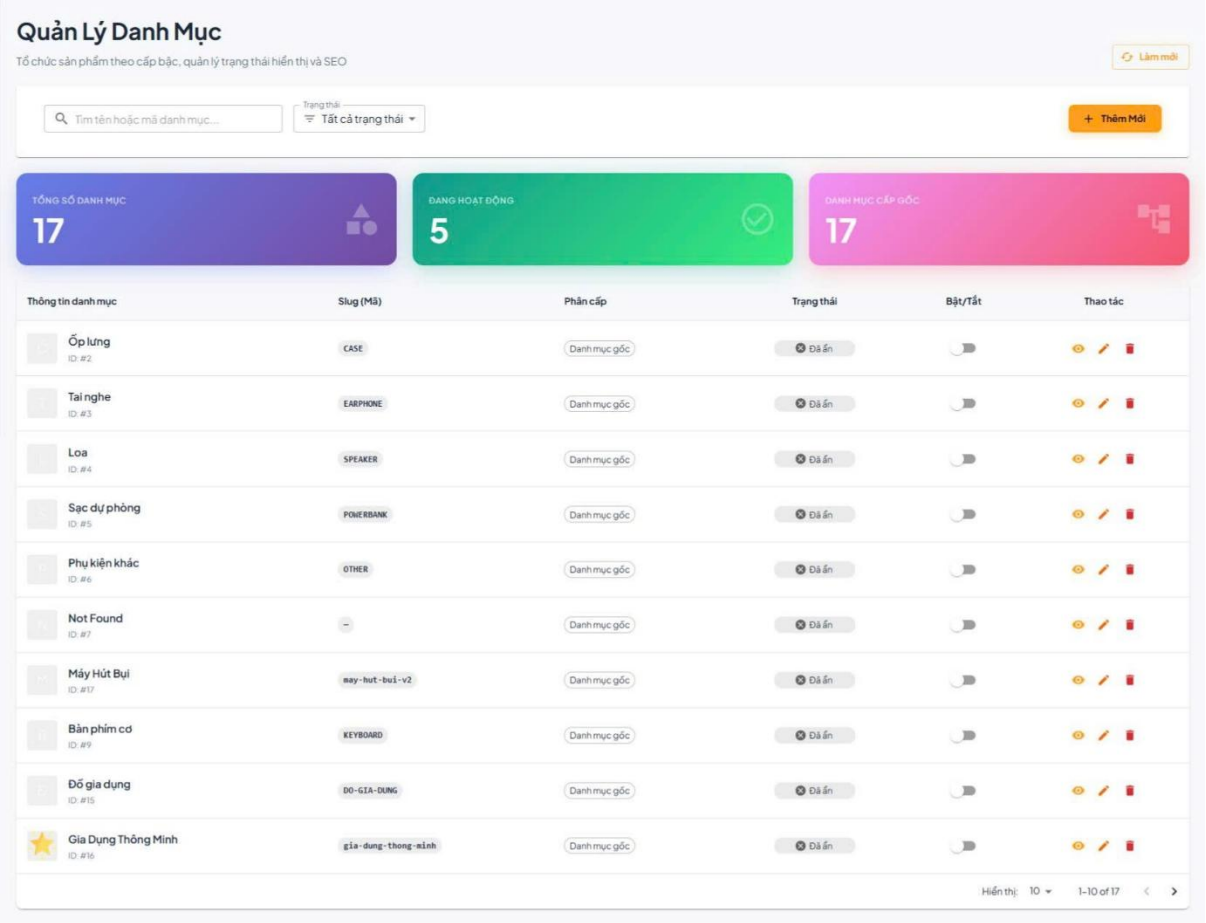
Bước 1: Quản trị viên chọn mục Danh mục ở thanh điều hướng bên trái. Hệ thống tải dữ liệu và hiển thị giao diện Quản Lý Danh Mục.

Bước 2: Phía trên giao diện có các thẻ thống kê: Tổng số danh mục, số danh mục Đang hoạt động và số Danh mục cấp gốc, giúp nắm nhanh cấu trúc danh mục hiện tại.

Bước 3: Admin có thể nhập tên hoặc mã danh mục vào ô tìm kiếm để tra cứu nhanh, đồng thời sử dụng bộ lọc theo trạng thái nhằm thu hẹp phạm vi kết quả hiển thị. Trong trường hợp cần cập nhật lại dữ liệu mới nhất, Admin chỉ cần nhấn nút **Làm mới** ở góc trên bên phải giao diện.

Bước 4: Khi cần mở rộng ngành hàng, quản trị viên (admin) có thể nhấn nút + Thêm Mới để tạo một danh mục sản phẩm mới.

Bước 5: Bảng danh sách bên dưới hiển thị mã định danh (Slug) và cấp danh mục (danh mục chính hoặc danh mục con) của từng mục. Quản trị viên có thể nhanh chóng bật hoặc tắt trạng thái hiển thị bằng công tắc, hoặc sử dụng các biểu tượng ở cột Thao tác để thực hiện các chức năng như xem chi tiết, chỉnh sửa thông tin hoặc xóa danh mục không còn sử dụng



Hình 4.19 : Màn Hình Quản Lý Danh Mục Sản Phẩm

A.2.7. Duyệt Đánh giá (Reviews)

Mục đích: Cung cấp công cụ giúp Quản trị viên xem xét, kiểm duyệt và phản hồi các ý kiến của khách hàng về sản phẩm trên hệ thống Electro Store, từ đó duy trì chất lượng dịch vụ và uy tín của cửa hàng.

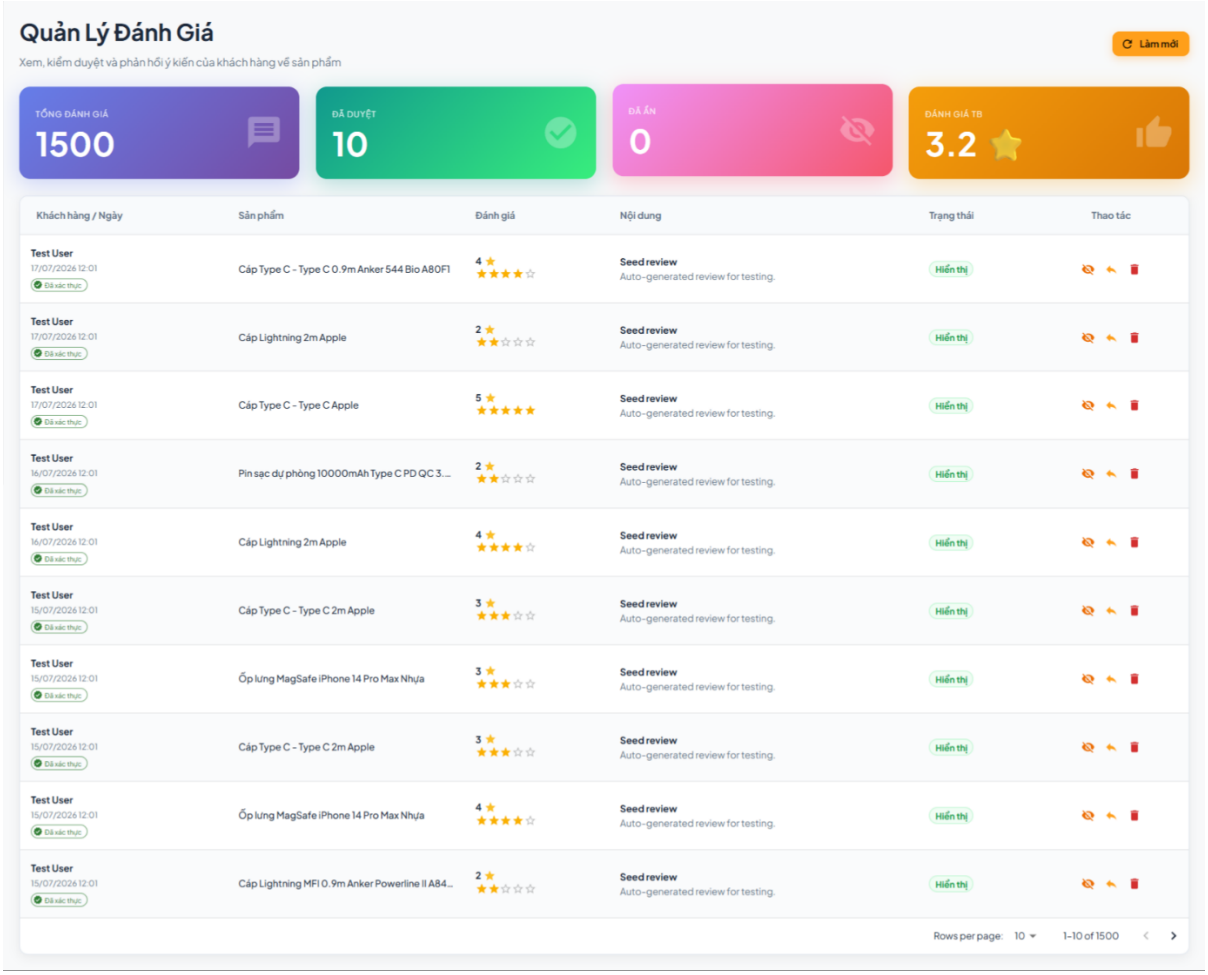
Điều kiện thực hiện: Quản trị viên đã đăng nhập thành công và đang ở khu vực quản trị của hệ thống.

Bước 1: Quản trị viên truy cập mục Đánh giá trong khu vực quản trị. Hệ thống tải và hiển thị toàn bộ dữ liệu đánh giá của người dùng nhằm phục vụ công tác kiểm duyệt.

Bước 2: Theo dõi thông tin tổng quan. Hệ thống cung cấp các chỉ số thống kê như tổng số đánh giá, số đánh giá đã được duyệt, số đánh giá bị ẩn và điểm đánh giá trung bình. Các thông tin này giúp Quản trị viên đánh giá nhanh tình trạng phản hồi của người dùng trên hệ thống.

Bước 3: Kiểm duyệt nội dung đánh giá. Quản trị viên xem danh sách chi tiết các đánh giá, bao gồm thông tin người dùng, sản phẩm được đánh giá, số sao, nội dung phản hồi và trạng thái hiển thị hiện tại.

Bước 4: Thực hiện xử lý đánh giá. Dựa trên nội dung và mức độ phù hợp, Quản trị viên có thể thay đổi trạng thái hiển thị (duyệt hoặc ẩn), phản hồi lại người dùng hoặc xóa các đánh giá vi phạm quy định, nội dung không phù hợp hoặc mang tính spam.



Hình 4.20 : Màn hình quản lý và phản hồi đánh giá khách hàng

A.2.8. Hướng dẫn Quy trình Bảo hành (Dành cho Sales Staff)

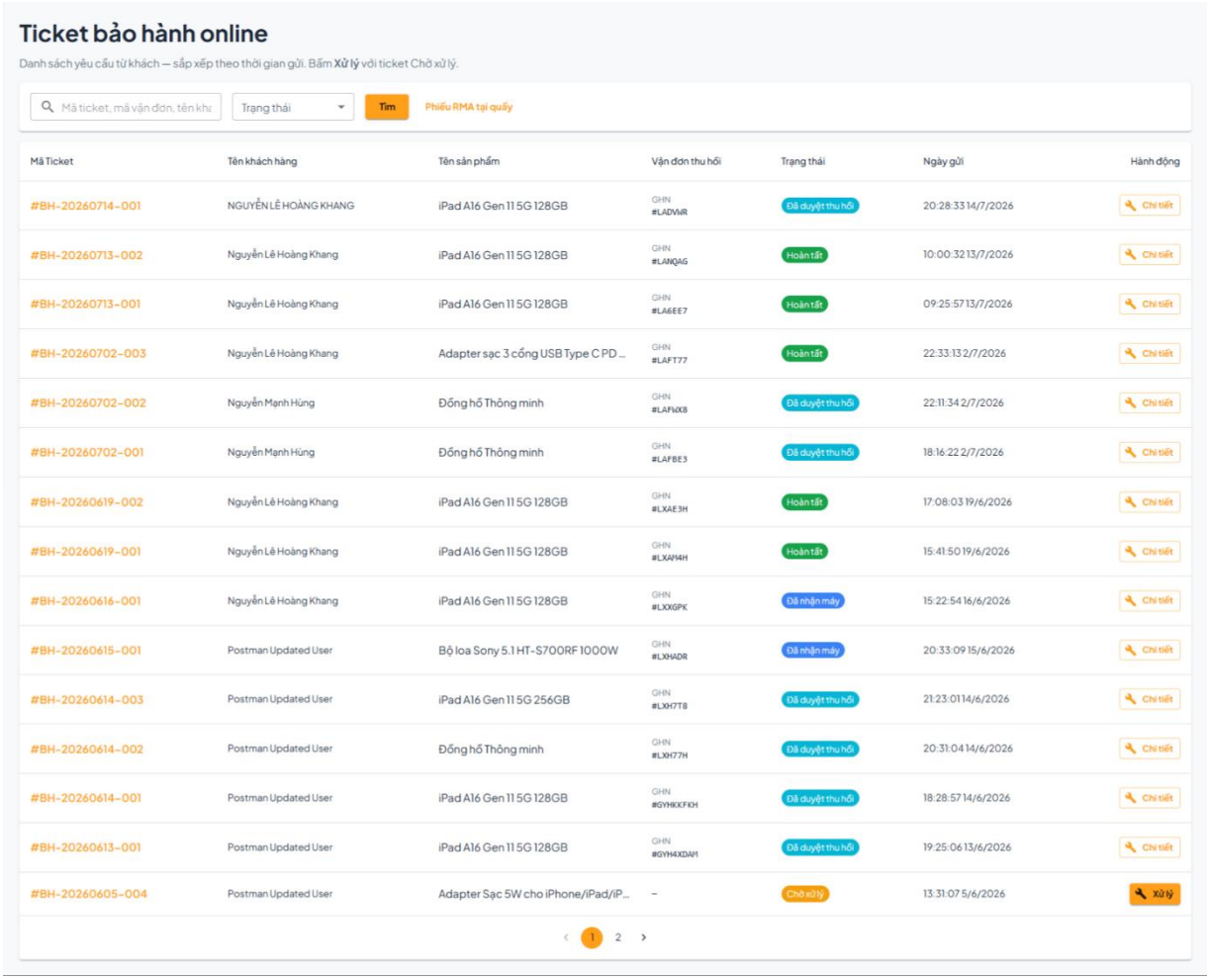
Mục đích: Bộ phận Sales là nơi đầu tiên tiếp nhận các yêu cầu bảo hành từ khách hàng gửi lên hệ thống. Công việc chính là kiểm tra thông tin sản phẩm, xem xét giấy tờ khách cung cấp và đưa ra quyết định chấp nhận hoặc từ chối yêu cầu. Sau khi Sales xử lý xong, các bước tiếp theo như nhận hàng và sửa chữa sẽ do bộ phận Kho và Kỹ thuật thực hiện. Khi khách hàng gửi yêu cầu bảo hành, hệ thống sẽ ghi nhận và đưa yêu cầu vào danh sách chờ xử lý. Nhân viên Sales thực hiện theo các bước sau:

Điều kiện thực hiện: Quản trị viên đã đăng nhập thành công và đang ở khu vực quản trị của hệ thống.

Bước 1: Nhân viên truy cập vào mục “Yêu cầu bảo hành” trên hệ thống và chọn danh sách các yêu cầu đang chờ xử lý để bắt đầu kiểm tra.

Bước 2: Mở chi tiết từng yêu cầu để xem thông tin sản phẩm mà khách hàng cung cấp, sau đó đối chiếu số IMEI hoặc số serial với dữ liệu trên hệ thống để xác định tình trạng bảo hành, đồng thời kiểm tra các hình ảnh, hóa đơn hoặc video đính kèm nhằm xác minh lỗi của sản phẩm.

Bước 3: Sau khi kiểm tra đầy đủ thông tin, nhân viên Sales sẽ đưa ra quyết định xử lý bằng cách lựa chọn "Duyệt thu hồi" nếu thông tin hợp lệ và sản phẩm đủ điều kiện bảo hành, khi đó hệ thống sẽ tạo thông tin vận chuyển để đơn vị giao nhận đến lấy sản phẩm từ khách hàng; ngược lại, nếu thông tin không hợp lệ hoặc sản phẩm không đủ điều kiện bảo hành, nhân viên sẽ chọn "Từ chối" và ghi rõ lý do để hệ thống phản hồi lại cho khách hàng.



Hình 4.21 : Giao diện quản lý và tiếp nhận các Yêu cầu bảo hành online

Bước 4: Sau khi đã duyệt hoặc từ chối, nhân viên có thể theo dõi lại danh sách để biết tình trạng của các yêu cầu đã xử lý, đặc biệt là các đơn đã được duyệt và đang trong quá trình vận chuyển.

A.2.9. Quản lý nhập/xuất kho và theo dõi thiết bị bằng mã IMEI/Serial

Mục đích : Chức năng này được xây dựng để hỗ trợ nhân viên kho quản lý thiết bị theo từng mã định danh riêng như IMEI hoặc Serial. Việc quản lý theo mã giúp hệ thống theo dõi chính xác vòng đời của từng sản phẩm, đồng thời đảm bảo hàng hóa được xuất kho theo đúng thứ tự nhập vào, hạn chế sai sót trong quá trình vận hành.

Điều kiện thực hiện: Quản trị viên đã đăng nhập thành công và đang ở khu vực quản trị của hệ thống.

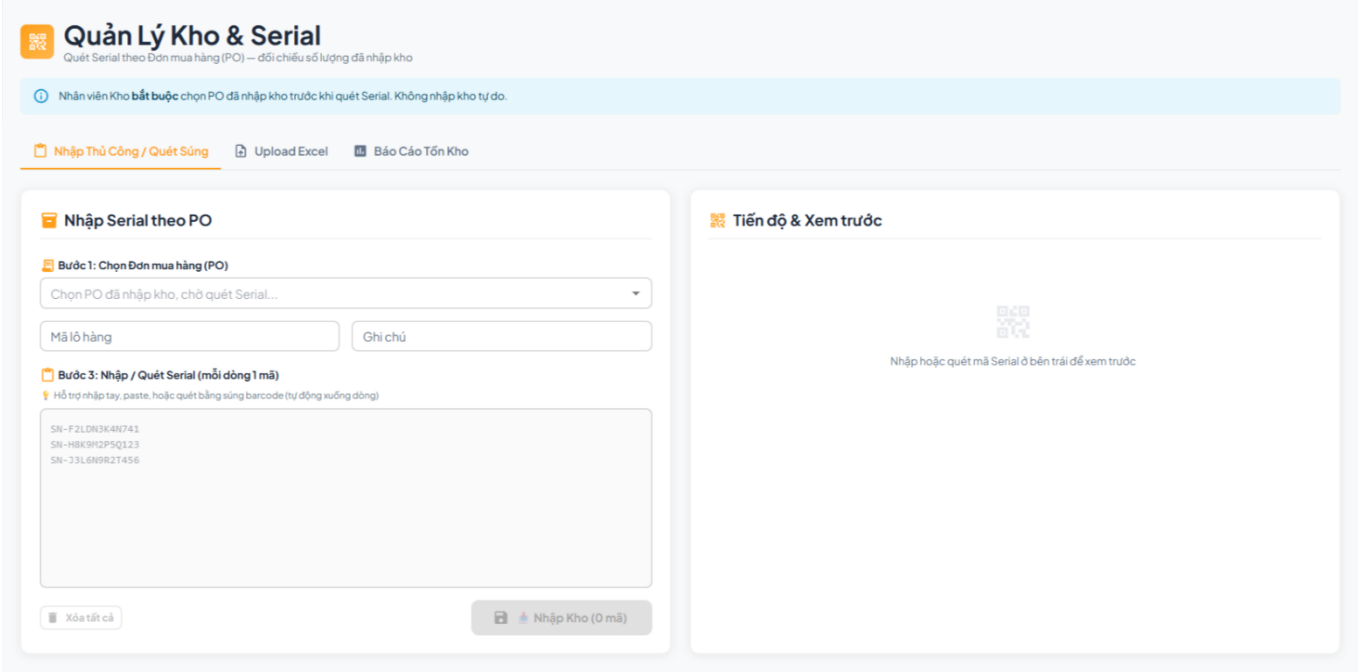
1. Quy trình nhập kho và gán mã định danh theo đơn đặt hàng

Bước 1: Nhân viên kho truy cập hệ thống và chọn mục nhập serial. Tại đây, hệ thống cho phép nhập dữ liệu theo hai cách là nhập trực tiếp hoặc sử dụng thiết bị quét mã vạch, tùy theo điều kiện thực tế tại kho.

Bước 2: Sau khi vào màn hình nhập mã định danh, nhân viên cần chọn đúng đơn mua hàng tương ứng. Mỗi mã định danh hoặc số serial chỉ được phép gán vào một đơn mua hàng cụ thể, điều này giúp đảm bảo dữ liệu nhập kho có thể truy vết rõ ràng khi cần kiểm tra.

Bước 3: Nhân viên tiến hành nhập hoặc quét lần lượt các mã của sản phẩm. Trong quá trình này, hệ thống sẽ hiển thị danh sách các mã đã ghi nhận để người dùng dễ dàng đối chiếu và kiểm tra lại nếu có sai sót.

Bước 4: Sau khi hoàn tất việc nhập danh sách mã, nhân viên xác nhận để hệ thống lưu dữ liệu và hoàn tất quá trình nhập kho cho lô hàng tương ứng.



Hình 4.22 : Màn hình nhập mã định danh Serial theo đơn mua hàng

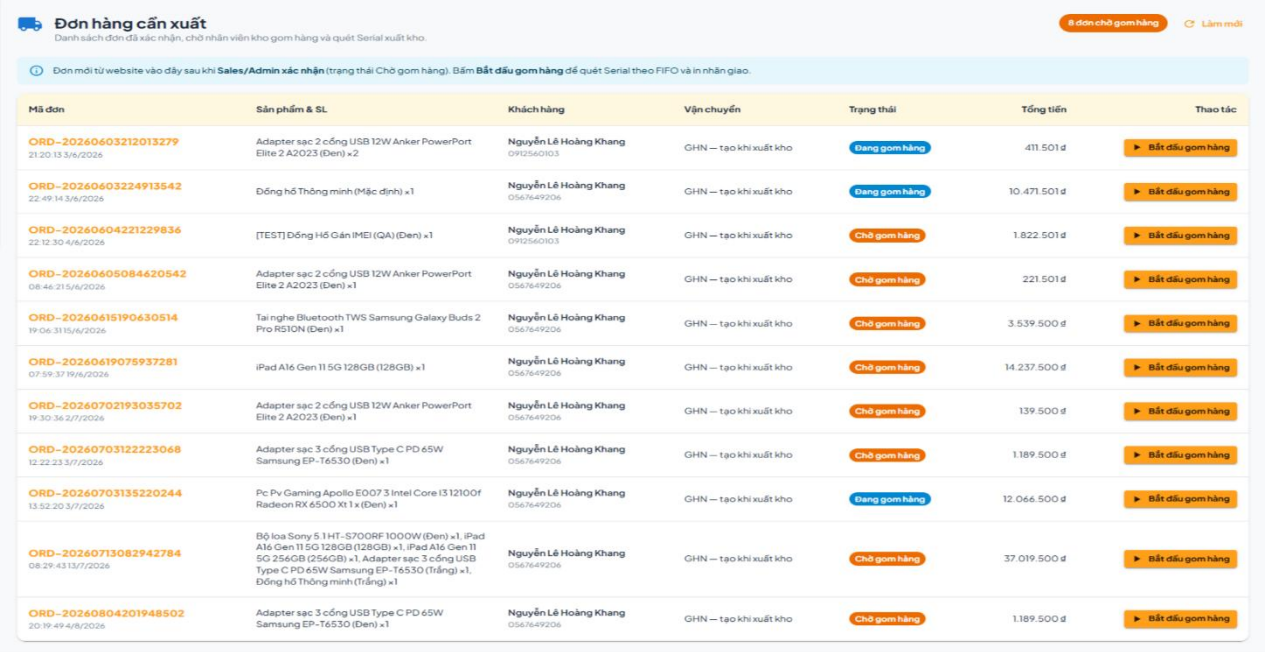
2. Quy trình xem và xử lý đơn hàng cần xuất kho

Bước 1: Nhân viên kho truy cập mục đơn hàng cần xuất để xem các đơn đã được xác nhận và chuyển sang trạng thái chờ xử lý.

Bước 2: Hệ thống hiển thị các đơn hàng đang ở trạng thái chờ gom hàng hoặc đang trong quá trình xử lý. Đây là các đơn đã được bộ phận liên quan xác nhận và sẵn sàng để xuất kho.

Bước 3: Nhân viên chọn một đơn cụ thể và bắt đầu quá trình gom hàng. Khi đó hệ thống sẽ chuyển sang giao diện chi tiết để hướng dẫn thao tác xuất kho

3. Quy trình gom hàng và xuất kho theo FIFO

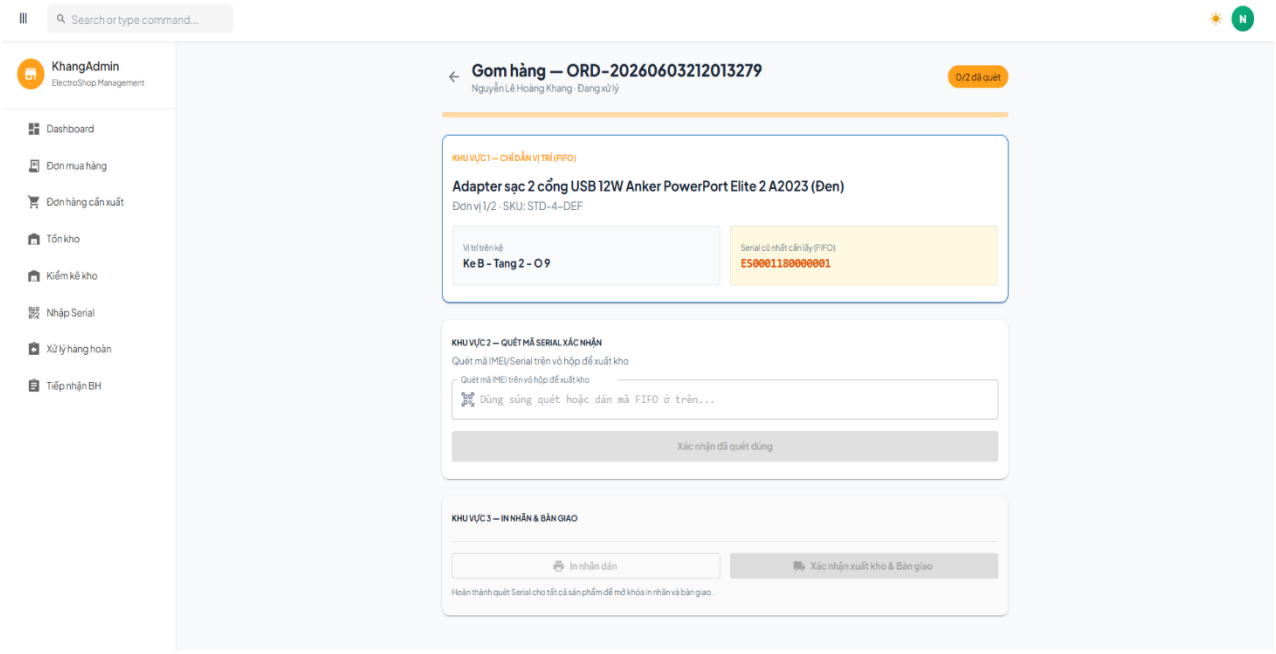


Hình 4.23 : Màn hình danh sách đơn hàng cần xuất kho

Sau khi bắt đầu xử lý đơn hàng, hệ thống hiển thị thông tin chi tiết sản phẩm cần xuất cùng vị trí lưu trữ trong kho. Dựa trên nguyên tắc FIFO, hệ thống ưu tiên các mã được nhập trước để đảm bảo tính nhất quán trong quản lý tồn kho.

Nhân viên kho di chuyển đến đúng vị trí được chỉ định, lấy sản phẩm theo mã Serial hiển thị trên hệ thống và tiến hành quét xác nhận. Hệ thống sẽ kiểm tra lại dữ liệu trước khi cho phép tiếp tục bước tiếp theo nhằm hạn chế sai lệch giữa thực tế và dữ liệu ghi nhận.

Khi số lượng sản phẩm đã được xác nhận đầy đủ, hệ thống cho phép in nhãn vận chuyển. Nhân viên dán nhãn lên kiện hàng và xác nhận hoàn tất quá trình xuất kho. Sau bước này, đơn hàng được chuyển sang đơn vị vận chuyển để tiếp tục quy trình giao hàng.



Hình 4.24 : Màn hình gom hàng và xuất kho theo FIFO

PHỤ LỤC B. HƯỚNG DẪN CÀI ĐẶT HỆ THỐNG

B.1. CẤU HÌNH HỆ THỐNG YÊU CẦU

Để cài đặt và vận hành hệ thống Electro Store trên môi trường phát triển cục bộ, máy tính cần đáp ứng các yêu cầu cơ bản sau:

+ **Hệ điều hành:** Windows 10 hoặc Windows 11, Linux Ubuntu 20.04 trở lên hoặc macOS.
+ **Bộ nhớ RAM:** Tối thiểu 8 GB; khuyến nghị từ 16 GB trở lên khi khởi chạy đồng thời toàn bộ các microservice.
+ **Java Development Kit:** JDK 17.
+ **Apache Maven:** Phiên bản 3.8 trở lên.
+ **Node.js:** Phiên bản 18 trở lên.
+ **Python:** Phiên bản 3.10 trở lên.
+ **MySQL Server:** Phiên bản 8.0 trở lên.
+ **Redis:** Hai instance hoạt động lần lượt tại cổng 6379 và 6380.
+ **Docker:** Khuyến nghị sử dụng Docker Desktop và Docker Compose để triển khai MySQL, Redis và các thành phần hạ tầng liên quan.

B.2. CÀI ĐẶT BACKEND

B.2.1. Công nghệ sử dụng

Backend của hệ thống Electro Store được xây dựng và vận hành dựa trên các công nghệ chính sau:

+ **Java 17:** Ngôn ngữ và nền tảng thực thi chính cho các dịch vụ Backend.
+ **Spring Boot và Spring Cloud:** Hỗ trợ xây dựng, cấu hình và vận hành các dịch vụ trong kiến trúc phân tán.
+ **Apache Maven 3.8 trở lên:** Quản lý thư viện, biên dịch, kiểm thử và đóng gói các module Java.
+ **Microservices và RESTful API Gateway:** Phân tách các miền nghiệp vụ thành dịch vụ độc lập và cung cấp điểm truy cập tập trung cho Client.
+ **MySQL 8:** Lưu trữ dữ liệu nghiệp vụ theo mô hình database-per-service.

+ **Redis:** Lưu trữ refresh token, dữ liệu tạm và hỗ trợ xử lý nhanh cho các tác vụ phù hợp.
+ **Eureka, Config Server và OpenFeign:** Hỗ trợ đăng ký và phát hiện dịch vụ, quản lý cấu hình tập trung và giao tiếp giữa các microservice.
+ **Spring Security và JWT:** Thực hiện xác thực, phân quyền và bảo vệ các API của hệ thống.
+ **Docker:** Triển khai các thành phần hạ tầng cục bộ như MySQL và Redis trong môi trường container.

B.2.2. Chuẩn bị môi trường

Trước khi biên dịch và khởi chạy các microservice Backend, cần hoàn tất các bước chuẩn bị sau:

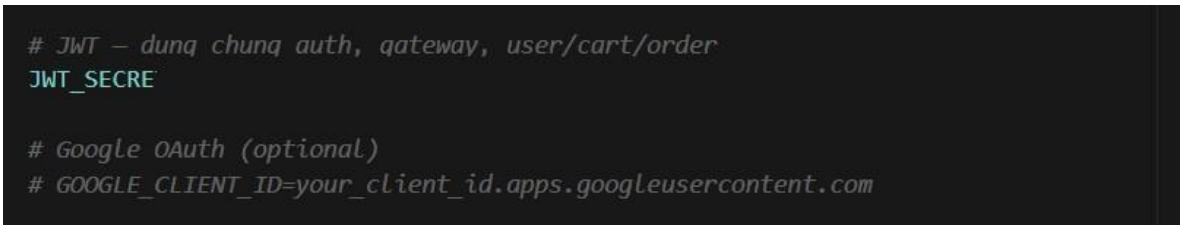
+ **Cài đặt Java:** Cài đặt JDK 17, cấu hình biến môi trường JAVA_HOME trỏ đến thư mục JDK và bổ sung thư mục bin vào biến PATH.
+ **Cài đặt Maven:** Sử dụng Apache Maven phiên bản 3.8 trở lên để tải thư viện, biên dịch và đóng gói các module Java.
+ **Chuẩn bị hạ tầng dữ liệu:** Khuyến nghị sử dụng Docker Desktop để triển khai MySQL và Redis thông qua Docker Compose nhằm đảm bảo tính đồng nhất và thuận tiện trong quá trình cài đặt. Trong trường hợp không sử dụng Docker, cần tiến hành cài đặt thủ công MySQL phiên bản 8 và cấu hình hai instance Redis chạy trên các cổng tương ứng theo yêu cầu của hệ thống.
+ **Tải mã nguồn:** Sử dụng Git để clone toàn bộ mã nguồn dự án về máy phát triển và mở thư mục Backend bằng IntelliJ IDEA hoặc công cụ phát triển tương đương.
+ **Tạo tệp cấu hình môi trường:** Sao chép tệp .env.example thành .env, sau đó cấu hình kết nối MySQL, Redis và khóa JWT_SECRET. Không đưa tệp .env chứa thông tin bí mật lên kho mã nguồn công khai.

B.2.3. Các bước thực hiện

+ **Kiểm tra tài nguyên và cổng dịch vụ:** Khuyến nghị máy có tối thiểu 16 GB

+ **RAM khi chạy toàn bộ hệ thống.** Các cổng 8080, 8761, 8888, 3306, 6379 và 6380 cần ở trạng thái chưa bị ứng dụng khác sử dụng.

Cấu hình JWT: Khóa JWT_SECRET được sử dụng chung bởi các dịch vụ liên quan đến xác thực và kiểm tra token. Giá trị cần đủ dài, khó dự đoán và được bảo vệ như một thông tin bí mật.



Hình 5.1 : Cấu hình biến môi trường JWT Secret đã che giá trị

Cấu hình MySQL: Các biến môi trường xác định địa chỉ máy chủ, cổng kết nối, tài khoản, mật khẩu và chế độ khởi tạo cơ sở dữ liệu cho các service.



Hình 5.2 : Minh họa cấu hình kết nối MySQL trong tệp môi trường

Cấu hình Redis: Các instance Redis cần được cấu hình tách biệt theo mục đích sử dụng, trong đó Redis xác thực thường hoạt động tại cổng 6379 và Redis thống kê hoạt động tại cổng 6380.



Hình 5.3 : Minh họa cấu hình kết nối Redis trong tệp môi trường

Các bước thực hiện

- **Bước 1: Mở dự án**

+ Clone hoặc giải nén mã nguồn dự án vào thư mục làm việc trên máy tính.
+ Mở thư mục gốc bằng IntelliJ IDEA và chờ công cụ tải cấu hình Maven.
+ Kiểm tra các module đã được nhận diện đầy đủ, đặc biệt là module dùng chung shared.

- **Bước 2: Cấu hình và khởi chạy hạ tầng**
  + Sao chép tệp .env.example thành .env, sau đó khai báo thông tin MySQL, Redis và JWT_SECRET.
  + Khởi động các container hạ tầng bằng lệnh docker compose up -d.
  + Trên Windows, import cơ sở dữ liệu bằng script Import-Databases.ps1. Trên Linux hoặc macOS, sử dụng script import-databases.sh.
- **Bước 3: Build và chạy hệ thống**
  + Build và cài đặt module dùng chung vào local Maven repository bằng lệnh mvn -f shared/pom.xml install -DskipTests.
  + Trên Windows, khởi chạy hệ thống bằng script START.ps1.
  + Trên Linux hoặc macOS, cấp quyền thực thi khi cần và chạy script ./start.sh.
- **Bước 4: Kiểm tra trạng thái**
  + Truy cập API Gateway tại địa chỉ http://localhost:8080.
  + Truy cập Eureka Dashboard tại địa chỉ http://localhost:8761 để kiểm tra các service đã đăng ký.
  + Có thể kiểm tra nhanh toàn bộ dịch vụ bằng script check-all-services.ps1 trên Windows hoặc check-all-services.sh trên Linux và macOS.

B.3. CÀI ĐẶT FRONTEND

B.3.1. Công nghệ sử dụng

Frontend của hệ thống Electro Store được xây dựng dựa trên các công nghệ chính sau:

+ **React:** Thư viện chính được sử dụng để xây dựng giao diện người dùng theo mô hình component.

+ **Vite:** Công cụ khởi tạo, phát triển và đóng gói ứng dụng Frontend với tốc độ xử lý nhanh.
+ **Material UI (MUI):** Cung cấp hệ thống component giao diện, bố cục và chủ đề thống nhất cho ứng dụng.
+ **Redux Toolkit:** Quản lý trạng thái dùng chung giữa các component và các luồng nghiệp vụ của ứng dụng.
+ **React Router:** Quản lý định tuyến và điều hướng giữa các màn hình phía Client.
+ **Axios và HTTP Client:** Gửi yêu cầu HTTP từ Frontend đến các dịch vụ Backend thông qua API Gateway.
+ **npm:** Quảnlý thư viện JavaScript dựa trên tệp khóa phiên bản package-lock.json, giúp quá trình cài đặt dependency nhất quán giữa các môi trường.

B.3.2. Chuẩn bị môi trường

Trước khi cài đặt và khởi chạy ứng dụng Frontend, cần bảo đảm các điều kiện sau:

+ **Cài đặt Node.js và npm:** Sử dụng Node.js phiên bản 18 trở lên và npm đi kèm. Môi trường CI của dự án sử dụng Node.js 18 để bảo đảm tính nhất quán trong quá trình build.
+ **Chuẩn bị mã nguồn:** Mã nguồn dự án cần được tải về máy và các lệnh Frontend được thực hiện trong thư mục frontend/.
+ **Kiểm tra tệp cấu hình:** Trong thư mục Frontend cần có các tệp chính gồm package.json, vite.config.js và .env.example.
+ **Chuẩn bị Backend:** Khi chạy đầy đủ giao diện và các chức năng nghiệp vụ,

Backend cùng API Gateway cần hoạt động tại địa chỉ http://localhost:8080 để Frontend có thể gửi yêu cầu API.

B.3.3. Các bước thực hiện

- **Bước 1:** Di chuyển vào thư mục Frontend**.** Mở Terminal tại thư mục gốc của dự án và chuyển vào thư mục frontend/.

- **Bước 2:** Cài đặt các gói phụ thuộc. Chạy lệnh npm install để cài đặt các dependency được khai báo trong package.json và đồng bộ phiên bản theo package-lock.json.
- **Bước 3:** Cấu hình biến môi trường. Sao chép tệp .env.example thành .env, sau đó kiểm tra cấu hình địa chỉ API. Trong môi trường phát triển, các yêu cầu API được chuyển tiếp thông qua proxy của Vite đến API Gateway tại cổng 8080.
- **Bước 4:** Khởi chạy ứng dụng. Chạy lệnh npm run dev. Sau khi Vite khởi động thành công, truy cập giao diện tại địa chỉ http://localhost:5173.

B.4. CÀI ĐẶT VÀ CHẠY MÔ HÌNH AI (PYTHON API)

B.4.1. Công nghệ sử dụng

Dịch vụ gợi ý sản phẩm được xây dựng dưới dạng Python API độc lập với các công nghệ chính sau:

+ **FastAPI:** Framework web hiệu năng cao được sử dụng để xây dựng các API huấn luyện và truy vấn danh sách sản phẩm gợi ý.
+ **Uvicorn:** ASGI Server chịu trách nhiệm khởi chạy và phục vụ ứng dụng FastAPI.
+ **scikit-surprise:** Thư viện xây dựng hệ thống gợi ý Collaborative Filtering, được sử dụng để triển khai thuật toán Item-based KNNBaseline.
+ **Pandas:** Hỗ trợ đọc, làm sạch, biến đổi và xử lý dữ liệu rating phục vụ quá trình huấn luyện mô hình.
+ **SQLAlchemy và PyMySQL:** Cung cấp cơ chế kết nối, truy vấn và lấy dữ liệu từ cơ sở dữ liệu MySQL.
+ **pickle:** Lưu mô hình đã huấn luyện và nạp lại mô hình từ tệp reco_model.pkl, giúp hạn chế chi phí huấn luyện lại khi dịch vụ khởi động.

Mô hình gợi ý**:** Hệ thống sử dụng phương pháp Item-based Collaborative Filtering với thuật toán KNNBaseline và độ tương đồng Pearson Baseline để xác định các sản phẩm có đặc điểm đánh giá tương tự.

B.4.2. Các bước thực hiện

- **Bước 1: Cài đặt Python**
  + Tải và cài đặt Python phiên bản 3.10 trở lên từ trang : https://www.python.org.
  + Mở Terminal và chạy lệnh python --version để kiểm tra phiên bản đã cài đặt.
- **Bước 2: Cài đặt các thư viện cần thiết**
  + Mở Terminal tại thư mục reco-service/.
  + Khuyến nghị tạo môi trường ảo bằng lệnh python -m venv .venv, sau đó kích hoạt môi trường phù hợp với hệ điều hành đang sử dụng.
  + Chạy lệnh pip install -r requirements.txt để cài đặt FastAPI, Uvicorn, Pandas, scikit-surprise, SQLAlchemy, PyMySQL và các thư viện liên quan.
- **Bước 3: Cấu hình môi trường và mô hình**
  + Sao chép tệp .env.example thành .env.
  + **Khai báo kết nối MySQL:**

```
MYSQL_HOST = os.getenv("MYSQL_HOST", "localhost")
MYSQL_PORT = os.getenv("MYSQL_PORT", "3306")
MYSQL_USER = os.getenv("MYSQL_USER", "root")
MYSQL_PASSWORD = os.getenv("MYSQL_PASSWORD", "")
MYSQL_DATABASE = os.getenv("MYSQL_DATABASE", "electro_review_db")
ORDER_DB_NAME = os.getenv("ORDER_DB_NAME", "electro_order_db")

DATABASE_URL = (
    f"mysql+pymysql://{MYSQL_USER}:{MYSQL_PASSWORD}"
    f"@{MYSQL_HOST}:{MYSQL_PORT}/{MYSQL_DATABASE}"
    f"?charset=utf8mb4"
)
```

Hình 5.4 : Cấu hình kết nối MySQL của Recommendation Service

  + Bảo đảm cơ sở dữ liệu MySQL đã có dữ liệu rating, đánh giá hoặc lịch sử đơn hàng để phục vụ quá trình huấn luyện.
  + Mô hình được lưu và nạp từ tệp models/reco_model.pkl. Khi chưa có mô hình, có thể gọi API /train để huấn luyện trước khi yêu cầu danh sách gợi ý.

- **Bước 4**: Khởi chạy Python API
  + Tại Terminal của reco-service/, sau khi kích hoạt môi trường Python, chạy lệnh uvicorn main:app --host 0.0.0.0 --port 5003 --reload.
  + Dịch vụ hoạt động tại địa chỉ http://localhost:5003.
  + Kiểm tra trạng thái dịch vụ tại http://localhost:5003/health và truy cập tài liệu API tại http://localhost:5003/docs.